

Master Problem Sheet:

CSE110

CSE111

CSE230

CSE220

CSE221

CSE 110

LAB ASSIGNMENT # 01

Introduction to Flowcharts and Java Programming



CSE110 | Programming Language I

	LAB TASKS	HOME TASKS
CODING	06	05
FLOWCHART	04	05

NOTE: You need to submit only the Home Tasks. Submit all the Flowchart or Tracing tasks hand drawn or handwritten respectively to your Lab Instructors before the next lab. Submit all the Homework Coding Tasks in the Google Form shared on buX.

Lab Policy: [Lab Policy Student Version - Summer 2025 Onward](#)
Must Submit the Lab Agreement Form: [Form Link](#)

LAB TASKS

[NO NEED TO SUBMIT]

FLOWCHART TASKS

Question: 1

Design a flowchart to take a number as input, store it in a variable, and print its cube.

Question: 2

Create a flowchart to convert a temperature in Fahrenheit to Celsius. The user should input the temperature in Fahrenheit, and the flowchart should display the corresponding temperature in Celsius. The formula to convert Fahrenheit to Celsius is: $C = (F - 32) \times 5/9$

Question: 3

Jamal is struggling with his math homework, which requires him to calculate the volume of water needed to fill a cylinder completely. Unable to solve the problem on his own, he turns to you, his friend who excels in mathematics and computer science. To assist him, you decide to create a flowchart that takes the cylinder's height (H), radius (R), and the current height of water (h) as user inputs and determines the additional volume of water needed to fill the cylinder to its full capacity.

$$[\text{Volume of Cylinder} = \pi * R^2 * H]$$

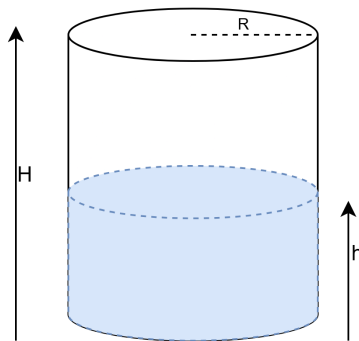


Fig. 1

Question: 4

Create a flowchart to find the average of three numbers. The flowchart should take three inputs from the user and display their average.

For this course, we'll be using **DrJava** as IDE for Java Coding:

[Link to JDK and DrJava](#)

DrJava Installation Guide:

[Windows Installation](#)

[MAC Installation](#)

CODING TASKS

Question: 1

Find out which of the following are legal identifiers in Java, and which are not. Also, explain why the invalid ones are invalid. You can try to define each of these (items a to l below) as a variable in the Dr java interaction pane and find out.

The first one is done for you as an Example. If you want to define hungry, you have to try

int hungry;

a) hungry	b) 2AB	c) 312.2	d) MOBILE
e) "Ans"	f) \$30	g) Yes/No	h) student-id
i) A+3	j) 'X'	k) public	l) student_id

Question: 2

Write the Java code for the following:

- 2.1 Declare an **integer** variable. Initialize it with some value of your choice and print it to check the value has been stored properly.
- 2.2 Declare and initialize another **integer** variable. Add this to the first one and print out the result. Verify that the addition has been done correctly.
- 2.3 Now print the product and division of the two **integer** numbers.
- 2.4 Repeat exercises 2.1, 2.2, and 2.3 for variables of data type **double**. Verify your answers.

2.5 Repeat exercises 2.1, 2.2, and 2.3 for one **double** data type and one **integer** datatype. Verify your answers.

2.6 Repeat exercises 2.1 and 2.2 for variables of data type **String**. How does the addition operator work for Strings? What if the first variable is an integer and the second is a String and vice versa?

Question: 3

Write a Java code that calculates and prints the circumference and area of a circle with a radius of 4 units. [Hint: use Math.PI to get the value of π]

Question: 4

Write a Java code where given an integer, we need to print the last 2 digits of that number.

Question: 5

A local tailor named Jamal often receives measurement details from his foreign customers in inches, but his measuring tools only use meters. To make his work easier, Jamal decides to ask his programmer friend (you) to write a simple Java program that will convert inches into meters.

[NOTE: 1 inch = 0.0254 meters]

Test Data:

Given a value for inch: 1000

Expected Output:

1000 inch is 25.4 meters

Question: 6

Write a Java program declaring two integer variables and initializing them. Your task is to swap the values of these two variables. You must complete it using two different approaches.

- a. By creating a third variable.
- b. Without creating any other variables.

Home TASKS

FLOWCHART TASKS

Question: 1

Design a flowchart to take an integer consisting of 4 digits as input and print the last 2 digits of that number.

Question: 2

Design a flowchart to take an integer consisting of 4 digits as input and print the first 2 digits of that number.

Question: 3

Design a flowchart to take three integers a , n , d as input where a is the first term of an arithmetic sequence, n is the number of terms and d is the common difference between the terms in the sequence, and print the n -th term of the arithmetic sequence.

Question: 4

Design a flowchart to take two numbers a and c as input (look at the image below), and print the area of the portion colored in green. In the following image, a is the radius of the circle, and c is the length of the sides of the square.

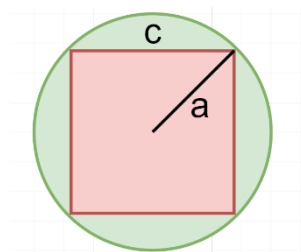


Fig. 2

Question: 5

Design a flowchart to take a number a as input (look at the image below), and print the area of the portion colored in red. In the following image, a is the diameter of the circle.

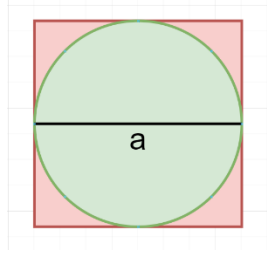


Fig. 3

CODING TASKS

Question: 1

NASA has recently shared the total time (in minutes) since one of its space missions was launched. As a software engineer working at the mission control center, your task is to help them calculate how many years and days have passed since the launch. Therefore, write a Java program that can do the conversion.

NOTE: For simplicity, assume each year has exactly **365 days**.

Test Data:

Given the number of minutes: 3456789

Expected Output:

3456789 minutes is approximately 6 years and 210 days

Question: 2

Suppose, you have three integer variables: a, b, c. Your first task is to assign the values 2, 5, 8 in these three variables. Next, you need to calculate and display the value of variable d using the following formula:

$$d = (2b \times \frac{c-a}{3}) + 7$$

Write a Java program based on this mentioned scenario that prints the value of d after calculation. **[Answer: 27]**

Question: 3

Rafi has just started learning Java programming. His teacher gives him a simple challenge to test his understanding of integer manipulation and modulus operations.

The task is to display the two rightmost digits of his student ID in reverse order.

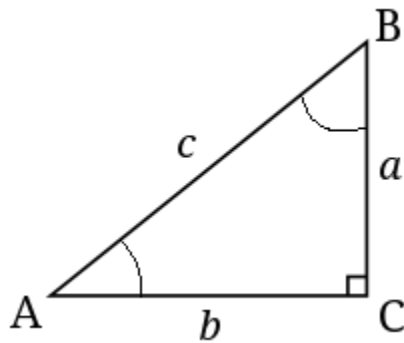
Rafi knows his student ID is **1000054943**, and he must write a Java program that prints the digits **3** and then **4**, since those are the two rightmost digits reversed.

Question: 4

Rafi is helping his friend place a Wi-Fi router in a rectangular room. He wants to know the shortest straight-line distance (diagonal) between two opposite corners of the room so that the Wi-Fi signal can reach the farthest corner efficiently. Write a Java program to calculate the diagonal distance using Pythagoras' Theorem, you can assume the length and width of the room is 10 and 13 respectively.

Question: 5

Design a Java program to calculate Sin and Cos values from a right-angled triangle.



Assume the values of a and b are 4.5 and 9.5 respectively. Finally, print the Sin and Cos values of angle A and angle B (SinA, CosA, SinB, CosB). The formulas to calculate these values are given below.

Hint: You need the values for all 3 sides to calculate both sin and cos. You are given only a and b. How would you get the value of c? You'll need the help of Math.sqrt().

Trigonometry formulas			
$\sin(A) = \frac{a}{c}$	$\cos(A) = \frac{b}{c}$	$\sin(B) = \frac{b}{c}$	$\cos(B) = \frac{a}{c}$

LAB ASSIGNMENT # 02

Branching



CSE110 | Programming Language I

	LAB TASKS	HOME TASKS
CODING	05	06
TRACING	01	02

NOTE: You need to submit only the Home Tasks. Submit all the Flowchart or Tracing tasks hand drawn or handwritten respectively to your Lab Instructors before the next lab. Submit all the Homework Coding Tasks in the Google Form shared on buX.

LAB TASKS
[NO NEED TO SUBMIT]

Question: 1

Write the Java code of a program to find the largest among three numbers entered by the user.

Sample Input	Sample Output
Enter Three Numbers: 100 23 -4	Largest Number: 100
Enter Three Numbers: -7 -36 -5	Largest Number: -5

Question: 2

You are developing a student grading system for a university portal. One of the features of this system is to automatically determine a student's letter grade based on their numerical score, which is entered by the user. The grading policy is defined as follows:

Scores	90-100	85-89	70-84	57-69	50-56	<50
Grades	A	A-	B	C	D	F

Sample Input	Output
45	Your grade is F
87	Your grade is A-

Question: 3

Write a Java code of a program that takes an integer number as user input and then determines if that number is divisible by both 5 and 7; otherwise display “No”. For example, numbers like 35, 70, 105, 140, 175, 210, 245, 280 etc. can be divisible by both 5 and 7.

Sample Input	Output
15	Invalid: Divisible by 5 Only
28	Invalid: Divisible by 7 Only
105	Divisible by Both
36	No

Question: 4

The Registrar’s Office at a University is preparing the academic calendar for upcoming sessions. Before finalizing the schedule, they need to determine whether a given year will have February 29 because leap years affect semester dates, holidays, and exam schedules.

To automate this, they’ve asked you to write a simple Java program that checks whether a year is a leap year or not, following the standard rules:

- A year is a leap year if it is divisible by 4.
- However, if the year is a century (divisible by 100), it must also be divisible by 400 to be a leap year.

Sample Input	Output
2020	2020 is a leap year
2001	2001 is not a leap year
1900	1900 is not a leap year
2000	2000 is a leap year

Question: 5

It's 11:59 PM, and Harun is desperately trying to finish a math assignment before the deadline. The final question involves a strange piecewise function, and Harun needs to write a program to verify the answers before submitting.

The function is defined as follows:

$$f(x) = \begin{cases} 2x, & x < 0 \\ x + 1, & 0 \leq x < 2 \\ x^2 - 1, & 2 \leq x < 5 \\ 3x^2 + 2, & x \geq 5 \end{cases}$$

Your task is to help Harun by writing a Java program that takes the value of x as input from the user and displays the corresponding value of f(x) according to the formula.

Sample Input	Output
-3	output: -6
1	output: 2
4	output: 15
10	output: 302

Question: 6

What will be the output of the following program? Show the workings.

1	<code>public class Tracing1 {</code>	Output
2	<code> public static void main(String[] args) {</code>	
3	<code> int num1 = 10;</code>	
4	<code> int num2 = -3;</code>	
5	<code> int num3 = -1;</code>	
6	<code> int sum = num1 + num2 + num3;</code>	
7	<code> if (num3 < 0) {</code>	
8	<code> System.out.println(num3 * (-2));</code>	
9	<code> }</code>	
10	<code> else {</code>	
11	<code> System.out.println(sum);</code>	
12	<code> }</code>	
13	<code> if (num1 < 5) {</code>	
14	<code> System.out.println(num1 + 10);</code>	
15	<code> }</code>	
16	<code> else if (num2 == -3) {</code>	
17	<code> num2 = num1;</code>	
18	<code> System.out.println(num2);</code>	
19	<code> }</code>	
20	<code> else {</code>	
21	<code> System.out.println(num1 + num2 + num3);</code>	
22	<code> }</code>	
23	<code> if (num1 > 15) {</code>	
24	<code> System.out.println(num1);</code>	
25	<code> }</code>	
26	<code> if (num2 == 0) {</code>	
27	<code> System.out.println(num2 + num3);</code>	

28	}
29	else {
30	System.out.println(num3);
31	}
32	if (sum != 0) {
33	System.out.println(100);
34	}
35	else {
36	System.out.println(sum + 100);
37	}
38	if (num1 > 0 && num2 < 0) {
39	System.out.println(num1 == num2);
40	}
41	else {
42	System.out.println("End");
43	}
44	}
45	}

HOME TASKS

Question: 1

Write a Java code of a program that takes an integer number as user input and then determines if that number is divisible by either 5 or 7 but not both; otherwise display “No”. For example, numbers like 35, 70, 105, 140, 175, 210, 245, 280 etc. can be divisible by both 5 and 7. These numbers are invalid.

Sample Input	Output
15	Divisible by 5 Only
28	Divisible by 7 Only
105	Invalid: Divisible by both
36	No

Question: 2

A new Smart Electric Car Charging Station has been set up in your city.

The station automatically calculates the total cost for charging based on the amount of electricity consumed (in kWh) and the type of user.

To make the billing fair and environmentally conscious, the station follows these rules:

Base rate: 15 Taka per kWh.

- Discounts and surcharges:
 - Students (under 18 years old) get a 20% discount on the total bill.
 - Regular users (18 to 60 years old) pay the normal rate.
 - Senior citizens (above 60 years old) get a 10% discount.
- High usage surcharge:
 - If total energy consumption exceeds 100 kWh, a 5% surcharge is added after applying any discount.

Sample Input	Output
Age: 16 Electricity Consumed: 50	Final Bill: 600.0 Taka
Age: 65 Electricity Consumed: 120	Final Bill: 1701.0 Taka

Question: 3

Write a Java program that takes 3 float numbers as input from the user and prints the maximum and minimum number from the inputs.

Sample Input	Output
18.83 4.02 83.12	Maximum number is 83.12 Minimum number is 4.02
-26.45 -0.02 -13.56	Maximum number is -0.02 Minimum number is -26.45

Question: 4

A triangle has 3 sides. Write a program which asks the users for input. Based on the input, your program should output whether it is an Equilateral, Isosceles or Scalene.

- Equilateral triangle has three sides with equal length
- Isosceles triangle has two sides with equal length and another side is different
- Scalene triangle has different lengths in each side

Sample Input	Output
5 2 4	This is a Scalene triangle
5 5 3	This is a Isosceles triangle
3 3 3	This is a Equilateral triangle

Question: 5

Suppose you are hired by a supershop named Bastob. Now, your task is to create a Java program that will help the cashier calculate the change to be returned.

The program takes two inputs. The first input is an integer number which is the amount of money to be paid in taka and the second integer is the amount of money the customer gave to the cashier. Your program should print the following:

- If the customer gave more money than the actual amount, print change the cashier should return in notes and coins.
- If the customer gave less money than the amount to be paid, then print the amount the customer needs to pay.

Consider the following denomination for notes and coins in taka:

Notes: 100, 50, 20, 10.

Coins: 5, 2, 1.

Sample Input	Output
Enter the amount the customer need to pay(Taka) 35 Enter the amount, customer gave(Taka) 53	The returned amount is 18 taka. 100 taka note: 0 50 taka note: 0 20 taka note: 0 10 taka note: 1 5 taka coin: 1 2 taka coin: 1 1 taka coin: 1
Enter the amount the customer need to pay(Taka) 60 Enter the amount, customer gave(Taka) 500	The returned amount is 440 taka. 100 taka note: 4 50 taka note: 0 20 taka note: 2 10 taka note: 0 5 taka coin: 0 2 taka coin: 0 1 taka coin: 0
Enter the amount the customer need to pay(Taka) 50 Enter the amount, customer gave(Taka) 50	The returned amount is 0 taka.
Enter the amount the customer need to pay(Taka) 550 Enter the amount, customer gave(Taka) 520	Please pay 30 taka more.

Question: 6

Write a Java program that reads three numbers and prints "All numbers are equal" if all three numbers are equal, "All numbers are different" if all three numbers are different and "Neither all are equal nor different" otherwise.

User Input	Output
Input the 1st number: 2345 Input the 2nd number: 2452 Input the 3rd number: 4532	All numbers are different
Input the 1st number: 230 Input the 2nd number: 230 Input the 3rd number: 112	Neither all are equal or different

Question: 7

What will be the output of the following program? Your answer will not be accepted without the workings.

1	public class Tracing2 {
2	public static void main(String[] args) {
3	boolean var1, var2, var3, var4, var5, var6;
4	boolean result1, result2, result3, result4, result5;
5	boolean result6, result7, result8, result9, result10;
6	var1 = var2 = var3 = var4 = var5 = var6 = false;
7	result1 = result2 = result3 = result4 = result5 = false;
8	result6 = result7 = result8 = result9 = result10 = false;
9	var1 = (!false false) && true;
10	var2 = var1 && true;
11	var3 = false && !true;
12	var4 = true;
13	var5 = false;
14	var6 = var3 && true;
15	result1 = (var1 && var2) && (40 % 3 > 45) (var5 && var6);
16	result2 = (var1 var2) (result1 && false);
17	result3 = (var1 && result1) result2 var5;
18	result4 = (var1 var2) ((var3 && var1) && false);
19	result5 = (var1 && var2) && (result3 var1);
20	result6 = ((var3 !var2) && result5) true;
21	result7 = (var4 && result1) && ((result1 && false) true);
22	result8 = ((var1 && result3) && (!var5 var6)) && true;
23	result9 = ((result2 && var2) (!result7 && var1)) && !false;
24	result10 = !(var1 && true);
25	System.out.println(result1 + " " + result2);
26	System.out.println(result3 + " " + result4);
27	System.out.println(result5 + " " + result6);
28	System.out.println(result7 + " " + result8);
29	System.out.println(result9 + " " + result10);
30	}
31	}

Question: 8

What will be the output of the following program? Your answer will not be accepted without the workings.

1	<code>public class Tracing3 {</code>
2	<code> public static void main(String[] args) {</code>
3	<code> int p = 5;</code>
4	<code> int q = 6;</code>
5	<code> int r = 9;</code>
6	<code> int sum = 0;</code>
7	<code> if (p < 12) {</code>
8	<code> System.out.println(r + 2);</code>
9	<code> }</code>
10	<code> else {</code>
11	<code> System.out.println(r + p);</code>
12	<code> }</code>
13	<code> if (q > 20){</code>
14	<code> System.out.println(r + 19);</code>
15	<code> }</code>
16	<code> else if (q <= 6) {</code>
17	<code> System.out.println(q + 3);</code>
18	<code> }</code>
19	<code> else{</code>
20	<code> System.out.println(p + q + r);</code>
21	<code> }</code>
22	<code> if (r > 15) {</code>
23	<code> System.out.println(r);</code>
24	<code> }</code>
25	<code> else if (r == 0) {</code>
26	<code> System.out.println(p + q);</code>
27	<code> }</code>
28	<code> else {</code>
29	<code> System.out.println(p);</code>
30	<code> }</code>
31	<code> if (sum != 0) {</code>
32	<code> System.out.println(3);</code>
33	<code> }</code>
34	<code> else {</code>
35	<code> System.out.println(sum + 32);</code>
36	<code> }</code>
37	<code> if(p > 0 && r < 10){</code>

38	<code>System.out.println(p + r);</code>
39	<code>}</code>
40	<code>else {</code>
41	<code>System.out.println(p - r);</code>
42	<code>}</code>
43	<code>}</code>
44	<code>}</code>

LAB ASSIGNMENT # 03

Linear Loop



CSE110 | Programming Language I

	LAB TASKS	HOME TASKS
CODING	07	08
TRACING	01	02

NOTE: You need to submit only the Home Tasks. Submit all the Flowchart or Tracing tasks hand drawn or handwritten respectively to your Lab Instructors before the next lab. Submit all the Homework Coding Tasks in the Google Form shared on buX.

LAB TASKS
[NO NEED TO SUBMIT]

Question: 1

Write a Java code that would print the following sequences using while/for loop:

- I. **24, 18, 12, 6, 0, -6**
- II. **18, -27, 36, -45, 54, -63**

Question: 2

Write a Java program that will take N numbers from the user and find their sum and average using a for loop.

Sample Input:

N = 5

Input the 5 numbers:

1
2
3
4
5

Expected Output:

The sum of 5 no is: 15

The Average is: 3.0

Question: 3

You are the engineer maintaining the power distribution system of NeoVolt City, a futuristic city powered by solar grids.

Each day the power grid generates and loses energy based on environmental conditions:

- If the day number is divisible by 3, a solar storm occurs, power drops by 45 units
- If divisible by 5, wind turbines overproduce, power increases by 25 units
- Otherwise, normal operation, power increases by 10 units

The grid starts with 40 units of power.

If the power ever exceeds 500, excess power is wasted and reduced by 50 units to protect it from overloading.

If power falls below 0, a blackout occurs, stop the simulation immediately.

Sample Input	Sample Output
Enter Number of Days: 5	Final Power: 50

Enter Number of Days: 12	Blackout on Day 9
--------------------------	-------------------

Question: 4

Write a Java program that will keep taking integer numbers as inputs from the user and print the square of those numbers until it gets a negative number and then stop.

Sample Input/Output: (The purple numbers are input.)

Enter Number: 2

$2^2 = 4$

Enter Number: 6

$6^2 = 36$

Enter Number: 1

$1^2 = 1$

Enter Number: 4

$4^2 = 16$

Enter Number: -5

Question: 5

Write a Java program that will calculate the value of y where the expression of y is as follows :

$$y = 1^2 - 2^2 + 3^2 - 4^2 + 5^2 \dots\dots\dots + n^2$$

Sample Input	Output	Explanation
5	15	$y = 1 - 4 + 9 - 16 + 25 = 15$
10	-55	$y = 1 - 4 + 9 - 16 + 25 - 36 + 49 - 64 + 81 - 100 = -55$
20	-210	$y = 1 - 4 + 9 - 16 + 25 \dots\dots\dots - 400 = -210$

Question: 6

Write a program in Java that asks the user for an **integer** input and counts the number of digits in the number.

Hint: You may keep dividing the number by ten and count how many times this can be done until the number becomes 0.

Sample Input	Output
7546	Total digits = 4

Question: 7

In a sunny garden, a gardener waters plants for H hours to help them grow. He starts with W water units. Each hour, he uses 3 units per plant, but there are 5 plants. If the hour is hot (hour number is a multiple of 2), use 4 units per plant instead. Subtract water each hour. If the water goes below 0 in any hour, stop and say the hour. If all hours are done, say the remaining water.

Sample Input	Sample Output
Enter Hour (H): 5 Enter Water Units (W): 10	Stopped at Hour 1
Enter Hour (H): 5 Enter Water Units (W): 100	All Watered 15 Water Unit Left
Explanation: Plants=5 (fixed) Water=100 (Starting with 100 Water Units) Hour-1 (odd), so $5 * 3 = 15$, Remaining Water: 85 Hour-2 (even), so $5 * 4 = 20$, Remaining Water: 65 Hour-3 (odd), so $5 * 3 = 15$, Remaining Water: 50 Hour-4 (even), so $5 * 4 = 20$, Remaining Water: 30 Hour-5 (odd), so $5 * 3 = 15$, Remaining Water: 15	

Question: 8

1	public class Tracel{
2	public static void main(String args[]){
3	int x = 0, p = 0, sum = 0;
4	p = 1;
5	x = 2;
6	double q;
7	sum = 0;
8	while (p < 12){
9	q = x + p-(sum+7/3)/3.0%2 ;
10	sum = sum + (++x) + (int)q;
11	System.out.println(sum++);
12	if (x > 5){
13	p += 4/2;
14	}
15	else {
16	p += 3%1;
17	}
18	}
19	sum = sum + p;
20	System.out.println(sum) ;
21	}
22	}

HOME TASKS

Question: 1

Write a Java program that will take a positive integer n as input and print all the numbers from 0 to n which are **divisible by 5 but not divisible by 3**.

Sample Input

40

Sample Output

5
10
20
25
35
40

Question: 2

In Hogwarts, apprentice wizards brew magic potions using glowing herbs and sparkling crystals collected from the Whispering Woods. You have a stash of H herbs and C crystals. Each successful potion requires exactly 3 herbs and 2 crystals. Brew as many potions as possible without running short mid-mix stop when you can't complete another full potion. After brewing, check the total potions: if even, the batch is perfectly balanced ("Stable Elixir"); if odd, it's unpredictably fizzy ("Volatile Brew"). Output the number of potions and the balance message.

Sample Input	Sample Output
Please Enter number of Herbs (H): 25 Please Enter number of Crystals (C): 13	Potion-1 created Remaining Herbs: 22, Remaining Crystals: 11 Potion-2 created Remaining Herbs: 19, Remaining Crystals: 9 Potion-3 created Remaining Herbs: 16, Remaining Crystals: 7 Potion-4 created Remaining Herbs: 13, Remaining Crystals: 5 Potion-5 created Remaining Herbs: 10, Remaining Crystals: 3 Potion-6 created Remaining Herbs: 7, Remaining Crystals: 1 Potions Created: 6

	Stable Elixir
Please Enter number of Herbs (H): 9 Please Enter number of Crystals (C): 6	Potion-1 created Remaining Herbs: 6, Remaining Crystals: 4 Potion-2 created Remaining Herbs: 3, Remaining Crystals: 2 Potion-3 created Remaining Herbs: 0, Remaining Crystals: 0 Potions Created: 3 Volatile Elixir

Question: 3

In a sunny mountain cave, a baby dragon named Charmander plays a fun game to evolve. Charmander starts with E energy points. He must blow fire across N small rivers. For each river i, the distance is D units. If D is 5 or less, it costs 2 energy. If D is more than 5, it costs $D / 2$ energy (whole numbers only). Take away the energy after each blow. If energy goes below 0, Charmander gets tired and stops. Tell the river where he stops, or if he finishes all, say how much energy is left.

Sample Input	Sample Output
Energy (E): 15 Number of River (N): 3 Enter River Distance D1: 4 Enter River Distance D2: 6 Enter River Distance D3: 3	All Done 8 energy Left
Energy (E): 19 Number of River (N): 8 Enter River Distance D1: 14 Enter River Distance D2: 12 Enter River Distance D3: 9 Enter River Distance D4: 2 Enter River Distance D5: 15	Tired at River 5
Explanations: Energy is 19 and needs to blow fire across 8 rivers In first river distance (D) is 14 which is greater than 5 so it costs 7 (half of D) so remaining energy (19-7=12) In second river distance (D) is 12 which is greater than 5 so it costs 6 (half of D) so remaining energy (12-6=6) In third river distance (D) is 9 which is greater than 5 so it costs 4 (half of D) so remaining energy (6-4=2)	

In fourth river distance (D) is 2 which is less than 5 so it costs 2 so remaining energy (2-2=0)
In fifth river distance (D) is 15, but there is no energy left, therefore it got tired at River 5

Question: 4

Write a program in Java that prints the individual digits of an integer number (user input) backward (From right to left).

Sample Input	Output
32768	8, 6, 7, 2, 3

Hint: First to get the digit from the right side, we can take the remainder of the number using the modulus (%) operator i.e. mod 10 to get the rightmost digit and print it. For dropping the last digit, we can perform division by 10 on the number and then continue the same to print the other digits as shown below.

Explanation:

$$32768 \% 10 = 8$$

$$32768 / 10 = 3276$$

Then,

$$3276 \% 10 = 6$$

$$3276 / 10 = 327$$

and so on

$$327 \% 10 = 7$$

$$327 / 10 = 32$$

$$32 \% 10 = 2$$

$$32 / 10 = 3$$

$$3 \% 10 = 3$$

$$3 / 10 = 0$$

Done! When the number becomes 0 you can end your loop.

Question: 5

In a secret city, you are on a mission to open a high-tech vault that protects a treasure!
The vault has a N-digit secret code (like 32768). To open it, you must say each digit one by one, from left to right, exactly as shown on the screen. Your smart watch can read the full number, but you need to break it into digits and speak them out loud like:

"3... 2... 7... 6... 8..."

If you get even one digit wrong, the alarm goes off!

Write a Java program that:

- Asks you: "Enter the N-digit vault code:"
- Reads the number
- Prints the digits forward, separated by commas

Rules:

- You cannot use **Math.pow()**
- You must count the digits first
- Then use division and remainder to pull out each digit
- Print them from left to right

[Instructions From Headquarters:

Start by finding how many digits (N). For 32768 → 5 digits. Then start with 10,000 (that's $10^{(N-1)}$).

Divide the code by 10,000 → get 3

Take remainder → left with 2768

Now divide the divider by 10 → 1000

Repeat: divide, print, remainder, shrink divider

Until you print all digits!]

Question: 6

Write a Java program that will take an integer as input and -

- A. Find out if the number is a prime number or not.
- B. Find out if the number is a perfect number or not.

[Prime Number: If a number has only two divisors, (1 and itself), then it is a prime number. Else, then it is not a prime number.

Perfect Number: A number is said to be a perfect number if the sum of its divisors, including 1 but not the number itself is equal to that number.]

Sample Input

6

Sample Output

6 is not a prime number

6 is a perfect number

Question: 7

Write a Java program which adds all numbers that are multiples of either 7 or 9 but not both, up to 600. Use only a single loop in your program. *[Ans: 39814]*

Question: 8

Write a Java code that asks an integer as input from the user and takes that many integer inputs. Your task is to count how many numbers are non-negative and negative.

Sample Input: (The purple numbers are input.)

Enter an integer: 9

Enter number 1: -8

Enter number 2: 33

Enter number 3: -100

Enter number 4: 10

Enter number 5: 0

Enter number 6: 5

Enter number 7: 10

Enter number 8: -4

Enter number 9: 4

Sample Output:

6 Non-negative Numbers

3 Negative Numbers

Question: 9

1	<code>public class Trace2 {</code>
2	<code> public static void main(String[] args) {</code>
3	<code> int x = 0, p = 0, sum = 0;</code>
4	<code> p = 1;</code>
5	<code> x = 2;</code>
6	<code> double q = 0.0;</code>
7	<code> sum = 5;</code>
8	<code> while (p < 15) {</code>
9	<code> q = x + p - (sum + (int) (7 / 4)) / 3.0 % 2;</code>
10	<code> sum = sum + x + (int) q;</code>
11	<code> x += 1;</code>
12	<code> System.out.println(sum);</code>
13	<code> if (x > 5) {</code>
14	<code> p += (int) (5 / 2);</code>
15	<code> }</code>
16	<code> else {</code>
17	<code> p += 10 % 3;</code>
18	<code> }</code>
19	<code> }</code>
20	<code> sum = sum + p;</code>
21	<code> System.out.println(sum);</code>
22	<code> }</code>
23	<code>}</code>

Question: 10

1	<code>public class Trace3 {</code>
2	<code> public static void main(String[] args) {</code>
3	<code> int m = 17, n = 13, p = 1, sum = 30;</code>
4	<code> while(0 < p%10){</code>
5	<code> if(m % 10 == 0){</code>
6	<code> sum = sum * m % n + p / n ;</code>
7	<code> }</code>
8	<code> else{</code>
9	<code> if(m % 4 == 0){</code>
10	<code> sum += n + (--m) ;</code>
11	<code> }</code>
12	<code> else{</code>
13	<code> sum -= m--;</code>
14	<code> }</code>
15	<code> }</code>
16	<code> p+=1;</code>
17	<code> System.out.println(sum) ;</code>
18	<code> }</code>
19	<code> System.out.println(!(n%13 == 0) && !false p>10);</code>
20	<code>}</code>
21	<code>}</code>

LAB ASSIGNMENT # 04

Nested Loop



CSE110 | Programming Language I

	LAB TASKS	HOME TASKS
CODING	04	04
TRACING	01	02

NOTE: You need to submit only the Home Tasks. Submit all the Tracing tasks handwritten respectively to your Lab Instructors before the next lab. Submit all the Homework Coding Tasks in the Google Form shared on buX.

LAB TASKS
[NO NEED TO SUBMIT]

Question: 1

In a virtual arena, there are N warriors numbered from 1 to N. Every warrior fights every other warrior exactly once. A warrior wins if their strength (their number) is divisible by the opponent's strength. Your job is to write a java code that calculates the total number of duels won in the entire tournament.

Sample Input	Sample Output
5	2 beats 1 3 beats 1 4 beats 1 4 beats 2 5 beats 1 Total wins: 5
8	2 beats 1 3 beats 1 4 beats 1 4 beats 2 5 beats 1 6 beats 1 6 beats 2 6 beats 3 7 beats 1 8 beats 1 8 beats 2 8 beats 4 Total wins: 12

Question: 2

Students at Hero Academia must complete missions and earn points in order to pass a grade. The homeroom teacher of class 1A wants to identify the top performer and give them a badge of honour. He provides you with the number of students (S), the number of missions (M) these students completed and for each student then gives you how many points they collected on each of these missions.

Write the java code to calculate each student's average points and help the teacher find the top performer. [N.B: All students receive points for all (M number of) missions.]

Sample Input	Sample Output
Students (S): 3 Missions (M): 3 Scores of Student 1: 10 9 8 Scores of Student 2: 9 10 10 Scores of Student 3: 7 8 9	Average point of Student 1: 9.0 Average point of Student 2: 9.67 Average point of Student 3: 8.0 Student 2 receives the badge of honour!!

Question: 3

You are testing SecureVault's safe locks. The program should ask for how many pairs (N) of codes to test, then for each pair, take two positive numbers and find their GCD using the Euclidean method. If any number is 0 or negative, print 0.

*[Hint: (Euclidean method) Two positive integers a and b . Assume a is greater than b , then divide a by b and find the remainder r . This can be expressed as $a = \text{quotient} * b + r$.*

If $r = 0$, b is the GCD of a and b and the loop terminated.

If $r \neq 0$, replace a with b and b with r and repeat the previous step.]

Example:

Steps	a	b	$a \% b$	Actions
1	48	18	12	$a \leftarrow 18, b \leftarrow 12$
2	18	12	6	$a \leftarrow 12, b \leftarrow 6$

3	12	6	0	Stop→GCD = 6
---	----	---	---	--------------

Input	Output
Enter number of pairs: 2 Pair 1: Enter first number: 40 Enter second number: 50 Pair 2: Enter first number: 0 Enter second number: 32	Pair 1: GCD = 10 Pair 2: GCD = 0

Question: 4

At Nimbus Tech, the HR analytics team is studying the “Stress Level” of a particular employee over “W” number of weeks. Each week the employee takes on a task more difficult than the previous week and adds on to his workload. This each week’s workload up to week “W”, is subtracted to find his Total Stress level.

For week 1, the workload is 1.

For week 2, it’s the sum of workload of previous week and this week’s new project (1+2).

For week 3, it’s (1+2+3). And so,

For week W, it’s (1+2+3+...+W)

The “Stress Level” also written as STRL is the result of subtracting all the weeks’ workload.

$$\text{STRL} = -(1) - (1+2) - (1+2+3) - \dots - (1+2+3+\dots+W)$$

Write a java code for the HR team to find the STRL of the employee when given “W” or number of weeks.

Sample Input	Sample Output
Enter number of Weeks, W: 4	STRL= -20
<u>Explanation:</u> Since W = 4, $\text{STRL} = - (1) - (1+2) - (1+2+3) - (1+2+3+4) = - 1 - 3 - 6 - 10 = -20$	

Question: 5

1	public class T1{
2	public static void main(String args[]){
3	int x = 0, y = 0;
4	int sum = 0;
5	while (x < 4){
6	y = x - 3;
7	while (y < 3){
8	sum = (sum % 3) + x - y * 3 ;
9	System.out.println(sum);
10	y = y + 1;
11	}
12	if (x > 5){
13	x++;
14	}
15	else{
16	x += 2;
17	}
18	}
19	}
20	}

HOME TASKS

Question: 1

NarcissiTech is a luxury telecom company that sells "Vanity Phone Numbers", special numbers that customers love because they are mathematically elegant. One of their flagship products is "Armstrong Vanity Numbers", where the number itself equals the sum of its digits each raised to the power of the total number of digits.

You need to write a Java program that asks the user to input a starting number and an ending number. The program must find and display all Armstrong numbers within this range (inclusive).

[An Armstrong number is a number whose digits' values, each raised to the power of the total number of digits, add up to the number itself.]

For example:

- 371 has 3 digits
- So we calculate:
 $3^3 + 7^3 + 1^3 = 27 + 343 + 1 = 371$
- Since the sum equals the original number, 371 is an Armstrong number.]

Question: 2

At Codewarts University of Magic and Programming, every student is assigned an unique numeric ID when they enroll. Professor Dumbledore, the head of the Department of Logical Arts, believes that students whose ID numbers are powers of 2 possess extraordinary logical abilities. Such students are said to have "Lucky IDs."

Your task is to help the university identify which students have Lucky IDs. You will first take an user input of the number of test cases (N) representing how many students' IDs need to be checked. For each student, take input of their ID number and determine whether it is a power of two. Display "Lucky ID" if the student's ID is a power of two, otherwise display "Not Lucky".

Sample Input	Sample Output
Enter number of students to check: 5 Enter student ID: 1 Enter student ID: 2 Enter student ID: 3 Enter student ID: 8	Lucky ID Lucky ID Not Lucky Lucky ID Not Lucky

Enter student ID: 10	
Explanation: Student ID: 1 = 2^0 Student ID: 2 = 2^1 Student ID: 3 = Not a power of 2. Student ID: 8 = 2^3 Student ID: 10 = Not a power of 2.	

Question: 3

At *ShopSmart Mart*, the owner keeps track of daily sales of three products for several days. At the end of each day, the shop calculates the total sales. To comply with local regulations, the shop applies taxes on total sales:

- If the total sales for the day equal to or greater than 100, a 2% tax is added.
- If the total sales equal to or greater than 200, a 5% tax is applied instead.
- If the total sales equal to or greater than 500, a 10% tax is applied instead.

The owner wants your help to display the sales table for each day, showing the total sales including any applicable tax.

You need to write a Java program that asks the user to input the number of days (N). Then, for each day, input the sales of the 3 products. For each day, calculate the total sales, apply the correct tax based on the total, and display the total sales including tax.

Sample Input	Sample Output
Enter number of days: 4 Enter sales for Day 1: 30 40 50 Enter sales for Day 2: 40 50 60 Enter sales for Day 3: 100 80 50 Enter sales for Day 4: 200	Day 1: Total Sales with Tax: 122.4 Day 2: Total Sales with Tax: 153.0 Day 3: Total Sales with Tax: 241.5 Day 4: Total Sales with Tax: 583.0

150 180	
------------	--

Explanation:

Sales for Day 1: $30+40+50 = 120$

Total sales= $120 + (2\% \text{ of } 120) = 122.4$

Sales for Day 2: $40+50+60 = 150$

Total sales= $150 + (2\% \text{ of } 150) = 153.0$

Sales for Day 3: $100+80+50 = 230$

Total sales= $230 + (5\% \text{ of } 230) = 241.5$

Sales for Day 4: $200+150+180 = 530$

Total sales= $530 + (10\% \text{ of } 530) = 583.0$

Question: 4

GymFit Elite uses an AI fitness system to calculate effective calorie burn per exercise with advanced rules. Your task is to write a java programme to calculate the average of earned calories. For each member, input the number of exercises and their raw calories. Each day a member can do the highest 3 exercises. If the calorie is greater than 350 then add 50% bonus, add 10% penalty to all if every exercise is less than 200, and give 50 calories flat bonus per exercise if the raw average is greater than 400. Now calculate the total calories earned and print the average calories earned a day.

Input	Output
Number of Members: 2 Exercises for Member-1: 3 Exercise-1: 300 Exercise-2: 400 Exercise-3: 800 Exercises for Member-2: 4 Exercise-1: 150 Exercise-2: 100 Exercise-3: 80	Average calories earned per day for Member-1: 750.000 (Can't do more than 3 exercise) Average calories earned per day for Member-2: 99.000

Explanation:

For first member (1), number of exercise is 3

Calorie for 3 exercises are 300, 400 and 800

So, rawSum = $300+400+800 = 1500$

As calorie burn for exercise 2 and 3 is more than 350 so 50% bonus is added to them

For 400, 200 will be added and for 800, 400 will be added, so total = $300 + 200 + 400 + 800 + 400 = 2100$

Average of rawSum is $1500 / 3 = 500$

The average of rawSum is greater than 400, therefore 50 bonus will be added for all three exercises. Therefore total is $2100 + 50 + 50 + 50 = 2250$

Finally total average is: $2250 / 3 = 750$

For second member (2), number of exercise is 4 which is greater than 3 so member need to perform maximum 3 exercises

Calorie for 3 exercises are 150, 100 and 80

So, rawSum = $150+100+80 = 330$

So, rawAvg = $330 / 3 = 110$

As calorie burn for exercise 1, 2 and 3 is not more than 200, 10% penalty will be given

$110 - 10\% = 99$

The average of rawSum is not greater than 400, therefore 50 bonus will not

be added for all three exercises.

Question: 5

1	public class T3
2	{
3	public static void main(String args[])
4	{
5	int x = 0, y = 0;
6	int sum = 0;
7	while (x < 10){
8	y = x - 3;
9	y = 40;
10	while (y > 22){
11	if ((sum > 30) && (sum < 40)){
12	sum = sum + x * 2 ;
13	}
14	else if ((sum > 40) && (sum < 50)){
15	sum = sum + x * 3;
16	}
17	else {
18	sum = sum + 23;
19	}
20	System.out.println(sum);
21	y = y - 10;
22	}
23	x += 2;
24	}
25	}
26	}

Question: 6

1	public class Tracing{
2	public static void main(String[] args) {
3	int sum = 0, r = 0, i = 10, j = -5000;
4	while (i < 10000) {
5	r = (i%2) + 1;
6	j = i + 2;
7	while (j < i){
8	r = 2-4+6-8+10-12;
9	j++;
10	}
11	sum = i % r + (i * r) - j + 5;
12	System.out.println(sum);
13	j--;
14	if (i > 18) {
15	sum += (++r) + i/3 + r + 12;
16	break;
17	}
18	else{
19	i += 3;
20	}
21	}
22	System.out.println(sum);
23	}
24	}

UNGRADED TASKS
(For Practice)

Questions: 1

In the ancient city of Luminallis, the Grand Scrollkeeper guards the Scroll of Know-it-all, a relic that grants you access to all knowledge in existence. To get the scroll you must pass the Scrollkeeper's test to find the sum of "unusual numbers". Unusual numbers are those that are not divisible by any number smaller than themselves, other than 1.

The scrollkeeper hands you two numbers: X (A starting point) and Y (The number of unusual numbers you must find). Write a java code to find the answer to the scrollkeeper's question.

Sample Input	Sample Output
X: 4 Y: 5	Sum: 53
<u>Explanation:</u> <i>5 unusual numbers starting from 4 are: 5, 7, 11, 13, 17</i> <i>Their sum: $5 + 7 + 11 + 13 + 17 = 53$</i>	

Question: 2

Read an integer N that is the number of test cases that follow. Each test case contains two integers X and Y. Print one output line for each test case that the sum of Y odd numbers from X including it if is the case. For example:

For the input 4 5, the output must be 45, that is: $5 + 7 + 9 + 11 + 13$

For the input 7 4, the output must be 40, that is: $7 + 9 + 11 + 13$

Sample Input	Sample Output
2 4 3 11 2	21 24
<u>Explanation:</u> Here, the 2 means there are two test cases. For each test case you have to take two inputs (X, Y) and print the sum of Y odd numbers starting from X.	

Question: 3

Write a Java program to take a positive integer N (where $N > 0$) as user input and print the first N prime numbers starting from 2. Your code should check all the positive integers starting from 2 and determine whether they are prime or not until N prime numbers are found.

Sample Input 1:

5

Sample Output 1:

2

3

5

7

11

Sample Input 2:

7

Sample Output 2:

2

3

5

7

11

13

17

Question: 4

Read one integer P ($12 \leq P \leq 1000$). Find all integer triples (a, b, c) with $1 \leq a < b < c$ such that $a + b + c = P$ and $a*a + b*b = c*c$. Print each triple on its own line as three numbers separated by single spaces, then print the total number of such triples on the last line. If there are no solutions, print 0 only.

Sample Input	Sample Output
60	10 24 26 15 20 25 Total- 2
120	20 48 52 24 45 51 30 40 50 Total- 120

LAB ASSIGNMENT # 05

PATTERN GENERATION



CSE110 | Programming Language I

	LAB TASKS	HOME TASKS
CODING	08	07
TRACING	00	00

NOTE: You need to submit only the Home Tasks. Submit all the Homework Coding Tasks in the Google Form shared on buX.

LAB TASKS
[NO NEED TO SUBMIT]

Rules:

- You are not allowed to use any array or String
- One only line where the word “String” may appear is in public static void main(**String** [] args) {
- The word “char” must not be anywhere in your solution

Overall hint: Imagine all outputs as a matrix of space and digit. Then count spaces and digits to find out the trend of increment or decrement of number of digits/spaces. Utilize several IF statements to control when ‘digit’ will be printed, when ‘space’ will be printed and when ‘enter’ will be printed. Nested loops may be utilized to print multiple lines.

Question: 1

Number Line

Sample Input	Sample Output
6	123456
7	1234567

Question: 2

Rectangle

Sample Input	Sample Output
4 6	123456 123456 123456 123456
8 4	1234 1234 1234 1234 1234 1234 1234 1234

Question: 3

Triangle - Left Justified

Sample Input	Sample Output
4	1 12 123 1234
6	1 12 123 1234 12345 123456

Question: 4

Triangle - Right Justified

Sample Input	Sample Output
4	4 34 234 1234
6	6 56 456 3456 23456 123456

Question: 5

Triangle - Isosceles

Sample Input	Sample Output
3	1 123 12345
4	1 123 12345 1234567

Question: 6

Hollow Rectangle

Hint 1: Print the character space (' ') in the middle.

Hint 2: You can re-use your solution to **PROBLEM 2) Rectangle** and use **IF** condition to selectively print the first and last digit of each line and all digits of the first and last line.

Sample Input	Sample Output
4 5	12345 1 5 1 5 12345
5 4	1234 1 4 1 4 1 4 1234

Question: 7

Hollow Triangle - Left Justified

Sample Input	Sample Output
5	1 12 1 3 1 4 12345
6	1 12 1 3 1 4 1 5 123456

Question: 8

Palindrome

Sample Input	Sample Output
5	123454321
7	1234567654321

HOME TASKS

Question: 1

Rhombus

Sample Input	Sample Output
3	1 123 12345 123 1
4	1 123 12345 1234567 12345 123 1

Question: 2

Hollow Triangle - Right Justified

Sample Input	Sample Output
5	5 45 3 5 2 5 12345
6	6 56 4 6 3 6 2 6 123456

Question: 3

Hollow Triangle – Isosceles

Sample Input	Sample Output
3	<pre> 1 1 3 12345 </pre>
4	<pre> 1 1 3 1 5 1234567 </pre>

Question: 4

Hollow Rhombus

Sample Input	Sample Output
3	<pre> 1 1 3 1 5 1 3 1 </pre>
4	<pre> 1 1 3 1 5 1 7 1 5 1 3 1 </pre>

Question: 5

Palindromic Triangle

Sample Input	Sample Output
5	1 121 12321 1234321 123454321
4	1 121 12321 1234321

Question: 6

Downward Palindromic Triangle

Sample Input	Sample Output
5	123454321 1234321 12321 121 1
4	1234321 12321 121 1

Question: 7

Hourglass

Sample Input	Sample Output
5	12345 123 1 123 12345
7	1234567 12345 123 1 123 12345 1234567

UNGRADED TASKS

Question: 1

You're a master baker at Golden Crust Bakery. Every morning, you make n loaves. But the dough must rise in stages. For each loaf, you check every 5 minutes:

- If it's not risen enough, print . (still rising).
- If it just rose, print * (ready!).
- Each loaf rises at a different minute (based on its number).

Write a program to print the rising status of the first n loaves over 20 minutes.

Sample Input:
Enter number of Loaves: 3
Sample Output:
<pre>.....****</pre>

Question: 2

You're a domino artist. You line up n dominoes in a row.

- When the first falls, it knocks the next every 1 second.
- Print the falling wave over n seconds.

Sample Input	Sample Output
Enter Time: 5	<pre>/ // /// //// /////</pre>
Enter Time: 8	<pre>/ // /// //// ///// //////// ///////// /////////</pre>

Question: 3

You're controlling the stage lights for a K-pop concert. There are 8 lights. Every second, the pattern shifts left. Print the first 8 seconds of the show. Light on = *, off = .

Sample Output

```
* * * * . . . .
* * * . . . . *
* * . . . . * *
* . . . . * * *
. . . . * * * *
. . . * * * * .
. . * * * * .
. * * * * .
* * * * . . . .
```

LAB ASSIGNMENT # 07

Arrays



CSE110 | Programming Language I

	LAB TASKS	HOME TASKS
CODING	05	05
TRACING	01	03

NOTE: You need to submit only the Home Tasks. Submit all the Flowchart or Tracing tasks hand drawn or handwritten, respectively to your Lab Instructors before the next lab. Submit all the Homework Coding Tasks in the Google Form shared on buX.

LAB TASKS
[NO NEED TO SUBMIT]

Question: 1

Write a Java program that will take N integer numbers from the user and create an array of length N.

- A. Print the elements of the array with their indices.
- B. Take another integer input from the user, resize the array by length 1, and add the new integer value to the array. Print the resized array.

Sample Input	Sample Output
N = 5 Enter a number: 11 Enter a number: 22 Enter a number: 33 Enter a number: 44 Enter a number: 55 Enter another number: 101	The elements of the array are: 0: 11 1: 22 2: 33 3: 44 4: 55 After resizing the array: 11 22 33 44 55 101

Question: 2

Write a Java program that asks the user for the length of an array and then creates an integer array of that length by taking inputs from the user. Then,

- A. Reverse the array by creating a new array of the same length and print it. (Out-of Place)
- B. Reverse the array without creating any new arrays and print it. (In-Place)

Sample Input	Sample Output
Enter the length of the array: 5 Enter a number: 7 Enter a number: -31 Enter a number: 344 Enter a number: 97 Enter a number: 100	Reversed using a new array: 100 97 344 -31 7 Reversed the original array: 100 97 344 -31 7

Question: 3

You are given an integer array with duplicate values. Write a Java program to update the array by replacing the duplicate values of the array with zero. Then print the updated array. [Your code should work for any given integer array]

Given Array	Sample Output
<pre>int arr [] = {9, -5, 7, 9, -5, 5, 7};</pre>	<pre>Before removing duplicates: 9 -5 7 9 -5 5 7 After replacing duplicates with 0: 9 -5 7 0 0 5 0</pre>

Question: 4

You are given an integer array. You need to create a new array that will contain only the unique (distinct) elements of the given array. Finally, print the new array.

Given Array	Sample Output
<pre>int arr [] = {23,100,23,56,100};</pre>	<pre>Input array: 23 100 23 56 100 New array: 23 100 56</pre>
<pre>int arr [] = {-5,10,-7,-5};</pre>	<pre>Input array: -5 10 -7 -5 New array: -5 10 -7</pre>

Question: 5

Take an integer N input from the user and create an array of length N by taking the elements as user input. Then, print the array. Next, sort the array in descending order using Selection Sort technique. Lastly, print the sorted array.

Sample Input	Sample Output
N = 6 Enter a number: 3 Enter a number: 6 Enter a number: 1 Enter a number: 2 Enter a number: 8 Enter a number: 5	Original Array: 3 6 1 2 8 5 Sorted Array: 8 6 5 3 2 1

Question: 6

Trace the following code, create a tracing table, and write the outputs.

1	class TraceA{
2	public static void main(String args[]) {
3	int [] myArray = new int[5];
4	int [] b;
5	int idx1 = 0, idx2 = 0;
6	b = myArray;
7	while (idx1 < 5){
8	myArray[idx1] = idx1 + 11;
9	idx2 = 1;
10	while (idx2 < idx1){
11	myArray[idx1] = b[idx2 - 1] + myArray[idx2] - idx1;
12	idx2 = idx2 + 1;
13	}
14	System.out.println(myArray[idx2]);
15	idx1 = idx1 + 1;
16	}
17	}
18	}

HOME TASKS

Question: 1

Take an integer N input from the user and create an integer array of N numbers by taking inputs from the user. Then, print the array. Next, modify the array by changing the positive numbers by 1 and the negative numbers by 0. If the element is zero, then it will be unchanged. Lastly, print the modified array.

Sample Input	Sample Output
N = 4 3 4 -2 1	Original array: 3 4 -2 1 After modifying: 1 1 0 1
N = 3 -4 0 2	Original array: -4 0 2 After modifying: 0 0 1

Question: 2

Write a Java program that will take N integer numbers from the user and create an array of length N. Take another number from the user and print the index of the number where it is found first. If not found then print 'Element not found'.

Note: Think about how to apply the concept of flag and break in this task.

Sample Input 1 N = 7 Enter a number: 45 Enter a number: 0 Enter a number: 17 Enter a number: 11 Enter a number: -34 Enter a number: -100 Enter a number: 17 17	Sample Input 2 N = 5 Enter a number: 4 Enter a number: 99 Enter a number: 23 Enter a number: -67 Enter a number: 34 55
Sample Output 1 17 is at index 2	Sample Output 2 Element not found

Question: 3

Write a Java program that asks the user for the length of an array then creates a double data-type array of that length by taking inputs from the user.

Then do the following:

- a. Show the maximum element and its index from the array.
- b. Show the minimum element and its index from the array.
- c. Show the summation of all the elements from the array.
- d. Show the average of all the elements from the array.

Sample Input	Sample Output
Enter the length of the array: 5 Enter a number: 7.5 Enter a number: -31.2 Enter a number: 344.0 Enter a number: 97.1 Enter a number: 100.4	Maximum element 344.0 found at index 2 Minimum element -31.2 found at index 1 Summation: 517.8 Average: 103.56

Question: 4

Write a Java program that will take input of two arrays and elements from the user and check whether the second array is a subset of the first array. A subset is a set that contains only elements found in the original set.

Sample Input - 1	Sample Output - 1
Please enter the length of array 1: 5 Please enter the elements of the arr1: 5 3 2 72 8 Please enter the length of array 2: 3 Please enter the elements of the arr2: 5 3 72	Array 2 is a subset of Array 1.

Sample Input - 2	Sample Output - 2
Please enter the length of array 1: 5 Please enter the elements of the arr1: 7 2 33 1 6 Please enter the length of array 2: 3 Please enter the elements of the arr2:	Array 2 is not a subset of Array 1.

1	
8	
2	

Question: 5

You are given two arrays of the same length. The first array contains the marks of the students and the second array contains the name of the students. You need to sort the marks array in ascending order while maintaining the corresponding names of the students in the names array aligned with their respective marks. Use the Bubble sort technique to solve this problem. [Your code should work for any given arrays]

Given Array 1	Sample Output 1
<pre>int [] marks = {85, 90, 75, 44, 99}; String [] names = {"Bob", "Alice", "Max", "Marry", "Rosy"};</pre>	<pre>Sorted Array: 44 75 85 90 99 Marry Max Bob Alice Rosy</pre>

Given Array 2	Sample Output 2
<pre>int [] marks = {100, 47, 85, 94, 5, 50}; String [] names = {"Henry", "Mari", "Herry", "Jack", "Lily", "Oliver"};</pre>	<pre>Sorted Array: 5 47 50 85 94 100 Lily Mari Oliver Herry Jack Henry</pre>

Question: 6

Trace the following code, create a tracing table, and write the outputs.

1	class TraceB{
2	public static void main(String args[]){
3	int [] myArray = new int[10];
4	int index1 = 0, index2 =0;
5	while (index1 < 10){
6	myArray[index1] = index1 + 3;
7	index2 = 1;
8	while (index2 < index1){
9	myArray[index1] = myArray[index1] + myArray[index2] - index1;
10	index2 = index2 + 1;
11	}
12	System.out.println(myArray[index1]);
13	index1 = index1 + 1;
14	}
15	}
16	}

Question: 7

Trace the following code, create a tracing table, and write the outputs.

1	class TraceC {
2	public static void main(String args[]){
3	int [] myArray = new int[10];
4	int [] b;
5	int index1 = 0, index2 =0;
6	index1 = 0;
7	b = myArray;
8	while (index1 < 10){
9	myArray[index1] = index1 + 4;
10	index2 = 1;
11	while (index2 < index1){
12	myArray[index1] = b[index1] + myArray[index2] - index1;
13	index2 = index2 + 1;
14	}
15	System.out.println(myArray[index1]);
16	index1 = index1 + 1;
17	}
18	}
19	}

Question: 8

Trace the following code, create a tracing table, and write the outputs.

1	class TraceD{
2	public static void main(String args[]){
3	int[] arr1 = {3,2,0,1,5,6,7};
4	int[] arr2 = {30,20,40,11,55,-34,100};
5	int a1 = 0, a2 = 0;
6	while (a1<arr1.length-1){
7	arr2[a1] = arr1[a2]+ a1 - arr2[a2];
8	a2 = 1;
9	while (a2 < a1){
10	arr2[a1] = arr2[a1] + arr1[a2] - a2;
11	a2 = a2 + 1;
12	}
13	System.out.println(arr2[a1]);
14	a1 = a1 + 1;
15	}
16	System.out.println(arr2[arr1[a2]]);
17	}
18	}

LAB ASSIGNMENT # 08

Method



CSE110 | Programming Language I

	LAB TASKS	HOME TASKS
CODING	03	03
TRACING	01	02

NOTE: You need to submit only the Home Tasks. Submit all the Flowchart or Tracing tasks hand drawn or handwritten, respectively to your Lab Instructors before the next lab. Submit all the Homework Coding Tasks in the Google Form shared on buX.

Lab Tasks

(No Need to Submit)

Task 01

[A,B,C,D should be written in a single java file]

- A. Write a method called **evenChecker** that takes an **integer** number as its argument and prints whether the number is even or odd **inside the method**.

Sample Method Call	Sample Output
<code>evenChecker(10);</code>	Even!!
<code>evenChecker(17);</code>	Odd!!

- B. Write a method called **isEven** that takes an **integer** number as an argument and **returns** boolean true if the number is even otherwise **returns** boolean false.

Sample Method Call	Sample Output
<code>boolean result = isEven(10); System.out.println(result);</code>	true
<code>boolean result = isEven(17); System.out.println(result);</code>	false

- C. Write a method called **isPos** that takes an **integer** number as an argument and **returns** boolean true if the number is positive otherwise **returns** boolean false.

Sample Method Call	Sample Output
<code>boolean result = isPos(-5);</code>	false

System.out.println(result);	
boolean result = isPos(12); System.out.println(result);	true

D. Write a method called **sequence()** that takes an **integer** in its parameter called n. Now, if n is **positive** then it prints all the **even** numbers from **0 to n**, otherwise if n is **negative**, it prints all the **odd** numbers from **n to -1**.

Note: **You must call** the methods from **1B** and **1C**, otherwise this task would be **considered invalid**.

Sample Method Call	Sample Output	Explanation
sequence(10);	0 2 4 6 8 10	Here, 10 is positive so 0,2,4,6,8,10 were printed.
sequence(-7);	-7 -5 -3 -1	Here, -7 is negative so -7,-5,-3,-1 were printed.
sequence(7);	0 2 4 6	Here, 7 is positive so 0,2,4,6 were printed
sequence(-8);	-7 -5 -3 -1	Here, -8 is negative so -7,-5,-3,-1 were printed.

Task02

[A,B,C should be written in a single java file]

A. Write a method called **circleArea** that takes an **integer** radius in its parameter and **returns** the **area** of the circle.

Note: area of a circle is πr^2

Sample Method Call	Sample Output
double area = circleArea(5); System.out.println(area);	78.5398

- B. Write a method called **sphereVolume** that takes a **double** radius in its parameter and **returns** the **volume** of the sphere.

Note: volume of a sphere is $\frac{4}{3}\pi r^3$

Sample Method Call	Sample Output
<code>double volume = sphereVolume(5.0); System.out.println(volume);</code>	523.5987

- C. Write a method called **findSpace** that takes two values in its parameters one is an **integer** diameter and another one is a String. Using the given diameter, this method should calculate the Area of a circle or the Volume of a sphere depending on the value of the second parameter. Finally, it should print the result **inside the method**.

Note: **You must call** the method written in task **2A & 2B**, otherwise this task would be **considered invalid**.

Sample Method Call	Sample Output
<code>findSpace(10,"circle");</code>	78.5398
<code>findSpace(5,"sphere");</code>	65.4498
<code>findSpace(10,"square");</code>	Wrong Parameter

Task03

[A,B should be written in a single java file]

- A. Write a method called **isTriangle** that takes 3 integer numbers as arguments. The method will **return** the boolean true if the 3 sides can form a valid triangle otherwise it'll **return** the boolean false.

Note: In a valid triangle, the sum of **any** two sides will be greater than the third side.

Sample Method Call	Sample Output	Explanation
<code>boolean res = isTriangle(7,5,10); System.out.println(res);</code>	true	Here, $7+5>10$, $5+10>7$ also, $10+7>5$. Thus, these 3 sides can form a valid triangle.
<code>boolean res = isTriangle(3,2,1); System.out.println(res);</code>	false	Here, $1+2\leq 3$, thus, these 3 sides can NOT form a valid triangle.

B. Write a method called **triArea** that takes 3 sides of a triangle as 3 **integer** arguments. The method should calculate and print the area of the triangle only if it's a valid triangle otherwise print that it's not a valid triangle.

Area of triangle = $\sqrt{s(s-a)(s-b)(s-c)}$, where 's' is the semi-perimeter of the triangle. So, semi-perimeter = double $s = \text{perimeter}/2 = (a + b + c)/2$.

Note: **You must call** the method written in task **3A**, otherwise this task would be **considered invalid**.

Sample Method Call	Sample Output	Explanation
<code>triArea(3,2,1);</code>	Can't form triangle	Here, $1+2\leq 3$, thus, these 3 sides can NOT form a valid triangle.
<code>triArea(7,5,10);</code>	16.248	Here, 7,5,10 is able to form a valid triangle so, using the formula we get the area as 16.248

Task04

Trace the following code to generate the outputs. Show the necessary trace table.

1	public class Trace1{
2	public static int f3(int a, int c){
3	System.out.println("F3 begins");
4	System.out.println(a+c);
5	System.out.println("F3 ends soon");
6	return a*5;
7	}
8	public static String f1(int n){
9	System.out.println("F1 begins");
10	f2(5,n);
11	System.out.println("F1 ends soon");
12	return n+1+" from F1";
13	}
14	public static void main(String[] args){
15	System.out.println("Starting soon...");
16	int c = 99;
17	String s = f1(c);
18	System.out.println(s);
19	System.out.println("The End");
20	}
21	public static void f2(int c, int d){
22	System.out.println("F2 begins");
23	System.out.println(f3(c+1,d-1));
24	System.out.println("F2 ends");
25	}
26	}

Home Tasks

Task01

[A,B,C should be written in a single java file]

- A. Write a method called **isPrime** which takes an integer in its parameter to check whether a number is prime or not. If the number is prime then the method returns boolean **true** otherwise it returns boolean **false**.

Sample Input	Sample Output
<code>boolean check = isPrime(7); System.out.println(check);</code>	true
<code>boolean check = isPrime(15); System.out.println(check);</code>	false

- B. Write a method called **isPerfect** which takes an integer in its parameter to check whether a number is perfect or not. If the number is perfect then the method returns boolean **true** otherwise it returns boolean **false**.

Sample Input	Sample Output
<code>boolean check = isPerfect(6); System.out.println(check);</code>	true
<code>boolean check = isPerfect(33); System.out.println(check);</code>	false

- C. Write a method called **special_sum** that calculates the sum of all numbers that are either prime numbers or perfect up till the integer value given in its parameter. This integer value must be taken as user input and passed into the method.

Note: **You must call** the methods written in task 1A & 1B, otherwise this task will be **considered invalid**.

Sample Input	Sample Output	Output
8	<pre>int result = special_sum(8); System.out.println(result);</pre>	23
Explanation:	Between 1 to 8 the Prime numbers are 2,3,5,7 and 6 is a Perfect number. So, the summation is 2+3+5+7+6=23.	

Task02

[A,B,C should be written in a single java file]

- A. Write a simple method called **showDots** that takes a number as an argument and then prints that amount of dots inside the method.

Note: You can use `System.out.print()` to avoid the next output being printed on the next line.

Sample Method Call	Sample Output
<code>showDots(5);</code>	
<code>showDots(3);</code>	...

- B. Write a method called **show_palindrome** that takes a number as an argument and then prints a palindrome inside the method.

Note: You can use `System.out.print()` to avoid the next output being printed on the next line

Sample Method Call	Sample Output
<code>show_palindrome(5)</code>	123454321
<code>show_palindrome(3)</code>	12321

C. Write a method called **showDiamond** that takes an integer number as an argument and then prints a **palindromic diamond shape**. Moreover, the empty spaces surrounding the diamonds are filled with dots(.) .

Note: **You must call** the methods written in task **2A & 2B**, otherwise this task would be **considered invalid**.

Sample Method Call	Sample Output
showDiamond(5)	1.... ...121... ..12321.. .1234321. 123454321 .1234321. ..12321.. ...121...1....
showDiamond(3)	..1.. .121. 12321 .121. ..1..

Task03

[A,B should be written in a single java file]

A. Write a method called **calcTax** that takes 2 arguments which are **your age** then **your salary**. The method must calculate and **return** the tax as per the following conditions:

- No tax if you are less than 18 years old.
- No tax if you get paid less than 10,000
- 7% tax if you get paid between 10,000 and 20,000 (both inclusive)
- 14% tax if you get paid more than 20,000

Sample Method Call	Output	Explanation
<code>double t = calcTax(16,20000); System.out.println(t);</code>	0.0	Here, the age is less than 18 so 0 tax.
<code>double t = calcTax(20,18000); System.out.println(t);</code>	1260.0	Here, the age is greater than 18 and income is between 10K-20K so tax is 7% of 18000 = 1260.

B. Write a method called **calcYearlyTax** that takes no arguments. Inside the method it should take **first input as your age** and then **12 other inputs** as income of each month of the year. The method must calculate and print Tax for each month and finally print the total Tax of the whole year based on the **6A conditions**.

Note: **You must call** the method written in task **3A**, otherwise this task would be **considered invalid**.

Sample Method Call	Sample Input	Sample Output
<code>calcYearlyTax()</code>	22 8000 15000	Month1 tax: 0 Month2 tax: 1050.0 Month3 tax: 3080.0

	22000	Month4 tax: 0
	2300	Month5 tax: 1071.0
	15300	Month6 tax: 2940.0
	21000	Month7 tax: 4760.0
	34000	Month8 tax: 0
	9000	Month9 tax: 3780.0
	27000	Month10 tax: 12320.0
	88000	Month11 tax: 4480.0
	32000	Month12 tax: 0
	7300	Total Yearly Tax: 33481.0

Task04

Trace the following code to generate the outputs. Show the necessary trace table.

1	public class Trace2{
2	public static void main (String[] args){
3	int a = 4, b = 7;
4	System.out.println(methodA(a,b));
5	}
6	public static double methodA(int m, int n){
7	int p = m+n-23, s = 0;
8	if (p<0){
9	System.out.println(p);
10	System.out.println(methodB(p+10));
11	s = methodB(p-10);
12	}
13	System.out.println(--s);
14	return p*m+s;
15	}
16	public static int methodB(int r){
17	int q = 6;
18	System.out.println(++r + q);
19	return q-r;
20	}
21	}

Task05

Trace the following code to generate the outputs. Show the necessary trace table.

1	public class Trace3{
2	public static boolean met1(int n1, int n2){
3	System.out.println("Method 1");
4	int n = n1+n2;
5	System.out.println(n);
6	return met3(n, n2)>n1;
7	}
8	public static int met2(int n, String s){
9	int p = 5;
10	System.out.println("Method 2");
11	System.out.println(met1(n,p));
12	return s.length();
13	}
14	public static double met3(int n, int p){
15	System.out.println("Method 3");
16	System.out.println(n/p);
17	return p;
18	}
19	public static void main (String[] args){
20	System.out.println("Main Method");
21	System.out.println(met2(6,"ABC"));
22	}
23	}

LAB ASSIGNMENT # 09

Recursive Method



CSE110 | Programming Language I

	LAB TASKS	HOME TASKS
CODING	04	04
TRACING	01	01

NOTE: You need to submit only the Home Tasks. Submit all the Flowchart or Tracing tasks hand drawn or handwritten, respectively to your Lab Instructors before the next lab. Submit all the Homework Coding Tasks in the Google Form shared on buX.

Lab Tasks

(No Need to Submit)

Task01

[A,B,C should be written in a single java file]

A. Write a method called **oneToN** that prints 1 till N recursively.

Hint: N is a number taken as input from the user and you need to print the numbers starting from 1 to N recursively.

Sample Input	Sample Method Call	Output
N = 5	oneToN(1,N);	1 2 3 4 5
N = 11	oneToN(1,N);	1 2 3 4 5 6 7 8 9 10 11

B. Write a method **nToOne** that prints from N to 1 recursively.

Hint: N is a number taken as input from the user and you need to print the numbers starting from N to 1.

Sample Input	Sample Method Call	Output
N = 6	nToOne(1,N);	6 5 4 3 2 1
N = 3	nToOne(1,N);	3 2 1

C. Write a method called **recursiveSum** to sum till N recursively.

Hint: N is a number taken as input from the user and you need to add the numbers starting from 1 to N recursively and print the sum.

Sample Input	Sample Method Call	Output
N = 4	System.out.println(recursiveSum(1,N));	10
N = 12	System.out.println(recursiveSum(1,N));	78

Task02

Write a **recursive method** called **reverseDigits** that takes an integer n as an argument and prints the digits of n in reverse order.

Hint: Think about how you solved it using loop

Sample Input	Sample Method Call	Output
n = 12345	reverseDigits(n);	5 4 3 2 1
n = 649	reverseDigits(n);	9 4 6
n = 1000	reverseDigits(n);	0 0 0 1

Task03

Write a **recursive method** called **sumDigits** that takes an integer n as an argument and sums up the digits of n then **returns** the result.

Hint: Think about how you would solve it using loop

Sample Input	Sample Method Call	Output
n = 12345	int x = sumDigits(n); System.out.println(x);	15
n = 649	int x = sumDigits(n); System.out.println(x);	19

Task04

Write a **recursive method** called **reverse_string(s, idx)** that returns the reverse of a given string **s**.

Sample Method Call	Output
System.out.println(reverse_string("Hello", 0))	olleH
System.out.println(reverse_string("swan", 0))	naws

Task05

Trace the following code to generate the outputs. Show the necessary trace table.

1	<code>public class ClassWork1{</code>
2	<code> public static int calculate(int n) {</code>
3	<code> if (n <= 0){</code>
4	<code> return 4;</code>
5	<code> }</code>
6	<code> else if (n % 2 != 0){</code>
7	<code> return n + calculate(n - 1);</code>
8	<code> }</code>
9	<code> else{</code>
10	<code> return n * calculate(n - 2);</code>
11	<code> }</code>
12	<code> }</code>
13	<code> public static void main(String[] args) {</code>
14	<code> int result = calculate(8);</code>
15	<code> System.out.println(result);</code>
16	<code> int result2 = calculate(5);</code>
17	<code> System.out.println(result2);</code>
18	<code> }</code>
19	<code>}</code>

Home Tasks

Task01

Write a **recursive method** called **factorial(n)** that returns the factorial of a number **n**. Assume $n \geq 0$.

Sample Input	Sample Method Call	Output
n = 5	int x = factorial(n); System.out.println(x);	120
n = 7	int x = factorial(n); System.out.println(x);	5040

Task02

Write a **recursive method** called **power(base, exponent)** that calculates base raised to the power of exponent (assume exponent is a non-negative integer).

Sample Method Call	Output
int x = power(5,3); System.out.println(x);	125
int x = power(8,4); System.out.println(x);	4096

Task03

Write a **recursive method** called **print_elements(arr, index)** that prints elements of an array starting from index to the end.

Given Array and Input	Sample Method Call	Output
<pre>int[] arr = {5,6,2,1,8,7}; int index = 2;</pre>	<pre>print_element(arr, index);</pre>	2 1 8 7
<pre>int[] arr = {13,12,19,21,31,55}; int index = 0;</pre>	<pre>print_element(arr, index);</pre>	13 12 19 21 31 55

Task04

The Fibonacci sequence is a series of numbers that starts with 0 & 1 and the rest of the numbers are generated by adding the immediate two numbers before it. It goes like this: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144 and so on.

In short, $\text{fibonacci}(0) = 0$, $\text{fibonacci}(1) = 1$ and $\text{fibonacci}(n) = \text{fibonacci}(n-1) + \text{fibonacci}(n-2)$.

Sample Method Call	Output
<pre>System.out.println(fibonacci(0));</pre>	0
<pre>System.out.println(fibonacci(1));</pre>	1
<pre>System.out.println(fibonacci(5));</pre>	5
<pre>System.out.println(fibonacci(9));</pre>	34

Task05

Trace the following code to generate the outputs. Show the necessary trace table.

1	public class ClassWork2{
2	public static String fun(String s, int n){
3	if(s.length()==4){
4	return n+s+n;
5	} else if(n%2==0){
6	System.out.println(s+n+n+3);
7	return fun(s+n, n+3);
8	} else {
9	System.out.println(s+n+(n-1));
10	return fun(s+n, n-1);
11	}
12	}
13	public static void main(String[] args){
14	String s = fun("",1);
15	System.out.println(s);
16	}
17	}

CSE 111

Lab Assignment 01



Inspiring Excellence

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	Loops, String, Arrays, OOP Basics
Number of Tasks:	10 [Classwork: 5, Homework: 5]

Lab Policy: [Lab Policy Student Version - Summer 2025 Onward](#)
Must Submit the Lab Agreement Form: [Form Link](#)

CLASSWORK

Task 1

Write a Java program that takes 10 inputs from the user in a loop, and displays the sum, average, minimum and maximum of **Only the positive odd numbers** from those numbers. If no such numbers are found, then display the message “No odd positive numbers found”.

Sample Input	Sample Output
1 4 2 9 2 -4 3 -1 0 1	Sum = 14 Minimum = 1 Maximum = 9 Average = 3.5
34 -11 50 24 -24 2 -4 0 8 12	No odd positive numbers found
23 2 -4 0 8 12 34 -11 53 21	Sum = 97 Minimum = 21 Maximum = 53 Average = 32.333333333333336

Task 2

Write a Java program that takes TWO string inputs (containing exactly one word in each string) from the user. Concatenate those two strings with a single space in between them. Generate a number **which is the sum of all the letters in that concatenated string** where A = 65, Z = 90, a = 97, and z = 122. Your task is to print that concatenated string and the number generated from that string.

Sample Input	Output
Hello123 Wo%%rld	Hello123 Wo%%rld 1020
Ja12-va CHOWD+ HURY	Ja12-va CHOWD+ HURY 1087

Task 3

Write a Java program that asks the user the length of an array (N) then takes N number of doubles as elements for the array as input. First, remove the consecutive duplicate elements from the original array **to form a new array**. Then print the number of elements removed from the original array.

Sample Input	Sample Output
N = 8 Please enter the elements of the array: 5.2 2.7 1.0 1.0 2.7 3.5 3.5 3.5	New Array: 5.2 2.7 1.0 2.7 3.5 Removed elements : 3

Task 4

Design the “**Student**” class so that the main method prints the following:

Tester Class	Output
<pre>public class StudentTester{ public static void main(String [] args){ Student s1 = new Student(); System.out.println("Name of the Student: "+s1.name); System.out.println("ID of the Student: "+s1.id); s1.name = "Bob"; s1.id = 123; System.out.println("Name of the Student: "+s1.name); System.out.println("ID of the Student: "+s1.id); } }</pre>	<pre>Name of the Student: Default ID of the Student: 0 Name of the Student: Bob ID of the Student: 123</pre>

Task 5

Consider the following class:

```
public class Human{
    public int age;
    public double height;
}
```

Show the output of the following sequence of statements:

[illegible]

For this course, we'll be using **DrJava** as IDE for Java Coding:

[Link to JDK and DrJava](#)

Drjava Installation Guide:

<https://www.youtube.com/watch?v=Gss9sL3Q-8s>

HOMEWORK

Task 1

Write a java program that takes 2 integer numbers as input and calculates how many prime numbers exist between them.

Sample Input	Sample Output
10 15	There are 2 prime numbers between 10 and 15.
150 100	There are 10 prime numbers between 100 and 150.

Task 2

Write a Java program that takes a string input in small letters from the user and prints the previous alphabet in sequence for each alphabet found in the input.

Sample Input	Output
wxyz	vwxxy
thecow	sgdbnv
abcd	zabc

Task 3

Write a Java program that will take an integer number N from the user and create an integer array by taking N numbers from the user. Print how many times each number appears in the array.

Sample Input	Sample Output
N = 5 6	6 - 2 times 15 - 2 times

15 14 15 6	14 - 1 times
N = 6 -5 10 14 10 -7 10	-5 - 1 times 10 - 3 times 14 - 1 times -7 - 1 times

Task 4

Design the **CSECourse** class to generate the correct output from the driver code provided below:

Driver Code	Output
<pre> public class CourseTester{ public static void main(String args []){ CSECourse c1 = new CSECourse(); System.out.println("Course Name: "+c1.courseName); System.out.println("Course Code: "+c1.courseCode); System.out.println("Credit: "+c1.credit); } } </pre>	Course Name: Programming Language II Course Code: CSE111 Credit: 3

Task 5

Consider the following class:

```
public class Student{
    public String name;
    public double cgpa;
}
```

Show the output of the following sequence of statements:

[illegible]

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Write a Java program that will take an integer number N from the user and create an integer array by taking N numbers from the user. Then take another number from the user and create a new array by removing that number from the input array. Finally, print the new array.

Sample Input	Sample Output
N = 5 23 100 0 56 -34 Remove Element = 100	Input array: 23 100 0 56 -34 New array: 23 0 56 -34
N = 4 -5 10 2 -7 Remove Element = 43	Input array: -5 10 2 -7 Element not found

Task 2

Write a program that reads 5 numbers into an array and prints the smallest and largest number and their location in the array.

Sample Input	Sample Output
7 13 2 10 6	The largest number 13 was found at location 1. The smallest number 2 was found at location 2.

2 4 -5 12 3	The largest number 12 was found at location 3. The smallest number -5 was found at location 2.
-------------------------	---

Task 3

Write a program that asks the user how many numbers to take. Then, it takes that many numbers in an array and prints the median value.

[How to Find the Median Value: <http://www.mathsisfun.com/median.html>]

Sample Input	Sample Output
5 10 50 40 20 30	The median is 30. Explanation: 30 falls in middle 10, 20, 30, 40, 50
4 30 10 40 20	The median is 25. Explanation: $(20+30)/2=25$ (average of two middle values from 10, 20, 30, 40).

Task 4

Write a Java program that asks the user for the length of an array and then creates an integer array of that length by taking inputs from the user. Then, reverse the **original array without** creating any new array and print it. [**In-place reverse**]

Sample Input	Sample Output
Enter the length of the array: 5 7 -31	100 97 344 -31 7

344	
97	
100	

Task 5

Design the **Player** class with the necessary properties so that the given Driver code produces the expected output.

Driver Code	Output
<pre> public class PlayerTester{ public static void main(String args[]){ Player player1 = new Player(); player1.name = "Ronaldo"; player1.jersey_number = 9; player1.position = "Striker"; System.out.println("Name of the Player: "+ player1.name); System.out.println("Jersey Number of player: "+ player1.jersey_number); System.out.println("Position of player: "+ player1.position); System.out.println("====="); Player player2 = new Player(); player2.name = "Neuer"; player2.jersey_number = 1; player2.position = "Goal Keeper"; System.out.println("Name of the player: "+ player2.name); System.out.println("Jersey Number of player: "+ player2.jersey_number); System.out.println("Position of player: "+ player2.position); } } </pre>	<pre> Name of the Player: Ronaldo Jersey Number of player: 9 Position of player: Striker ===== Name of the player: Neuer Jersey Number of player: 1 Position of player: Goal Keeper </pre>

Lab Assignment 02



Inspiring Excellence

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	OOP Basics, Instance Variable and Instance Method
Number of Tasks:	12 (Classwork: 06, Homework: 06)

[Submit all the Coding Tasks (Homework: Task 1 to 4) in the Google Form shared on buX before the next lab. Submit the Tracing Tasks (Homework: Task 5 to 6) handwritten to your Lab Instructors at the beginning of the lab]

[You are not allowed to change the driver codes of any of the tasks]

CLASSWORK

Task 1

You are given the following “**University**” class:

```
public class University{  
    public String name;  
    public String country;  
}
```

Now write a Java **tester** class named “**UniversityTester**”.

- a. Write the main method and create 2 objects of **University** class and print the location of the objects and print the instance variables of the objects. Are the location of the objects the same?
- b. Now change the instance variables of the first object.
 name = “Imperial College London”
 country = “England”

Now change the instance variables of the second object.

name = “BRAC University”
country = “Bangladesh”

Now check if the instance variables of both objects have changed or not and whether the instance variables of both objects are of the same value or not.

Task 2

Complete the “Cat” class so the main method produces the following output:

Test Class	Output
<pre>public class Test7{ public static void main(String [] args){ Cat c1 = new Cat(); System.out.println("1====="); c1.printCat(); c1.color = "Black"; System.out.println("2====="); c1.printCat(); c1.color = "Brown"; c1.action = "jumping"; System.out.println("3====="); c1.printCat(); } }</pre>	<pre>1===== White cat is sitting 2===== Black cat is sitting 3===== Brown cat is jumping</pre>

Task 3

Design the **Course** class to generate the correct output from the driver code provided below:

Driver Code	Output
<pre>public class Tester3{ public static void main(String[] args) { Course c1 = new Course(); Course c2 = new Course(); c1.updateDetails("Programming Language I","CSE110", 3); System.out.println("===== 1 ====="); c1.displayCourse(); c2.updateDetails("Data Structures","CSE220", 3); System.out.println("===== 2 ====="); c2.displayCourse(); c1.updateDetails("Programming Language II","CSE111", 3); System.out.println("===== 3 ====="); c1.displayCourse(); } }</pre>	<pre>===== 1 ===== Course Name: Programming Language I Course Code: CSE110 Course Credit: 3 ===== 2 ===== Course Name: Data Structures Course Code: CSE220 Course Credit: 3 ===== 3 ===== Course Name: Programming Language II Course Code: CSE111 Course Credit: 3</pre>

Task 4

Design the **CellPhone** class so that the **main** method of tester class can produce the following output:

Tester Class	Output
<pre>public class Tester4{ public static void main(String[]args){ CellPhone phone1 = new CellPhone(); phone1.printDetails(); phone1.model = "Nokia 1100"; System.out.println("1#####"); phone1.storeContact("Joy - 01834"); System.out.println("====="); phone1.printDetails(); System.out.println("2#####"); phone1.storeContact("Toya - 01334"); phone1.storeContact("Aayan - 01135"); System.out.println("====="); phone1.printDetails(); System.out.println("3#####"); phone1.storeContact("Sani - 01441"); System.out.println("====="); phone1.printDetails(); } }</pre>	<pre>Phone Model unknown Contacts Stored 0 1##### Contact Stored ===== Phone Model Nokia 1100 Contacts Stored 1 Stored Contacts: Joy - 01834 2##### Contact Stored Contact Stored ===== Phone Model Nokia 1100 Contacts Stored 3 Stored Contacts: Joy - 01834 Toya - 01334 Aayan - 01135 3##### Memory full. New contact can't be stored. ===== Phone Model Nokia 1100 Contacts Stored 3 Stored Contacts: Joy - 01834 Toya - 01334 Aayan - 01135</pre>

Task 5

1	<code>public class Task5 {</code>
2	<code> public int p = 3, y = 2, sum;</code>
3	<code> public void methodA(){</code>
4	<code> int x = 0, y = 0;</code>
5	<code> y = y + this.y;</code>
6	<code> x = sum + 2 + p;</code>
7	<code> sum = x + methodB(p, y) + y;</code>
8	<code> System.out.println(x + " " + y+ " " + sum);</code>
9	<code> }</code>
10	<code> public int methodB(int p, int n){</code>
11	<code> int x = 0;</code>
12	<code> y = y + (++p);</code>
13	<code> x = x + 2 + n;</code>
14	<code> sum = sum + x + y;</code>
15	<code> System.out.println(x + " " + y+ " " + sum);</code>
16	<code> return sum;</code>
17	<code> }</code>
18	<code>}</code>

Driver code:

<pre> public class Tester5 { public static void main(String [] args){ Task5 t1 = new Task5(); t1.methodA(); t1.methodA(); } } </pre>	Outputs		
	x	y	Sum

Task 6

1	public class Test6{
2	public int sum;
3	public int y;
4	public void methodA(){
5	int x=2, y =3;
6	int [] msg = {3, 7};
7	y = this.y + msg[0];
8	methodB(msg[1]++, msg[0]);
9	x = x + this.y + msg[1];
10	sum = x + y + msg[0];
11	System.out.println(x + " " + y+ " " + sum);
12	}
13	public void methodB(int mg2, int mg1){
14	int x = 0;
15	y = this.y + mg2;
16	x = x + 19 + mg1;
17	sum = this.sum + x + y;
18	mg2 = y + mg1;
19	mg1 = mg2 + x + 2;
20	System.out.println(x + " " + y+ " " + sum);
21	}
22	}

Driver code:

<pre> public class Tester6{ public static void main (String args[]){ Test6 t1 = new Test6(); t1.methodB(5,-8); Test6 t2 = new Test6(); t2.methodA(); } } </pre>	Outputs		

HOMEWORK

Task 1

Design the “**ImaginaryNumber**” class to generate the **output** given below:

Tester Class	Output
<pre>public class Tester7{ public static void main(String [] args){ ImaginaryNumber num1 = new ImaginaryNumber(); String p = num1.printNumber(); System.out.println(p); System.out.println("1*****"); num1.realPart=3; num1.imaginaryPart=7; System.out.println(num1.printNumber()); System.out.println("2*****"); ImaginaryNumber num2 = new ImaginaryNumber(); num2.realPart=1; num2.imaginaryPart=9; System.out.println(num2.printNumber()); } }</pre>	<pre>0 + 0i 1***** 3 + 7i 2***** 1 + 9i</pre>

Task 2

Implement the “**Assignment**” class with necessary properties, so that the given output is produced for the provided driver code.

Driver Class	Output
<pre>public class AssignmentTester{ public static void main(String [] args){ Assignment as1 = new Assignment(); as1.printDetails(); System.out.println("1-----"); as1.tasks = 11; as1.difficulty = "Moderate"; as1.submission = true; as1.printDetails(); System.out.println("2-----"); } }</pre>	<pre>Number of tasks: 0 Difficulty level: null Submission required: false 1----- Number of tasks: 11 Difficulty level: Moderate Submission required: true 2----- Assignment will not require submission</pre>

<pre> System.out.println(as1.makeOptional()); System.out.println("3-----"); as1.printDetails(); System.out.println("4-----"); Assignment as2 = new Assignment(); as2.tasks = 12; as2.difficulty = "Hard"; as2.submission = false; as2.printDetails(); System.out.println("5-----"); System.out.println(as2.makeOptional()); } } </pre>	<pre> 3----- Number of tasks: 11 Difficulty level: Moderate Submission required: false 4----- Number of tasks: 12 Difficulty level: Hard Submission required: false 5----- Submission is already not required </pre>
--	--

Task 3

Create an **Employee** class to provide the expected output.

- An employee will have a name, salary and designation.
- The name will be assigned inside the newEmployee() method
- Whenever a New Employee joins his/her salary will be **Tk. 30,000** and the designation will be **junior**.
- Employees with salaries greater than **Tk. 50,000** and **Tk. 30,000** need to pay **30%** and **10%** of salary as tax respectively.
- Employees can be promoted to **senior**, **lead** and **manager** positions. Based on their promotion they will get an increment of **Tk. 25,000**, **Tk. 50,000** and **Tk. 75,000** respectively.

Driver Code	Expected Output
<pre> public class Tester9{ public static void main(String[] args){ Employee emp1 = new Employee(); Employee emp2 = new Employee(); Employee emp3 = new Employee(); emp1.newEmployee("Harry Potter"); emp2.newEmployee("Hermione Granger"); emp3.newEmployee("Ron Weasley"); System.out.println("1 ====="); emp1.displayInfo(); System.out.println("2 ====="); emp2.displayInfo(); System.out.println("3 ====="); emp3.displayInfo(); System.out.println("4 ====="); emp1.calculateTax(); System.out.println("5 ====="); emp1.promoteEmployee("lead"); System.out.println("6 ====="); emp1.calculateTax(); System.out.println("7 ====="); emp1.displayInfo(); System.out.println("8 ====="); emp3.promoteEmployee("manager"); System.out.println("9 ====="); emp3.calculateTax(); System.out.println("10 ====="); emp3.displayInfo(); } } </pre>	<pre> 1 ===== Employee Name: Harry Potter Employee Salary: 30000.0 Tk Employee Designation: junior 2 ===== Employee Name: Hermione Granger Employee Salary: 30000.0 Tk Employee Designation: junior 3 ===== Employee Name: Ron Weasley Employee Salary: 30000.0 Tk Employee Designation: junior 4 ===== No need to pay tax 5 ===== Harry Potter has been promoted to lead New Salary: 80000.00 Tk 6 ===== Harry Potter Tax Amount: 24000.0 Tk 7 ===== Employee Name: Harry Potter Employee Salary: 80000.0 Tk Employee Designation: lead 8 ===== Ron Weasley has been promoted to manager New Salary: 105000.00 Tk 9 ===== Ron Weasley Tax Amount: 31500.0 Tk 10 ===== Employee Name: Ron Weasley Employee Salary: 105000.0 Tk Employee Designation: manager </pre>

Task 4

Implement the “**MobilePhone**” class with necessary properties to produce the given output for the provided driver code.

Driver Class	Output
<pre>public class MobilePhoneTester{ public static void main(String args []){ MobilePhone m1 = new MobilePhone(); MobilePhone m2 = new MobilePhone(); m1.setContactCapacity(5); m2.setContactCapacity(100); m1.details(); System.out.println("1-----"); m1.addContact("John", 9866); m1.addContact("Maria", 7865); System.out.println("2-----"); m1.details(); System.out.println("3-----"); m1.makeCall(9866); System.out.println("4-----"); m1.addContact("Henry", 2365); System.out.println("5-----"); m1.makeCall(7552); m1.makeCall(2365); System.out.println("6-----"); m1.addContact("Gomes", 4589); m1.addContact("Antony", 8421); m1.addContact("Tony", 5789); System.out.println("7-----"); m1.details(); } }</pre>	<pre>Total Contacts: 0 Contact List: 1----- The contact of John is added. The contact of Maria is added. 2----- Total Contacts: 2 Contact List: John:9866 Maria:7865 3----- Calling John . . . 4----- The contact of Henry is added. 5----- Calling 7552 . . . Calling Henry . . . 6----- The contact of Gomes is added. The contact of Antony is added. Storage Full!! 7----- Total Contacts: 5 Contact List: John:9866 Maria:7865 Henry:2365 Gomes:4589 Antony:8421</pre>

Task 5

1	public class B {
2	public int temp = 4;
3	public int sum, y, x;
4	public void methodA(int m){
5	int [] n = {2,5};
6	int x = 0;
7	y = m + this.methodB(x++,m)+(temp++);
8	x = this.x + 2 + n[0];
9	sum = sum + x + y;
10	n[0] = sum + 2;
11	System.out.println(n[0]+" " + x+ " " + sum);
12	}
13	public int methodB(int m, int n){
14	int y = 4 + this.y + m;
15	x = this.y + y + (++temp) - n;
16	sum = x + y + this.sum;
17	System.out.println(y+ " " + this.x + " " +sum);
18	return x;
19	}
20	}

<pre> public class Tester11 { public static void main(String [] args){ B t1 = new B(); t1.methodA(5); B t2 = new B(); t2.methodB(12, 2); } } </pre>	Outputs		

Task 6

1	public class A{
2	public int x = 3, y = 5, sum = 9;
3	public int methodA(int temp, int x){
4	this.x += (x++) + temp;
5	if(temp % 5 == 2){
6	sum += (this.y++) + temp;
7	}
8	else{
9	sum += 3;
10	if(y > 5) ++y;
11	}
12	System.out.println(this.x + " " + y + " " + sum);
13	return this.x;
14	}
15	public void methodB(int y){
16	int temp = (y++) + this.y;
17	this.y = (++temp) + methodA(temp, y) + x;
18	sum = y + (++this.x) + (temp++);
19	System.out.println(x + " " + y + " " + (sum++));
20	}
21	public void methodC(int y){
22	y = (this.x++) + sum + 3;
23	System.out.println(x + " " + y + " " + sum);
24	}
25	}

Driver code:

<pre> public class Test12{ public static void main(String [] args) { A a1 = new A(); a1.methodA(2, 4); a1.methodB(3); A a2 = new A(); a2.methodC(7); a1.methodB(3); } } </pre>	Outputs		

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Complete the **Bird** class so that main method produces the following **output**:

Test class	Output
<pre>public class BirdTest{ public static void main(String args[]) { Bird b1 = new Bird(); b1.name = "Parrot"; b1.flyUp(3); b1.makeNoise(); b1.flyDown(5); b1.flyDown(2); b1.flyDown(1); Bird b2 = new Bird(); b2.name = "Eagle"; b2.flyUp(5); b2.flyDown(5); b2.makeNoise(); } }</pre>	<pre>Parrot has flown up 3 feet. Squawk Parrot cannot fly down 5 feet. Parrot has flown down 2 feet. Parrot has flown down 1 feet and landed. Eagle has flown up 5 feet. Eagle has flown down 5 feet and landed. Squee</pre>

Task 2

Implement the “**ChickenBurger**” class with necessary properties, so that the given output is produced for the provided driver code.

[Note:

1. There are four available spice levels: **Mild**, **Spicy**, **Naga** and **Extreme**. You can store these values in a String array.
2. You might need to use the `.equals()` method to compare two string values.]

Driver Class	Output
<pre>public class BurgerMaker{ public static void main(String [] args){ ChickenBurger b1 = new ChickenBurger(); System.out.println(b1.bun); } }</pre>	<pre>Sesame 200 Less Not Set -----1----- Cannot serve now. Customize Spice</pre>

```

System.out.println(b1.price);
System.out.println(b1.sauceOption);
System.out.println(b1.spiceLevel);
System.out.println("-----1-----");
System.out.println(b1.serveBurger());
System.out.println("-----2-----");
b1.customizeSpiceLevel("Extreme Jhaal");
b1.customizeSpiceLevel("Spicy");
System.out.println("-----3-----");
System.out.println(b1.serveBurger());
System.out.println("-----4-----");
ChickenBurger b2 = new ChickenBurger();
b2.bun = "Brioche";
b2.price += 50;
b2.sauceOption = "Regular";
b2.customizeSpiceLevel("Naga");
System.out.println("-----5-----");
System.out.println(b2.serveBurger());
}
}

```

```

Level first.
-----2-----
This spice level is unavailable.
Spice level set to Spicy.
-----3-----
The burger is being served:-
Bun Type: Sesame
Price: 200
Sauce Option: Less
Spice Level: Spicy
-----4-----
Spice level set to Naga.
-----5-----
The burger is being served:-
Bun Type: Brioche
Price: 250
Sauce Option: Regular
Spice Level: Naga

```

Task 3

Design the **MagicPotion** class so that the following Tester class generates the expected output

- Potion cannot be activated if potion strength below 5

Driver Code	Outputs
<pre>public class MagicTester { public static void main(String[] args) { MagicPotion potion = new MagicPotion(); potion.setDetails("blue", 4); System.out.println("====="); potion.activate(); potion.describe(); System.out.println("====="); potion.boost(); potion.describe(); System.out.println("====="); potion.activate(); potion.describe(); System.out.println("====="); System.out.println("Is potion strong? " + potion.isStrong()); System.out.println("====="); potion.boost(); potion.activate(); potion.describe(); System.out.println("====="); System.out.println("Is potion strong? " + potion.isStrong()); System.out.println("====="); potion.boost(); System.out.println("Is potion strong? " + potion.isStrong()); System.out.println("====="); potion.weaken(); potion.weaken(); potion.weaken(); potion.describe(); } }</pre>	<pre>===== Potion colour: blue Potion strength: 4 Is active: false ===== Potion colour: blue Potion strength: 5 Is active: false ===== Potion colour: blue Potion strength: 5 Is active: true ===== Is potion strong? false ===== Already activated Potion colour: blue Potion strength: 6 Is active: true ===== Is potion strong? false ===== Is potion strong? true ===== Potion colour: blue Potion strength: 4 Is active: false</pre>

Task 4

1	public class TraceA{
2	public int x =0, y = 8, sum = 0;
3	public void methodA(int y, int x){
4	int [] arr = {4, 7};
5	this.x += x;
6	arr[0] = y++;
7	arr[1] += arr[1] % arr[0] * x;
8	if (y % 2 == 0){
9	sum = x + methodB(arr[0]++, arr[1], y) + this.x;
10	}
11	else{
12	sum = this.y + methodB(++arr[0], arr[1], x) + this.y;
13	}
14	System.out.println(x+ " " + arr[0] + " " + sum);
15	}
16	public int methodB(int a, int b, int x){
17	sum = sum % x;
18	if(a % b == x){
19	return this.y--;
20	}
21	this.x = a * b / x;
22	y += this.x % this.y;
23	System.out.println(a + " " + this.x + " " + y);
24	return y;
25	}
26	}

Driver code:

<pre> public class TesterX{ public static void main(String [] args){ TraceA t1 = new TraceA(); t1.methodA(4, 7); int x = t1.methodB(3,2,1); System.out.println(t1.y + " " + t1.sum + " " + x); } } </pre>	Output		

Lab Assignment 03



Inspiring Excellence

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	Method Overloading, Constructor & Overloading
Number of Tasks:	12 (Classwork: 06, Homework: 06)

[Submit all the Coding Tasks (Homework: Task 1 to 4) in the Google Form shared on buX before the next lab. Submit the Tracing Tasks (Homework: Task 5 to 6) handwritten to your Lab Instructors at the beginning of the lab]

[You are not allowed to change the driver codes of any of the tasks]

CLASSWORK

Task 1

Design the **Shape** class with necessary properties to produce the given output for the provided driver code.

Driver Class	Output
<pre>public class ShapeTester{ public static void main(String args []){ Shape circle = new Shape(); Shape triangle = new Shape(); Shape rectangle = new Shape(); circle.setParameters("Circle", 5); triangle.setParameters("Triangle", 4, 7); rectangle.setParameters("Rectangle", 2.4, 4.4); System.out.println(circle.details()); System.out.println("1-----"); System.out.println(triangle.details()); System.out.println("2-----"); System.out.println(rectangle.details()); } }</pre>	<pre>Shape Name: Circle Area: 78.54 1----- Shape Name: Triangle Area: 14.0 2----- Shape Name: Rectangle Area: 10.56</pre>

Task 2

Design the **Student** class in such a way that it produces the following output.

Driver Code	Expected Output
<pre>public class StudentTester{ public static void main(String[] args){ Student s1 = new Student("Harry", "CSE"); System.out.println(s1.name); s1.updateName("Harry Potter"); System.out.println(s1.name); System.out.println(s1.prog); s1.updateProgram("CS"); String var = s1.accessProgram(); System.out.println(var); } }</pre>	<pre>Harry Harry Potter CSE CS</pre>

Task 3

Design the **Shape2D** class in such a way that it produces the following output.

Driver Code	Expected Output
<pre>public class Shape2DTester { public static void main(String[] args) { Shape2D sq = new Shape2D(); System.out.println("-----1-----"); sq.area(); System.out.println("-----2-----"); Shape2D rectangle = new Shape2D(5, 6); System.out.println("-----3-----"); rectangle.area(); System.out.println("-----4-----"); Shape2D tri1 = new Shape2D(5, 6, "Triangle"); System.out.println("-----5-----"); tri1.area(); System.out.println("-----6-----"); Shape2D tri2 = new Shape2D(5, 6, 7); System.out.println("-----7-----"); tri2.area(); } }</pre>	<pre>A Square has been created with length: 5 -----1----- The area of the Square is: 25.00 -----2----- A Rectangle has been created with length: 5 and breath: 6 -----3----- The area of the Rectangle is: 30.00 -----4----- A Triangle has been created with height: 5 and base: 6 -----5----- The area of the Triangle is: 15.00 -----6----- A Triangle has been created with the following sides: 5, 6, 7 -----7----- The area of the Triangle is: 14.70</pre>

Task 4

Design the **Library** class with the necessary properties so that the given output is produced for the provided driver code.

Driver Code	Output
<pre>public class Tester5{ public static void main(String[] args) { Library a1 = new Library(3); System.out.println("1-----"); a1.addBook("Ice"); System.out.println("2-----"); a1.printDetail(); System.out.println("3-----"); a1.addBook("Emma"); a1.addBook("Wings"); a1.addBook("Next"); System.out.println("4-----"); a1.printDetail(); System.out.println("5-----"); Library a2 = new Library(4); System.out.println("6-----"); a2.addBook("Onnobhubon"); a2.addBook("Ami"); System.out.println("7-----"); a2.printDetail(); System.out.println("8-----"); a2.addBook("Deyal"); a2.addBook("Himu"); a2.addBook("Megher Upor Bari"); System.out.println("9-----"); a2.printDetail(); } }</pre>	<pre>A library has been created with capacity 3 1----- Book 'Ice' added to the library 2----- Maximum Capacity: 3 Total Books: 1 Book list: Ice 3----- Book 'Emma' added to the library Book 'Wings' added to the library Exceeds maximum capacity. You can't add more than 3 books 4----- Maximum Capacity: 3 Total Books: 3 Book list: Ice Emma Wings 5----- A library has been created with capacity 4 6----- Book 'Onnobhubon' added to the library Book 'Ami' added to the library 7----- Maximum Capacity: 4 Total Books: 2 Book list: Onnobhubon Ami 8----- Book 'Deyal' added to the library Book 'Himu' added to the library Exceeds maximum capacity. You can't add more than 4 books 9----- Maximum Capacity: 4 Total Books: 4 Book list: Onnobhubon Ami Deyal Himu</pre>

Task 5

Design the **TaxiLagbe** class with necessary properties to produce the given output for the provided driver code.

Driver Code	Output
<pre>public class TaxiTester{ public static void main(String[] args) { TaxiLagbe taxi1 = new TaxiLagbe("1010-01","Dhaka"); System.out.println("1-----"); taxi1.printDetails(); System.out.println("2-----"); taxi1.addPassenger("Wilson", 105); System.out.println("3-----"); taxi1.printDetails(); System.out.println("4-----"); taxi1.addPassenger("Walker", 100, "Wood", 200); System.out.println("5-----"); taxi1.printDetails(); System.out.println("6-----"); taxi1.addPassenger("Karen", 200); taxi1.addPassenger("Donald", 130); System.out.println("7-----"); taxi1.printDetails(); System.out.println("8-----"); TaxiLagbe taxi2 = new TaxiLagbe("1010-02"); System.out.println("9-----"); taxi2.addPassenger("Don", 115, "Parker", 215); System.out.println("10-----"); taxi2.printDetails(); } }</pre>	<pre>A new taxi has been created 1----- Taxi number: 1010-01 This taxi can cover Dhaka area Total Passenger: 0 Passenger List: Total collected fare: 0 Taka 2----- Dear Wilson! Welcome to TaxiLagbe 3----- Taxi number: 1010-01 This taxi can cover Dhaka area Total Passenger: 1 Passenger List: Wilson Total collected fare: 105 Taka 4----- Dear Walker! Welcome to TaxiLagbe Dear Wood! Welcome to TaxiLagbe 5----- Taxi number: 1010-01 This taxi can cover Dhaka area Total Passenger: 3 Passenger List: Wilson Walker Wood Total collected fare: 405 Taka 6----- Dear Karen! Welcome to TaxiLagbe Taxi Full! No more passengers can be added 7----- Taxi number: 1010-01 This taxi can cover Dhaka area Total Passenger: 4 Passenger List: Wilson Walker Wood Karen Total collected fare: 605 Taka 8----- A new taxi has been created 9----- Dear Don! Welcome to TaxiLagbe Dear Parker! Welcome to TaxiLagbe 10----- Taxi number: 1010-02 This taxi can cover Khulna area Total Passenger: 2 Passenger List: Don Parker Total collected fare: 330 Taka</pre>

Task 6

1	public class A{
2	public int temp = 3, sum = 9, y = 4, x = 0;
3	public A(){
4	int sum = 7;
5	y = temp - 5;
6	sum = temp + 2;
7	temp -= 2;
8	this.x = sum + temp + y;
9	}
10	public A(int y, int temp){
11	y = temp - 1 + x;
12	sum = temp + 2 -x;
13	temp-=2;
14	}
15	public void methodB(int m, int [] n){
16	int x = 0;
17	y = y + m + this.methodB(x,m) ;
18	x = this.x + 2 + (++n[0]) ;
19	sum = sum + x + y;
20	n[0] = sum + 2;
21	System.out.println(n[0] + " " + y+ " " + sum) ;
22	}
23	public int methodB(int m, int n){
24	int [] y = {10};
25	this.y = y[0] + this.y + m;
26	x = this.y + 2 + temp - n;
27	sum = x + y[0] + this.sum;
28	System.out.println(y[0]+ " "+ temp + " " + sum) ;
29	return y[0] ;
30	}
31	}

Driver Code	Output		
<pre> public class Tester6 { public static void main(String args[]){ int[] x = {35}; A a1; a1 = new A(); A a2 = new A(-5,-7); a1.methodB(1, x); a2.methodB(1, x); } } </pre>			

HOMEWORK

Task 1

Design the **Toy** class in such a way that it produces the following output

Driver Code	Expected Output
<pre>public class ToyTester{ public static void main(String[] args){ Toy t1 = new Toy("Car", 230); System.out.println("1====="); t1.updatePrice(250); System.out.println("2====="); System.out.println(t1.name); t1.showPrice(); System.out.println("3====="); Toy t2 = new Toy("Robot", 450); System.out.println("4====="); t2.updateName("Autobot"); System.out.println("5====="); System.out.println(t2.name); t2.showPrice(); } }</pre>	<pre>A new toy has been made! 1===== 2===== Car price: 250 Taka 3===== A new toy has been made! 4===== Changing old name: Robot new name: Autobot 5===== Autobot price: 450 Taka</pre>

Task 2

Complete the following **Cart** class to generate the given output from the tester code:

- A cart will have a cart number which will be assigned in *create_cart()* method.
- Each cart can hold up to 3 items (at max).
- Each cart must have two arrays to store items and their respective prices.
- The items inside a cart will be added in *addItem()* method only if the cart items do not exceed 3.
- The *giveDiscount()* method saves the discount given to that cart object and updates the price accordingly.

Driver Code	Output
<pre> public class CartTester{ public static void main(String [] args){ Cart c1 = new Cart (); Cart c2 = new Cart (); Cart c3 = new Cart (); c1.create_cart(1); c2.create_cart(2); c3.create_cart(3); System.out.println("====1===="); c1.addItem("Table", 3900.5); c1.addItem("Chair", 1400.76); c1.addItem(5400.87, "Television"); c1.addItem(5000.0, "Refrigerator"); System.out.println("====2===="); c2.addItem("Stove", 439.90); System.out.println("====3===="); c3.addItem("Chair", 1400.5); c3.addItem(3400.0, "Chair"); System.out.println("====4===="); c1.cartDetails(); System.out.println("====5===="); c2.cartDetails(); System.out.println("====6===="); c3.cartDetails(); c1.giveDiscount(10); System.out.println("====7===="); c1.cartDetails(); } } </pre>	<pre> ====1==== Table added to cart 1. You have 1 item(s) in your cart now. Chair added to cart 1. You have 2 item(s) in your cart now. Television added to cart 1. You have 3 item(s) in your cart now. You already have 3 items on your cart ====2==== Stove added to cart 2. You have 1 item(s) in your cart now. ====3==== Chair added to cart 3. You have 1 item(s) in your cart now. Chair added to cart 3. You have 2 item(s) in your cart now. ====4==== Your cart(c1) : Table - 3900.5 Chair - 1400.76 Television - 5400.87 Discount Applied: 0.0% Total price: 10702.130000000001 ====5==== Your cart(c2) : Stove - 439.9 Discount Applied: 0.0% Total price: 439.9 ====6==== Your cart(c3) : Chair - 1400.5 Chair - 3400.0 Discount Applied: 0.0% Total price: 4800.5 ====7==== Your cart(c1) : Table - 3900.5 Chair - 1400.76 Television - 5400.87 Discount Applied: 10.0% Total price: 9631.917000000001 </pre>

Task 3

Design the **Reader** class in such a way so that the following code provides the expected output.

- A reader will have a name, capacity to read and an array of books they are reading.
- The initial capacity of a reader will be 2. The initial name will be “New user”.

Driver Code	Expected Output
<pre>public class Reader_tester { public static void main(String[] args){ System.out.println("1 ====="); Reader r1 = new Reader("Messi"); Reader r2 = new Reader("Ronaldo", 3); System.out.println("2 ====="); r1.readerInfo(); System.out.println("3 ====="); r2.addBook("Java"); r2.addBook("Python"); r2.addBook("C++"); r2.readerInfo(); System.out.println("4 ====="); r1.addBook("C#"); r1.addBook("Rust"); r1.addBook("GoLang"); System.out.println("5 ====="); r2.addBook("Python"); System.out.println("6 ====="); r1.readerInfo(); System.out.println("7 ====="); r1.updateCapacity(3); System.out.println("8 ====="); r1.addBook("GoLang"); System.out.println("9 ====="); r1.readerInfo(); } }</pre>	<pre>1 ===== A new reader is created! A new reader is created! 2 ===== Name: Messi Capacity: 2 Books: No books added yet 3 ===== Name: Ronaldo Capacity: 3 Books: Book 1: Java Book 2: Python Book 3: C++ 4 ===== No more capacity 5 ===== No more capacity 6 ===== Name: Messi Capacity: 2 Books: Book 1: C# Book 2: Rust 7 ===== Capacity has changed to 3 8 ===== 9 ===== Name: Messi Capacity: 3 Books: Book 1: C# Book 2: Rust Book 3: GoLang</pre>

Task 4

You are building a tracker system that will keep track of a person's income and expenses.

- When the ***createTracker()*** method is invoked it sets the balance to 1.0 taka.
- The ***info()*** method **returns** a String with the trackers information.
- If the total balance becomes 0 after the ***expense()*** method is called it prints "You're broke!". Again if the available balance is less than the expense it prints "Not enough balance.". Otherwise the method prints "Balance updated" after updating the balance.
- The last expense and income history can be seen by using the ***history()*** method.

Driver Code	Output
<pre>public class Tester4{ public static void main(String[] args) { MoneyTracker tr1 = new MoneyTracker(); System.out.println(tr1.info()); tr1.createTracker("John"); System.out.println("1 ====="); System.out.println(tr1.info()); System.out.println("2 ====="); tr1.income(1000); System.out.println(tr1.info()); System.out.println("3 ====="); tr1.expense(800); tr1.expense(100); System.out.println(tr1.info()); System.out.println("4 ====="); tr1.showHistory(); System.out.println("5 ====="); tr1.expense(101); System.out.println("6 ====="); tr1.expense(200); System.out.println("7 ====="); tr1.income(200); tr1.showHistory(); System.out.println("8 ====="); } }</pre>	<pre>A new money tracker has been launched. Name: null Current Balance: 0.0 1 ===== Name: John Current Balance: 1.0 2 ===== Balance Updated! Name: John Current Balance: 1001.0 3 ===== Balance Updated. Balance Updated. Name: John Current Balance: 101.0 4 ===== Last added: 1000.0 Last spent: 100.0 5 ===== You're broke! 6 ===== Not enough balance. 7 ===== Balance Updated! Last added: 200.0 Last spent: 101.0 8 =====</pre>

Task 5

1	public class A{
2	public int x, y, z = 5;
3	public double p = 0.0;
4	public void methodA(int x, int m) {
5	this.x = methodB(this.x);
6	p = x + this.x % m * 3.0;
7	y = y + methodB(x++, this.x);
8	System.out.println(this.x + " " + x + y + " " + p) ;
9	}
10	public int methodB(int q, int n) {
11	int arr[] = {3,4,5};
12	arr[0] = arr[0] + this.x + n;
13	arr[1] = q + arr[1];
14	System.out.println(arr[0] + " " + arr[1] + " " + arr[2]) ;
15	return arr[1] + arr[2];
16	}
17	public int methodB(int y) {
18	if(y % 2 == 0) {
19	int temp = this.methodB(2, y);
20	return temp;
21	}
22	else{
23	return 4;
24	}
25	}
26	}

Driver Code	Output		
<pre> public class Tester11 { public static void main(String [] args){ A a1 = new A(); a1.methodA(2,3); a1.methodB(5,4); } } </pre>			

Task 6

1	public class Maze{
2	public int x;
3	public Maze(){
4	int x = 17;
5	this.methodB(x, -this.x);
6	}
7	public void methodA(){
8	int m = 0, x = 9;
9	m = methodB(m-3)+x;
10	this.x = ++x;
11	System.out.println(this.x+" "+m);
12	this.methodB(x,m);
13	System.out.println(x+" "+(m+this.x));
14	methodB(m);
15	}
16	public int methodB(int y){
17	x=y*y;
18	System.out.println(x+" "+y);
19	return x-11;
20	}
21	public void methodB(int z, int x){
22	z=z-2;
23	x=this.x-2*x;
24	System.out.println(z+" "+this.x);
25	}
26	}

[illegible]

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Driver Code	Output
<pre>public class CustomerTester { public static void main(String[] args) { Customer c1 = new Customer(); c1.createCustomer("John"); System.out.println("1====="); c1.showCart(); System.out.println("2====="); c1.addItem("Apple", 2); c1.addItem("Bread", 5); c1.addItem("Milk"); c1.addItem("Eggs", 2); System.out.println("3====="); c1.showCart(); System.out.println("4====="); c1.calculatePrice(); System.out.println("5====="); Customer c2 = new Customer(); c2.createCustomer("Jane"); c2.addItem("Apple", 2, "Orange", 5); c2.addItem("Chocolates", 15, "Bread", 5); c2.addItem("Milk", 3); System.out.println("6====="); c2.showCart(); System.out.println("7====="); c2.calculatePrice(); } }</pre>	<pre>1===== Customer: John 2===== Apple added to cart Orange added to cart Bread added to cart Milk added to cart Cart is full 3===== Customer: John Item: Apple Price: 2 Item: Orange Price: 5 Item: Bread Price: 5 Item: Milk Price: 3 4===== Total: 15 5===== Apple and Orange added to cart Chocolates and Bread added to cart Cart is full 6===== Customer: Jane Item: Apple Price: 2 Item: Orange Price: 5 Item: Chocolates Price: 15 Item: Bread Price: 5 7===== Total: 27</pre>

Task 2

Driver Code	Sample Output
<pre>public class CalculatorTester { public static void main(String[] args) { Calculator calc = new Calculator(); System.out.println("1====="); calc.add(10, 20); System.out.println("2====="); calc.add(5, 15, 25); System.out.println("3====="); calc.multiply(6, 7); System.out.println("4====="); calc.multiply(2, 3, 4); System.out.println("5====="); calc.multiply("Hello", 3); System.out.println("6====="); calc.multiply("Java", 5); } }</pre>	<pre>1===== 30 2===== 45 3===== 42 4===== 24 5===== Hello-Hello-Hello 6===== Java-Java-Java-Java-Java</pre>

Task 3

Driver Code	Sample Output
<pre>public class LibraryTest { public static void main(String[] args) { Book book1 = new Book(); book1.createBook("The Great Gatsby"); Book book2 = new Book(); book2.createBook("1984", "George Orwell"); Book book3 = new Book(); book3.createBook("To Kill a Mockingbird", "Harper Lee", "Fiction"); System.out.println(" ---Book Customization--- "); book1.customizeGenre("Classic"); book1.customizePages(180); book2.customizeGenre("Dystopian"); book2.customizePages(328); book3.customizePages(281); System.out.println(); System.out.println(" ---Library Inventory--- "); book1.displayDetails(); book2.displayDetails(); book3.displayDetails(); } }</pre>	<pre> ---Book Customization--- Updated genre of "The Great Gatsby" to Classic. Updated pages of "The Great Gatsby" to 180 pages. Updated genre of "1984" to Dystopian. Updated pages of "1984" to 328 pages. Updated pages of "To Kill a Mockingbird" to 281 pages. ---Library Inventory--- Title: The Great Gatsby, Author: Unknown, Genre: Classic, Pages: 180 Title: 1984, Author: George Orwell, Genre: Dystopian, Pages: 328 Title: To Kill a Mockingbird, Author: Harper Lee, Genre: Fiction, Pages: 281</pre>

Task 4

Driver Code	Sample Output
<pre>public class MovieManagerTest { public static void main(String[] args) { Movie inception = new Movie(); inception.setMovieDetails("Inception", "Christopher Nolan", 8.8); System.out.println("1====="); inception.addActors("Leonardo DiCaprio", "Joseph Gordon-Levitt"); inception.addActors("Ellen Page"); inception.showInfo(); System.out.println("2====="); Movie avengers = new Movie(); avengers.setMovieDetails("Avengers: Endgame", "Anthony Russo", 8.4); avengers.addActors("Robert Downey Jr.", "Chris Evans", "Scarlett Johansson"); avengers.showInfo(); System.out.println("3====="); Movie parasite = new Movie(); parasite.setMovieDetails("Parasite", "Bong Joon-ho"); parasite.addActors("Song Kang-ho", "Choi Woo-shik"); parasite.updateRating(8.6); parasite.showInfo(); System.out.println("4====="); parasite.updateRating(8.9); parasite.showInfo(); } }</pre>	<pre>1===== Added actor "Leonardo DiCaprio" to "Inception". Added actor "Joseph Gordon-Levitt" to "Inception". Added actor "Ellen Page" to "Inception". Title: Inception Director: Christopher Nolan Rating: 8.8 Actors: Leonardo DiCaprio, Joseph Gordon-Levitt, Ellen Page 2===== Added actor "Robert Downey Jr." to "Avengers: Endgame". Added actor "Chris Evans" to "Avengers: Endgame". Added actor "Scarlett Johansson" to "Avengers: Endgame". Title: Avengers: Endgame Director: Anthony Russo Rating: 8.4 Actors: Robert Downey Jr., Chris Evans, Scarlett Johansson 3===== Added actor "Song Kang-ho" to "Parasite". Added actor "Choi Woo-shik" to "Parasite". Updated rating of "Parasite" to 8.6 Title: Parasite Director: Bong Joon-ho Rating: 8.6 Actors: Song Kang-ho, Choi Woo-shik 4===== Updated rating of "Parasite" to 8.9 Title: Parasite Director: Bong Joon-ho Rating: 8.9 Actors: Song Kang-ho, Choi Woo-shik</pre>

Task 5

1	public class Test3 {
2	int x = 2, y = 4, sum = 3;
3	int arr[] = {x, y, sum};
4	public void methodA(int x) {
5	arr[0] += methodB(y, this.x) + methodC(x);
6	System.out.println(x + " " + this.x + " " + sum);
7	arr[1] += this.x * (++y) / (sum % x);
8	System.out.println(y + " " + sum + " " + this.x);
9	arr[2] += methodC(x) + methodB(this.x, sum);
10	System.out.println(arr[0] + " " + arr[1] + " " + arr[2]);
11	}
12	public int methodB(int q, int n) {
13	int arr2[] = {7, 8};
14	int a = (arr2[0]++) - q;
15	int b = (++arr2[1]) - n;
16	return a + b;
17	}
18	public int methodC(int z) {
19	z = sum + methodB(x, sum) - z;
20	return z/2;
21	}
22	}

<pre> public class Tester3{ public static void main(String [] args){ Test3 t3 = new Test3(); t3.methodA(7); } } </pre>	Outputs		

Lab Assignment 04



Inspiring Excellence

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	Reference Passing, Multiclass Problem & Encapsulation
Number of Tasks:	13 (Classwork: 06, Homework: 07)

[Submit all the Coding Tasks (Homework: Task 1 to 4) in the Google Form shared on buX before the next lab. Submit the Tracing Tasks (Homework: Task 5 to 6) handwritten to your Lab Instructors at the beginning of the lab]

[You are not allowed to change the driver codes of any of the tasks]

CLASSWORK

Task 1

Design the **Shop** class with necessary properties so that the following output is produced.

Driver Code	Output
<pre>public class ShopTester{ public static void main(String[] args){ Shop s1 = new Shop("Agora", 4); Shop s2 = new Shop(3); System.out.println("1====="); s1.info(); System.out.println("2====="); s2.info(); System.out.println("3====="); s1.addItem("KitKat", 25); s1.addItem("Hershey's", 100); s1.addItem("Dairy Milk", 150); s1.addItem("Feastables", 500); s1.addItem("Prime", 1200); System.out.println("4====="); s1.info(); System.out.println("5====="); s1.purchase("KitKat"); s1.purchase("Snickers"); System.out.println("6====="); s2.info(); System.out.println("7====="); s2.addItem(s1, "KitKat"); s2.addItem("Toblerone", 80); s2.addItem(s1, "Mars"); System.out.println("8====="); s2.info(); } }</pre>	<pre>Agora shop created! Mega shop created! 1===== Shop Name: Agora Item Details: No items in shop 2===== Shop Name: Mega Item Details: No items in shop 3===== Prime, could not be added 4===== Shop Name: Agora Item Details: 4 / 4 KitKat - 25 Tk Hershey's - 100 Tk Dairy Milk - 150 Tk Feastables - 500 Tk 5===== Purchase Complete! Snickers, is not available in this shop 6===== Shop Name: Mega Item Details: No items in shop 7===== Mars, not found in Agora! 8===== Shop Name: Mega Item Details: 2 / 3 KitKat - 25 Tk Toblerone - 80 Tk</pre>

Task 2

Write the **Teacher** and **Course** classes so that the TestTeacher class produces the outputs given.
Hint: A teacher can add a maximum of 3 courses.

[The attribute of Course class must be Private]

Driver Code	Output
<pre>public class TestTeacher{ public static void main(String [] args){ Teacher t1 = new Teacher("Matin Saad Abdullah","MSA"); Teacher t2 = new Teacher("Mumit Khan","MMK"); Teacher t3 = new Teacher("Sadia Hamid Kazi","SKZ"); Course c1 = new Course("CSE 110"); Course c2 = new Course("CSE 111"); Course c3 = new Course("CSE 220"); Course c4 = new Course("CSE 221"); Course c5 = new Course("CSE 230"); Course c6 = new Course("CSE 310"); Course c7 = new Course("CSE 320"); Course c8 = new Course("CSE 340"); t1.addCourse(c1); t1.addCourse(c2); t2.addCourse(c3); t2.addCourse(c4); t2.addCourse(c5); t3.addCourse(c6); t3.addCourse(c7); t3.addCourse(c8); System.out.println("1====="); t1.printDetail(); System.out.println("2====="); t2.printDetail(); System.out.println("3====="); t3.printDetail(); } }</pre>	<pre>A new teacher has been created A new teacher has been created A new teacher has been created 1===== Name: Matin Saad Abdullah Initial: MSA List of courses: CSE 110 CSE 111 2===== Name: Mumit Khan Initial: MMK List of courses: CSE 220 CSE 221 CSE 230 3===== Name: Sadia Hamid Kazi Initial: SKZ List of courses: CSE 310 CSE 320 CSE 340</pre>

Task 3

Design the **Team** and **Player** classes to get the output as shown.

Hints:

- Create an array in the **Team** class to store the **Player**'s object
- Use constructor overloading technique for **Team** class
- A team can add maximum 11 players
- All attributes of **Player** class must be kept **Private**.

Driver Code	Output
<pre>public class TeamTester { public static void main(String[] args) { Team b = new Team(); b.updateName("Bangladesh"); Player mashrafi = new Player("Mashrafi", 42, 100); b.addPlayer(mashrafi); Player tamim = new Player("Tamim", 35, 70); b.addPlayer(tamim); System.out.println("====="); b.printDetail(); System.out.println("====="); Team a = new Team("Australia"); Player ponting = new Player("Ponting", 50, 300); a.addPlayer(ponting); Player lee = new Player("Lee", 49, 200); a.addPlayer(lee); a.printDetail(); } }</pre>	<pre>===== Team: Bangladesh List of players: Name: Mashrafi Age: 42, Total Matches: 100 Name: Tamim Age: 35, Total Matches: 70 ===== Team: Australia List of players: Name: Ponting Age: 50, Total Matches: 300 Name: Lee Age: 49, Total Matches: 200</pre>

Task 4

Design the required class/es so that the following output is generated. Read the following description:

1. You may assume that to board a bus, a student must have the bus pass, and his/her location must match the route of the bus.
2. Additionally, the default maximum capacity of the bus is 2.
3. The location attribute of the **Student** class will be **private**

Driver Code	Output
<pre> public class BracuStudentTester { public static void main(String[] args) { BracuStudent st1 = new BracuStudent("Afif", "Mirpur"); System.out.println("1====="); BracuStudent st2 = new BracuStudent("Shanto", "Motijheel"); BracuStudent st3 = new BracuStudent("Taskin", "Mirpur"); st1.showDetails(); st2.showDetails(); System.out.println("2====="); st3.showDetails(); System.out.println("3====="); BracuBus bus1 = new BracuBus("Mirpur"); BracuBus bus2 = new BracuBus("Azimpur", 5); bus1.showDetails(); bus2.showDetails(); System.out.println("4====="); st2.collectPass(); st3.collectPass(); System.out.println("5====="); st2.showDetails(); st3.showDetails(); System.out.println("6====="); bus1.board(); System.out.println("7====="); bus1.board(st1, st2); System.out.println("8====="); st1.collectPass(); st2.setLocation("Mirpur"); st1.showDetails(); st2.showDetails(); System.out.println("9====="); bus1.board(st1); bus1.board(st2, st3); System.out.println("10====="); bus1.showDetails(); } } </pre>	<pre> 1===== Student Name: Afif Lives in Mirpur Have Bus Pass? false Student Name: Shanto Lives in Motijheel Have Bus Pass? false 2===== Student Name: Taskin Lives in Mirpur Have Bus Pass? false 3===== Bus Route: Mirpur Passenger Count: 0 (Max: 2) Passengers on Board: Bus Route: Azimpur Passenger Count: 0 (Max: 5) Passengers on Board: 4===== 5===== Student Name: Shanto Lives in Motijheel Have Bus Pass? true Student Name: Taskin Lives in Mirpur Have Bus Pass? true 6===== No passengers 7===== You don't have a bus pass! You got on the wrong bus! 8===== Student Name: Afif Lives in Mirpur Have Bus Pass? true Student Name: Shanto Lives in Mirpur Have Bus Pass? true 9===== Afif boarded the bus. Shanto boarded the bus. Bus is full! 10===== Bus Route: Mirpur Passenger Count: 2 (Max: 2) Passengers on Board: Afif Shanto </pre>

Task 5

1	public class Trace {
2	public int sum, temp;
3	public Trace(int sum, int temp){
4	this.sum = sum;
5	this.temp = temp;
6	}
7	}
8	class Quiz5{
9	public int sum = 11, x = -2, y = 16;
10	public Trace trace = null;
11	public Quiz5(Trace t){
12	trace = t;
13	int x = trace.temp + y;
14	sum = sum + (t.sum++) + y;
15	System.out.println(trace.sum + " " + sum + " " + x);
16	sum -= 10;
17	}
18	public void methodA(int sum, int temp){
19	sum = -3 + this.sum - trace.sum;
20	x = sum + 13 + y;
21	y = trace.temp + temp + sum;
22	this.sum = y + methodB(trace.temp, trace) + trace.temp;
23	System.out.println(sum + " " + y + " " + this.sum);
24	}
25	public int methodB(int x, Trace temp){
26	int sum = x + temp.sum + this.x;
27	temp.temp = sum + this.sum;
28	System.out.println(x + " " + temp.temp + " " + sum);
29	return sum;
30	}
31	}

<pre>Trace p = new Trace(3, 4); Quiz5 q = new Quiz5(p); q.methodA(4, 8); q.methodA(5, 10);</pre>	Output		

Task 6

1	public class TracingX {
2	public int x, y = 1;
3	public int metA(int y){
4	y += x + 3;
5	int temp = y + this.y;
6	if (temp % 2 == 0){
7	return temp;
8	}
9	TracingX t = new TracingX();
10	t.y = this.x - (++x) + t.x;
11	this.y = y + t.metA(t.x);
12	System.out.println(x + " " + y + " " + temp);
13	return temp + this.y;
14	}
15	}

Driver code:

```
public class TesterX {  
    public static void main(String[] args) {  
        TracingX t1 = new TracingX();  
        t1.y = t1.x = 5;  
        TracingX t2 = new TracingX();  
        t2.x = t1.metA(2);  
        t2.y = t2.metA(4);  
        System.out.println(t1.y + t1.x + " " + t2.x + " " + t2.y);  
    }  
}
```

Output:

HOMEWORK

Task 1

Design the **Triangle** Class that will produce the following output. We will consider both triangles to have the same sides if all sides are equal in the same orientation/sequence only.

Types of Triangle:

- Equilateral: When all sides in the same orientation are equal.
- Isosceles: When any two sides of a triangle in the same orientation are equal.
- Scalene: When all sides are of different lengths.

Driver Code	Output
<pre>public class TriangleTester{ public static void main(String args[]){ Triangle t1 = new Triangle(4, 4, 4); Triangle t2 = new Triangle(4, 5, 6); Triangle t3 = new Triangle(4, 5, 6); Triangle t4 = new Triangle(5, 4, 6); t1.triangleDetails(); System.out.println("-----1-----"); System.out.println(t1.printTriangleType()); System.out.println("-----2-----"); t3.triangleDetails(); System.out.println(t3.printTriangleType()); System.out.println("-----3-----"); t4.triangleDetails(); System.out.println(t4.printTriangleType()); System.out.println("-----4-----"); t2.compareTriangles(t3); System.out.println("-----5-----"); t1.compareTriangles(t2); System.out.println("-----6-----"); t1 = t2; t1.compareTriangles(t2); System.out.println("-----7-----"); t3.compareTriangles(t4); } }</pre>	<pre>Three sides of the triangle are: 4, 4, 4 Perimeter: 12 -----1----- This is an Equilateral Triangle. -----2----- Three sides of the triangle are: 4, 5, 6 Perimeter: 15 This is a Scalene Triangle. -----3----- Three sides of the triangle are: 5, 4, 6 Perimeter: 15 This is a Scalene Triangle. -----4----- Addresses are different but the sides of the triangles are equal. -----5----- Addresses, length of the sides and perimeter all are different. -----6----- These two triangle objects have the same address. -----7----- Only the perimeter of both triangles is equal.</pre>

Task 2

Design the **Player** class to obtain the following output from the driver code. You can assume, each player can defeat at most 5 villains.

Driver Code	Expected Output
<pre>public class PlayerTester { public static void main(String[] args) { Player ben = new Player("Ben", 10); System.out.println("=====1====="); ben.viewInfo(); System.out.println("=====2====="); ben.defeatVillain("Vilgax", 100); System.out.println("=====3====="); ben.defeatVillain("Yamcha", 10); System.out.println("=====4====="); ben.viewInfo(); System.out.println("=====5====="); ben.defeatVillain('8', "Vilgax", 100); System.out.println("=====6====="); ben.viewInfo(); System.out.println("=====7====="); Player kevin = new Player("Kevin"); System.out.println("=====8====="); kevin.viewInfo(); System.out.println("=====9====="); ben.defeatVillain(kevin); System.out.println("=====10====="); ben.viewInfo(); System.out.println("=====11====="); Player goku = new Player("Goku", 9000); System.out.println("=====12====="); ben.defeatVillain(goku); } }</pre>	<pre>Ben joined the game HP: 10 =====1===== Player Name: Ben Current HP: 10 =====2===== failed to defeat Vilgax =====3===== defeated Yamcha =====4===== Player Name: Ben Current HP: 20 Defeated: Yamcha, =====5===== HP with 8x boost: 160 defeated Vilgax =====6===== Player Name: Ben Current HP: 260 Defeated: Yamcha, Vilgax, =====7===== Kevin joined the game HP: 100 =====8===== Player Name: Kevin Current HP: 100 =====9===== defeated Kevin =====10===== Player Name: Ben Current HP: 360 Defeated: Yamcha, Vilgax, Kevin, =====11===== Goku joined the game HP: 9000 =====12===== failed to defeat Goku</pre>

Task 3

Design the **Student** and the **Connect** class so that the following output is produced.

Note:

- A student's email, password, and login status are null by default while creating an object of the Student class.
- The password and login status attributes of the Student class will be **private**
- Your code should satisfy the conditions mentioned in the output only.
- Connect class will have two instance variables: totalAdvisee and an array of Student type to store the student object. The array will be updated inside the advising() method only when the advising is successful.
- Connect can take at most 5 advisees.

Driver Code	Expected Output
<pre>public class ConnectTester { public static void main(String[] args) { Student rakib = new Student("Rakib", 12301455, "CSE"); Student roy = new Student("Roy", 12501345, "CS"); System.out.println("1*****"); Connect connectObj = new Connect(); System.out.println("2*****"); connectObj.login(rakib); System.out.println("3*****"); connectObj.advising(rakib); System.out.println("4*****"); rakib.email = "rakib@hotmail.com"; rakib.setPassword("1234"); System.out.println("5*****"); connectObj.login(rakib); System.out.println("6*****"); connectObj.advising(rakib); System.out.println("7*****"); connectObj.advising(rakib, "CSE110", "PHY111", "MAT110", "CSE260"); System.out.println("8*****"); connectObj.advising(rakib, "CSE110", "PHY111", "MAT110"); System.out.println("9*****"); connectObj.allAdviseeInfo(); System.out.println("10*****"); roy.email = "roy@hotmail.com"; roy.setPassword("abcd"); connectObj.login(roy); System.out.println("11*****"); connectObj.advising(roy, "CSE110", "ENG101", "PHY112"); System.out.println("12*****"); connectObj.allAdviseeInfo(); } }</pre>	<pre>Student object is created Student object is created 1***** Connect is ready to use! 2***** Email and password need to be set. 3***** Please login to advise courses! 4***** 5***** Login successful 6***** You haven't selected any courses. 7***** You need special approval to take more than 3 courses. 8***** Advising successful! 9***** Total Advisee: 1 Name: Rakib ID: 12301455 Department: CSE Advised Courses: CSE110 PHY111 MAT110 ===== 10***** Login successful 11***** Advising successful! 12***** Total Advisee: 2 Name: Rakib ID: 12301455 Department: CSE Advised Courses: CSE110 PHY111 MAT110 ===== Name: Roy ID: 12501345 Department: CS Advised Courses: CSE110 ENG101 PHY112 =====</pre>

Task 4

Spaceship: This class represents a spaceship. Each spaceship has a **name** and a **capacity** (the maximum weight it can carry).

Cargo: This class represents a piece of cargo. Each cargo item has a **name** and a **weight**. Both attributes should be **private** which means they cannot be accessed directly from outside of the class.

A **Spaceship** contains **Cargo**. That means each spaceship can carry multiple cargo items, but the total weight of the cargo cannot exceed the spaceship's capacity. Also, the maximum number of cargo items is 100. Your task is to design the **Spaceship** and **Cargo** class with necessary properties so that the given output is produced for the provided driver code.

Driver Code	Output
<pre>public class ShipTester { public static void main(String[] args) { Spaceship falcon = new Spaceship("Falcon", 50000); Spaceship apollo = new Spaceship("Apollo", 100000); Spaceship enterprise = new Spaceship("Enterprise",220000); System.out.println("1.====="); Cargo gold = new Cargo("Gold", 20000); Cargo platinum = new Cargo("Platinum", 25000); Cargo dilithium = new Cargo("Dilithium", 50000); Cargo trilithium = new Cargo("Trilithium", 70000); Cargo neutronium = new Cargo("Neutronium", 80000); System.out.println("2.====="); falcon.loadCargo(gold); falcon.loadCargo(platinum); falcon.displayDetails(); System.out.println("3.====="); apollo.loadCargo(gold); apollo.displayDetails(); System.out.println("4.====="); falcon.loadCargo(neutronium); System.out.println("5.====="); enterprise.loadCargo(dilithium); enterprise.loadCargo(trilithium); enterprise.loadCargo(neutronium); enterprise.displayDetails(); } }</pre>	<pre>1.===== 2.===== Spaceship Name: Falcon Capacity: 50000 Current Cargo Weight: 45000 Cargo:Gold Platinum 3.===== Spaceship Name: Apollo Capacity: 100000 Current Cargo Weight: 20000 Cargo:Gold 4.===== Warning: Unable to load Neutronium inside Falcon. Exceeds capacity by 75000. 5.===== Spaceship Name: Enterprise Capacity: 220000 Current Cargo Weight: 200000 Cargo:Dilithium Trilithium Neutronium</pre>

Task 5

1	public class TestX{
2	public int result = 17, i, n = 7, m = 5;
3	public int [] num = {6,8,11};
4	public void compute(int [] nums){
5	n = nums[0] + this.m;
6	this.result += nums[i];
7	TestX temp = new TestX();
8	boolean check = !(temp.modify(temp, num));
9	if (check) {
10	m = nums[1] + temp.result;
11	} else {
12	m = result - nums[3];
13	}
14	System.out.println(m + " " + n + " " + result);
15	}
16	public boolean modify(TestX obj, int[] arr){
17	arr[1]++;
18	i=i+1;
19	obj.m = this.result + arr[i];
20	obj.result = obj.m - obj.n + n;
21	System.out.println(obj.m + " " + obj.n + " " + result);
22	return arr[1] % 2 == 0;
23	}
24	public boolean modify(TestX obj){
25	num[i]++;
26	i=i+1;
27	obj.m = this.result + num[i];
28	obj.result = num[2] - obj.n + n;
29	System.out.println(obj.m + " " + obj.n + " " + result);
30	return num[1] % 2 != 0;
31	}
32	}

<pre> public class tester12{ public static void main(String[] args){ TestX tx = new TestX(); int[] data = {4, 6, 2, 3, 5}; tx.compute(data); tx.modify(tx); } } </pre>	Outputs		

Task 6

1	public class msgClass{
2	public int content;
3	}
4	class FinalT5A{
5	public int sum = 2, y = 1, x = 1;
6	public void methodA(){
7	int x = 6, y = 0;
8	msgClass myMsg = new msgClass();
9	myMsg.content = this.x;
10	x = x + myMsg.content;
11	this.y = this.y + methodB(myMsg, myMsg.content);
12	System.out.println(x + " " + this.y+ " " + sum);
13	y = this.y/2 + this.x;
14	x = y + sum/2;
15	sum = x + y + myMsg.content;
16	System.out.println(x + " " + y+ " " + sum);
17	}
18	public int methodB(msgClass mg2, int mg1){
19	int x = 0;
20	y = y + mg2.content;
21	mg2.content = y + mg1;
22	x = this.x + 3 + mg1;
23	sum = sum + x + y;
24	System.out.println(this.x + " " + this.y+ " " + sum);
25	mg2.content = sum - mg1 ;
26	return sum;
27	}
28	}

DRIVER CODE	OUTPUTS		
<pre> public class Tester5{ public static void main(String args []){ FinalT5A fT5A = new FinalT5A(); fT5A.methodA(); } } </pre>			

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Design the Company and Employee classes so that the Tester1 class produces the given outputs.

Restriction: Company class can't have more than 1 array and all attributes of Employee class must be Private.

Driver Code	Output
<pre>public class Tester1{ public static void main(String args[]){ Employee e1 = new Employee(); Employee e2 = new Employee("Alif", 34, "Fulltime"); System.out.println("1-----"); Company c1 = new Company(); c1.details(); System.out.println("2-----"); Employee e3 = new Employee("Akter", 36,"Part-time"); Employee e4 = new Employee("Ria", 38, "Fulltime"); System.out.println("3-----"); c1.addEmployee(e2); c1.addEmployee(e3); System.out.println("4-----"); c1.details(); System.out.println("5-----"); c1.addEmployee(e4); c1.addEmployee(e1); System.out.println("6-----"); c1.details(); System.out.println("7-----"); c1.removeEmployee(e4); System.out.println("6-----"); c1.details(); } }</pre>	<pre>A default employee has been created 1----- Company Name: ABC Company Total Employee: 0 Fulltime Employees: Part-Time Employees: 2----- 3----- Alif has joined the company Akter has joined the company 4----- Company Name: ABC Company Total Employee: 2 Fulltime Employees: Name: Alif, ID: 34 Part-Time Employees: Name: Akter, ID: 36 5----- Ria has joined the company No more vacancy 6----- Company Name: ABC Company Total Employee: 3 Fulltime Employees: Name: Alif, ID: 34 Name: Ria, ID: 38 Part-Time Employees: Name: Akter, ID: 36 7----- Ria has left the company 6----- Company Name: ABC Company Total Employee: 2 Fulltime Employees: Name: Alif, ID: 34 Part-Time Employees: Name: Akter, ID: 36</pre>

Task 2

Write “**Student**” class to show the following expected outputs

Note:

- ❖ A student can't take any course until the CGPA is set.
- ❖ A student cannot take more than 4 courses.
- ❖ A student with CGPA below 3 cannot take more than 3 courses.

Driver Code	Expected Output
<pre>public class StudentDriver { public static void main(String[] args){ Student student1 = new Student(12345678); System.out.println("1-----"); student1.addCourse("CSE110"); System.out.println("2-----"); student1.storeCG(2.5); student1.addCourse("CSE110"); student1.addCourse("ENG101"); student1.showAdvisee(); System.out.println("3-----"); student1.removeAllCourse(); student1.showAdvisee(); System.out.println("4-----"); student1.storeID(54652365); String[] courses = {"SOC101","CSE111","ENG102"}; student1.addCourse(courses); student1.showAdvisee(); System.out.println("5-----"); student1.addCourse("CSE230"); student1.showAdvisee(); System.out.println("6-----"); Student student2 = new Student(975738383,3.7); System.out.println("7-----"); String[] courses2 = {"CSE220","PHY112","MAT120","BUS101","CHN101"}; student2.addCourse(courses2); student2.showAdvisee(); } }</pre>	<pre>A student with ID 12345678 has been created. 1----- Failed to add CSE110 Set CG first 2----- Student ID: 12345678, CGPA: 2.5 Added courses are: CSE110 ENG101 3----- Student ID: 12345678, CGPA: 2.5 No courses added. 4----- Student ID: 54652365, CGPA: 2.5 Added courses are: SOC101 CSE111 ENG102 5----- Failed to add CSE230 CG is low. Can't add more than 3 courses. Student ID: 54652365, CGPA: 2.5 Added courses are: SOC101 CSE111 ENG102 6----- A student with ID 975738383 and cgpa 3.7 has been created. 7----- Failed to add CHN101 Maximum 4 courses allowed. Student ID: 975738383, CGPA: 3.7 Added courses are: CSE220 PHY112 MAT120 BUS101</pre>

Task 3

Design the **necessary** classes for the NetFlix system with the necessary properties to produce the given output for the provided driver code.

Driver code	Output
<pre>public class NetflixAccount{ public static void main(String[] args){ Movie m1 = new Movie("Avengers", "Sci-fi", "1:59:08"); Movie m2 = new Movie("Lord of the Rings", "Fantasy", "2:58:05"); Movie m3 = new Movie("Final Destination","Horror"); Movie m4 = new Movie("Fast and Furious","Action", "2:05:04"); System.out.println("====="); NetflixUser user1 = new NetflixUser("Martin Freeman", "Free"); user1.addToFavourites(m1); user1.showFavourites(); user1.upgradePlan(); user1.addToFavourites(m1); user1.showFavourites(); System.out.println("====="); NetflixUser user2 = new NetflixUser("Vin Diesel", "Premium"); user2.addToFavourites(m1); user2.addToFavourites(m2); user2.addToFavourites(m3); user2.addToFavourites(m4); user2.showFavourites(); System.out.println("====="); NetflixUser user3 = new NetflixUser("Chris Evans"); user3.addToFavourites(m1); user3.upgradePlan(); user3.addToFavourites(m2); user3.addToFavourites(m3); user1.upgradePlan(); } }</pre>	<pre>===== New user account created! Cannot add movies to favourites. Please upgrade to Netflix Premium. No available favourites. Please upgrade to Netflix Premium. Welcome to Netflix Premium! Movie added to favourites. Showing Martin Freeman's Favourites: Avengers, Sci-fi, Duration: 1:59:08 ===== New user account created! Movie added to favourites. Movie added to favourites. Movie added to favourites. Cannot add new movies. Favourites list is full. Showing Vin Diesel's Favourites: Avengers, Sci-fi, Duration: 1:59:08 Lord of the Rings, Fantasy, Duration: 2:58:05 Final Destination, Horror, Duration: Unknown ===== New user account created! Cannot add movies to favourites. Please upgrade to Netflix Premium. Welcome to Netflix Premium! Movie added to favourites. Movie added to favourites. You already have Netflix Premium!</pre>

Task 4

Please write the **Student** and **Department** class with the necessary properties so that the provided driver code generates the output given below. The id of the Student class will be **private**. For simplicity, assume that a department can add a maximum of 5 students.

Driver Code	Output
<pre>public class Tester1 { public static void main(String[] args) { Student s1 = new Student("Akib", 10, 3.29); Student s2 = new Student("Reza", 15, 3.45); Student s3 = new Student("Kabir", 20,4.0); System.out.println("1====="); Department cse = new Department("CSE"); cse.findStudent(-100); System.out.println("2====="); cse.addStudent(s1, s2, s3); System.out.println("3====="); cse.details(); System.out.println("4====="); cse.findStudent(15); System.out.println("5====="); Student s4 = new Student("Nakib", 15,3.22); cse.addStudent(s4); System.out.println("6====="); s4.setId(25); cse.addStudent(s4); System.out.println("7====="); cse.details(); System.out.println("8====="); Student s5 = new Student("Sakib", 30,2.29); cse.addStudent(s5); System.out.println("9====="); cse.details(); } }</pre>	<pre>1===== Student with this ID doesn't exist, Please give a valid ID 2===== Welcome to CSE department, Akib Welcome to CSE department, Reza Welcome to CSE department, Kabir 3===== Department Name: CSE Number of student:3 Details of the students: Student name: Akib, ID: 10, cgpa: 3.29 Student name: Reza, ID: 15, cgpa: 3.45 Student name: Kabir, ID: 20, cgpa: 4.0 4===== Student info: Student Name: Reza ID: 15 CGPA: 3.45 5===== Student with the same ID already exists, Please try with another ID 6===== Welcome to CSE department, Nakib 7===== Department Name: CSE Number of student:4 Details of the students: Student name: Akib, ID: 10, cgpa: 3.29 Student name: Reza, ID: 15, cgpa: 3.45 Student name: Kabir, ID: 20, cgpa: 4.0 Student name: Nakib, ID: 25, cgpa: 3.22 8===== Welcome to CSE department, Sakib 9===== Department Name: CSE Number of student:5 Details of the students: Student name: Akib, ID: 10, cgpa: 3.29 Student name: Reza, ID: 15, cgpa: 3.45 Student name: Kabir, ID: 20, cgpa: 4.0 Student name: Nakib, ID: 25, cgpa: 3.22 Student name: Sakib, ID: 30, cgpa: 2.29</pre>

Task 5

1	public class QuizA {
2	public int x, y;
3	public int sum = 1;
4	public QuizA(int x, int y) {
5	this.x = y;
6	this.y = x;
7	}
8	public void methodA() {
9	int x = 3;
10	y = this.y + x;
11	QuizA exam = new QuizA(5, 11);
12	exam.sum = x;
13	exam.y = this.y;
14	x = this.x + x + exam.sum;
15	this.y = this.sum + methodB(exam.sum, exam);
16	System.out.println(exam.x + " " + this.y + " " + sum);
17	sum = x % 2 + this.x;
18	y = x + y + exam.sum;
19	System.out.println(x + " " + y + " " + sum);
20	}
21	public int methodB(int x1, QuizA x2) {
22	int y = 0;
23	y = this.y + x2.sum;
24	x2.sum = x1 + x2.x;
25	sum = sum + x + y;
26	System.out.println(this.x + " " + this.y + " " + sum);
27	return x2.sum;
28	}
29	}

Driver Code	Output		
<pre> public class QuizTesterA{ public static void main(String []args){ QuizA q1 = new QuizA(3,4); q1.methodA(); } } </pre>			

Task 6

1	public class Trace{
2	public int x, y = 5, temp = -4, sum = 3;
3	public Trace(){
4	y = temp + 3 ;
5	sum = 3 + temp + 2;
6	temp -= 2;
7	}
8	public Trace(Trace t){
9	sum = t.sum++;
10	x = t.x + 2;
11	t.methodB(2,3);
12	}
13	public void methodA(int m, int n){
14	int x = 2;
15	y = y + m + (temp++);
16	x = x + 5 + n;
17	sum = sum + x + y;
18	System.out.println(x + " " + y+ " " + sum);
19	}
20	public void methodB(int m, int n){
21	int y = 0;
22	y = y + this.y;
23	x = this.y + 2 + temp;
24	methodA(x, y);
25	sum = x + y + sum;
26	System.out.println(x + " " + y+ " " + sum);
27	}
28	}

<pre> public class TraceTester { public static void main(String args []){ Trace t1 = new Trace(); Trace t2 = new Trace(t1); t1.methodA(1, 2); t2.methodB(3, 2); } } </pre>	Outputs		

Task 7

1	public class A {
2	public int x, y = 2, sum = 5;
3	public A(){
4	x = sum - y;
5	this.methodB(1);
6	}
7	public A(int x){
8	this.x = x;
9	}
10	public void methodB(int y) {
11	int temp = (y++) + (++this.y);
12	sum = y + temp - ++this.x;
13	System.out.println(y + " " + temp + " " + (sum++));
14	this.y = (++temp) + methodC(temp, y) + x;
15	}
16	public void methodA(int temp, int x) {
17	A a2 = new A(6);
18	a2.sum = this.methodC(a2);
19	System.out.println(this.x + " " + a2.sum + " " + y);
20	this.sum = a2.methodC(this);
21	System.out.println(a2.x + " " + this.sum + " " + a2.y);
22	}
23	public int methodC(A a) {
24	return this.x + a.y + a.sum;
25	}
26	public int methodC(int sum, int y) {
27	y = (this.x++) + sum + 3;
28	return y;
29	}
30	}

<pre> public class TesterA{ public static void main(String [] args) { A a1 = new A(); a1.methodA(4, 3); } } </pre>	Outputs		

Lab Assignment 05



Inspiring Excellence

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	Encapsulation, Static variable and Static Method
Number of Tasks:	13 (Classwork: 06, Homework: 07)

[Submit all the Coding Tasks (Homework: Task 1 to 5) in the Google Form shared on buX before the next lab. Submit the Tracing Tasks (Homework: Task 6 to 7) handwritten to your Lab Instructors at the beginning of the lab]

[You are not allowed to change the driver codes of any of the tasks]

CLASSWORK

Task 1

Design the **Account** class such that it produces the following output.

- Each Account will have a name, address and a balance variable.
- The balance must be kept private.
- Use Encapsulation to access and modify balance.

After finishing the class design to get the following output, try to directly print the balance variable. Can you print the private balance variable in the tester code?

Tester Code	Output
<pre>public class AccountTester { public static void main(String[] args) { System.out.print("All accounts belong to "); System.out.println(Account.bankName); System.out.println("Accounts created: " + Account.count); System.out.print("Total Money stored "); System.out.println(Account.totalBalance + " taka"); System.out.println("1-----"); Account a1 = new Account("Abir", "Dhanmondi"); System.out.println("2-----"); a1.printInfo(); System.out.println("3-----"); Account a2 = new Account("Mira", "Mirpur"); a2.printInfo(); System.out.println("4-----"); a1.setBalance(1000); System.out.println("5-----"); a1.printInfo(); a2.setBalance(2000); System.out.println("6-----"); System.out.println(a2.getBalance()); System.out.println("7-----"); a2.setBalance(500); System.out.println("8-----"); a2.printInfo(); System.out.println("9-----"); System.out.print("All accounts belong to "); System.out.println(Account.bankName); System.out.println("Accounts created: " + Account.count); System.out.print("Total Money stored "); System.out.println(Account.totalBalance + " taka"); } }</pre>	<pre>All accounts belong to Badda Bank Accounts created: 0 Total Money stored 0.0 taka 1----- 2----- Name: Abir, address: Dhanmondi Balance: 0.0 Taka 3----- Name: Mira, address: Mirpur Balance: 0.0 Taka 4----- 5----- Name: Abir, address: Dhanmondi Balance: 1000.0 Taka 6----- 2000.0 7----- 8----- Name: Mira, address: Mirpur Balance: 2500.0 Taka 9----- All accounts belong to Badda Bank Accounts created: 2 Total Money stored 3500.0 taka</pre>

Task 2

Design the **Student** class such that it produces the following output.

Note: A Student needs at least 50 marks to pass.

Tester Code	Output
<pre>public class MarksTester { public static void main(String[] args) { System.out.println("Total Students: " + Student.total_students); System.out.println("Average Marks: " + Student.averageMarks()); System.out.println("-----1-----"); Student mike = new Student("Mike"); mike.setCodingMarks(35); mike.setTracingMarks(10); mike.individualDetail(); System.out.println("-----2-----"); System.out.println("Total Students: " + Student.total_students); System.out.println("Average Marks: " + Student.averageMarks()); System.out.println("-----3-----"); Student eleven = new Student("Eleven", 70, 20); eleven.individualDetail(); System.out.println("-----4-----"); Student will = new Student("Will"); will.setCodingMarks(51); will.individualDetail(); System.out.println("-----5-----"); System.out.println("Total Students: " + Student.total_students); System.out.println("Average Marks: " + Student.averageMarks()); } }</pre>	<pre>Total Students: 0 Average Marks: 0.0 -----1----- Name: Mike ID: 1 Coding Marks: 35 Tracing Marks: 10 Mike has failed with 45 marks -----2----- Total Students: 1 Average Marks: 45.0 -----3----- Name: Eleven ID: 2 Coding Marks: 70 Tracing Marks: 20 Eleven has passed with 90 marks -----4----- Name: Will ID: 3 Coding Marks: 51 Tracing Marks: 0 Will has passed with 51 marks -----5----- Total Students: 3 Average Marks: 62.0</pre>

Task 3

From the given Tester Code write the **Trainer** class that uses the **Pokemon** class to produce the given output.

- YOU CANNOT change the given Pokemon class.
- Each Trainer can catch at max 7 pokemons.

Tester Code	Output
<pre>public class PokemonTrainerTester { public static void main(String[] args) { Pokemon pikachu = new Pokemon("Pikachu", 65.0); Pokemon caterpie = new Pokemon("Caterpie", 45.0); Pokemon squirtle = new Pokemon("Squirtle", 70.0); Pokemon eevee = new Pokemon("Eevee", 60.0); System.out.println("1-----"); Trainer ash = new Trainer("Ash"); System.out.println("2-----"); ash.catchPokemon(pikachu); System.out.println("3-----"); ash.catchPokemon(caterpie); ash.catchPokemon(squirtle); System.out.println("4-----"); ash.viewPokeDex(); System.out.println("5-----"); Trainer gary = new Trainer("Gary"); System.out.println("6-----"); gary.catchPokemon(squirtle); gary.catchPokemon(eevee); System.out.println("7-----"); gary.viewPokeDex(); System.out.println("8-----"); Trainer.battle(ash, gary); } }</pre>	<pre>1----- Trainer ID: 1, Name: Ash - created 2----- Ash caught: Pikachu 3----- Ash caught: Caterpie Ash caught: Squirtle 4----- Ash's Trainer code: T1 Pikachu: 65.0 points Caterpie: 45.0 points Squirtle: 70.0 points Average of HP: 60.0 5----- Trainer ID: 2, Name: Gary - created 6----- Gary caught: Squirtle Gary caught: Eevee 7----- Gary's Trainer code: T2 Squirtle: 70.0 points Eevee: 60.0 points Average of HP: 65.0 8----- Battle Info Ash's HP average: 60.0 Gary's HP average: 65.0 Gary wins</pre>
Given Class	
<pre>public class Pokemon { private String name; private double hp; public Pokemon(String n, double hp) { this.name = n; this.hp = hp; } public String getName() { return name; } public double getHP() { return hp; } }</pre>	

Task 4

Suppose you have opened a new library, from where your friends can borrow books. Initially you have bought 3 books (Pather Panchali, Durgesh Nandini & Anandmath) each of 3 copies only. Design the **Borrower** class in such a way that the following code provides the expected output.

- You are given the arrays **book_count** and **book_name** to keep track of the number of books available. For simplicity, assume that there will be no other books in the library.
- You must reuse the **remainingBooks()** method when needed.

Given Code	Expected Output
<pre>public class BorrowerTester{ public static void main(String args[]){ Borrower.bookStatus(); System.out.println("*****1*****"); Borrower b1 = new Borrower("Nabila"); b1.borrowBook("Pather Panchali"); b1.borrowBook("Anandmath"); b1.borrowerDetails(); System.out.println("*****2*****"); Borrower b2 = new Borrower("Sadia"); b2.borrowBook("Anandmath"); b2.borrowBook("Durgesh Nandini"); b2.borrowBook("Pather Panchali"); b2.borrowerDetails(); System.out.println("*****3*****"); System.out.println(Borrower.remainingBooks("Anandmath")+ copies of Anandmath is remaining."); System.out.println("*****4*****"); Borrower b3 = new Borrower("Anika"); b3.borrowBook("Anandmath"); Borrower.bookStatus(); System.out.println("*****5*****"); Borrower b4 = new Borrower("Oishi"); b4.borrowBook("Anandmath"); b4.borrowBook("Durgesh Nandini"); b4.borrowerDetails(); } } public class Borrower{ public static int book_count[] = {3, 3, 3}; public static String book_name[] = {"Pather Panchali", "Durgesh Nandini", "Anandmath"}; // Your Code here }</pre>	<pre>Available Books: Pather Panchali: 3 Durgesh Nandini: 3 Anandmath: 3 *****1***** Name: Nabila Books Borrowed: Pather Panchali Anandmath *****2***** Name: Sadia Books Borrowed: Anandmath Durgesh Nandini Pather Panchali *****3***** 1 copies of Anandmath is remaining. *****4***** Available Books: Pather Panchali: 1 Durgesh Nandini: 2 Anandmath: 0 *****5***** This book is not available. Name: Oishi Books Borrowed: Durgesh Nandini</pre>

Task 5

1	public class B{
2	public static int x;
3	public int y = 4;
4	public int temp = -5;
5	public int sum = 2;
6	public B(){
7	y = temp + 3 ;
8	sum = 3 + temp + 3;
9	temp-=2;
10	}
11	public B(B b){
12	sum = b.sum;
13	x = b.x;
14	b.methodB(1,3);
15	}
16	public void methodA(int m, int n){
17	int x = 2;
18	y = y + m + (temp++);
19	x = x + 7 + n;
20	sum = sum + x + y;
21	System.out.println(x + " " + y+ " " + sum);
22	}
23	public void methodB(int m, int n){
24	int y = 0;
25	y = y + this.y;
26	x = this.y + 3 + temp;
27	methodA(x, y);
28	sum = x + y + sum;
29	System.out.println(x + " " + y+ " " + sum);
30	}
31	}

Consider the following driver code and find the output.

B b1 = new B();	x	y	sum
B b2 = new B(b1);			
b1.methodA(3, 2);			
b2.methodB(1, 2);			

Task 6

1	class Trace6CW {
2	public static int temp = 10;
3	public int sum = 0;
4	public static int[] y = {2, 5};
5	public Trace6CW() {
6	temp -= 2;
7	sum = temp + y[0];
8	y[1] = sum - temp;
9	}
10	public void methodA(int m, int n) {
11	int temp = 0;
12	temp = Trace6CW.temp + m;
13	this.sum = this.sum + temp + Trace6CW.y[1];
14	Trace6CW.y[0] = this.sum - n;
15	System.out.println(this.sum + " " + temp + " " + Trace6CW.y[0]);
16	}
17	public static void methodB(Trace6CW s1, int m) {
18	int sum = 5;
19	y[0] = s1.sum + sum;
20	s1.sum = temp + y[1] + m;
21	System.out.println(s1.sum + " " + y[0] + " " + temp);
22	}
23	}

<pre> class Test5CW { public static void main(String[] args) { Trace6CW s1 = new Trace6CW(); Trace6CW s2 = new Trace6CW(); s1.methodA(3, 2); Trace6CW.methodB(s2, 4); s2.methodA(1, 1); } } </pre>	OUTPUT		

HOMEWORK

Task 1

Design the **Product** class such that it produces the following output.

Tester Code	Output
<pre>public class ProductTester{ public static void main(String[] args) { Product p1 = new Product("Table", 10); Product p2 = new Product("Chair", 15); Product p3 = new Product("Sofa", 20); Product p4 = new Product("Divan", 8); System.out.println("-----1-----"); Product.displayProducts(); System.out.println("-----2-----"); Product.buy("Chair", 5); System.out.println("-----3-----"); Product.displayProducts(); System.out.println("-----4-----"); Product.buy("Sofa", 25); System.out.println("-----5-----"); Product.displayProducts(); System.out.println("-----6-----"); Product.buy("Bed", 10); } }</pre>	<pre>Stored: Table Stored: Chair Stored: Sofa Storage is full! Cannot add Divan -----1----- === Stored Products === Table - Qty: 10 Chair - Qty: 15 Sofa - Qty: 20 -----2----- Product Sold -----3----- === Stored Products === Table - Qty: 10 Chair - Qty: 10 Sofa - Qty: 20 -----4----- Quantity low -----5----- === Stored Products === Table - Qty: 10 Chair - Qty: 10 Sofa - Qty: 20 -----6----- Product not found</pre>

Task 2

Design the **Character** class such that it produces the following output.

Tester Code	Output
<pre>public class WeirderStuffTester { public static void main(String[] args) { Character.printStats(); System.out.println("-----1-----"); Character twelve = new Character("Twelve", "Kid", 100); twelve.printDetails(); System.out.println("-----2-----"); Character spike = new Character("Spike", "Kid", 50); spike.printDetails(); System.out.println("-----3-----"); Character.printStats(); System.out.println("-----4-----"); Character reeve = new Character("Reeve", 70); reeve.printDetails(); System.out.println("-----5-----"); Character chopper = new Character("Chopper", "Adult", 120); chopper.printDetails(); System.out.println("-----6-----"); Character.printStats(); } }</pre>	<pre>Total Characters: 0 Kids: 0 Teens: 0 Adults: 0 Average Health: 0 Strongest Character: None -----1----- ID: 1, Name: Twelve Group: Kid Health: 100 -----2----- ID: 2, Name: Spike Group: Kid Health: 50 -----3----- Total Characters: 2 Kids: 2 Teens: 0 Adults: 0 Average Health: 75.0 Strongest Character: Twelve (Health 100) -----4----- ID: 3, Name: Reeve Group: Teen Health: 70 -----5----- ID: 4, Name: Chopper Group: Adult Health: 120 -----6----- Total Characters: 4 Kids: 2 Teens: 1 Adults: 1 Average Health: 85.0 Strongest Character: Chopper (Health 120)</pre>

Task 3

Design the **Artifact** class where all the attributes kept inside the class are classified i.e. private.

The “Vault” can only store a maximum of 4 artifacts. The power of the artifacts are calculated as:

For artifact with,

- Even length name => Summation of the ascii values of all characters in even index of name.
- Odd length name => Summation of the ascii values of all characters in odd index of name.

Tester Code	Output
<pre>public class TesterArtifact{ public static void main(String[] args) { Artifact a = new Artifact("Phone Microwave", "Kurusu"); System.out.println("-----1-----"); Artifact.AddtoVault(a); Artifact.AddtoVault(new Artifact("D-Mail Capsule", "Mayuri")); System.out.println("-----2-----"); Artifact c = new Artifact("C204 Chip"); Artifact d = new Artifact("Divergence Meter"); Artifact e = new Artifact("M4A2 Robot", "Okabe"); Artifact.AddtoVault(c); Artifact.AddtoVault(d); Artifact.AddtoVault(e); System.out.println("-----3-----"); Artifact.labReport(); System.out.println("-----4-----"); System.out.println("Power of "+c.GetName()+" is "+c.CalcPower()); System.out.println("-----5-----"); System.out.println("Strongest Artifact: "+Artifact.strongest()); System.out.println("-----6-----"); a.revealArtifact(); System.out.println("-----7-----"); a.changeName("Banana Microwave"); System.out.println("-----8-----"); Artifact.labReport(); System.out.println("-----9-----"); System.out.println("Strongest Artifact: "+Artifact.strongest()); } }</pre>	<pre>-----1----- Kurusu added Phone Microwave successfully to the vault. Mayuri added D-Mail Capsule successfully to the vault. -----2----- Okabe added C204 Chip successfully to the vault. Okabe added Divergence Meter successfully to the vault. !!Okabe unsuccessful in adding artifact to the vault!! -----3----- === Future Gadget Lab === Phone Microwave added by Kurisu has power of 702. D-Mail Capsule added by Mayuri has power of 602. C204 Chip added by Okabe has power of 274. Divergence Meter added by Okabe has power of 734. -----4----- Power of C204 Chip is 274 -----5----- Strongest Artifact: Divergence Meter -----6----- Phone Microwave added by Kurisu has power of 702. -----7----- Name changed and power recalculated. -----8----- === Future Gadget Lab === Banana Microwave added by Kurisu has power of 774. D-Mail Capsule added by Mayuri has power of 602. C204 Chip added by Okabe has power of 274. Divergence Meter added by Okabe has power of 734. -----9----- Strongest Artifact: Banana Microwave</pre>

Task 4

Design the **AnimalKeepers** class with the following requirements:

- The Animal Keepers have private IDs starting from 101.
- The Safari has an array named, Animals = {"Lion", "Tiger", "Seal", "Gorilla", "Deer"}
- Only one task is assigned per animal and so the tasks are overridden when reassigned for the same animal.

[Hint: You can call the static method printTasks() from inside details()]

Tester Code	Output
<pre>import java.util.Arrays; public class KeeperTester { public static void main(String[] args) { Animalkeepers.details(); System.out.println("-----1-----"); System.out.println(Arrays.toString(Animalkeepers.Animals)); System.out.println("-----2-----"); Animalkeepers leo = new Animalkeepers("Leo"); Animalkeepers theo = new Animalkeepers("Theo"); Animalkeepers mochi = new Animalkeepers("Mochi"); System.out.println("-----3-----"); Animalkeepers.printTasks(); System.out.println("-----4-----"); leo.doTask("Lion", "Feed"); System.out.println("-----5-----"); leo.doTask("Monkey", "Feed"); System.out.println("-----6-----"); Animalkeepers.details(); System.out.println("-----7-----"); theo.doTask("Tiger", "Bathe"); mochi.doTask("Seal", "Clean Pen"); mochi.doTask("Deer", "Add Food"); System.out.println("-----8-----"); Animalkeepers.printTasks(); System.out.println("-----9-----"); leo.doTask("Deer", "Play"); System.out.println("-----10-----"); Animalkeepers.details(); } }</pre>	<pre>No Animal Keepers working yet. -----1----- [Lion, Tiger, Seal, Gorilla, Deer] -----2----- Leo with ID 101 got the job! Theo with ID 102 got the job! Mochi with ID 103 got the job! -----3----- No tasks assigned. -----4----- Task assigned to Leo -----5----- Animal not in the Safari -----6----- Total Animal Keeper: 3 Total Task assigned: 1 Feed (Keeper - Leo) === Lion -----7----- Task assigned to Theo Task assigned to Mochi Task assigned to Mochi -----8----- Feed (Keeper - Leo) === Lion Bathe (Keeper - Theo) === Tiger Clean Pen (Keeper - Mochi) === Seal Add Food (Keeper - Mochi) === Deer -----9----- Task assigned to Leo -----10----- Total Animal Keeper: 3 Total Task assigned: 4 Feed (Keeper - Leo) === Lion Bathe (Keeper - Theo) === Tiger Clean Pen (Keeper - Mochi) === Seal Play (Keeper - Leo) === Deer</pre>

Task 5

Design the **Event** and **Organizer** classes in such a way that the following code provides the expected output. Hint:

- Make the name instance variable of the Event class **private**
- For simplicity assume that the Event class can create a maximum of 5 event objects and an Organizer can organize a maximum of 4 events.

Driver Code	Output
<pre>public class EventTester{ public static void main(String args []){ Event.allEventInfo(); System.out.println("1-----"); Event ev1 = new Event("HP Day", "7/12/24"); Event ev2 = new Event("TechConnect", "10/12/24"); System.out.println(ev1.details()); System.out.println("2-----"); Organizer uni = new Organizer(); Organizer bracu = new Organizer("BRACU"); Organizer buet = new Organizer("BUET"); System.out.println("3-----"); Event.allEventInfo(); System.out.println("4-----"); bracu.organizeEvent(ev1); bracu.organizeEvent(ev2); System.out.println("5-----"); Event ev3 = new Event("From Earth to Orbit", "15/12/24"); Event ev4 = new Event("NSysS 2024","21/12/24"); System.out.println("6-----"); buet.organizeEvent(ev4); bracu.organizeEvent(ev3); System.out.println("7-----"); bracu.searchEventByDate("21/12/24"); System.out.println("8-----"); bracu.searchEventByDate("15/12/24"); System.out.println("9-----"); Event.allEventInfo(); } }</pre>	<pre>Total Events: 0 Event Details: 1----- Name: HP Day Date: 7/12/24 2----- Please provide the organizer's name 3----- Total Events: 2 Event Details: Event 1: Name: HP Day Date: 7/12/24 Event 2: Name: TechConnect Date: 10/12/24 4----- BRACU successfully organized HP Day BRACU successfully organized TechConnect 5----- 6----- BUET successfully organized NSysS 2024 BRACU successfully organized From Earth to Orbit 7----- No event is scheduled for 21/12/24 8----- From Earth to Orbit 9----- Total Events: 4 Event Details: Event 1: Name: HP Day Date: 7/12/24 Event 2: Name: TechConnect Date: 10/12/24 Event 3: Name: From Earth to Orbit Date: 15/12/24 Event 4: Name: NSysS 2024 Date: 21/12/24</pre>

Task 6

1	class Trace {
2	public static int[] x = {3, -4};
3	public int y = 4;
4	public static int temp = -5;
5	private int sum = 2;
6	public Trace(){
7	y = temp + 3 ;
8	sum = 3 + temp + x[1];
9	temp-=2;
10	x[0] = ++x[1] - 2;
11	}
12	public Trace(Trace trace){
13	sum = trace.sum;
14	x = trace.x;
15	trace.methodB(1,3);
16	}
17	public void methodA(int m, int n){
18	int x = 2 - this.x[0] - Trace.x[1];
19	y = y + m + (temp++);
20	x = x + 7 + n;
21	sum = sum + x + y;
22	System.out.println(x + " " + y+ " " + sum);
23	}
24	public void methodB(int m, int n){
25	int y = 0;
26	y = y + this.y;
27	Trace.x[0] = this.y + 3 + temp;
28	methodA(x[1], y);
29	sum = Trace.x[1] + y + sum;
30	System.out.println(this.x[0] + " " + y+ " " + sum);
31	}
32	public static void methodC(Trace trace1, Trace trace2){
33	temp = x[0] - Trace.x[1];
34	x = new int[]{trace1.y, trace2.y};
35	}
36	}

Consider the following driver code and find the output.

Trace trace1 = new Trace();	Output 1	Output 2	Output 3
Trace trace2 = new Trace(trace1);			
trace1.methodA(3, 2);			
Trace.methodC(trace1, trace2);			
trace2.methodB(1, 2);			

Task 7

1	class Tracing {
2	public static int x = 0, y = 0;
3	public int a;
4	private int b = 3;
5	public Tracing(int a, int b) {
6	this.a = a;
7	this.b = b - this.b;
8	x += 1;
9	y += Tracing.y - 2;
10	}
11	public void set_b(int b) {
12	this.b = b;
13	}
14	public int get_b() {
15	return this.b;
16	}
17	public void methodA(int x) {
18	this.a = x + this.x - Tracing.x;
19	this.b = this.a + this.methodB() - this.b;
20	System.out.println(this.a + " " + this.b + " " + x);
21	}
22	public int methodB() {
23	int y = -3;
24	this.b = y - this.y + this.a;
25	System.out.println(this.a + " " + this.b + " " + x);
26	this.y -= y;
27	x += this.b + this.y;
28	return this.b;
29	}
30	public void methodB(Tracing t1) {
31	int t = this.y - t1.get_b() + this.b;
32	t1.set_b(t);
33	t1.a = this.x - t1.a + this.a;
34	System.out.println(t1.a + " " + t1.get_b() + " " + x);
35	}
36	}

Consider the following driver code and find the output.

Tracing t1 = new Tracing(2, 3);	Output 1	Output 2	Output 3
t1.methodA(1);			
Tracing t2 = new Tracing(3, 4);			
t2.methodA(2);			
t1.methodB(t2);			
t2.methodB(t2);			

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Design the **SultansDine** class with the necessary property to produce the output from the given driver code.

Subtasks:

1. Create SultansDine class
2. Create 2 static variable and 1 static array
3. Create 1 static method
4. Calculation of branch sell is given below
 - a. If sellQuantity < 10:
 - i. Branch_sell = quantity * 300
 - b. Else if sellQuantity < 20:
 - i. Branch_sell = quantity * 350
 - c. Else
 - i. Branch_sell = quantity * 400
5. Calculation of branch's sell percentage = (branch's sell / total sell) * 100

Driver Code	Output
<pre>public class SultansDineTester { public static void main(String[] args) { SultansDine.details(); System.out.println("1====="); SultansDine dhanmondi = new SultansDine("Dhanmondi"); dhanmondi.sellQuantity(25); dhanmondi.branchInformation(); System.out.println("2====="); SultansDine.details(); System.out.println("3====="); SultansDine baily_road = new SultansDine("Baily Road"); baily_road.sellQuantity(15); baily_road.branchInformation(); System.out.println("4====="); SultansDine.details(); System.out.println("5====="); SultansDine gulshan = new SultansDine("Gulshan"); gulshan.sellQuantity(9); gulshan.branchInformation(); System.out.println("6====="); SultansDine.details(); } }</pre>	<pre>Total Number of branch(s): 0 Total Sell: 0 Taka 1===== Branch Name: Dhanmondi Branch Sell: 10000 Taka 2===== Total Number of branch(s): 1 Total Sell: 10000 Taka Branch Name: Dhanmondi, Branch Sell: 10000 Taka Branch consists of total sell's 100.00 3===== Branch Name: Baily Road Branch Sell: 5250 Taka 4===== Total Number of branch(s): 2 Total Sell: 15250 Taka Branch Name: Dhanmondi, Branch Sell: 10000 Taka Branch consists of total sell's 65.57 Branch Name: Baily Road, Branch Sell: 5250 Taka Branch consists of total sell's 34.43 5===== Branch Name: Gulshan Branch Sell: 2700 Taka 6===== Total Number of branch(s): 3 Total Sell: 17950 Taka Branch Name: Dhanmondi, Branch Sell: 10000 Taka Branch consists of total sell's 55.71 Branch Name: Baily Road, Branch Sell: 5250 Taka Branch consists of total sell's 29.25 Branch Name: Gulshan, Branch Sell: 2700 Taka Branch consists of total sell's 15.04</pre>

Task 2

Design a **Student** class in such a way that the following code provides the expected output.

Driver Code	Output
<pre>public class StudentTester { public static void main(String[] args) { Student.printDetails(); System.out.println("-----"); Student mikasa = new Student("Mikasa", 3.75); mikasa.individualDetail(); System.out.println("-----"); Student.printDetails(); System.out.println("-----"); Student harry = new Student("Harry", 2.5, "Charms"); harry.individualDetail(); System.out.println("-----"); Student.printDetails(); System.out.println("-----"); Student levi = Student.createStudent("Levi", 3.33); levi.individualDetail(); System.out.println("-----"); Student.printDetails(); } }</pre>	<pre>Total Student(s): 0 CSE Student(s): 0 Other Department Student(s): 0 ----- ID: 1 Name: Mikasa CGPA: 3.75 Department: CSE ----- Total Student(s): 1 CSE Student(s): 1 Other Department Student(s): 0 ----- ID: 2 Name: Harry CGPA: 2.5 Department: Charms ----- Total Student(s): 2 CSE Student(s): 1 Other Department Student(s): 1 ----- ID: 3 Name: Levi CGPA: 3.33 Department: CSE ----- Total Student(s): 3 CSE Student(s): 2 Other Department Student(s): 1</pre>

Task 3

For this task, you need to design the **Cargo** class with appropriate static and non-static variables and methods to produce this given output for the given tester code.

Note: .load() method marks an object as selected for transport, and .unload() method unmarked it. At a time, the transport capacity is 10.0 tonnes. Each Cargo object is initialized with 2 attributes from the constructor - the contents and the weight. Carefully observe the outputs to identify the other attributes and design the class.

Given Code	Expected Output
<pre>public class CargoTester { public static void main(String[] args) { System.out.println("Cargo Capacity: "+ Cargo.capacity()); System.out.println("1====="); Cargo a = new Cargo("Industrial Machinery", 4.5); a.details(); System.out.println("2====="); a.load(); System.out.println("3====="); Cargo b = new Cargo("Steel Ingot", 2.7); b.details(); System.out.println("4====="); System.out.println("Cargo Capacity: "+ Cargo.capacity()); System.out.println("5====="); b.load(); System.out.println("Cargo Capacity: "+ Cargo.capacity()); System.out.println("6====="); Cargo c = new Cargo("Tree Trunks", 3.6); c.load(); System.out.println("7====="); c.details(); b.details(); System.out.println("8====="); Cargo d = new Cargo("Processed Goods", 1.8); d.load(); System.out.println("Cargo Capacity: "+ Cargo.capacity()); System.out.println("9====="); b.unload(); System.out.println("Cargo Capacity: "+ Cargo.capacity()); System.out.println("10====="); c.load(); System.out.println("11====="); b.details(); System.out.println("Cargo Capacity: "+ Cargo.capacity()); } }</pre>	<pre>Cargo Capacity: 10.0 1===== Cargo ID: 1, Contents: Industrial Machinery, Weight: 4.5, Loaded: false 2===== Cargo 1 loaded for transport. 3===== Cargo ID: 2, Contents: Steel Ingot, Weight: 2.7, Loaded: false 4===== Cargo Capacity: 5.5 5===== Cargo 2 loaded for transport. Cargo Capacity: 2.8 6===== Cannot load cargo, exceeds weight capacity. 7===== Cargo ID: 3, Contents: Tree Trunks, Weight: 3.6, Loaded: false Cargo ID: 2, Contents: Steel Ingot, Weight: 2.7, Loaded: true 8===== Cargo 4 loaded for transport. Cargo Capacity: 1.0 9===== Cargo 2 unloaded. Cargo Capacity: 3.7 10===== Cargo 3 loaded for transport. 11===== Cargo ID: 2, Contents: Steel Ingot, Weight: 2.7, Loaded: false Cargo Capacity: 0.09999999999999964</pre>

Task 4

1.	public class Maze{	Output	
2.	public static int x;		
3.	public void methodA(){		
4.	int m = 5;		
5.	x=11;		
6.	System.out.println(x+" "+m);		
7.	m=methodB(m-3)+x;		
8.	System.out.println(x+" "+(m));		
9.	methodB(x,m);		
10.	System.out.println(x+" "+m+x);		
11.	}		
12.	public int methodB(int y){		
13.	x=y*y;		
14.	System.out.println(x+" "+y);		
15.	return x+3;		
16.	}		
17.	public void methodB(int z, int x){		
18.	z=z-2;		
19.	x=x*1%z;		
20.	System.out.println(z+" "+x);		
21.	}		
22.	}		
23.	public class TestU3{		
24.	public static void main(String [] args){		
25.	Maze c = new Maze();		
26.	c.methodA();		
27.	c.methodB(-11, 45);		
28.	}		
29.	}		

Task 5

1	class TraceTask {
2	public static int x = 3;
3	public int y = 5;
4	public int z = 2;
5	public TraceTask(int z) {
6	this.z = z;
7	x += 2;
8	this.y = x - this.z;
9	}
10	public void methodA(int x) {
11	this.y = x + this.y + TraceTask.x;
12	x = this.z + 4;
13	TraceTask.x = this.y - x;
14	System.out.println(x + " " + this.y + " " + TraceTask.x);
15	}
16	public int methodB(TraceTask t, int z) {
17	int y = 2;
18	t.z = this.z + y;
19	t.methodA(y);
20	this.x = t.y + this.z;
21	System.out.println(this.z + " " + t.z + " " + y);
22	return this.x;
23	}
24	}

<pre> class TraceTaskTester{ public static void main(String[] args) { TraceTask t1 = new TraceTask(4); TraceTask t2 = new TraceTask(2); t1.methodA(3); int res = t2.methodB(t1, 5); System.out.println(res + " " + TraceTask.x + " " + t1.z); } } </pre>	OUTPUT		

Lab Assignment 06



Inspiring Excellence

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	Inheritance
Number of Tasks:	12 (Classwork: 06, Homework: 06)

[Submit all the Coding Tasks (Homework: Task 1 to 5) in the Google Form shared on buX before the next lab.]

[You are not allowed to change the driver codes of any of the tasks]

CLASSWORK

Task 1

Complete the class **Circle** so that the desired outputs are generated properly.

Given Code	Expected Output
<pre>public class shapeTester { public static void main(String[] args) { Shape s = new Shape(); s.name = "Mobius Strip"; s.color = "Blue"; s.displayInfo(); System.out.println("1====="); Circle c = new Circle(); System.out.println("2====="); c.name = "Circle"; c.color = "Red"; c.radius = 5; c.displayInfo(); System.out.println("3====="); c.area(); } } public class Shape { public String name; public String color; public void displayInfo() { System.out.printf("Name: %s\nColor: %s\n", name, color); } } public class Circle extends Shape { //Your Code Here }</pre>	<pre>Name: Mobius Strip Color: Blue 1===== 2===== Name: Circle Color: Red 3===== Area of Red Circle: 78.54</pre>

Task 2

Given the following classes, write the code for the **BBAStudent** class so that the following output is printed when we run the TestStudent class.

Driver Code and Parent Class	Output
<pre>public class TestStudent{ public static void main(String [] args){ BBAStudent b1 = new BBAStudent(); BBAStudent b2 = new BBAStudent("Humty Dumty"); BBAStudent b3 = new BBAStudent("Little Bo Peep"); b1.details(); System.out.println("1-----"); b2.details(); System.out.println("2-----"); b3.details(); } } public class Student{ private String name = "Just a Student"; private String department = "nothing"; public void setDepartment(String dpt){ this.department = dpt; } public void setName(String name){ this.name = name; } public void details(){ System.out.println("Name : " + name + " Department: " + department); } }</pre>	<pre>Name: Default Department: BBA 1----- Name: Humty Dumty Department: BBA 2----- Name: Little Bo Peep Department: BBA</pre>

Task 3

Given the following classes, write the code for the **Vehicle2010** class to print the following output when we run the Vehicle2010User class.

Driver Code and Parent Class	Output
<pre>public class Vehicle2010User{ public static void main(String[] args){ Vehicle2010 car1 = new Vehicle2010(); System.out.println(car1.getPosition()); car1.moveLowerLeft(); System.out.println(car1.getPosition()); Vehicle2010 car2 = new Vehicle2010(); System.out.println(car2.getPosition()); car2.moveUpperRight(); System.out.println(car2.getPosition()); car2.moveLowerRight(); System.out.println(car2.getPosition()); } } public class Vehicle{ public int x; public int y; public void moveUp(){ y = y+1; } public void moveDown(){ y = y-1; } public void moveLeft(){ x = x-1; } public void moveRight(){ x = x+1; } public String getPosition(){ return "("+ x + ","+ y + ")"; } }</pre>	<pre>(0,0) (-1,-1) (0,0) (1,1) (2,0)</pre>

Task 4

Given the following classes, write the code for the **Cricket_Tournament** and the **Tennis_Tournament** classes derived from **Tournament** so that the following output is generated.

Given Code	Expected Output
<pre>public class TournamentTester { public static void main(String[] args) { Cricket_Tournament ct1 = new Cricket_Tournament(); System.out.println(ct1.info()); System.out.println("-----"); Cricket_Tournament ct2 = new Cricket_Tournament("IPL", 10, "t20"); System.out.println(ct2.info()); System.out.println("-----"); Tennis_Tournament tt = new Tennis_Tournament("Roland Garros", 128); System.out.println(tt.info()); } } public class Tournament { private String name; public Tournament() { this.name = "Default"; } public Tournament(String name) { this.name = name; } public String getName(){ return "Tournament Name: "+name; } }</pre>	<pre>Cricket Tournament Name: Default Number of Teams: 0 Type: No type ----- Cricket Tournament Name: IPL Number of Teams: 10 Type: t20 ----- Tennis Tournament Name: Roland Garros Number of Players: 128</pre>

Task 5

Design the **Dog** and **Cat** class derived from the **Animal** class with appropriate attributes and properties so that the driver code can generate the output given below.

Driver Code and Parent Class	Output
<pre>public class AnimalTester1 { public static void main(String[] args) { Dog dog = new Dog("Buddy", 5, "Brown", "Bulldog"); Cat cat = new Cat("Kitty", 3, "White", "Persian"); System.out.println("1.====="); System.out.println(dog.dogInfo()); System.out.println("2.====="); System.out.println(cat.catInfo()); System.out.println("3.====="); dog.makeSound(); System.out.println("4.====="); cat.makeSound(); } } public class Animal { public String name; public int age; public String color; public Animal(String name, int age, String color) { this.name = name; this.age = age; this.color = color; } public String info() { return "Name: "+name+"\nAge: "+age+"\nColor: "+color+"\n"; } }</pre>	<pre>1.===== Name: Buddy Age: 5 Color: Brown Breed: Bulldog 2.===== Name: Kitty Age: 3 Color: White Breed: Persian 3.===== Brown color Buddy is barking 4.===== White color Kitty is meowing</pre>

Task 6

1	public class A{
2	public int temp = 4;
3	public int sum = 1;
4	public int y = 2;
5	public void methodA(int m, int n){
6	int x = 0;
7	y = y + m + (temp++);
8	x = x + 2 + n;
9	sum = sum + x + y;
10	System.out.println(x + " " + y+ " " + sum);
11	}
12	}
13	public class B extends A {
14	public int x = 6;
15	public void methodB(int m, int n){
16	int y = 0;
17	y = ++y + this.y;
18	x = this.y + 2 + temp;
19	methodA(x, y);
20	sum = x + y + super.sum;
21	System.out.println(x + " " + y+ " " + sum);
22	}
23	}

<pre> A a1 = new A(); a1.methodA(1, 1); B b1 = new B(); b1.methodB(1, 2); </pre>	x	y	sum

HOMEWORK

Task 1

Complete the class **Cow** so that the desired outputs are generated properly.

Given Code	Expected Output
<pre>public class AnimalTester2 { public static void main(String args[]){ Animal a1 = new Animal(); System.out.println("1-----"); a1.details(); System.out.println("2-----"); Cow c1 = new Cow(); c1.name = "Pammy"; System.out.println("3-----"); System.out.println("Name: " + c1.getName()); c1.details(); System.out.println("4-----"); c1.updateSound("Moo"); System.out.println("5-----"); c1.details(); } } public class Animal{ public int legs = 4; public String sound = "Not defined"; public void details(){ System.out.println("Legs: "+legs); System.out.println("Sound: "+sound); } } public class Cow extends Animal{ //Your Code Here }</pre>	<pre>1----- Legs: 4 Sound: Not defined 2----- The cow says hello! 3----- Name: Pammy Legs: 4 Sound: Not defined 4----- 5----- Legs: 4 Sound: Moo</pre>

Task 2

Design the **CheckingAccount** class derived from the **Account** class with appropriate attributes and properties so that the driver code can generate the output given below.

Driver Code and Parent Class	Output
<pre>public class Account{ public double balance = 0.0; public Account(double balance){ this.balance = balance; } public double showBalance(){ return balance; } } //Tester Class public class TestAccount{ public static void main(String [] args){ System.out.println("Total Checking Accounts: "+CheckingAccount.count); CheckingAccount c1 = new CheckingAccount(); System.out.println("Account Balance: " + c1.showBalance()); CheckingAccount c2 = new CheckingAccount(100.0); System.out.println("Account Balance: " + c2.showBalance()); CheckingAccount c3 = new CheckingAccount(200.0); System.out.println("Account Balance: " + c3.showBalance()); System.out.println("Total Checking Accounts: "+CheckingAccount.count); } }</pre>	<pre>Total Checking Accounts: 0 Account Balance: 0.0 Account Balance: 100.0 Account Balance: 200.0 Total Checking Accounts: 3</pre>

Task 3

Given the following classes, write the code for the **Book** and the **CD** class so that the following output is printed.

Driver Code and Parent Class	Expected Output
<pre>public class Tester6 { public static void main(String[] args) { Book book = new Book(1, "The Alchemist", 500, "97806", "HarperCollins"); System.out.println(book.printDetail()); System.out.println("-----"); CD cd = new CD(2, "Shotto", 300, "Warfaze", 50, "Hard Rock"); System.out.println(cd.printDetail()); } } class Product { private int id; private String title; private int price; public Product(int id, String title, int price) { this.id = id; this.title = title; this.price = price; } public String getIdTitlePrice() { return "ID: " + id + " Title: " + title + " Price: " + price; } }</pre>	<pre>ID: 1 Title: The Alchemist Price: 500 ISBN: 97806 Publisher: HarperCollins ----- ID: 2 Title: Shotto Price: 300 Band: Warfaze Duration: 50 minutes Genre: Hard Rock</pre>

Task 4

Design the **ComplexNumber** class with the necessary property to produce the output from the given driver code.

Driver Code and Parent Class	Output
<pre>public class ComplexNumberTester { public static void main(String[] args) { ComplexNumber cn1 = new ComplexNumber(); System.out.println(cn1.details()); System.out.println("-----"); ComplexNumber cn2 = new ComplexNumber(5.0, 7.0); System.out.println(cn2.details()); } } public class RealNumber { public double realValue; public RealNumber() { this(0.0); } public RealNumber(double realValue) { this.realValue = realValue; } public String getReal(){ return "RealPart: " + realValue; } }</pre>	<pre>RealPart: 1.0 ImaginaryPart: 1.0 ----- RealPart: 5.0 ImaginaryPart: 7.0</pre>

Task 5

Given the following classes, write the code for the **CSEStudent** class derived from **Student** so that the following output is generated.

Given Code	Expected Output
<pre>public class StudentTester{ public static void main (String args[]){ CSEStudent.details(); System.out.println("1-----"); CSEStudent s1 = new CSEStudent("Bob", 23); s1.info(); System.out.println("2-----"); CSEStudent s2 = new CSEStudent("Don", 33); s2.info(); System.out.println("3-----"); s1.addLabBasedCourse("CSE220"); s1.addLabBasedCourse("CSE221"); System.out.println("4-----"); s1.info(); System.out.println("5-----"); CSEStudent.details(); System.out.println("6-----"); s1.addLabBasedCourse("CSE230"); System.out.println("7-----"); s1.info(); System.out.println("8-----"); s2.addLabBasedCourse("CSE110"); s2.info(); } } class Student{ public String name; public int id; public String courses = ""; public Student(String n, int i){ name = n; id = i; } public void info(){ System.out.println("Name: "+name); System.out.println("ID: "+id); System.out.println("Courses: "+courses); } }</pre>	<pre>Total CSE Students: 0 Available Lab Based Courses: CSE110 CSE111 CSE220 CSE221 1----- Name: Bob ID: 23 Courses: 2----- Name: Don ID: 33 Courses: 3----- 4----- Name: Bob ID: 23 Courses: CSE220 CSE221 5----- Total CSE Students: 2 Available Lab Based Courses: CSE110 CSE111 CSE220 CSE221 6----- It is not a lab based course! 7----- Name: Bob ID: 23 Courses: CSE220 CSE221 8----- Name: Don ID: 33 Courses: CSE110</pre>

Task 6

1	public class A{
2	public int m = 14, n = -1, p = 3;
3	public static int temp = 11;
4	public void methodA(int m, int n){
5	int x = 0;
6	p = this.p + m + (this.temp++);
7	x = x + 2 + n;
8	this.n = n + B.x + p;
9	System.out.println(this.m + " " + p+ " " + this.n);
10	}
11	}
12	public class B extends A {
13	public static int x = 6;
14	public void methodB(int x, int y){
15	y = ++y + this.p + n;
16	super.p = x + 2 + temp;
17	methodA(temp, x);
18	B.x = this.x + x + super.temp;
19	System.out.println(B.x + " " + y+ " " + A.temp);
20	}
21	}

<pre> B b1 = new B(); b1.methodB(3, 5); A a1 = new A(); a1.methodA(12, -9); </pre>	x	y	sum

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Given the following classes, write the code for the **Player** and the **Manager** classes derived from SportsPerson class so that the following output is printed. To calculate the match earnings use the following formula:

1. Player: $(\text{total_goal} * 1000) + (\text{total_match} * 10)$
2. Manager: $\text{match_win} * 1000$

Given Code	Expected Output
<pre>public class PlayerTester { public static void main(String[] args) { Player playerOne = new Player("Al-Nassr", "Ronaldo", "Striker", 25, 32); playerOne.calculateRatio(); playerOne.printDetails(); System.out.println("-----"); Manager managerOne = new Manager("Real Madrid", "Zidane", "Manager", 25); managerOne.printDetails(); } } class SportsPerson { private String team; private String name; public String role; public double earningPerMatch; public SportsPerson(String teamName, String name, String role){ this.team = teamName; this.name = name; this.role = role; this.earningPerMatch = 0; } public String getNameTeam() { return "Name: " + name + ", Team Name: " + team; } }</pre>	<pre>Name: Ronaldo, Team Name: Al-Nassr Team Role: Striker Total Goal: 25, Total Played: 32 Goal Ratio: 0.78 Match Earning: 25320K ----- Name: Zidane, Team Name: Real Madrid Team Role: Manager Total Win: 25 Match Earning: 25000K</pre>

Lab Assignment 07



Inspiring Excellence

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	Inheritance
Number of Tasks:	11 (Classwork: 05, Homework: 06)

[Submit all the Coding Tasks (Homework: Task 1 to 5) in the Google Form shared on buX before the next lab.]

[You are not allowed to change the driver codes of any of the tasks]

CLASSWORK

Task 1

A retail company only sells two types of products and wants to store them in a catalog. You are given the Tester class and Product class. Design the Clothing and Electronics class by observing the given classes and their outputs.

Given Code	Expected Output
<pre>public class ShoppingCartTester{ public static void main(String[] args) { Product p1 = new Product("Notebook", 4.99); Clothing c1 = new Clothing("T-Shirt", 19.99, "M"); Electronics e1 = new Electronics("Laptop", 999.99, 24); p1.displayInfo(); c1.displayInfo(); e1.displayInfo(); System.out.println("Total products created : " + Product.totalProducts); Electronics e2 = new Electronics("Smartphone", 799.99, 12); e2.displayInfo(false); } } public class Product { public String productName; public double price; public static int totalProducts = 0; public Product(String productName, double price) { this.productName = productName; this.price = price; totalProducts++; } public void displayInfo() { System.out.println("Product: " + productName + ", Price: \$" + price); } }</pre>	<pre>Product: Notebook, Price: \$4.99 Clothing: T-Shirt, Price: \$19.99, Size: M Electronics: Laptop, Price: \$999.99, Warranty: 24 months Total products created: 3 Electronics: Smartphone, Warranty: 12 months</pre>

Task 2

Given the following base class and driver code, write the code for the Warrior and Mage classes derived from the GameCharacter class so that the following output is printed.

Damage Calculation Formulas:

Warrior: (level * strength) + (armor * 50)

Mage: (level * intelligence) + (mana * 10)

Driver Code and Parent Class	Output
<pre>public class CharacterTester { public static void main(String[] args) { Warrior warriorOne = new Warrior("Conan", 100, 150, 15, "Two-Handed Sword"); System.out.println("-----1-----"); warriorOne.calculateDamage(); System.out.println(warriorOne); System.out.println("-----2-----"); Mage mageOne = new Mage("Merlin", 80, 200, 18, "Arcane Staff"); mageOne.calculateDamage(); System.out.println(mageOne); System.out.println("-----3-----"); GameCharacter.printAllCharacters(); } } class GameCharacter { public static String [] allCharacters= new String [5]; public static int characterCount=0; private String characterName; private int level; public String weaponType; public double totalDamage; public GameCharacter(String name, int level, String weapon) { this.characterName = name; this.level = level; this.weaponType= weapon; } public String getCharacterInfo() { return "Name: " + characterName + ", Level: " + level; } public int getLevel(){ return level; } public static void printAllCharacters(){ if (characterCount>0){ System.out.println("Available Characters:"); for(int i=0;i<characterCount;i++){ System.out.println(allCharacters[i]); } } else{ System.out.println("No available character"); } } }</pre>	<pre>A new warrior has arrived -----1----- Name: Conan, Level: 100 Character Type: Warrior Weapon: Two-Handed Sword Strength: 150, Armor: 15 Total Damage: 15750.0 -----2----- A new mage came to live Name: Merlin, Level: 80 Character Type: Mage Weapon: Arcane Staff Intelligence: 200, Mana: 18 Total Damage: 16180.0 -----3----- Available Characters: Conan Merlin</pre>

Task 3

Design the PremiumRide derived from Ride class to generate the given output.

fare += (fare * 20) / 100.0

An extra 20% charge is applied to the total fare if surge is active(true).

If calculateFare(int amount) is called, the fare is discounted with the amount.

Driver Code and Parent Class	Output
<pre>public class RideTester { public static void main(String[] args) { Ride r1 = new Ride(7); PremiumRide r2 = new PremiumRide(15, true); PremiumRide r3 = new PremiumRide(12, false); System.out.println("1====="); System.out.println(r1); System.out.println(r2); System.out.println(r3); System.out.println("2====="); System.out.println("Regular Ride: "+r1.calculateFare()+" TK"); System.out.println("Premium Ride "+r2.getId()+": +r2.calculateFare()+" TK"); System.out.println("Premium Ride "+r3.getId()+": +r3.calculateFare(40)+" TK"); System.out.println("3====="); System.out.println(r3); } } public class Ride { private int distance; public Ride(int distance) { this.distance = distance; } public double calculateFare() { return distance * 10; } public double getDistance() { return distance; } public String toString(){ return "Distance: "+this.distance+" km"; } }</pre>	<pre>1===== Distance: 7 km Distance: 15 km ID: 1-15 Service Charge: 50 TK Surge: true Discount: false Distance: 12 km ID: 2-12 Service Charge: 50 TK Surge: false Discount: false 2===== Regular Ride: 70.0 TK Premium Ride 1-15: 240.0 TK Premium Ride 2-12: 130.0 TK 3===== Distance: 12 km ID: 2-12 Service Charge: 50 TK Surge: false Discount: true</pre>

Task 4

Design the **CinemexTicket** class derived from the **MovieTicket** Class so that the given output is produced:

- ❖ The `seatTypes` and `seatPrices` arrays contain the type of the seat and its corresponding price
- ❖ Night show charge (15% of ticket price) will be applicable if the time is between 6:00 PM - 11:00 PM
- ❖ Unique id for a ticket is generated by:
`MovieName-FirstLetterOfSeatType-TicketCount`
- ❖ You may need to use `.split()` and `Integer.parseInt()` built-in methods

Parent Class

```
public class MovieTicket {
    public static String [] seatTypes = {"Regular", "Premium", "IMAX 3D"};
    public static double [] seatPrices = {300.0, 450.0, 600.0};
    public static int nightShowCharge = 15;
    private String movie;
    public String showtime;
    public String date;
    private double price;
    public String seat;

    public MovieTicket (String movie, String date, String showtime, double price) {
        this.movie = movie;
        this.showtime = showtime;
        this.date = date;
        this.price = price;
        this.seat = "Not Selected";
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public double getPrice() {
        return price;
    }

    public String getMovie() {
        return movie;
    }

    public String toString() {
        return "Movie: " + movie + "\nShowtime: " + showtime + "\nDate: " + date;
    }
}
```

Driver Code	Output
<pre> public class Tester { public static void main(String[] args) { CinemexTicket ticket1 = new CinemexTicket("Deadpool and Wolverine", "18:30", "Action-Comedy", "July 24, 2024"); System.out.println("Total movie ticket(s): " + CinemexTicket.getTotalTickets()); System.out.println("1====="); ticket1.calculateTicketPrice(); System.out.println("2====="); System.out.println(ticket1); System.out.println("3====="); System.out.println(ticket1.confirmPayment()); System.out.println("4====="); System.out.println(ticket1); System.out.println("5====="); CinemexTicket ticket2 = new CinemexTicket("Twisters", "10:00", "Sci-Fi", "August 10, 2024", "Premium"); System.out.println("Total movie ticket(s): " + CinemexTicket.getTotalTickets()); System.out.println("6====="); ticket2.calculateTicketPrice(); System.out.println("7====="); System.out.println(ticket2.confirmPayment()); System.out.println("8====="); System.out.println(ticket2); System.out.println("9====="); System.out.println(ticket2.confirmPayment()); } } </pre>	<pre> Total movie ticket(s): 1 1===== Ticket price is calculated successfully. 2===== Ticket ID: Deadpool and Wolverine-R-1 Movie: Deadpool and Wolverine Showtime: 18:30 Date: July 24, 2024 Genre: Action-Comedy Seat Type: Regular Price(tk): 345.0 Status: Not Paid 3===== Payment Successful. 4===== Ticket ID: Deadpool and Wolverine-R-1 Movie: Deadpool and Wolverine Showtime: 18:30 Date: July 24, 2024 Genre: Action-Comedy Seat Type: Regular Price(tk): 345.0 Status: Paid 5===== Total movie ticket(s): 2 6===== Ticket price is calculated successfully. 7===== Payment Successful. 8===== Ticket ID: Twisters-P-2 Movie: Twisters Showtime: 10:00 Date: August 10, 2024 Genre: Sci-Fi Seat Type: Premium Price(tk): 450.0 Status: Paid 9===== Ticket price is already paid! </pre>

Task 5

1	public class A {
2	public static int temp = 4;
3	public static int x = -10;
4	public int sum = 0;
5	public int y = 0;
7	public A() {
8	y = temp - 2;
9	sum = temp + 1;
10	temp -= 2;
11	}
13	public void methodA(int m, int n) {
14	int x = 0;
15	y = y + m + (temp++);
16	x = x + 1 + n;
17	sum = sum + x + y;
18	System.out.println(x + " " + y + " " + sum);
19	}
20	}
22	public class B extends A {
23	public static int x = 0;
24	public int sum = -6;
25	public B() {
26	sum = 0;
27	y = temp + 3;
28	super.sum = 3 + temp + 2;
29	temp -= 2;
30	}
31	public B(B b) {
33	sum = b.sum + super.sum;
34	x = b.x + 1;
35	b.methodB(2, 3);
36	}
37	public void methodB(int m, int n) {
38	int y = 0;
39	y = y + this.y;
40	x = y + 2 + (++temp);
41	methodA(x, y);
42	sum = x + y + sum;
43	System.out.println(x + " " + y + " " + sum);
44	}
45	}

Write the output of the following code:

<pre>public class Tester { public static void main(String[] args) { A a1 = new A(); B b1 = new B(); B b2 = new B(b1); b1.methodA(2, 3); b2.methodB(3, 8); } }</pre>	Output:		
	x	y	sum

HOMEWORK

Task 1

Implement the design of the Butterfly class and the Moth class so that these classes extend from the Caterpillar class and generate the output as follows.

- Butterfly loses 5 units of energy due to flying and gains 5 energy units per food amount
- Moth loses 2.5 units of energy due to nocturnal activity and gains 2.5 energy units per food amount

Driver Code and Given Class	Expected Output
<pre>public class Caterpillar_Tester { public static void main(String[] args) { Caterpillar c = new Caterpillar("Leaf", 5); c.showDetails(); System.out.println("-----1-----"); c.eat(); c.eat(3); c.showDetails(); System.out.println("-----2-----"); Butterfly b = new Butterfly("Flower", 2); Moth m = new Moth("Cotton", 4); System.out.println("-----3-----"); b.transform(); m.transform(); System.out.println("-----4-----"); b.eat(2); m.eat(5); System.out.println("-----5-----"); b.showDetails(); m.showDetails(); } } public class Caterpillar { public String food; public int age; public double energy; public Caterpillar(String food, int age) { this.food = food; this.age = age; this.energy = 0; } public void eat() { energy += 1; System.out.println("Caterpillar ate 1 unit of food"); } public void eat(int amount) { energy += amount; System.out.println("Caterpillar ate " + amount + " units of food"); } }</pre>	<pre>Food: Leaf Age: 5 Energy: 0.0 -----1----- Caterpillar ate 1 unit of food Caterpillar ate 3 units of food Food: Leaf Age: 5 Energy: 4.0 -----2----- -----3----- Caterpillar transforms into Butterfly Caterpillar transforms into Moth -----4----- Butterfly lost energy while flying and absorbed nectar Moth lost energy due to nocturnal activity -----5----- Food: Flower Age: 2 Energy: 5.0 joules Food: Cotton Age: 4 Energy: 10.0 joules</pre>

<pre> public void transform() { System.out.println("Caterpillar transforms"); } public void showDetails() { System.out.println("Food: " + food); System.out.println("Age: " + age); System.out.println("Energy: " + energy); } } </pre>	
---	--

Task 2

You are given the Audience class. You need to design the Concert (Parent) and VIPConcert (Child) classes so that the following tester code generates the desired output.

Hint: Assume a maximum of 5 artists can be added to a concert. Ticket Pricing:

- Zone A: 500 per ticket
- Zone B: 1,000 per ticket
- Zone VIP: 2,000 per ticket

Driver Code and Given Class	Output
<pre> public class ConcertTester{ public static void main(String[] args) { Concert concert = new Concert("ABC Conference Center","7:00 PM"); concert.showDetails(); System.out.println("-----1-----"); concert.addArtist("Tahsan Khan"); concert.addArtist("Habib Wahid"); concert.showDetails(); System.out.println("-----2-----"); Audience a1 = new Audience("Sarah"); Audience a2 = new Audience("Alex"); System.out.println("-----3-----"); a1.buyConcertTicket(concert, "A", 3); a2.buyConcertTicket(concert, "B"); System.out.println("-----4-----"); concert.showDetails(); System.out.println("-----5-----"); VIPConcert vipConcert = new VIPConcert("Army Stadium","10:00 PM"); vipConcert.addArtist("Atif Aslam"); Audience a3 = new Audience("Emily"); a3.buyConcertTicket(vipConcert); System.out.println("-----6-----"); vipConcert.showDetails(); System.out.println("-----7-----"); Concert.showTotalTicketsSold(); } } public class Audience { private String name; </pre>	<pre> Venue: ABC Conference Center Showtime: 7:00 PM Artist: Tickets sold in this concert: 0 -----1----- Venue: ABC Conference Center Showtime: 7:00 PM Artist: -Tahsan Khan -Habib Wahid Tickets sold in this concert: 0 -----2----- -----3----- Sarah bought 3 ticket(s) in Zone A Total price: 1500 Alex bought 1 ticket(s) in Zone B Total price: 1000 -----4----- Venue: ABC Conference Center Showtime: 7:00 PM Artist: -Tahsan Khan -Habib Wahid Tickets sold in this concert: 4 -----5----- Emily bought 1 ticket(s) in Zone VIP Total price: 2000 -----6----- Venue: Army Stadium Showtime: 10:00 PM </pre>

```
public Audience(String name) {  
    this.name = name;  
}  
public void buyConcertTicket(Concert concert, String zone, int  
quantity) {  
    System.out.println(name + " bought " + quantity + " ticket(s) in  
Zone " + zone);  
    concert.buyTicket(zone, quantity);  
}  
public void buyConcertTicket(Concert concert, String zone) {  
    System.out.println(name + " bought 1 ticket(s) in Zone " + zone);  
    concert.buyTicket(zone);  
}  
public void buyConcertTicket(VIPConcert concert) {  
    System.out.println(name + " bought 1 ticket(s) in Zone VIP");  
    concert.buyTicket();  
}  
}
```

Artist:
-Atif Aslam
Tickets sold in this concert: 1
-----7-----
Total tickets sold (all venues): 5

Task 3

Given the following classes, write the code for the **CarDriver** and the **BikeDriver** class so that the following output is printed.

Driver Code and Parent Class	Expected Output
<pre>import java.util.Arrays; public class PathaoRideManager{ public static void main(String[] args) { System.out.println("1.====="); GenericDriver d = new GenericDriver(); System.out.println("2.====="); d.hasSafetyTraining(); System.out.println("3.====="); BikeDriver d1 = new BikeDriver("John", "Not Premium"); System.out.println("4.====="); System.out.println(d1); System.out.println("5.====="); System.out.println(d1.acceptRide(false)); System.out.println("6.====="); d1.hasSafetyTraining(); System.out.println("7.====="); CarDriver d2 = new CarDriver("Max"); System.out.println("8.====="); System.out.println(d2); System.out.println("9.====="); d2.hasSafetyTraining(); System.out.println("10.====="); System.out.println(d2.acceptRide(true)); System.out.println("11.====="); System.out.println(d2); System.out.println("12.====="); BikeDriver.restrictedAreas("Airport Road"); BikeDriver.restrictedAreas("Defense Area"); BikeDriver.restrictedAreas("Navy Base"); System.out.println(Arrays.toString(BikeDriver.restrictedAreas)); System.out.println("13.====="); d1.fightRestriction(new String[]{"Defense Area", "Banani", "Uttara"}); System.out.println("14.====="); CarDriver.restrictedAreas("Cantonment"); CarDriver.restrictedAreas("Road 27"); System.out.println(Arrays.toString(CarDriver.restrictedAreas)); System.out.println("15.====="); d2.fightRestriction(new String[]{"Defense Area", "Road 27"}); } } public class GenericDriver { private String name; public GenericDriver() { System.out.println("Welcome to Pathao!"); } public GenericDriver(String name) { this.name = name; System.out.println("Welcome to Pathao Driver Panel!"); } }</pre>	<pre>1.===== Welcome to Pathao! 2.===== All drivers must have safety training. 3.===== Welcome to Pathao Driver Panel! John has been registered as a Pathao driver! 4.===== John's driver profile is Not Premium 5.===== All Pathao drivers can accept rides. Driver does not have a verified vehicle. John's driver profile is Not Premium 6.===== All drivers must have safety training. 7.===== Welcome to Pathao Driver Panel! Max has been registered as a Pathao driver! 8.===== Max's driver profile is a Premium 9.===== All drivers must have safety training. Premium drivers receive extra safety briefings. 10.===== All Pathao drivers can accept rides. Driver has a verified vehicle. Max's driver profile is Premium 11.===== Max's driver profile is a Premium 12.===== [Airport Road, Defense Area, Navy Base] 13.===== John cannot enter Defense Area John can enter Banani John can enter Uttara 14.===== [Cantonment, Road 27] 15.===== Max can enter Defense Area Max cannot enter Road 27</pre>

```

        System.out.println(this.name + " has been registered as a
Pathao driver!");
    }
    public String getName() {
        return name;
    }
    public void hasSafetyTraining() {
        System.out.println("All drivers must have safety training.");
    }
    public String acceptRide(boolean hasVehicle) {
        String s = "All Pathao drivers can accept rides. ";
        if (!hasVehicle)
            s += "Driver does not have a verified vehicle. ";
        else
            s += "Driver has a verified vehicle. ";
        return s;
    }
}

```

Task 4

Design the Car and ElectricCar classes so that the following output is produced. The ElectricCar class and Car class should inherit from the Vehicle class.

Driver Code and Parent Class	Output
<pre> public class VehicleShowroom { public static void main(String[] args) { Car c1 = new Car("Toyota Camry", 25000, 4); System.out.println("-----1-----"); c1.vehicleDetail(); System.out.println("-----2-----"); Car.showAllAvailableCars(); System.out.println("-----3-----"); Car c2 = new Car("Honda Civic", 22000, 4); Car c3 = new Car("Ford Mustang", 35000, 2); Car.markAsSold(c1); Car.markAsSold(c2); System.out.println("-----4-----"); c2.vehicleDetail(); System.out.println("-----5-----"); Car.showAllCars(); System.out.println("-----6-----"); ElectricCar e1 = new ElectricCar("Tesla Model 3", 45000, 75); ElectricCar e2 = new ElectricCar("Nissan Leaf", 32000, 60); ElectricCar e3 = new ElectricCar("Nissan Leaf", 32000, 60); System.out.println("-----7-----"); e1.vehicleDetail(); System.out.println("-----8-----"); e3.vehicleDetail(); System.out.println("-----9-----"); ElectricCar.markAsSoldEV(e1); ElectricCar.markAsSoldEV(e2); ElectricCar.markAsSoldEV(e3); } } </pre>	<pre> Vehicle ID: CAR001 created -----1----- Model: Toyota Camry, Price: \$25000 Status: Available Type: Regular Car Seats: 4 -----2----- Total Car: 1 Available Cars: CAR001 : Toyota Camry -----3----- Vehicle ID: CAR002 created Vehicle ID: CAR003 created -----4----- Model: Honda Civic, Price: \$22000 Status: Sold Type: Regular Car Seats: 4 -----5----- Total Car: 3 CAR001 : Toyota Camry - sold CAR002 : Honda Civic - sold CAR003 : Ford Mustang - available -----6----- Vehicle ID: EV001 created Vehicle ID: EV002 created Vehicle ID: EV003 created -----7----- Model: Tesla Model 3, Price: \$45000 </pre>

<pre> public class Vehicle { public String model; public int price; public boolean sold; public String vehicleId; public Vehicle(String model, int price) { this.model = model; this.price = price; this.sold = false; this.vehicleId = ""; } public void vehicleDetail() { System.out.println("Model: " + model + ", Price: \$" + price); System.out.print("Status: "); if(sold){ System.out.print("Sold\n"); } else{ System.out.print("Available\n"); } } } </pre>	<pre> Status: Available Type: Electric Vehicle Battery Capacity: 75 kWh -----8----- Model: Nissan Leaf, Price: \$32000 Status: Available Type: Electric Vehicle Battery Capacity: 60 kWh -----9----- </pre>
--	---

Task 5

Design the **Manager** and **Developer** class derived from the **Employee** class with appropriate attributes and properties so that the driver code can generate the output given below. [Hint:

Manager:

1. Adds a bonus to the base salary if the manager works more than 40 hours.
2. If the manager works more than 100 hours, the full amount is approved; if they work more than 80 hours, half the amount is approved. Otherwise, the increment is denied.

Developer:

1. Adds \$700 to the base salary if the developer works with Java programming language.]

Driver Code and Parent Class	Output
<pre> public class Employee { public String name; private double baseSalary; private int hoursWorked; public Employee(String name, double baseSalary, int hoursWorked){ this.name = name; this.baseSalary = baseSalary; this.hoursWorked = hoursWorked; } public double getBaseSalary() { return baseSalary; } } </pre>	<pre> 1.===== Name: Neymar Base Salary: \$1000.0 Work Hours: 45 Bonus: 10.0 % Final Salary: \$1100.0 2.===== Increment denied. 3.===== \$50 Increment approved. 4.===== 5.===== Name: Neymar </pre>

```

    public void setBaseSalary(double baseSalary) {
        this.baseSalary = baseSalary;
    }
    public int getHoursWorked() {
        return hoursWorked;
    }
    public void setHoursWorked(int hoursWorked) {
        this.hoursWorked = hoursWorked;
    }
    public void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Base Salary: $" + baseSalary);
        System.out.println("Work Hours: " + hoursWorked);
    }
}

public class EmployeeTester {
    public static void main(String[] args) {
        Manager neymar = new Manager("Neymar", 1000, 45, 10);
        Developer messi = new Developer("Messi", 1000, 50, "Java");
        Developer chiesa = new Developer("Chiesa", 1000, 50, "Javascript");
        neymar.calculateSalary();
        System.out.println("1.=====");
        neymar.displayInfo();
        System.out.println("2.=====");
        neymar.requestIncrement(100);
        System.out.println("3.=====");
        neymar.setHoursWorked(85);
        neymar.requestIncrement(100);
        System.out.println("4.=====");
        neymar.calculateSalary();
        System.out.println("5.=====");
        neymar.displayInfo();
        System.out.println("6.=====");
        messi.calculateSalary();
        System.out.println("7.=====");
        messi.displayInfo();
        System.out.println("8.=====");
        chiesa.calculateSalary();
        System.out.println("9.=====");
        chiesa.displayInfo();
    }
}

```

```

Base Salary: $1050.0
Work Hours: 85
Bonus: 10.0 %
Final Salary: $1155.0
6.=====
7.=====
Name: Messi
Base Salary: $1000.0
Work Hours: 50
Language: Java
Final Salary: $1700.0
8.=====
9.=====
Name: Chiesa
Base Salary: $1000.0
Work Hours: 50
Language: Javascript
Final Salary: $1000.0

```


Task 6

1	public class A {
2	public static int temp = 4;
3	public static int x = -10;
4	public int sum, y;
5	public A() {
6	y = temp - 2;
7	sum = temp + 1 + this.x;
8	temp -= 2;
9	}
10	public A(int x){
11	this.methodA(3,5);
12	}
13	public void methodA(int m, int n) {
14	y = y + m + (temp++);
15	x = x + 1 + n;
16	sum = sum + this.x + y;
17	System.out.println(x + " " + y + " " + sum);
18	}
19	}
20	public class B extends A {
21	public static int x = 0;
22	public int sum = -6;
23	public B() {
24	sum = 0;
25	y = temp + 3;
26	super.sum = this.x + super.x + A.x;
27	B.x -= 2;
28	}
29	public B(B b) {
30	super(5);
31	sum = b.sum + super.sum;
32	x = b.x + 1;
33	b.methodB(2, 3);
34	}
35	public void methodA(int m, int n) {
36	y = this.y + n + sum;
37	x = x + 4 + n;
38	sum = super.sum + x + y;
39	System.out.println(x + " " + y + " " + sum);
40	}
41	public void methodB(int m, int n) {
42	x = y + 2 + (++temp);
43	sum = x + y + sum;
44	System.out.println(x + " " + y + " " + sum);

45	<code>super.methodA(x, y);</code>
46	<code>}</code>
47	<code>}</code>

<code>B b1 = new B(); B b2 = new B(b1); b1.methodA(2, 3);</code>	x	y	sum

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Design the **KKTea** (parent) and **KKFlavouredTea** (child) classes so that the following output is produced. The **KKFlavouredTea** class should inherit **KKTea** and **KKTea** should inherit the **Tea** class. Note that:

- An object of either class represents a single box of teabags.
- Each tea bag weighs 2 grams.
- The status of an object refers to whether it is sold or not

Driver Code and Parent Class	Output
<pre>public class Tea { public String name; protected int price; protected boolean status; public Tea(String name, int price) { this.name = name; this.price = price; this.status = false; } public void productDetail() { System.out.println("Name: " + name + ", Price: " + price); System.out.println("Status: " + status); } } //Driver Code public class TeaTester{ public static void main(String[] args) { KKTea t1 = new KKTea(250, 50); System.out.println("-----1-----"); t1.productDetail(); System.out.println("-----2-----"); KKTea.totalSales(); System.out.println("-----3-----"); KKTea t2 = new KKTea(470, 100); KKTea t3 = new KKTea(360, 75); KKTea.updateSoldStatusRegular(t1); KKTea.updateSoldStatusRegular(t2); System.out.println("-----4-----"); t2.productDetail(); System.out.println("-----5-----"); KKTea.totalSales(); System.out.println("-----6-----"); KKFlavouredTea t4 = new KKFlavouredTea("Jasmine", 260, 50); KKFlavouredTea t5 = new KKFlavouredTea("Honey Lemon", 270, 45); KKFlavouredTea t6 = new KKFlavouredTea("Honey Lemon", 270, 45); System.out.println("-----7-----"); t4.productDetail(); System.out.println("-----8-----"); t6.productDetail(); System.out.println("-----9-----"); KKFlavouredTea.updateSoldStatusFlavoured(t4); KKFlavouredTea.updateSoldStatusFlavoured(t5); KKFlavouredTea.updateSoldStatusFlavoured(t6);</pre>	<pre>-----1----- Name: KK Regular Tea, Price: 250 Status: false Weight: 100, Tea Bags: 50 -----2----- Total Sales: 0 KK Regular Tea: 0 -----3----- -----4----- Name: KK Regular Tea, Price: 470 Status: true Weight: 200, Tea Bags: 100 -----5----- Total Sales: 2 KK Regular Tea: 2 -----6----- -----7----- Name: KK Jasmine Tea, Price: 260 Status: false Weight: 100, Tea Bags: 50 -----8----- Name: KK Honey Lemon Tea, Price: 270 Status: false Weight: 90, Tea Bags: 45 -----9----- -----10----- Total Sales: 5 KK Regular Tea: 2 KK Flavoured Tea: 3</pre>

<pre> System.out.println("-----10-----"); KKTea.totalSales(); } } </pre>	
--	--

Task 2

1	public class A{
2	public static int temp = 4;
3	public int sum = 1;
4	public int y = 2;
5	public A(){
6	y = temp - 2;
7	sum = temp + 3;
8	temp-=2;
9	}
10	public void methodA(int m, int n){
11	int x = 0;
12	y = y + m + (temp++);
13	x = x + 2 + n;
14	sum = sum + x + y;
15	System.out.println(x + " " + y+ " " + sum);
16	}
17	}
18	public class B extends A {
19	public int x = 5, temp = 10;
20	public B(){
21	y = temp + 3 ;
22	sum = 3 + temp + 2;
23	temp-=1;
24	}
25	public B(B b){
26	sum = b.sum;
27	x = b.x;
28	}
29	public void methodB(int m, int n){
30	int y = 0;
31	y = y + this.y;
32	x = this.y + 2 + temp;
33	methodA(x, y);
34	sum = x + y + super.sum;
35	System.out.println(x + " " + y+ " " + sum);
36	}
37	}

A a1 = new A();	x	y	sum
B b1 = new B();			
B b2 = new B(b1);			
a1.methodA(1, 1);			

b1.methodA(1, 2); b2.methodB(3, 2);			

Lab Assignment 08

[HOMEWORK SUBMISSION DEADLINE: 10 MAY, 11:59 PM]



I n s p i r i n g E x c e l l e n c e

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	Polymorphism
Number of Tasks:	8 (Classwork: 05, Homework: 03)

[Submit all the Coding Tasks (Homework: Task 1 to 2) in the Google Form shared on buX before the deadline.]

[You are not allowed to change the given codes of any of the tasks]

CLASSWORK

Task 1

1.a. Write the **Pokemon** class so that the following code generates the output below:

Driver Code	Output
<pre>public class PokemonTester{ public static void main(String[] args){ Pokemon pikachu = new Pokemon("Pikachu"); pikachu.attack(); pikachu.attack("Thunderbolt"); pikachu.attack("Iron Tail", 90); } }</pre>	<pre>Pikachu attacks with a basic move! Pikachu uses Thunderbolt! Pokemon uses Iron Tail with power 90!</pre>

1.b. What type of polymorphism is depicted in the code above?

Task 2

Write the **Mango** and the **Jackfruit** classes derived from Fruit class so that the following code generates the output below:

Parent Class	
<pre>public class Fruit{ private boolean formalin = false; private String name = ""; public Fruit(boolean formalin, String name){ this.formalin = formalin; this.name = name; } public String getName(){ return name; } public boolean hasFormalin(){ return formalin; } }</pre>	
Driver Code	Output
<pre>public class FruitTester{ public static void testFruit(Fruit f){ System.out.println("----Printing Detail-----"); if(f.hasFormalin()){ System.out.println("Do not eat the "+f.getName()+"."); System.out.println(f); }else{ System.out.println("Eat the "+f.getName()+"."); System.out.println(f); } } public static void main(String [] args){ Mango m = new Mango(); testFruit(m); Jackfruit j = new Jackfruit(); testFruit(j); } }</pre>	<pre>----Printing Detail----- Do not eat the Mango. Mangos are bad for you ----Printing Detail----- Eat the Jackfruit. Jackfruits are good for you</pre>

Task 3

Design a set of classes for a Fantasy Game Character System. There is a parent class called **Character**. From it, there are two different child classes: **Warrior** and **Mage**. Additionally, there is a subclass called **Paladin** that extends **Warrior**.

Each character has:

- name (String)
- level (int)

Lastly, you need to Override the `.equals()` method inside the parent class

Parent Class	
<pre>public class Character { public String name; public int level; public Character(String name, int level) { this.name = name; this.level = level; } public void specialMove() { System.out.println("Character uses a generic move."); } // Override the .equals() method }</pre>	
Driver Code	Output
<pre>public class GameTester { public static void main(String[] args) { Character c1 = new Paladin("Arthur", 10); Character c2 = new Mage("Merlin", 12); Character c3 = new Warrior("Leon", 10); c1.specialMove(); c2.specialMove(); c3.specialMove(); + if (c1 instanceof Paladin) { Paladin p = (Paladin) c1; p.specialMove(); } Warrior w1 = new Warrior("Leon", 10); System.out.println("c3 equals w1? " + c3.equals(w1)); Mage m1 = new Mage("Merlin", 15); System.out.println("c2 equals m1? " + c2.equals(m1)); } }</pre>	<pre>Arthur unleashes a holy strike! Merlin casts a powerful fireball! Leon performs a heavy sword slash! Arthur unleashes a holy strike! c3 equals w1? true c2 equals m1? false</pre>

Task 4

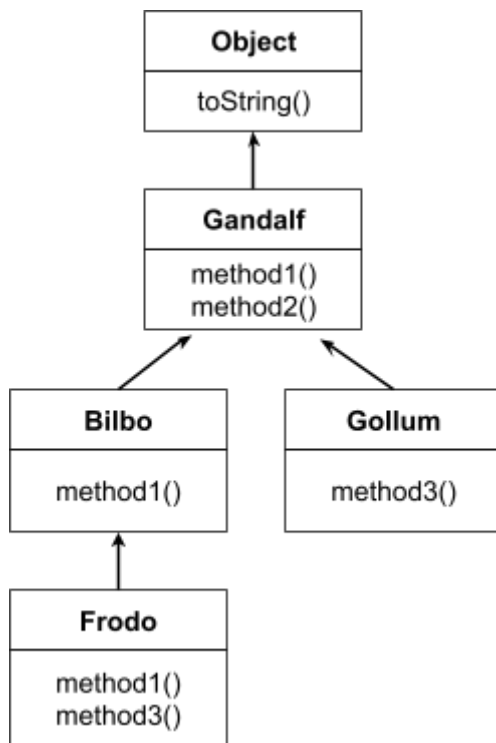
1	public class Gandalf {
2	public void method1(){
3	System.out.println("Gandalf 1");
4	}
5	
6	public void method2(){
7	System.out.println("Gandalf 2");
8	method1();
9	}
10	}
11	public class Bilbo extends Gandalf{
12	public void method1(){
13	System.out.println("Bilbo 1");
14	}
15	}
16	public class Gollum extends Gandalf{
17	public void method3(){
18	System.out.println("Gollum 3");
19	}
20	}
21	public class Frodo extends Bilbo{
22	public void method1(){
23	System.out.println("Frodo 1");
24	super.method1();
25	}
26	
27	public void method3(){
28	System.out.println("Frodo 3");
29	}
30	}

Assuming the following variables have been defined:

```
Gandalf var1 = new Frodo();
Gandalf var2 = new Bilbo();
Gandalf var3 = new Gandalf();
Object var4 = new Bilbo();
Bilbo var5 = new Frodo();
Object var6 = new Gollum();
```

A class diagram for the above classes has been added with this question.

Note: The diagram may not always be provided, so students should know how to draw it from the code.



In the table below,

- The output produced by the statement in the left-hand column, should be written in the right-hand column
- If the statement produces more than one line of output, indicate the line breaks with slashes as in "a/b/c" to indicate three lines of output with "a" followed by "b" followed by "c".
- If the statement causes an error, fill in the right-hand column with either the phrase "compiler error" or "runtime error" to indicate when the error would be Detected.

	Statement	Output
1	var1.method1();	
2	var2.method1();	
3	var4.method1();	
4	var6.method1();	
5	var1.method2();	
6	var3.method2();	
7	var4.method2();	
8	var5.method2();	
9	var6.method2();	

10	((Frodo)var4).method3();	
11	((Frodo)var6).method2();	
12	((Gollum)var1).method3();	
13	((Gollum)var4).method1();	
14	((Gandalf)var1).method2();	
15	((Frodo)var4).method1();	
16	((Gollum)var6).method2();	
17	((Gandalf)var2).method1();	
18	((Bilbo)var6).method2();	
19	((Frodo)var1).method3();	
20	((Gandalf)var5).method3();	

Task 5

1	public class Caramel extends SilkOreo{
2	String texture = "Softy";
3	public void method1() {
4	System.out.println("Caramel m1");
5	}
6	public void method4() {
7	System.out.println("Caramel m4");
8	}
9	public String toString(){
10	method2();
11	return "Caramel is "+ texture;
12	}
13	}
14	public class Chocolate{
15	String texture = "Chocolaty";
16	public void method1() {
17	method2();
18	System.out.println("Chocolate m1");
19	}
20	public void method2() {
21	System.out.println("Chocolate m2");
22	}
23	public String toString(){
24	method2();
25	return "Chocolate is "+ texture;
26	}
27	}
28	public class DairyMilk extends Chocolate{
29	String texture = "Yummy";
30	public void method2() {
31	System.out.println(this.texture);
32	System.out.println("DairyMilk m2");
33	}
34	public void method3() {
35	System.out.println("DairyMilk m3");
36	}
37	}
38	public class KitKat extends Chocolate{
39	String texture = "Crunchy";
40	public void method1() {
41	System.out.println("KitKat m1");
42	}
43	public void method4() {
44	System.out.println("KitKat m4");
45	}

46	public String toString(){
47	method2();
48	return "KitKat is "+ texture;
49	}
50	}
51	public class SilkOreo extends DairyMilk{
52	String texture = "Silky";
53	public void method1() {
54	super.method1();
55	System.out.println("SilkOreo m1");
56	}
57	public void method3() {
58	System.out.println("SilkOreo m3");
59	System.out.println(this);
60	}
61	}

Assuming the following variables have been defined:

```
Chocolate choco1 = new Chocolate();
KitKat kit = new KitKat();
DairyMilk dairyMilk1 = new DairyMilk();
DairyMilk dairyMilk2 = new SilkOreo();
Object obj1 = new DairyMilk();
Object obj2 = new KitKat();
Chocolate caramel1 = new Caramel();
```

In the table below,

- The output produced by the statement in the left-hand column, should be written in the right-hand column
- If the statement produces more than one line of output, indicate the line breaks with slashes as in "a/b/c" to indicate three lines of output with "a" followed by "b" followed by "c".
- If the statement causes an error, fill in the right-hand column with either the phrase "compiler error" or "runtime error" to indicate when the error would be detected.-

	Statement	Output
1	choco1.method1();	
2	dairyMilk1.method1();	
3	dairyMilk2.method4();	
4	caramel1.method1();	

5	System.out.println(caramel1);	
6	System.out.println(caramel1.texture);	
7	((Chocolate)kit).method2();	
8	((SilkOreo)dairyMilk2).method3();	
9	((DairyMilk)kit).method2();	
10	((Chocolate)kit).method2();	
11	((Chocolate)dairyMilk2).method1();	
12	((Chocolate)obj1).method2();	
13	((Caramel)obj1).method2();	
14	((SilkOreo)obj2).method3();	
15	System.out.println(((Object)choco1).toString());	
16	System.out.println(((Chocolate)kit).texture);	

HOMEWORK

[SUBMISSION DEADLINE: 10 MAY, 11:59 PM]

Task 1

Write the **CSEStudent** and **CSE111Student** classes derived from **Student** class so that the following code generates the output below:

Parent Class	
<pre>public class Student{ public String msg = "I love BU"; public String shout(){ return msg; } }</pre>	
Driver Code	Output
<pre>public class StudentTester{ public static void printShout(Student s){ System.out.println("-----"); System.out.println(s.msg); System.out.println(s.shout()); } public static void main(String [] args){ Student s = new Student(); CSEStudent cs = new CSEStudent(); CSE111Student cs111 = new CSE111Student(); System.out.println(s.msg); System.out.println(cs.msg); System.out.println(cs111.msg); printShout(s); printShout(cs); printShout(cs111); } }</pre>	<pre>I love BU I want to transfer to CSE I love Java Programming ----- I love BU I love BU ----- I love BU I want to transfer to CSE ----- I love BU I love Java Programming</pre>

Task 2

Write the **PlatinumCard** and **SignatureCard** classes derived from **CreditCard** class so that the following code generates the output below.

Note: Platinum card users initially have 100 reward points and will get 2 reward points for spending 100 taka each. Signature card users initially have 200 reward points and will get 4 reward points for spending 100 taka each. Signature card users are allowed to bring upto 5 companions at lounges.

Parent Class	
<pre>public class CreditCard { public String cardHolder; public String accountNo; public int rewardPoints; public CreditCard(String cardHolder, String accountNo, int rewardPoints){ this.cardHolder = cardHolder; this.accountNo = accountNo; this.rewardPoints = rewardPoints; } public void cardDetails(){ System.out.println("Card Holder Name: " + cardHolder); System.out.println("Account Number: " + accountNo); System.out.println("Reward point gained: " + rewardPoints); } }</pre>	
Driver Code	Output
<pre>public class CardTester { public static void main(String[] args) { CreditCard card1 = new PlatinumCard("Ali", "345 127"); CreditCard card2 = new SignatureCard("Rahul", "514 123"); CreditCard card3 = new SignatureCard("Rohan", "147 965"); CreditCard [] cards = {card1, card2, card3}; for (int i = 0; i<cards.length; i++) { System.out.println("====="); if (cards[i] instanceof SignatureCard) { SignatureCard new_card = (SignatureCard) cards[i]; new_card.spendCash(500); } else if (cards[i] instanceof PlatinumCard) { PlatinumCard new_card = (PlatinumCard) cards[i]; new_card.spendCash(200); } System.out.println("====="); cards[i].cardDetails(); } } }</pre>	<pre>===== Previous Reward Points: 100 Reward points after spending 200 taka: 104 ===== Card Holder Name: Ali Account Number: 345 127 Reward point gained: 104 ===== Previous Reward Points: 200 Reward points after spending 500 taka: 220 ===== Card Holder Name: Rahul Account Number: 514 123 Reward point gained: 220 Possible Number of Companions for Lounge: 5 ===== Previous Reward Points: 200 Reward points after spending 500 taka: 220 ===== Card Holder Name: Rohan Account Number: 147 965 Reward point gained: 220 Possible Number of Companions for Lounge: 5</pre>

Task 3

1	public class Sue {
2	void method1() {
3	System.out.println("sue 1");
4	}
5	void method3() {
6	System.out.println("sue 3");
7	}
8	}
9	
10	public class Blue {
11	void method1() {
12	System.out.println("blue 1");
13	method3();
14	}
15	void method3() {
16	System.out.println("blue 3");
17	}
18	}
19	
20	public class Moo extends Blue {
21	void method2() {
22	super.method3();
23	System.out.println("moo 2");
24	this.method3();
25	}
26	void method3() {
27	System.out.println("moo 3");
28	}
29	}
30	
31	public class Crew extends Moo {
32	void method1() {
33	System.out.println("crew 1");
34	}
35	void method3() {
36	System.out.println("crew 3");
37	}
38	}

Assuming the following variables have been defined:

```
Moo var1 = new Crew();  
Blue var2 = new Moo();  
Object var3 = new Sue();  
Sue var4 = new Sue();
```

```
Blue var5 = new Crew();
Blue var6 = new Blue();
```

In the table below,

- The output produced by the statement in the left-hand column, should be written in the right-hand column
- If the statement produces more than one line of output, indicate the line breaks with slashes as in "a/b/c" to indicate three lines of output with "a" followed by "b" followed by "c".
- If the statement causes an error, fill in the right-hand column with either the phrase "compiler error" or "runtime error" to indicate when the error would be detected.

	Statement	Output
1	var1.method1();	
2	var2.method1();	
3	var3.method1();	
4	var4.method1();	
5	var5.method1();	
6	var6.method1();	
7	var1.method3();	
8	var2.method3();	
9	var3.method3();	
10	((Blue)var1).method1();	
11	((Crew)var1).method2();	
12	((Sue)var1).method3();	
13	((Blue)var3).method1();	
14	((Crew)var3).method1();	
15	((Sue)var3).method3();	
16	((Moo)var2).method2();	
17	((Crew)var3).method2();	
18	((Moo)var5).method2();	
19	((Moo)var6).method2();	
20	((Moo)var2).method1();	

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Write the Garage, Bike and Car class. **Car**, **Bike** are child classes of **Vehicle** class. But **Garage** is neither a parent nor a child class. The Garage class has **two arrays as instance variables** called *cars* and *bikes* that can store **Car and Bike objects**.

Hint: In this task you'll need to use the **instanceof** keyword and **downcasting**.

Parent Class

```
public class Vehicle {  
    private String brand;  
    private int year, wheels;  
  
    public Vehicle(String b, int y){  
        this.brand = b;  
        this.year = y;  
    }  
  
    public String getBrand(){  
        return this.brand;  
    }  
  
    public int getYear(){  
        return this.year;  
    }  
  
    public void setWheels( int w ){  
        this.wheels = w;  
    }  
  
    public int getWheels(){  
        return this.wheels;  
    }  
  
    public String toString(){  
        return "Brand: "+this.brand+", Year: "+this.year+", Wheels: "+this.wheels;  
    }  
}
```

DRIVER CODE	OUTPUT
<pre> Garage g = new Garage(2, 3); System.out.println("=====0====="); Vehicle vC1 = new Car("Ford", "Mustang", 2022, 2, 4, false); Vehicle vC2 = new Car("Tesla", "Model S", 2025, 4, 4, true); Vehicle vC3 = new Car("Reliant", "Robin", 1981, 2, 3, false); System.out.println("=====1====="); System.out.println(vC1); System.out.println("=====2====="); g.addVehicle(vC1); g.addVehicle(vC2); g.addVehicle(vC3); System.out.println(g.cars[1]); System.out.println("=====3====="); g.cars[0].startAutoPilot(); g.cars[1].startAutoPilot(); System.out.println("=====4====="); Vehicle vB1 = new Bike("Honda", "Gold Wing", 2022, 3, true); System.out.println(vB1); g.addVehicle(vB1); System.out.println("=====5====="); Vehicle vB2 = new Bike("Royal Enfield", "Classic 350", 2021, 2, false); g.addVehicle(vB2); System.out.println(g.bikes[1]); System.out.println("=====6====="); Vehicle vB3 = new Bike("Harley-Davidson", "Street 750", 2022, 2, false); g.addVehicle(vB3); Vehicle vB4 = new Bike("Yamaha", "MT-15", 2023, 2, false); g.addVehicle(vB4); System.out.println("=====7====="); g.bikes[0].doAWheelie(); g.bikes[1].doAWheelie(); </pre>	<pre> Welcome to the Garage! Car Capacity: 2 Bike Capacity: 3 =====0===== =====1===== Car Brand: Ford, Year: 2022, Wheels: 4, Model: Mustang, Doors: 2, AI: false =====2===== A Ford CAR has been added to the Garage A Tesla CAR has been added to the Garage Can't add more Cars! Capacity: 2 Car Brand: Tesla, Year: 2025, Wheels: 4, Model: Model S, Doors: 4, AI: true =====3===== Ford:Mustang has NO AutoPilot Tesla:Model S AutoPilot Started =====4===== Bike Brand: Honda, Year: 2022, Wheels: 3, Model: Gold Wing, SideCar: true A Honda BIKE has been added to the Garage =====5===== A Royal Enfield BIKE has been added to the Garage Bike Brand: Royal Enfield, Year: 2021, Wheels: 2, Model: Classic 350, SideCar: false =====6===== A Harley-Davidson BIKE has been added to the Garage Can't add more bikes! Capacity: 3 =====7===== Wheelie Failed. Honda:Gold Wing has SideCar Royal Enfield:Classic 350 is doing Wheelie!! </pre>

Task 2

Design the **SmartSecurityCamera** class derived from SmartDevice class to generate the following output.

Tester Code and Parent Class	Output
<pre>public class SmartHomeTester { public static void main(String[] args) { SmartSecurityCamera cam1 = new SmartSecurityCamera("Garden-Cam", 100, 64); cam1.powerOn(); System.out.println("====="); cam1.record(true); System.out.println("====="); cam1.powerOff(); System.out.println("====="); cam1.powerOn(); System.out.println("====="); cam1.record(); System.out.println("====="); cam1.formatCard("0000"); System.out.println("====="); cam1.formatCard("ADMIN123"); System.out.println("====="); SmartSecurityCamera cam2 = new SmartSecurityCamera("Indoor-Cam", 80, 1); cam2.powerOn(); System.out.println("====="); cam2.record(); System.out.println("====="); cam2.powerOff(); System.out.println("====="); cam2.powerOn(); System.out.println("====="); cam2.formatCard("ADMIN123"); System.out.println("====="); cam2 = new SmartSecurityCamera("Indoor-Cam", 2, 10); cam2.powerOn(); } } class SmartDevice { public String deviceName; private double batteryLevel; protected boolean isActive; SmartDevice(String name, double battery) { this.deviceName = name; this.batteryLevel = battery; this.isActive = false; } public void powerOn() { if (batteryLevel > 5) { isActive = true; batteryLevel -= 2; System.out.println(deviceName + " is now ONLINE."); } } }</pre>	<pre>Garden-Cam is now ONLINE. ===== [IR SENSORS ACTIVE] Recording standard footage. ===== Turning off Night Vision. Garden-Cam has shut down. ===== Garden-Cam is now ONLINE. ===== Recording standard footage. ===== REQUEST: Format SD Card initiated. ACCESS DENIED: Incorrect PIN. ===== REQUEST: Format SD Card initiated. Auth Success. Wiping data. SUCCESS: Storage restored to 64GB. ===== Indoor-Cam is now ONLINE. ===== Recording standard footage. ===== Indoor-Cam has shut down. ===== Error: Indoor-Cam storage full. Recording disabled. ===== REQUEST: Format SD Card initiated. ERROR: Device must be ON to format. ===== Power Low: Indoor-Cam cannot start.</pre>

```
    } else {  
        System.out.println("Power Low: " + deviceName + "  
cannot start.");  
    }  
}  
  
public void powerOff() {  
    this.isActive = false;  
    System.out.println(deviceName + " has shut down.");  
}  
  
public double getBattery() {  
    return batteryLevel;  
}  
}
```

Task 3

1	public class Foo {
2	String name = "foo";
3	public void call1() {
4	System.out.println("Foo 1");
5	}
6	public void call2() {
7	call1();
8	System.out.println("Foo 2");
9	}
10	}
11	
12	public class Bar extends Foo {
13	public void call2() {
14	System.out.println("Bar 2");
15	}
16	public void call3() {
17	System.out.println("Bar 3");
18	}
19	}
20	
21	public class Buzz extends Bar {
22	String name = "Buzz";
23	public void call1() {
24	System.out.println("Buzz 1");
25	}
26	public void call4() {
27	call3();
28	System.out.println("Buzz 4");
29	}
30	}
31	public class Bux extends Foo {
32	String name = "Bux";
33	public void call1() {
34	System.out.println("Bux 1");
35	}
36	public void call3() {
37	System.out.println("Bux 3");
38	}
39	}

Assuming the following variables have been defined:

```
Foo foo1 = new Foo();  
Bar bar1 = new Bar();
```

```

Bux bux1 = new Bux();
Foo foo2 = new Buzz();
Bar bar2 = new Buzz();
Object obj1 = new Foo();

```

In the table below,

- The output produced by the statement in the left-hand column, should be written in the right-hand column
- If the statement produces more than one line of output, indicate the line breaks with slashes as in "a/b/c" to indicate three lines of output with "a" followed by "b" followed by "c".
- If the statement causes an error, fill in the right-hand column with either the phrase "compiler error" or "runtime error" to indicate when the error would be detected.

	Statement	Output
1	bar1.call1();	
2	foo2.call1();	
3	foo2.call2();	
4	bar2.call3();	
5	System.out.println(bar1.name);	
6	System.out.println(bar2.name);	
7	System.out.println(((Buzz)bar2).name);	
8	((Buzz)bar1).call4();	
9	((Bar)foo1).call3();	
10	((Foo)bux1).call1();	
11	((Bux)foo1).call1();	
12	bux1.call1();	
13	bux1.call2();	
14	((Foo)foo2).call2();	
15	((Buzz)obj1).call3();	
16	((Buzz)obj1).call2();	
17	((Bux)foo2).call2();	
18	((Buzz)obj1).call1();	
19	System.out.println(foo2.name);	
20	System.out.println(((Bux)foo2).name);	

Task 4

Write the **UsedBattery** and the **PlasticBottle** classes derived from **Waste** class so that the following code generates the output below.

You also need to **complete the SorterBot class** for this code to work. You cannot create any additional methods in the **SorterBot** class.

Tester	Output
<pre>public class RecyclingPlant { public static void main(String[] args) { SorterBot robot = new SorterBot(); UsedBattery bat1 = new UsedBattery("Duracell-X", true); PlasticBottle bot1 = new PlasticBottle("Coke-Zero", 500); Waste unknown = new Waste("Mystery-Box"); System.out.println("#####"); robot.processItem(bat1); robot.processItem(bot1); robot.processItem(unknown); robot.processItem(new UsedBattery("Energizer", false)); } } //Parent Class class Waste { String id; Waste(String id) { this.id = id; } } //Disjoint Class class SorterBot { public void processItem(Waste item) { //WRITE YOUR CODE HERE System.out.println("-----"); } }</pre>	<pre>##### Scanning Item Duracell-X [Type: BATTERY] Duracell-X is leaking. Sealing in concrete container. ----- Scanning Item Coke-Zero [Type: PLASTIC] Coke-Zero compressed from 500ml to flat disk. ----- Scanning Item Mystery-Box [Type: UNKNOWN] Item sent to generic incinerator. ----- Scanning Item Energizer [Type: BATTERY] Energizer stored in dry cell. -----</pre>

Task 5

Your task is to design the **StudyRoom** class with appropriate variables and methods such that the following tester code produces the expected output. Note:

- Assume that each **StudyRoom** can add two books.
- You cannot use any arrays in the **StudyRoom** class.
- You should use the given **Library** and **SilentStudyRoom** classes' variables and methods as needed.
- You cannot modify the given **Library** and **SilentStudyRoom** classes.

Tester Code	Expected Output
<pre>public class TestLibrary{ public static void main(String[] args) { Library library = new Library("The Mind Maze"); library.showRoomInfo(); System.out.println("===== 1 ====="); StudyRoom room9 = new StudyRoom("Study Hub"); SilentStudyRoom room9A = new SilentStudyRoom("Focus Room"); room9.addBook("Data Structures"); System.out.println("===== 2 ====="); room9.addBook("Operating Systems"); room9.showRoomInfo(); System.out.println("===== 3 ====="); System.out.println("Total books: " +StudyRoom.totalBooks); System.out.println("===== Add Book ====="); room9A.addBook(library, room9); System.out.println("===== 4 ====="); room9A.showRoomInfo(); System.out.println("Total books: " +StudyRoom.totalBooks); System.out.println("===== Add Book Again ====="); room9A.addBook(library, room9); System.out.println("===== 5 ====="); room9A.showRoomInfo(); System.out.println("Total books: " + StudyRoom.totalBooks); } }</pre>	<pre>Library Name: The Mind Maze ===== 1 ===== ===== 2 ===== Study Hub Details: Book 1: Data Structures Book 2: Operating Systems ===== 3 ===== Total books: 2 ===== Add Book ===== Library Name: The Mind Maze Adding book: Data Structures Book 1 Removed from Study Hub Book added successfully! ===== 4 ===== Focus Room Details: Book 1: Data Structures No book added Total books: 1 ===== Add Book Again ===== Library Name: The Mind Maze Adding book: Operating Systems Book 2 Removed from Study Hub Book added successfully! ===== 5 ===== Focus Room Details: Book 1: Data Structures Book 2: Operating Systems Total books: 0</pre>
<pre>// Grand Parent Class class Library{ public String name; public Library(String name) { this.name = name; } public void showRoomInfo() { System.out.println("Library Name: " + name); } }</pre>	
<pre>// Parent Class</pre>	

```
class StudyRoom extends Library{  
    // Write Your Code Here  
}
```

```
// Child Class  
class SilentStudyRoom extends StudyRoom{  
  
    public SilentStudyRoom(String name) {  
        super(name);  
    }  
  
    public void addBook(Library lib, StudyRoom room) {  
        lib.showRoomInfo();  
        if (room.getBook1() != null) {  
            System.out.println("Adding book: " + room.getBook1());  
            this.setBook1(room.getBook1());  
            room.removeBook(1);  
            System.out.println("Book added successfully!");  
        }  
        else if (room.getBook2() != null) {  
            System.out.println("Adding book: " + room.getBook2());  
            this.setBook2(room.getBook2());  
            room.removeBook(2);  
            System.out.println("Book added successfully!");  
        }  
        else {  
            System.out.println("No books available in "+room.name);  
        }  
    }  
}
```

Task 6

Your task is to design the **UpsideDown** class with appropriate variables and methods such that the following tester code produces the expected output. Note:

- Assume that each gate of **UpsideDown** can connect with two bridges.
- You cannot use any arrays in the **UpsideDown** class.
- You should use the given **Hawkins** and **DarkDimension** classes' variables and methods as needed.
- You cannot modify the given **Hawkins** and **DarkDimension** classes.

Tester Code	Expected Output
<pre> public class HawkinsLabTester { public static void main(String[] args) { Hawkins place1 = new Hawkins("Hawkins Lab"); Hawkins place2 = new Hawkins("Palace Arcade"); UpsideDown gate1 = new UpsideDown("The Nina Project"); UpsideDown gate2 = new UpsideDown("Brimborn Steel Works"); DarkDimension world = new DarkDimension("The Dark World"); gate1.open(); System.out.println("Total bridges: " + UpsideDown.totalBridges); System.out.println("===== [1] ====="); gate1.connect(place1); gate1.connect(place2); Hawkins place3 = new Hawkins("Starcourt Mall"); gate2.connect(place3); gate1.details(); gate2.details(); System.out.println("===== [2] ====="); world.runExperiment(gate1); world.runExperiment(gate2); System.out.println("===== [3] ====="); System.out.println("Total bridges: " + UpsideDown.totalBridges); System.out.println("===== [4] ====="); Hawkins place4 = new Hawkins("Byers new house"); gate1.connect(place4); gate1.disconnect(2); gate2.disconnect(3); System.out.println("===== [5] ====="); gate1.details(); } } </pre>	<pre> Bridge from The Nina Project is Open Total bridges: 0 ===== [1] ===== The Nina Project Details: Bridge 1: Hawkins Lab Bridge 2: Palace Arcade Brimborn Steel Works Details: Bridge 1: Starcourt Mall ===== [2] ===== Bridge present at The Nina Project Activating the door of Hawkins Lab Experiment executed successfully! No Bridge present at Brimborn Steel Works Cannot run experiment. ===== [3] ===== Total bridges: 3 ===== [4] ===== No further bridges with The Nina Project Invalid bridge number! ===== [5] ===== The Nina Project Details: Bridge 1: Hawkins Lab </pre>
<pre> // Grand Parent Class class Hawkins{ public String name; public boolean status=false; public Hawkins(String name) { this.name = name; } public boolean checkBridge(Hawkins h) { if (h.status==true) { System.out.println("Bridge present at " + h.name); return true; } else { System.out.println("No Bridge present at " + h.name); return false; } } } </pre>	

```

    }
}

public void open() {
    if (status==false){
        status = true;
        System.out.println("Bridge from "+name+" is Open");
    }
}
}

```

```

// Parent Class
class UpsideDown extends Hawkins{
    // Write Your Code Here
}

```

```

// Child Class
class DarkDimension extends UpsideDown {
    public DarkDimension(String name) {
        super(name);
    }

    public void runExperiment(UpsideDown portal) {
        if (!this.checkBridge(portal)) {
            System.out.println("Cannot run experiment.");
        }
        else {
            if (portal.getBridge1() != null) {
                portal.activate(portal.getBridge1());
                System.out.println("Experiment executed successfully!");
            } else if (portal.getBridge2() != null) {
                portal.activate(portal.getBridge2());
                System.out.println("Experiment executed successfully!");
            } else {
                System.out.println("No experiment found!");
            }
        }
    }
}
}

```

CSE 230

CSE230: Discrete Mathematics

Practice Sheet 1: Propositional Logic Basics

Q1	Let p, q and r be the propositions "The student has submitted the assignment", "The professor has graded the assignment", "The student receives a passing grade" respectively. Express each of these compound propositions as an English sentence. $p \wedge \neg q$ $p \vee r$ $p \rightarrow q \wedge r$ $p \leftrightarrow q$ $\neg r \wedge (\neg p \vee \neg q)$
Q2	Let p, q, r and s be the propositions p: The student has submitted the research paper. q: The student has attended all workshops. r: The student has participated in group projects. s: The student will receive a degree with honors. Express each of these compound propositions as an English sentence. $p \vee (q \wedge \neg s)$ $\neg p \vee (q \wedge r)$ $(p \wedge r) \rightarrow s$ $(p \wedge q) \leftrightarrow r$ $(p \rightarrow q) \wedge (r \rightarrow s)$ $\neg(p \wedge s) \vee (q \wedge r)$
Q3	Let p and q be the propositions: p: The weather is sunny. q: We will go to the beach. Write these propositions using p and q and logical connectives (including negations and exclusive OR): - It is sunny and we go to the beach. - It is sunny but we do not go to the beach. - It is not sunny and we do not go to the beach. - Either it is sunny, or we go to the beach (or both). - If it is sunny, then we go to the beach. - Either it is sunny or we go to the beach, but not both, and if it is sunny, then we do not go to the beach - That it is sunny is necessary and sufficient for going to the beach.
Q4	Let p, q, and r be the propositions: p: The campground is open for visitors. q: Campfires are allowed in the campground. r: The river nearby is safe for swimming. Write these propositions using p, q, and r and logical connectives (including negations). - The campground is open for visitors, but campfires are not allowed. - Campfires are allowed, and the river is safe for swimming, but the campground is not open for visitors. - If the campground is open, then campfires are allowed if and only if the river is safe for swimming. - Campfires are not allowed, but the campground is open and the river is safe for swimming. - For campfires to be allowed, it is necessary and sufficient that the campground be open and that the river is not safe for swimming. - Campfires are not allowed whenever the campground is open and the river is safe for swimming.

Q5	Let p and q be the propositions: p: The project is complete. q: The team has been informed. Express each of these compound propositions as an English sentence. $\neg p$ $p \vee q$ $\neg p \wedge q$ $q \rightarrow p$ $\neg q \rightarrow \neg p$ $\neg p \rightarrow \neg q$ $p \leftrightarrow q$ $\neg q \vee (\neg p \wedge q)$
Q6	Let p, q, and r be the propositions: p: You solve the practice sheet. q: You pass the test. r: You get a good grade. Express each of these compound propositions as an English sentence. $\neg q \leftrightarrow r$ $q \rightarrow \neg r$ $p \vee q \vee r$ $(p \rightarrow \neg r) \vee (q \rightarrow \neg r)$ $(p \wedge q) \vee (\neg q \wedge r)$ $(q \wedge r) \leftrightarrow (p \oplus \neg r)$
Q7	Determine the converse, contrapositive, and inverse of each of these conditional statements (Apply De Morgan's Laws for conjunction and disjunction where necessary): - If it rains tomorrow, the outdoor concert will be canceled. - An angle is a right angle only if it measures 90 degrees. - A polygon is a triangle only if it has three sides. - You are eligible for the scholarship if your CGPA is at least 3.7 and you have completed 30 credits. - A figure is a square only if it has four equal sides and four right angles. - The machine will shut down if the temperature exceeds 100°C and the system pressure drops below the minimum threshold.
Q8	How many rows appear in a truth table for each of these compound propositions? $p \rightarrow \neg p(p \vee \neg r) \wedge (q \vee \neg s)$ $q \vee p \vee \neg s \vee \neg r \vee \neg t \vee u$ $(p \wedge r \wedge t) \leftrightarrow (q \wedge t)$
Q9	Construct truth tables for each of the following compound propositions: $p \wedge \neg p$ $p \vee \neg p$ $(p \vee \neg q) \rightarrow q$ $(p \vee q) \wedge r$ $(p \wedge q) \vee r$ $(p \rightarrow q) \leftrightarrow (\neg q \rightarrow \neg p)$ $(p \vee (q \wedge r)) \rightarrow (r \vee p)$ $\neg(p \wedge q) \leftrightarrow (\neg p \vee \neg q)$ $(p \rightarrow (q \vee r)) \wedge (q \rightarrow r)$ $((p \vee q) \wedge (r \vee s)) \rightarrow (p \wedge s)$

Q10	Show that each of the following compound propositions is a tautology using the truth table. a) $p \vee \neg p$ b) $(p \rightarrow q) \wedge (q \rightarrow r) \rightarrow (p \rightarrow r)$ c) $(p \leftrightarrow q) \vee (\neg p \leftrightarrow q)$ d) $(p \vee q) \wedge (\neg p \vee r) \rightarrow (q \vee r)$ e) $(\neg p \rightarrow q) \rightarrow ((\neg q \rightarrow p) \vee r)$ f) $\neg(p \wedge \neg p) \vee (q \wedge r)$
Q11	Show that each of these pairs of propositions are logically equivalent using truth tables. a) $p \rightarrow q$ and $\neg q \rightarrow \neg p$ b) $p \leftrightarrow q$ and $\neg p \leftrightarrow \neg q$ c) $(p \wedge q) \vee (\neg p \wedge \neg q)$ and $p \leftrightarrow q$ d) $p \rightarrow (q \wedge r)$ and $(p \rightarrow q) \wedge (p \rightarrow r)$ e) $\neg(p \leftrightarrow q)$ and $p \leftrightarrow \neg q$ f) $p \wedge (q \vee r)$ and $(p \wedge q) \vee (p \wedge r)$ g) $\neg(p \vee q)$ and $\neg p \wedge \neg q$ h) $p \vee (q \wedge r)$ and $(p \vee q) \wedge (p \vee r)$ i) $p \rightarrow (q \rightarrow r)$ and $(p \wedge q) \rightarrow r$ j) $(p \vee q) \rightarrow r$ and $\neg r \rightarrow (\neg p \wedge \neg q)$
Q12	Show that, "In the Olympic Games, either the USA will win the gold medal or both Canada and Australia will win medals" and "The USA will win a medal or Canada will win a medal, and the USA will win a medal or Australia will win a medal" are logically equivalent using truth tables.
Q13	Let $D(x)$ denote "x donates to charity regularly," where the domain consists of all people in a city. Express each of these statements in English: a) $\exists x D(x)$ b) $\forall x D(x)$ c) $\exists x \neg D(x)$ d) $\forall x \neg D(x)$
Q14	Translate these statements into English, where $T(x)$ means "x is a teacher" and $R(x)$ means "x reads every day." The domain consists of all individuals in a neighborhood. a) $\forall x (T(x) \rightarrow R(x))$ b) $\exists x (T(x) \wedge R(x))$ c) $\forall x (T(x) \wedge R(x))$ d) $\exists x (T(x) \rightarrow R(x))$
Q15	Let $E(x)$ represent "x is enrolled in an art class," $B(x)$ means "x is in a beginner's course," $G(x)$ means "x has a good grade," and $S(x)$ represent "x likes studying." The domain consists of some students in an art class. Translate each of these statements into English. a) $\forall x (E(x) \rightarrow G(x))$ b) $\exists x (B(x) \wedge S(x))$ c) $\neg \forall x (B(x) \rightarrow (G(x) \vee S(x)))$
Q16	Translate each of these statements into logical expressions using predicates, quantifiers, and logical connectives. Let $F(x)$ denote "x is a family member," and $K(x)$ denote "x is kind." a) At least one family member is kind. b) All of your family members are kind. c) Not every family member is kind. d) Everyone is either a family member or is kind. e) Not everyone is a family member or there is someone who is not kind.

Q17	Let $L(x)$ represent "x knows how to play an instrument," and let $M(x)$ represent "x is a member of a band." Express each of these statements in terms of $L(x)$, $M(x)$, quantifiers, and logical connectives. The domain for quantifiers consists of all students at a school. - There is a student at your school who knows how to play an instrument and is a member of a band. - There is a student at your school who knows how to play an instrument but is not a member of a band. - Every student at your school either knows how to play an instrument or is a member of a band. - No student at your school knows how to play an instrument or is a member of a band.
Q18	Let $P(x)$, $Q(x)$, and $R(x)$ be the statements "x is a scientist," "x is curious," and "x likes to experiment," respectively. Express each of these statements using quantifiers, logical connectives, and $P(x)$, $Q(x)$, and $R(x)$, where the domain consists of all people. - All scientists are curious. - All curious people like to experiment. - No scientists like to experiment.
Q19	In a university, we define the following predicates: $S(x)$: x is a student. $P(x)$: x is a professor. $C(y)$: y is a course. $T(x,y)$: x takes course y. $E(x,y)$: x teaches course y. Express each of these statements using quantifiers, logical connectives. - Every student takes at least one course. - For every professor, there exists a course that they teach. - There exists a student who takes every course. - For every course, there exists a student who is enrolled in that course. - Every professor teaches at least one student in their course. - For every student, there exists a professor who teaches at least one course they are taking.
Q20	State the converse, contrapositive, and inverse of each of these conditional statements: - The ground will be wet unless it is not raining. - Having fuel is necessary for running the car. - Being sunny is sufficient for the picnic to be enjoyable. - I wake up whenever the alarm rings. - The streets get wet when it rains.
Q21	Let p and q be the propositions: p: The security system is activated. q: The alarm will sound. r: There is a breach. Write these propositions using p, q, and logical connectives (including negations). - The security system is activated, but the alarm does not sound. - The alarm sounds, and the security system is not activated. - If the security system is activated, then the alarm will sound if and only if there is a breach. - For the alarm to sound, it is necessary and sufficient that the security system is activated and there is a breach.

Q22	Let r and s be the propositions: r: I studied for the exam. s: I passed the exam. Express each of these propositions as an English sentence: a) $\neg r$ b) $r \leftrightarrow s$ c) $\neg r \oplus s$ d) $(r \leftrightarrow s) \wedge (\neg r \oplus s)$
Q23	Let p and q be the propositions p: You drive over 65 miles per hour. q: You get a speeding ticket. Write these propositions using p and q and logical connectives (including negations). a) You do not drive over 65 miles per hour. b) You drive over 65 miles per hour, but you do not get a speeding ticket. c) You will get a speeding ticket if you drive over 65 miles per hour. d) If you do not drive over 65 miles per hour, then you will not get a speeding ticket. e) Driving over 65 miles per hour is sufficient for getting a speeding ticket. f) You get a speeding ticket, but you do not drive over 65 miles per hour. g) Whenever you get a speeding ticket, you are driving over 65 miles per hour.
Q24	State the converse, contrapositive, and inverse of each of these conditional statements. a) If it snows today, I will ski tomorrow. b) I come to class whenever there is going to be a quiz. c) A positive integer is a prime only if it has no divisors other than 1 and itself.
Q25	Show that $\neg(p \wedge q) \equiv \neg p \vee \neg q$.
Q26	Show that the proposition is tautology $p \rightarrow (p \wedge (q \rightarrow p))$
Q27	Construct the truth table for $(p \rightarrow q) \leftrightarrow (\neg q \rightarrow \neg p)$
Q28	Construct the truth table for $(p \oplus q) \rightarrow (p \oplus \neg q)$
Q29	Show that, "In the FIFA World Cup either Germany will reach the final or England and Argentina will reach the final" and "Germany or England will reach the final, and Germany or Argentina will reach the final" are logically equivalent.
Q30	Let $P(x)$ be the statement "x spends more than five hours every weekday in class," where the domain for x consists of all students. Express each of these quantifications in English. a) $\exists x P(x)$ b) $\forall x P(x)$ c) $\exists x \neg P(x)$ d) $\forall x \neg P(x)$
Q31	Translate these statements into English, where $C(x)$ is "x is a comedian" and $F(x)$ is "x is funny" and the domain consists of all people. a) $\forall x (C(x) \rightarrow F(x))$ b) $\forall x (C(x) \wedge F(x))$ c) $\exists x (C(x) \rightarrow F(x))$ d) $\exists x (C(x) \wedge F(x))$

Q32	Translate these statements into English, where $A(x)$: x teaches CSE230 $T(x)$: x likes STA201 $F(x)$: x has a Facebook page $C(x)$: x likes to cook The domain consists of all faculties in a university. a) $\forall x(T(x) \rightarrow F(x))$ b) $\exists x(T(x) \wedge A(x))$ c) $\neg \forall x(T(x) \rightarrow (F(x) \vee C(x)))$
Q33	Translate each of these statements into logical expressions using predicates, quantifiers, and logical connectives. a) No one is perfect. b) Not everyone is perfect. c) All your friends are perfect. d) At least one of your friends is perfect. e) Everyone is your friend and is perfect. f) Not everybody is your friend or someone is not perfect.
Q34	Let $P(x)$ be the statement "x can speak English" and let $Q(x)$ be the statement "x knows programming language Python." Express each of these sentences in terms of $P(x)$, $Q(x)$, quantifiers, and logical connectives. The domain for quantifiers consists of all students at your school. (a) There is a student at your school who can speak English and who knows Python. (b) There is a student at your school who can speak English but who doesn't know Python. (c) Every student at your school either can speak English or knows Python. (d) No student at your school can speak English or knows Python.
Q35	Let $P(x)$, $Q(x)$, and $R(x)$ be the statements "x is a professor," "x is ignorant," and "x is vain," respectively. Express each of these statements using quantifiers; logical connectives; and $P(x)$, $Q(x)$, and $R(x)$, where the domain consists of all people. a) No professors are ignorant. b) All ignorant people are vain. c) No professors are vain.
Q36	Let $C(x)$ means "x is a core course," $T(x,y)$ means "Student y has taken course x," and $G(y)$ means "Student y has met graduation requirements." The domain consists of all students and courses. Formulate the following statements using quantifiers: a) For every student, if they have taken all core courses, they meet the graduation requirements. b) There exists a student who has not taken every core course but still meets the graduation requirements. c) For every core course, there is at least one student who has not taken it.
Q37	Let $S(x)$ mean "x submitted a research paper," $R(x,y)$ mean "Reviewer y read x's paper," and $A(x,y)$ mean "Reviewer y approved x's paper." The domain consists of all researchers and reviewers. Express the following statements using nested quantifiers: a) Every researcher has at least one reviewer who has read and approved their paper. b) There exists a reviewer who has read every paper but has not approved at least one of them. c) For every research paper, if it was read by a reviewer, then there exists another reviewer who did not read it.

Q38	Let $P(x, y)$ mean "Person x can work on a project with Person y," $Q(x)$ means "Person x knows Python," and $R(x)$ means "Person x is a project manager." The domain consists of all people in a software development team. Express the following statements using quantifiers and logical connectives: - There exists a project manager who can work with everyone on the team who knows Python. - For every person who knows Python, there is at least one other person they can work with who is not a project manager. - For every person, if they are a project manager, there exists at least one person on the team with whom they cannot work.
Q39	Let $S(x)$ represent "x is skilled in software development," $M(x)$ represent "x has managerial experience," and $Q(x, y)$ represent "Person x qualifies for Job y." The domain consists of all applicants for the job, and there are multiple types of jobs. - Describe, using quantifiers, the condition where there exists a job such that only applicants with both software skills and managerial experience qualify. - Write the condition, using quantifiers, for a job that requires at least one applicant without software development skills but with managerial experience to qualify.
Q40	Let $A(x)$ represent "x is an applicant," $S(x)$ represent "x has strong academic scores," and $E(x)$ represent "x participates in extracurricular activities." The domain consists of all college applicants. - Express with quantifiers the requirement that all applicants participate in extracurricular activities only if they have strong academic scores, but there is at least one applicant who has strong academic scores and does not participate in extracurricular activities. - Write the condition that for an applicant to be admitted, they must either have strong academic scores or participate in extracurricular activities, but not necessarily both.

CSE230: Discrete Mathematics

Practice Sheet 2: Proofs: Direct, Indirect, Contradiction, Induction

Q1	Prove that, for all integers n , if $n^3 + 5$ is odd then n is even.
Q2	Prove the following: Suppose $a, b \in \mathbb{Z}$. If $a + b \geq 19$, then $a \geq 10$ or $b \geq 10$.
Q3	Prove the following: Suppose a, b , and c are positive real numbers. If $ab = c$ then $a \leq \sqrt{c}$ or $b \leq \sqrt{c}$.
Q4	Prove that the sum of a rational number and an irrational number is irrational.
Q5	Prove that $\sqrt{5}$ is irrational. (Same as: Prove that if x is a rational number and $x^2 = 5$, then a contradiction arises)
Q6	Prove that for any two odd integers a and b , their sum $a+b$ is even.
Q7	Prove the following: Suppose $a, b, c \in \mathbb{Z}$. If $a^2 + b^2 = c^2$, then a or b is even.
Q8	Prove the following: If a and b are positive real numbers, then $a + b \geq 2\sqrt{ab}$.
Q9	Prove by mathematical induction that if n is a positive integer then $1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4}$
Q10	Prove by mathematical induction that if n is a positive integer then $(1 \times 2) + (2 \times 3) + (3 \times 4) + \dots + (n \times (n+1)) = \frac{n(n+1)(n+2)}{3}$
Q11	Prove by mathematical induction that $\sum_{k=1}^n k^2 = \frac{1}{6}n(n+1)(2n+1)$ for all $n \in \mathbb{N}$.
Q12	Prove by mathematical induction that if n is a positive integer then $1 + 5 + 9 + 13 + \dots + (4n-3) = \frac{n(4n-2)}{2}$
Q13	Prove by mathematical induction that, $\sum_{r=1}^n r(r+3) = \frac{1}{3}n(n+1)(n+5)$ where $n > 0$ and $n \in \mathbb{N}$.
Q14	Prove by mathematical induction that, $\sum_{r=1}^n (r-1)(r+1) = \frac{1}{6}n(n-1)(2n+5)$ where $n > 0$ and $n \in \mathbb{N}$.
Q15	Prove by mathematical induction that, $\sum_{r=2}^n r^2(r-1) = \frac{1}{12}n(n-1)(n+1)(3n+2)$ where $n > 1$ and $n \in \mathbb{N}$.

Q16	Prove by mathematical induction that, $\sum_{r=1}^n (2r-1)^2 = \frac{1}{3}n(2n-1)(2n+1)$ where $n > 0$ and $n \in \mathbb{N}$.
Q17	Prove by mathematical induction that, $\sum_{r=1}^n r(3r-1) = n^2(n+1)$ where $n > 0$ and $n \in \mathbb{N}$.
Q18	Prove by mathematical induction that, $\sum_{r=1}^n \frac{1}{r(r+1)} = \frac{n}{n+1}$ where $n > 0$ and $n \in \mathbb{N}$.
Q19	Prove by mathematical induction that, $\sum_{r=1}^n (3^{r-1}) = \frac{3^n - 1}{2}$ where $n > 0$ and $n \in \mathbb{N}$.
Q20	Prove by mathematical induction that, $\sum_{r=1}^n r \cdot 2^r = 2 + (n-1)2^{n+1}$ where $n > 0$ and $n \in \mathbb{N}$.
Q21	Prove by mathematical induction that, $\sum_{r=1}^n (r+1) \cdot 2^{(r-1)} = n \cdot 2^n$ where $n > 0$ and $n \in \mathbb{N}$.
Q22	Prove by mathematical induction that, $-1 + 1 + 5 + 11 + \dots + \{n(n-1) - 1\} = \frac{1}{3}n(n+2)(n-2)$ where $n > 0$ and $n \in \mathbb{N}$.
Q23	Prove by mathematical induction that, $4^n + 6n + 8$ is divisible by 18 where $n > 0$ and $n \in \mathbb{N}$.
Q24	Prove by mathematical induction that, $5^{2n} + 3n - 1$ is divisible by 9 where $n > 0$ and $n \in \mathbb{N}$.
Q25	Prove by mathematical induction that, $7^n + 5$ is divisible by 6 where $n \in \mathbb{N}$.
Q26	Prove by mathematical induction that, $7^{2n-1} + 1$ is divisible by 8 where $n \in \mathbb{N}$.
Q27	Prove by mathematical induction that, $4^n + 6n - 1$ is divisible by 9 where $n \in \mathbb{N}$.
Q28	Prove by mathematical induction that, $5^n + 8n + 3$ is divisible by 4 where $n \in \mathbb{N}$.

Q29	Prove by mathematical induction that, $3^{4n} + 2^{4n+2}$ is divisible by 5 where $n \in \mathbb{N}$.
Q30	Prove by mathematical induction that, $9^n - 5^n$ is divisible by 4 where $n \in \mathbb{N}$.
Q31	Prove by mathematical induction that, $(4n + 3) \times 5^n - 3$ is divisible by 16 where $n \in \mathbb{N}$.
Q32	Prove by mathematical induction that, the sum of the cubes of any three consecutive positive integers is always divisible by 9 .
Q33	Prove by mathematical induction that, $15^n - 8^{n-2}$ is divisible by 7 where $n > 1$ and $n \in \mathbb{N}$.
Q34	Prove by mathematical induction that, $(2n + 1)7^n + 11$ is divisible by 4 where $n > 1$ and $n \in \mathbb{N}$.
Q35	Prove by mathematical induction that, $24 \times 2^{4n} + 3^{4n}$ is divisible by 5 where $n > 1$ and $n \in \mathbb{N}$.
Q36	Prove by mathematical induction that, $4 \times 7^n + 3 \times 5^n + 5$ is divisible by 12 where $n \in \mathbb{N}$.
Q37	Prove by mathematical induction that, $(2n + 1)7^n - 1$ is divisible by 4 where $n > 1$ and $n \in \mathbb{N}$.
Q38	Prove by mathematical induction that, $4^{n+1} + 5^{2n-1}$ is divisible by 21 where $n > 0$ and $n \in \mathbb{N}$.
Q39	Prove by mathematical induction that, $2^n + 6^n$ is divisible by 8 where $n > 0$ and $n \in \mathbb{N}$.
Q40	Prove by mathematical induction that, $5^{n-1} + 11^n$ is divisible by 6 where n is a positive integer.
Q41	Prove by mathematical induction that, $5^{n+1} - 4n - 5$ is divisible by 16 where $n \in \mathbb{N}$.

CSE230: Discrete Mathematics

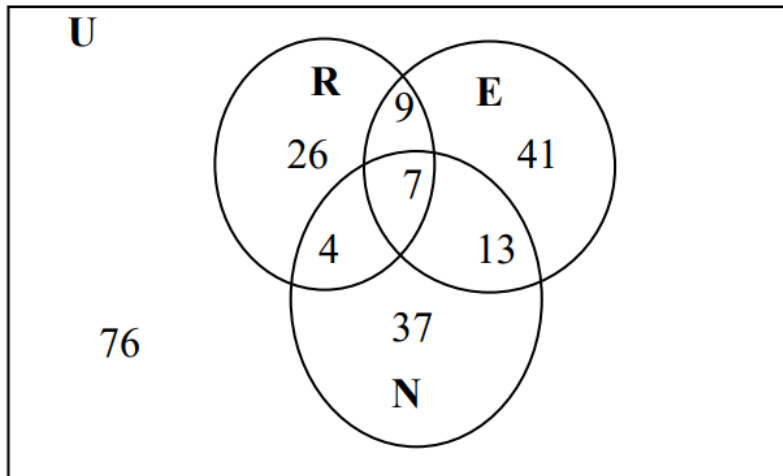
Practice Sheet 3: Sets

Q1	Determine whether each of these pairs of sets are equal. a) $\{1, 3, 3, 3, 5, 5, 5, 5, 5\}, \{5, 3, 1\}$ b) $\{\{1\}\}, \{1, \{1\}\}$ c) $\emptyset, \{\emptyset\}$
Q2	How many elements does each of these sets have where a and b are distinct elements? Here P represents the power set. a) $P(\{a, b, \{a, b\}\})$ b) $P(\{\emptyset, a, \{a\}, \{\{a\}\}\})$ c) $P(P(\emptyset))$
Q3	List all the subsets of: a) $\{1, 2, 3\}$ b) $\{\phi, \lambda, \Delta, \mu\}$ c) $\{\emptyset\}$
Q4	What is the cardinality of each of these sets? a) $\{a\}$ b) $\{\{a\}\}$ c) $\emptyset$ d) $\{\emptyset\}$ e) $\{\emptyset, \{\emptyset\}\}$ f) $\{\emptyset, \{\emptyset\}, \{\emptyset, \{\emptyset\}\}\}$
Q5	Find the power set of each of these sets, where p and q are distinct elements. a) $\{q\}$ b) $\{p, q\}$ c) $\{\emptyset, \{\emptyset\}\}$
Q6	Find if the following statements are True or False. a) $7 \in \{6, 7, 8, 9\}$ b) $5 \notin \{6, 7, 8, 9\}$ c) $\{2\} \not\subseteq \{1, 2\}$ d) $\emptyset \not\subseteq \{\alpha, \beta, x\}$ e) $\emptyset = \{\emptyset\}$
Q7	Let $A = \{1, 5, 31, 56, 101\}$, $B = \{22, 56, 5, 103, 87\}$, $C = \{41, 13, 7, 101, 48\}$ and $D = \{1, 3, 5, 7, \dots\}$ Give the sets resulting from: a) $A \cap B$ b) $C \cup A$ c) $C \cap D$

	d) $(A \cup B) \cup (C \cup D)$
Q8	Assume that the universal set is the set of all integers. Let, $A = \{-7, -5, -3, -1, 1, 3, 5, 7\}$ $B = \{x \in \mathbb{Z} \mid x^2 < 9\}$ $C = \{2, 3, 4, 5, 6\}$ $D = \{x \in \mathbb{Z} \mid x \leq 9\}$ In each of the following fill in the blank with most appropriate symbol from $\in, \notin, \subset, =, \neq, \subseteq$, so that resulting statement is true. $A \underline{\hspace{1cm}} D$ $3 \underline{\hspace{1cm}} B$ $9 \underline{\hspace{1cm}} D$ $\{2\} \underline{\hspace{1cm}} C^c$ $\emptyset \underline{\hspace{1cm}} D$ $A \underline{\hspace{1cm}} C$ $B \underline{\hspace{1cm}} C$ $C \underline{\hspace{1cm}} D$ $0 \underline{\hspace{1cm}} A \cap D$ $0 \underline{\hspace{1cm}} A \cup D$
Q9	Let $A = \{r, e, a, s, o, n, i, g\}$, $B = \{m, a, t, h, e, t, i, c, l\}$ and C = the set of vowels. Calculate: $A \cup B \cup C$. $A \cap B$. C^c.
Q10	Given sets A and B in a universe U, draw the Venn diagrams of each of these sets. $A \rightarrow B = \{x \in U \mid x \in A \rightarrow x \in B\}$ $A \leftrightarrow B = \{x \in U \mid x \in A \leftrightarrow x \in B\}$
Q11	Find A^3 if : $A = \{0\}$. $A = \{0, a\}$
Q12	Let $A = \{a, b, c\}$, $B = \{x, y\}$, and $C = \{0, 1\}$. Find $A \times B \times C$. $C \times B \times A$. $C \times A \times B$. $B \times B \times B$.
Q13	Given sets $A = \{1, 2, 3\}$ and $B = \{x, \{y\}\}$, find $A \times B$ and $B \times A$. Are the sets equal? What is the cardinality of the cartesian product $B \times A$?
Q14	Use a Venn diagram to illustrate the relationship $A \subseteq B$ and $B \subseteq C$.
Q15	Use a Venn diagram to illustrate the relationships $A \subset B$ and $B \subset C$.

Q1 6	Suppose that A, B, and C are sets such that $A \subseteq B$ and $B \subseteq C$. Show that $A \subseteq C$.
Q1 7	Let $A = \{1, 2, 3, 4, 5\}$ and $B = \{0, 3, 6\}$. Find a) $A \cup B$. b) $A \cap B$. c) $A - B$. d) $B - A$.
Q1 8	Let $A = \{a, b, c, d, e\}$ and $B = \{a, b, c, d, e, f, g, h\}$. Find a) $A \cup B$. b) $A \cap B$. c) $A - B$. d) $B - A$.
Q1 9	Prove the second De Morgan law that if A and B are sets, then $(A \cup B)^c = A^c \cap B^c$ by a) showing each side is a subset of the other side. b) using a membership table.
Q2 0	Use a membership table to show that $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$.
Q2 1	Using set identities, show that $(A \cup (B \cap C))^c = (C^c \cup B^c) \cap A^c$
Q2 2	Prove the complement laws using membership table (Consider True = Universal set, False = Empty set) a) $A \cup A^c = U$. b) $A \cap A^c = \emptyset$.
Q2 3	Let A and B be two finite sets such that $n(A) = 20$, $n(B) = 28$ and $n(A \cup B) = 36$, find $n(A \cap B)$ where $n(S)$ representing the number of elements in the set S.
Q2 4	Let A, B, and C be sets. Show that a) $(A \cup B) \subseteq (A \cup B \cup C)$. b) $(A \cap B \cap C) \subseteq (A \cap B)$. c) $(A - B) - C \subseteq A - C$.
Q2 5	In a group of 200 students, 85 students are attending leadership training programs, 50 students are involved in a team-building workshop . There are 15 students who are both attending leadership training programs and participating in the team-building workshop. Determine the number of students who are involved in exactly one of the activities.

Q2
6



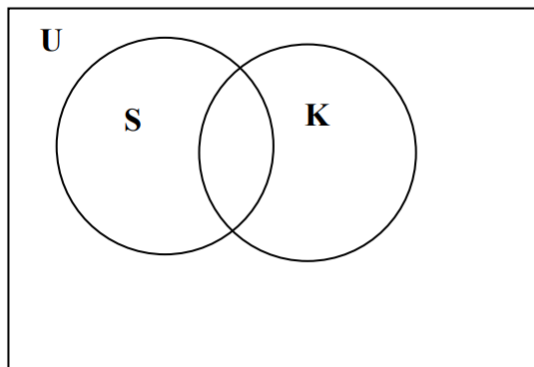
Use the diagram above to find the cardinality for each problem.

1. $n(N)$
2. $n[(R \cap E) \cup N]$
3. $n(E')$
4. $n(E \setminus R)$
5. $n(U)$
6. $n(R \cup E \cup N)$

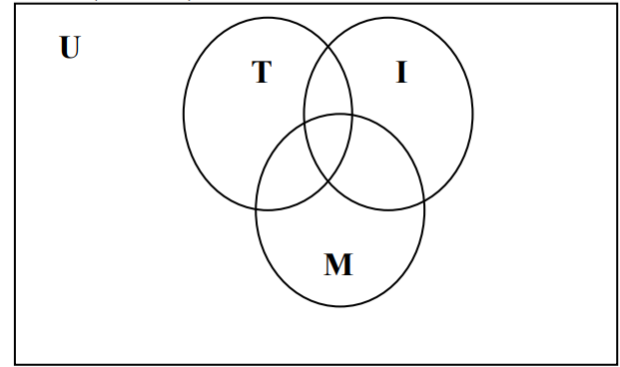
Q2
7

Illustrate each of the following by shading the Venn diagrams below.

a. $S' \cap K$

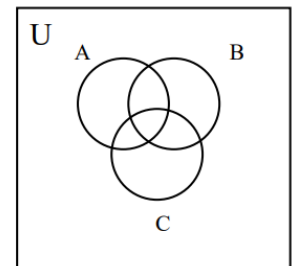
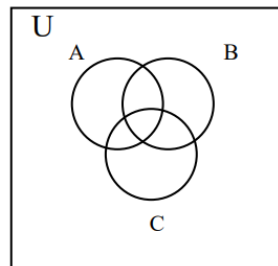
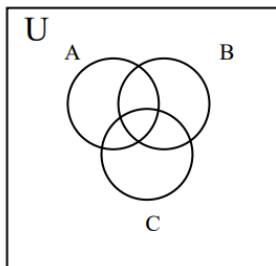


b. $(T \cup M)'$



Q2
8

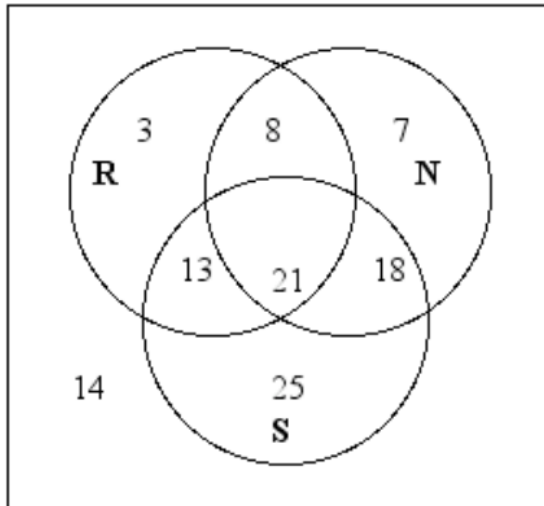
Use the Venn diagram to work the following problem in three steps $A' \cup (B \cap C)$.



Q29

A teacher surveyed her class of 43 students to find out how they prepared for their last test. She found that 24 students made flash cards, 14 studied their notes, and 27 completed the review assignment. Of the entire class of 43 students, 12 completed the review and made flash cards, 9 completed the review and studied their notes, and 7 made flash cards and studied their notes, while only 5 students completed all three of these tasks. The remaining students did not do any of these tasks. Represent this information in a Venn Diagram.

A number of deer were surveyed about activities that they enjoy. The results are summarized in the Venn diagram below:



R: enjoy running
N: enjoy nibbling
S: enjoy staring into headlights

Q30

How many deer :

- Enjoy running or staring into headlights?
- Enjoy nibbling and running?
- Enjoy exactly one of these activities?
- Enjoy staring into headlights but not running?
- Enjoy nibbling?
- Don't enjoy any of the activities?
- Enjoy any two activities only?

Q31

Draw a Venn diagram using 3 sets Q, T and P. None of these 3 sets are pairwise disjoint. Moreover, $P \cap Q \cap T \neq \phi$. How many disjoint regions are there? Indicate which regions fall under $(T \cap Q)^c - P$.

Q32

A survey of 1,200 people was conducted on three types of sports they enjoy: soccer, basketball, and baseball. Out of these, 670 enjoy soccer, 510 enjoy basketball, and 760 enjoy baseball. It was found that 320 people enjoy all three sports, while 45 people do not enjoy any of the three sports.

- How many people enjoy exactly two of these sports?
- How many people enjoy at least two of these sports?
- How many people enjoy exactly one of these sports?

CSE230: Discrete Mathematics

Practice Sheet 4: Sequence, Summation, Recurrence Solution

Q1	What is the term a_8 of the sequence $\{a_n\}$ if a_n equals a) 2^{n-1} ? b) 7 ? c) $1 + (-1)^n$? d) $-(-2)^n$?
Q2	What are the terms a_0, a_1, a_2 and a_3 of the sequence $\{a_n\}$, where a_n equals a) $(-2)^n$? b) 3 ? c) $7 + 4^n$? d) $2^n + (-2)^n$?
Q3	List the first 10 terms of each of these sequences. a) The sequence that begins with 2 and in which each successive term is 3 more than the preceding term. b) The sequence that lists each positive integer three times, in increasing order. c) The sequence that lists the odd positive integers in increasing order, listing each odd integer twice. d) The sequence whose n th term is $n! - 2^n$. e) The sequence that begins with 3, where each succeeding term is twice the preceding term. f) The sequence whose first term is 2, second term is 4, and each succeeding term is the sum of the two preceding terms. g) The sequence where the n th term is the number of letters in the English word for the index n . h) The sequence obtained by starting with 10 and obtaining each term by subtracting 3 from the previous term. i) The sequence whose n th term is the sum of the first n positive integers. j) The sequence whose n th term is $3^n - 2^n$.
Q4	Find the first five terms of the sequence defined by each of the following recurrence relations and initial conditions given below: a) $a_n = 6a_{n-1}, a_0 = 2$. b) $a_n = a_{n-1}^2, a_1 = 2$. c) $a_n = a_{n-1} + 3a_{n-2}, a_0 = 1, a_1 = 2$. d) $a_n = na_{n-1} + n^2a_{n-2}, a_0 = 1, a_1 = 1$. e) $a_n = a_{n-1} + a_{n-3}, a_0 = 1, a_1 = 2, a_2 = 0$.

Q5	Show that the sequence $\{a_n\}$ is a solution of the recurrence relation $a_n = -3a_{n-1} + 4a_{n-2}$ if a) $a_n = 0$. b) $a_n = 1$. c) $a_n = (-4)^n$. d) $a_n = 2(-4)^n + 3$
Q6	Is the sequence $\{a_n\}$ a solution of the recurrence relation $a_n = 8a_{n-1} - 16a_{n-2}$ if a) $a_n = 0$? b) $a_n = 1$? c) $a_n = 2^n$? d) $a_n = 4^n$? e) $a_n = n4^n$? f) $a_n = 2 \cdot 4^n + 3n4^n$? g) $a_n = (-4)^n$? h) $a_n = n^24^n$?
Q7	Show that the sequence $\{a_n\}$ is a solution of the recurrence relation $a_n = a_{n-1} + 2a_{n-2} + 2n - 9$ if a) $a_n = -n + 2$. b) $a_n = 5(-1)^n - n + 2$. c) $a_n = 3(-1)^n + 2^n - n + 2$. d) $a_n = 7 \cdot 2^n - n + 2$.
Q8	Find the solution to each of these recurrence relations with the given initial conditions. Use an iterative approach. a) $a_n = -a_{n-1}$, $a_0 = 5$. b) $a_n = a_{n-1} + 3$, $a_0 = 1$. c) $a_n = a_{n-1} - n$, $a_0 = 4$. d) $a_n = 2a_{n-1} - 3$, $a_0 = -1$. e) $a_n = (n + 1)a_{n-1}$, $a_0 = 2$. f) $a_n = 2na_{n-1}$, $a_0 = 3$. g) $a_n = -a_{n-1} + n - 1$, $a_0 = 7$.

Q9	A person deposits \$1000 in an account that yields 9% interest compounded annually. a) Set up a recurrence relation for the amount in the account at the end of n years. b) Find an explicit formula for the amount in the account at the end of n years. c) How much money will the account contain after 100 years?
Q10	Suppose that the number of bacteria in a colony triples every hour. a) Set up a recurrence relation for the number of bacteria after n hours have elapsed. b) If 100 bacteria are used to begin a new colony, how many bacteria will be in the colony in 10 hours?
Q11	Assume that the population of the world in 2017 was 7.6 billion and is growing at the rate of 1.12% a year. a) Set up a recurrence relation for the population of the world n years after 2017. b) Find an explicit formula for the population of the world n years after 2017. c) What will the population of the world be in 2050?
Q12	A factory makes custom sports cars at an increasing rate. In the first month only one car is made, in the second month two cars are made, and so on, with n cars made in the nth month. a) Set up a recurrence relation for the number of cars produced in the first n months by this factory. b) How many cars are produced in the first year? c) Find an explicit formula for the number of cars produced in the first n months by this factory.
Q13	An employee joined a company in 2017 with a starting salary of \$50,000 per year. Every year this employee receives a raise of \$1000 plus 5% of the salary of the previous year. a) Set up a recurrence relation for the salary of this employee n years after 2017. b) What will the salary of this employee be in 2025? c) Find an explicit formula for the salary of this employee n years after 2017. (non-homogeneous recurrence)

Q14	For each of these lists of integers, provide a simple formula or rule that generates the terms of an integer sequence that begins with the first term in each given list. Assuming that your formula or rule is correct, determine the next three terms of the sequence. a) 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, ... b) 1, 2, 2, 3, 4, 4, 5, 6, 6, 7, 8, 8, ... c) 1, 0, 2, 0, 4, 0, 8, 0, 16, 0, ... d) 3, 6, 12, 24, 48, 96, 192, ... e) 15, 8, 1, -6, -13, -20, -27, ... f) 3, 5, 8, 12, 17, 23, 30, 38, 47, ... g) 2, 16, 54, 128, 250, 432, 686, ... h) 2, 3, 7, 25, 121, 721, 5041, 40321, ...
Q15	For each of these lists of integers, provide a simple formula or rule that generates the terms of an integer sequence that begins with the first term in each given list. Assuming that your formula or rule is correct, determine the next three terms of the sequence. a) 3, 6, 11, 18, 27, 38, 51, 66, 83, 102, ... b) 7, 11, 15, 19, 23, 27, 31, 35, 39, 43, ... c) 1, 10, 11, 100, 101, 110, 111, 1000, 1001, 1010, 1011, ... d) 1, 2, 2, 2, 3, 3, 3, 3, 3, 5, 5, 5, 5, 5, 5, 5, ... e) 0, 2, 8, 26, 80, 242, 728, 2186, 6560, 19682, ... f) 1, 3, 15, 105, 945, 10395, 135135, 2027025, 34459425, ... g) 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, ... h) 2, 4, 16, 256, 65536, 4294967296, ...
Q16	Find the closed formula for each of the following sequences by relating them to a well known sequence. Assume the first term given is a_1. (a) 2, 5, 10, 17, 26, ... (b) 0, 2, 5, 9, 14, 20, ... (c) 8, 12, 17, 23, 30, ... (d) 1, 5, 23, 119, 719, ...
Q17	Write down the first 5 terms (starting with a_0) for each of the sequences described below. Then give either a closed formula or a recursive definition for the sequence (whichever is NOT given in the problem). (a) $a_n = \frac{1}{2}(n^2 + n)$. (b) $a_n = 2a_{n-1} - a_{n-2}$ with $a_0 = 0$ and $a_1 = 1$. (c) $a_n = na_{n-1}$ with $a_0 = 1$.

Q18	Consider the three sequences below. For each sequence, find a recursive definition. How are these sequences related? (a) 2, 4, 6, 10, 16, 26, 42, . . . (b) 5, 6, 11, 17, 28, 45, 73, . . . (c) 0, 0, 0, 0, 0, 0, 0, . . .
Q19	Write down the first few terms of the sequence given by $a_1 = 3$; $a_n = 2a_{n-1} + 4$. Then find a recursive definition for the sequence 10, 24, 52, 108, . . .
Q20	Show that $a_n = 3 \cdot 2^n + 7 \cdot 5^n$ is a solution to the recurrence relation $a_n = 7a_{n-1} - 10a_{n-2}$. What would the initial conditions needed for this to be the closed formula of the sequence?
Q21	Find a closed formula of the sequence with recursive definition $a_n = 2a_{n-1} - a_{n-2}$ and with $a_1 = 1$ and $a_2 = 2$.
Q22	Suppose you draw n lines in the plane so that every pair of lines cross (no lines are parallel) and no three lines cross at the same point. This will create some number of regions in the plane, including some unbounded regions. Call the number of regions R_n. Find a recursive formula for the number of regions created by n lines, and justify why your recursion is correct.
Q23	Consider the sequence 5, 9, 13, 17, 21, . . . with $a_1 = 5$. (a) Give a recursive definition for the sequence. (b) Give a closed formula for the nth term of the sequence. (c) Is 2013 a term in the sequence? Explain. (d) How many terms does the sequence 5, 9, 13, 17, 21, . . . , 533 have? (e) Find the sum: $5 + 9 + 13 + 17 + 21 + \cdots + 533$.
Q24	Consider the sum $4 + 11 + 18 + 25 + \cdots + 249$. How many terms (summands) are in the sum?
Q25	Consider the sequence 1, 7, 13, 19, . . . , $6n + 7$. (a) How many terms are there in the sequence? Your answer will be in terms of n. (b) What is the second-to-last term? (c) Find the sum of all the terms in the sequence, in terms of n.
Q26	Find $5 + 15 + 45 + \cdots + 5 \cdot 3^{20}$.
Q27	Solve the recurrence relation $a_n = a_{n-1} + 2^n$ with $a_0 = 5$.
Q28	Show that 4^n is a solution to the recurrence relation $a_n = 3a_{n-1} + 4a_{n-2}$.

Q29	Find the solution to the recurrence relation $a_n = 3a_{n-1} + 4a_{n-2}$ with initial terms $a_0 = 2$ and $a_1 = 3$.
Q30	Think back to the magical candy machine at your neighborhood grocery store. Suppose that the first time a quarter is put into the machine 1 Skittle comes out. The second time, 4 Skittles, the third time 16 Skittles, the fourth time 64 Skittles, etc. Find a recursive formula for how many Skittles the n th customer gets.
Q31	Let a_n be the number of $1 \times n$ tile designs you can make using 1×1 squares available in 4 colors and 1×2 dominoes available in 5 colors. (a) First, find a recurrence relation to describe the problem. Explain why the recurrence relation is correct (in the context of the problem). (b) Write out the first 6 terms of the sequence $a_1, a_2, \dots$ (c) Solve the recurrence relation.
Q32	Solve the recurrence relation $a_n = 2a_{n-1} - a_{n-2}$. (a) What is the solution if the initial terms are $a_0 = 1$ and $a_1 = 2$? (b) What do the initial terms need to be in order for $a_9 = 30$?
Q33	The population of Utopia increases 5 percent each year. In 2000 the population was 10,000. What was the population in 1970?
Q34	Assume that a Database table has 0 records at time $n=0$. Suppose that at time n , $100n$ new records are populated into that table. The number of records increases 20 percent per minute. Write a recurrence relation and an initial condition that defines the number of records in the database table.
Q35	A wildlife reserve starts with 200 deers. The population increases by 15% annually due to reproduction and natural growth. Write a recurrence relation for the deer population at the end of n years.
Q36	A cup of coffee is initially at 90°C and is left to cool down in a room at a constant temperature of 25°C. The temperature of the coffee decreases by 10% of the difference between its current temperature and the room temperature every minute. Write a recurrence relation for the temperature of the coffee at the end of n minutes.
Q37	A new machine has been purchased for \$50,000. Its value depreciates by 12% of its current value each year. Generate a recurrence relation for the value of the machine at the end of n years.

Q38	A television channel holds a contest everyday. On day 1, the prize money is \$1000. For each day, the prize money increases by 10%, plus an additional \$5 from the CEO. Model recurrence relations for the contest. (Mention the base case). Solve the recurrence relation and find what will be the prize money on the 11th day according to the Contest.
Q 39	A person drops a tennis ball from the top of a 100 meter tall building. After each drop on the ground, the ball jumps up to the two-third of its previous height. Find a recurrence relation expressing the total distance covered by the ball before its nth drop on the ground.
Q40	Solve the following recurrence relation: $2a_{n+2} = 4a_{n+1} + 126a_n + 2 \cdot 5^n$. Here $a_0 = 0$, $a_1 = 5$.
Q41	Expand the following sums and products. That is, write them out the long way. (a) $\sum_{k=1}^{100} (3 + 4k).$ (b) $\sum_{k=0}^n 2^k.$ (c) $\sum_{k=2}^{50} \frac{1}{(k^2 - 1)}.$ (d) $\prod_{k=2}^{100} \frac{k^2}{(k^2 - 1)}.$ (e) $\prod_{k=0}^n (2 + 3k).$

CSE230: Discrete Mathematics

Practice Sheet 5: Counting

Q1	A password consists of 3 letters followed by 2 digits. How many different passwords can be created if letters and digits can repeat?
Q2	A club has 3 types of snacks and 2 types of drinks. Alternatively, they can offer 4 types of combo meals. How many choices does a customer have?
Q3	How many three-digit numbers can be formed using the digits 1, 2, 3, 4, 5 if repetition is not allowed and the number must be divisible by 5?
Q4	A committee of 3 people is to be formed from 5 men and 4 women such that there is at least one woman. How many such committees are possible?
Q5	Some companies offer 4 models of phones in 3 colors each. How many different phone options are available?
Q6	A library has 5 sections. Each section has 3 types of books. How many ways can a student choose one book if they are restricted to one section?
Q7	How many ways can you arrange the letters in the word "PERMUTATIONS" if the vowels must be together?
Q8	How many ways can you arrange the letters in the word "CALCULUS" if the first and the last character must be the same?
Q9	A 4-digit lock code is created using digits 1 to 6. How many such codes are possible if no digit can appear more than once?
Q10	In a row of 6 chairs, how many ways can 3 people sit if no two people can sit next to each other?
Q11**	How many ways can a 5-digit number be formed using the digits 0, 1, 2, 3, 4, 5 if the number must not start with 0 and repetition is not allowed? How many of those numbers will be even?
Q12	In a race of 8 people, how many ways can the top 3 positions be awarded if no ties are allowed?
Q13	In the last BRACU chess competition, there were a total of 105 participants. The rules were simple. A game of chess will be played between two of the participants randomly. The loser will be eliminated immediately. What was the total number of games played in that competition?

Q14	There are 17 points in a plane. 5 of them lie on the same straight line. No three other points lie on a single straight line. a) How many straight lines can be drawn using the points? b) How many different triangles can be drawn?
Q15*	In how many ways can the letter of the words "MATHEMATICS" be arranged maintaining the following conditions? a) Keeping the vowels together. b) Keeping the relative position of vowels and consonants unchanged. c) Starting and ending with the same letters. d) Starting with M. e) Ending with T. f) Starting with M or ending with T. g) Starting with a vowel. h) Keeping the relative position of C and S unchanged

CSE230: Discrete Mathematics

Practice Sheet 5: Pigeonhole Principle

Q16 *	An opaque container has 7 red balls, 12 yellow balls, 8 green balls, 5 blue balls and 21 black balls. What is the least number of balls you need to pick up from that container to ensure that, a) you have at least 3 balls of the same color b) you have at least 9 balls of the same color c) you have at least 2 red balls d) you have one ball of each color
Q17	The population of BRAC University students will become 300 million by 2030. There are 30 departments and each department allocates 10 million students. While forming a club, what is the minimum number of students that needs to be selected randomly so that at least 5 of the students come from the same department?
Q18	What is the minimum number of students, each of whom comes from one of the 64 districts of Bangladesh, who must be enrolled in a university to guarantee that there are at least 100 who come from the same district?
Q19	In one of the admission tests, a university decided to intake students such that at least 10 newly admitted students share the same birthday. What is the minimum number of students the university needs to admit in that intake? [Based on a real-life university]
Q20	How many numbers must be selected from the set $\{1, 3, 5, 7, 9, 11, 13, 15\}$ to guarantee that at least one pair of these numbers add up to 16?
Q21	Suppose that there are nine students in a discrete mathematics class at a small college. a) Show that the class must have at least five male students or at least five female students. b) Show that the class must have at least three male students or at least seven female students.
Q22	What is the minimum number of students required in a CSE230 class to be sure that at least six will receive the same grade if there are 13 possible grades, A+, A, A-, B+, B, B-, C+, C, C-, D+, D, D- and F?
Q23	What is the size of the smallest group of people where at least 9 people share the same birth month?
Q24 **	Show that for every integer n there is a multiple of n that has only 0s and 1s in its decimal expansion.

* symbol indicates the degree of difficulty

CSE230: Discrete Mathematics

Practice Sheet 6: Permutations and Combinations

Q1	In a class, there are 27 boys and 14 girls. The teacher wants to select 1 boy and 1 girl to represent the class for a function. In how many ways can the teacher make this selection?
Q2	(i) How many numbers are there between 99 and 1000 having 7 in the unit's place? (ii) How many numbers are there between 99 and 1000 having at least one of their digits 7?
Q3	In how many ways can 5 children be arranged in a line such that (i) two particular children of them are always together (ii) two particular children of them are never together.
Q4	How many rearrangements of the following words are possible? a. carrier, mississippi b. betterment , so that the vowels stay together. c. endeavour , so that not all the vowels are together. d. TRIANGLE so that no vowels are together. e. INTERMEDIATE such that no two vowels come together
Q5	In how many ways 3 mathematics books, 4 history books, 3 chemistry books and 2 biology books can be arranged on a shelf so that all books of the same subjects are together.
Q6	A student has to answer 10 questions, choosing at least 4 from each of Parts A and B. If there are 6 questions in Part A and 7 in Part B, in how many ways can the student choose 10 questions?
Q7	Three married couples are to be seated in a row having six seats in a cinema hall. If spouses are to be seated next to each other, in how many ways can they be seated? Find also the number of ways of their seating if all the ladies sit together.
Q8	In a small village, there are 87 families, of which 52 families have at most 2 children. In a rural development programme 20 families are to be chosen for assistance, of which at least 18 families must have at most 2 children. In how many ways can the choice be made?
Q9	A boy has 3 library tickets and 8 books of his interest in the library. Of these 8, he does not want to borrow Mathematics Part II, unless Mathematics Part I is also borrowed. In how many ways can he choose the three books to be borrowed?
Q10	There are four bus routes between A and B; and three bus routes between B and C. A man can travel round-trip in number of ways by bus from A to C via B. If he does not

	want to use a bus route more than once, in how many ways can he make a round trip?
Q11	How many committees of 4 members (2 men, 2 women) are possible if 5 men and 6 women are available?
Q12	Ten different letters of the alphabet are given. How many 5-letter words can be formed using these letters such that at least one letter is repeated?
Q13	How many signals can be sent using 6 flags of different colors, taking one or more at a time?
Q14	In an examination with three multiple-choice questions, each having 4 options, in how many ways can a student fail to get all the answers correct?
Q15	Eight chairs are numbered 1 to 8. Two women and 3 men wish to occupy one chair each. First the women choose the chairs from amongst the chairs 1 to 4 and then men select from the remaining chairs. Find the total number of possible arrangements.
Q16	In how many ways can you arrange 5 boys and 8 girls in a line so that no two boys are next to each other?
Q17	A candidate is required to answer 7 questions out of 12 questions, which are divided into two groups, each containing 6 questions. He is not permitted to attempt more than 5 questions from either group. Find the number of different ways of doing questions.
Q18	We wish to select 6 persons from 8, but if the person A is chosen, then B must be chosen. In how many ways can selections be made?
Q19	In how many ways can you divide 12 people into 3 teams A, B, C of equal members?
Q20	There are 10 people named P1 , P2 , P3 , ... P10. Out of 10 persons, 5 persons are to be arranged in a line such that in each arrangement P1 must occur whereas P4 and P5 do not occur. Find the number of such possible arrangements.
Q21	How many committees of five persons with a chairperson can be selected from 12 persons?
Q22	Find the number of permutations of n distinct things taken r together, in which 3 particular things must occur together.
Q23	18 mice were placed in two experimental groups and one control group, with all groups equally large. In how many ways can the mice be placed into three groups?
Q24	How many total words can be formed by selecting 2 vowels and 3 consonants from 4 vowels and 5 consonants?

Q25	In how many ways can a football team of 11 players be selected from 16 players? How many of them will (i) include 2 particular players? (ii) exclude 2 particular players?
Q26	A sports team of 11 students is to be constituted, choosing at least 5 from Class XI and at least 5 from Class XII. If there are 20 students in each of these classes, in how many ways can the team be constituted?
Q27	A group consists of 4 girls and 7 boys. In how many ways can a team of 5 members be selected if the team has (i) no girls (ii) at least one boy and one girl (iii) at least three girls.
Q28	A five-digit number divisible by 3 is to be formed using the digits 0,1,2,3,4,5 without repetitions. How many such numbers can be formed?
Q29	In a room, everybody shakes hands with everybody else. The total number of handshakes is 66. How many people are in the room?
Q30	How many triangles can be formed by choosing the vertices from a set of 12 points, seven of which lie on the same line?
Q31	How many ways can a team of 11 players be selected from 22 players, always including 2 specific players and excluding 4 specific players?
Q32	In a football championship, 153 matches were played. Each team played one match with every other team. How many teams participated?
Q33	A committee of 6 members is to be chosen from 10 men and 7 women such that it contains at least 3 men and 2 women. If two particular women refuse to serve on the same committee, how many such committees can be formed?
Q34	There are 3 books on Mathematics, 4 on Physics and 5 on English. How many different collections can be made such that each collection consists of : (a) One book of each subject (b) At least one book of each subject (c) At least one book of English
Q35	Five boys and five girls form a line. Find the number of ways of making the seating arrangement under the following condition: (a) Boys and girls alternate (b) No two girls sit together (c) All the girls sit together (d) All the girls are never together
Q36	There are 10 professors and 20 lecturers out of whom a committee of 2 professors and 3 lecturers is to be formed. Find (a) In how many ways committees can be formed ? (b) In how many ways a particular professor is included?

	(c) In how many ways a particular lecturer is included ? (d) In how many ways a particular lecturer is excluded ?
Q37	Using the digits 1, 2, 3, 4, 5, 6, 7, a number of 4 different digits is formed. Find (a) how many numbers are formed? (b) how many numbers are exactly divisible by 2? (c) how many numbers are exactly divisible by 25? (d) how many of these are exactly divisible by 4?
Q38	How many words (with or without dictionary meaning) can be made from the letters of the word MONDAY, assuming that no letter is repeated, if (a) 4 letters are used at a time (b) All letters are used at a time (c) All letters are used but the first is a vowel

CSE230: Discrete Mathematics

Practice Sheet 7: Number Theory

Q1	Determine whether $4 \mid 7$ and whether $4 \mid 24$
Q2	What are the quotient and remainder when a) 193 is divided by 7? b) -42 is divided by 5? c) 597 is divided by 23? d) 30201 is divided by 33? e) 0 is divided by 18? f) 3 is divided by 8?
Q3	What are the possible remainders (according to the Division Algorithm) when an integer is a) Divided by 4? b) Divided by 9?
Q4	For each of the following, find the quotient and remainder (guaranteed by the Division Algorithm) a. When 17 is divided by 3. b. When -17 is divided by 3. c. When 73 is divided by 7. d. When -73 is divided by 7. e. When 436 is divided by 27. f. When 539 is divided by 110.
Q5	Today is Tuesday. Your uncle will come after 45 days. On which day (of the week) your uncle will be coming?
Q6	Find the value of $(177 \bmod 31 \cdot 270 \bmod 31) \bmod 31$
Q7	Show that if $a \mid b$ and $b \mid a$, where a and b are integers, then $a = b$ or $a = -b$.
Q8	Prove or disprove that if $a \mid bc$, where a , b and c are positive integers and $a \neq 0$, then $a \mid b$ or $a \mid c$.
Q9	Prove that if a is an integer that is not divisible by 3, then $(a + 1)(a + 2)$ is divisible by 3.
Q10	Let m be a positive integer. Show that $a \bmod m = b \bmod m$ if $a \equiv b \pmod{m}$.
Q11	Considering a and b are integers, and $a \equiv b + 7 \pmod{19}$, show that $(a^2 - 11) \equiv (b^2 - 5b) \pmod{19}$

Q12	If we consider $50 \equiv 23 \pmod{9}$, then prove that $34 \equiv 16 \pmod{9}$ [Hint: Use the Theorem 5]
Q13	28. Find $a \text{ div } m$ and $a \bmod m$ when a) $a = -991, m = 99$. b) $a = -119, m = 101$. c) $a = 10299, m = 999$. d) $a = 12346, m = 101$
Q14	Find the integer a such that a) $a \equiv 17 \pmod{29}$ and $-14 \leq a \leq 14$. b) $a \equiv -11 \pmod{21}$ and $90 \leq a \leq 110$. c) $a \equiv 24 \pmod{31}$ and $-15 \leq a \leq 15$. d) $a \equiv 99 \pmod{41}$ and $100 \leq a \leq 140$.
Q15	Use modular exponentiation to find: i) $7^{64} \bmod 645$. ii) $32^{203} \bmod 99$. iii) $22^{3219} \bmod 243$.
Q16	i) What is the octal and hexadecimal expansion of $(11011000110\ 01011)_2$. ii) What is the hexadecimal expansion of $(125110)_{10}$. iii) What is the decimal expansion of $(7716)_8$. iv) What is the binary expansion of $(\text{BADDAD})_{16}$. v) What is the octal expansion of $(12344321)_{10}$.
Q17	Expand the decimal number $(506070)_{10}$ to the base x , where $x = ((10 \cdot 2^2)^2 + 71) \bmod 23$.
Q18	If $(ab)_4$ is a base-4 integer and $(ba)_7$ is a base-7 integer, what is the largest possible value of $(a+b)_{10}$ and why? Find non-zero values for a and b such that $(ab)_4 = (ba)_7$, or prove that there are none.
Q19	Determine whether the integers in each of these sets are pairwise relatively prime : i) 14, 17, 85. ii) 21, 34, 55. iii) 25, 41, 49, 64. iv) 17, 18, 19, 23.
Q20	Consider two positive integers 5271 and 48714. Calculate the Greatest Common Divisor for the integers using Euclidean Algorithm. Also show the Least Common Multiple.
Q21	Find the greatest common divisor of the following pair of numbers using the Euclidean

	Algorithm: i) 11111, 111111. ii) 1529, 14038. iii) 750, 900. iv) 414, 662.
Q22	How many divisions are required to find $\gcd(21, 34)$ using the Euclidean algorithm ?
Q23	Find $\gcd(92928, 123552)$ and $\text{lcm}(92928, 123552)$, and verify that $\gcd(92928, 123552) \cdot \text{lcm}(92928, 123552) = 92928 \cdot 123552$
Q24	If the product of two integers is $2^2 \cdot 3^5 \cdot 5^2 \cdot 7^1 \cdot 23^2 \cdot 29^1$ and their greatest common divisor is 23345, what is their least common multiple?
Q25	Determine whether the integers in each of these sets are pairwise relatively prime. a) 11, 15, 19 b) 14, 15, 21 c) 12, 17, 31, 37 d) 7, 8, 9, 11
Q26	If p is a prime number, $p > 3$, then show that $p^2 \equiv p^4 \equiv 1 \pmod{3}$
Q27	Work out the GCD of 7105, 3185 and 2898 only using the Euclidean Algorithm . Note that, $\gcd(a, b, c) = \gcd(\gcd(a, b), c)$
Q28	Given that, the square root of a number n when divided by 11 gives a remainder of 6 where $6 < \sqrt{n} < 28$. Find the number n . Then, using modular exponentiation, calculate: $n^{125} \pmod{27}$.

CSE 220

LAB 1: 2D Matrix

[CO5]

Instructions for students:

- Complete the following methods based on Array & 2D Matrix.
- You may use Java / Python to complete the tasks.
- DO NOT CREATE a separate folder for each task.
- If you are using JAVA, then follow the [Java template](#).
- If you are using PYTHON, then follow the [Python template](#).

NOTE:

- YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN ARRAYS.
 - YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.
- [Make changes to the Sample Inputs and check whether your program works correctly]

<u>The Lab Tasks should be completed during the lab class</u> YOU HAVE TO SUBMIT ONLY THE <u>ASSIGNMENT TASK</u>

Total Assignment Tasks: 4

Total Marks: 20

Converted Marks: 4

Intro Lab Tasks

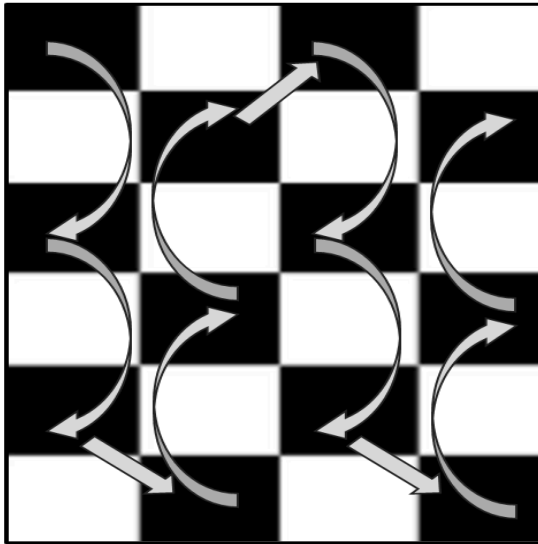
The following tasks are here to give you a brief idea about basic 2D Matrix operations. Once you're comfortable with basic coding of 2D Matrix then you can move on to the Main Lab Tasks. No coding template for these intro Lab Tasks.

- I. Print a given 2D matrix row wise.
- II. Print a given 2D matrix column wise.
- III. Explain the difference between I and II to yourself.
- IV. Find the absolute difference between the sum of primary diagonal and secondary diagonal of a given 2D matrix.
- V. Transpose a given 2D Matrix.

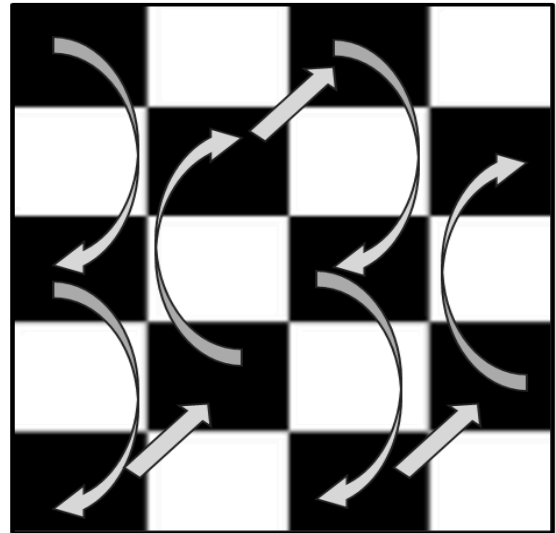
Main Lab Tasks [NO NEED TO SUBMIT]

1. Zigzag Walk:

As a child, you often played this game while walking on a tiled floor. You walked avoiding a certain color, for example white tiles (it almost felt like if you stepped on a white tile, you would die!). Now you are in a room of $m \times n$ dimension. The room has $m \times n$ black and white tiles. You step on the black tiles only. Your movement is like this:



OR



Now suppose you are given a 2D array which resembles the tiled floor. Each tile has a number. Can you write a method that will print your walking sequence on the floor?

Hint 1: The first tile of the room is always black.

Hint 2: Look out for the number of rows in the matrix Notice the transition from column 0 to column 1 in the above figures

Sample Given Matrix	Sample Output																									
<table><tr><td>3</td><td>8</td><td>4</td><td>6</td><td>1</td></tr><tr><td>7</td><td>2</td><td>1</td><td>9</td><td>3</td></tr><tr><td>9</td><td>0</td><td>7</td><td>5</td><td>8</td></tr><tr><td>2</td><td>1</td><td>3</td><td>4</td><td>0</td></tr><tr><td>1</td><td>4</td><td>2</td><td>8</td><td>6</td></tr></table>	3	8	4	6	1	7	2	1	9	3	9	0	7	5	8	2	1	3	4	0	1	4	2	8	6	3 9 1 1 2 4 7 2 4 9 1 8 6
3	8	4	6	1																						
7	2	1	9	3																						
9	0	7	5	8																						
2	1	3	4	0																						
1	4	2	8	6																						
<table><tr><td>3</td><td>8</td><td>4</td><td>6</td><td>1</td></tr><tr><td>7</td><td>2</td><td>1</td><td>9</td><td>3</td></tr><tr><td>9</td><td>0</td><td>7</td><td>5</td><td>8</td></tr><tr><td>2</td><td>1</td><td>3</td><td>4</td><td>0</td></tr></table>	3	8	4	6	1	7	2	1	9	3	9	0	7	5	8	2	1	3	4	0	3 9 1 2 4 7 4 9 1 8					
3	8	4	6	1																						
7	2	1	9	3																						
9	0	7	5	8																						
2	1	3	4	0																						

2. Decryption Process:

Suppose you're working as a cryptographer for a secret intelligence agency. You are given an encrypted matrix as an input. You have to decrypt it after some processing of the given matrix.

To decrypt this message efficiently, you have a task to construct a method called **decrypt_matrix(matrix)** which takes an encrypted matrix as input and returns a decrypted linear array. The process of finding the decrypted linear array is given below:

You have to find out the column-wise summations for each column and store the difference of subsequent column-wise summations in a new linear array.

Sample Given Matrix	Sample Output	Explanation																			
<table><tr><td>1</td><td>3</td><td>1</td></tr><tr><td>6</td><td>4</td><td>2</td></tr><tr><td>5</td><td>1</td><td>7</td></tr><tr><td>9</td><td>3</td><td>3</td></tr><tr><td>8</td><td>5</td><td>4</td></tr></table>	1	3	1	6	4	2	5	1	7	9	3	3	8	5	4	<table><tr><td>-13</td><td>1</td></tr></table>	-13	1	Sum of 0th column = 29 Sum of 1st column = 16 Sum of 2nd column = 17 Therefore, the size of the resulting array is 2 and the array is: <table><tr><td>16-29 = -13</td><td>17-16 = 1</td></tr></table>	16-29 = -13	17-16 = 1
1	3	1																			
6	4	2																			
5	1	7																			
9	3	3																			
8	5	4																			
-13	1																				
16-29 = -13	17-16 = 1																				

Assignment Tasks [NEED TO SUBMIT]

Won't be explained in lab class

1. Row Rotation Policy of Your Classroom [5 marks]

You are no longer permitted to choose your own seat on the new campus of your university. You must abide by the new regulations, which state that if you sit in the first row for the first week, you must shift to the second row for Week2, the third row for the Week3, and so on. In your classroom, there are a total of 6 rows and 5 columns. Your friend “AA” wants to know from you that on the upcoming exam week in which row he will be in. Your task is to implement a function `row_rotation(exam_week, seat_status)` that takes the exam_week and a 2D array of current seat status as input and returns the row number in which your friend “AA” will be seated and print the seat status for that week.

NOTE: If the shape of the given matrix is $M \times N$ then the space complexity of your solution must not be greater than $O(N)$.

Sample User Input & Given Matrix	Output																																																												
User Input: exam_week = 3 Given Matrix: current_seat_status = <table><tr><td> A </td><td> B </td><td> C </td><td> D </td><td> E </td></tr><tr><td> F </td><td> G </td><td> H </td><td> I </td><td> J </td></tr><tr><td> K </td><td> L </td><td> M </td><td> N </td><td> O </td></tr><tr><td> P </td><td> Q </td><td> R </td><td> S </td><td> T </td></tr><tr><td> U </td><td> V </td><td> W </td><td> X </td><td> Y </td></tr><tr><td> Z </td><td> AA </td><td> BB </td><td> CC </td><td> DD </td></tr></table>	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	AA	BB	CC	DD	<table><tr><td> U </td><td> V </td><td> W </td><td> X </td><td> Y </td></tr><tr><td> Z </td><td> AA </td><td> BB </td><td> CC </td><td> DD </td></tr><tr><td> A </td><td> B </td><td> C </td><td> D </td><td> E </td></tr><tr><td> F </td><td> G </td><td> H </td><td> I </td><td> J </td></tr><tr><td> K </td><td> L </td><td> M </td><td> N </td><td> O </td></tr><tr><td> P </td><td> Q </td><td> R </td><td> S </td><td> T </td></tr></table> Your friend AA will be on row 2	U	V	W	X	Y	Z	AA	BB	CC	DD	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
A	B	C	D	E																																																									
F	G	H	I	J																																																									
K	L	M	N	O																																																									
P	Q	R	S	T																																																									
U	V	W	X	Y																																																									
Z	AA	BB	CC	DD																																																									
U	V	W	X	Y																																																									
Z	AA	BB	CC	DD																																																									
A	B	C	D	E																																																									
F	G	H	I	J																																																									
K	L	M	N	O																																																									
P	Q	R	S	T																																																									
Explanation: According to the example the exam is 3 weeks away from the current seat status. So, during the exam day, the row containing AA will move 2 times (basically the whole matrix will rotate downward vertically).																																																													

2. Matrix Compression [5 marks]

Complete the function `compress_matrix` that takes a 2D array as a parameter and return a new compressed 2D array. In the given array the number of row and column will always be even. **Compressing a matrix means grouping elements in 2x2 blocks and sums the elements within each block. Check the sample input output for further clarification.**

Hint: Generally the block consists of the (i,j), (i+1,j), (i,j+1) and (i+1, j+1) elements for 2x2 blocks.

You cannot use any built-in function except `len()` and `range()`. You can use the `np` variable to create an array.

Python Notation	Java Notation
<pre>import numpy as np def compress_matrix (mat): # To Do</pre>	<pre>public int[][] compress_matrix (int[][] mat) { // To Do }</pre>

Sample Input array	All Box (No need to create these arrays)	Returned Array	Explanation
<pre>[[1, 2, 3, 4], [5, 6, 7, 8], [1, 3, 5, 2], [-2, 0, 6,-3]]</pre>	<pre>[[[1, 2], [[3, 4], [5, 6]] [[7, 8]] [[[1, 3], [[5, 2], [-2, 0]] [[6, -3]]</pre>	<pre>[[[14, 22], [2, 10]]</pre>	<pre>[[[1+2+5+6, 3+4+7+8], [1+3+-2+0, 5+2+6+-3]]</pre>

3. Game Arena [5 marks]

Suppose you and your friends are in the world of '*Alice in Borderland*' where you decided to take part in a game and entered the *game arena*. As a team, you need to gain **at least 10 points** in order to keep surviving in the borderland. Otherwise, you will be out of the game and your team will be banished for good. Now, the arena has a 2D array like structure where players of a team are given certain positions with values that are **multiples of 50**. By staying in these positions, every player can gain points from the cells above, below, left and right (not diagonally) only if those cells **contain 2** [The cells containing 1s and 0s are to be avoided]. For each player, add from these cells containing 2s to your total points for the team to keep on surviving in borderland. **Be careful about corner cases.**

Your task is to write a method which tells us whether your team is out or has survived the game.

Sample Given Matrix 1	Sample Output 1																								
<table><tr><td>0</td><td>2</td><td>2</td><td>0</td><td></td></tr><tr><td>50</td><td>1</td><td>2</td><td>0</td><td></td></tr><tr><td>2</td><td>2</td><td>2</td><td>0</td><td></td></tr><tr><td>1</td><td>100</td><td>2</td><td>0</td><td></td></tr></table>	0	2	2	0		50	1	2	0		2	2	2	0		1	100	2	0		Points Gained: 6. Your team is out.				
0	2	2	0																						
50	1	2	0																						
2	2	2	0																						
1	100	2	0																						
Explanation: Player with value 50 has 2 in the cell below him (1 cell). Player with value 100 has 2 in the cell above and in the right cell (2 cells). So in total, they got $(1+2)*2 = 6$ points which was not enough to survive the game.																									
Sample Given Matrix 2	Sample Output 2																								
<table><tr><td>0</td><td>2</td><td>2</td><td>0</td><td>2</td><td></td></tr><tr><td>1</td><td>50</td><td>2</td><td>1</td><td>100</td><td></td></tr><tr><td>2</td><td>2</td><td>2</td><td>0</td><td>2</td><td></td></tr><tr><td>0</td><td>200</td><td>2</td><td>0</td><td>0</td><td></td></tr></table>	0	2	2	0	2		1	50	2	1	100		2	2	2	0	2		0	200	2	0	0		Points Gained: 14. Your team has survived the game.
0	2	2	0	2																					
1	50	2	1	100																					
2	2	2	0	2																					
0	200	2	0	0																					
Explanation: Player with value 50 has 2 in the cell above, cell below and the right cell (3 cells). Player with value 100 has 2 in the cell above and the cell below (2 cells). Player with value 200 has 2 in the above cell and right cell (2 cells). So in total, they gained $(3+2+2)*2 = 14$ points and survived the game.																									
Note: For the cell with value 2 that is common between 2 players in position (2,1), both gained 2 points each, so it's not like if one player already added those 2 points, another player cannot. In the Given Matrix 2, Player with value 50 and player with value 200 are sharing a common 2 but both of them got points.																									

4. Rotate Secret [5 marks]

Your friend sent you a secret message scattered across a square board. The letters are scrambled, making the message unreadable. To recover it, you must rotate the concentric layers of the board clockwise, starting from the innermost layer and moving outward. You are given a square $N \times N$ matrix of characters, where both N (rows and columns) are even. Rotate each layer clockwise starting from the innermost layer, the innermost rotates once, and each outer layer rotates one time more than the one inside it. After performing all rotations, the hidden message will be revealed.

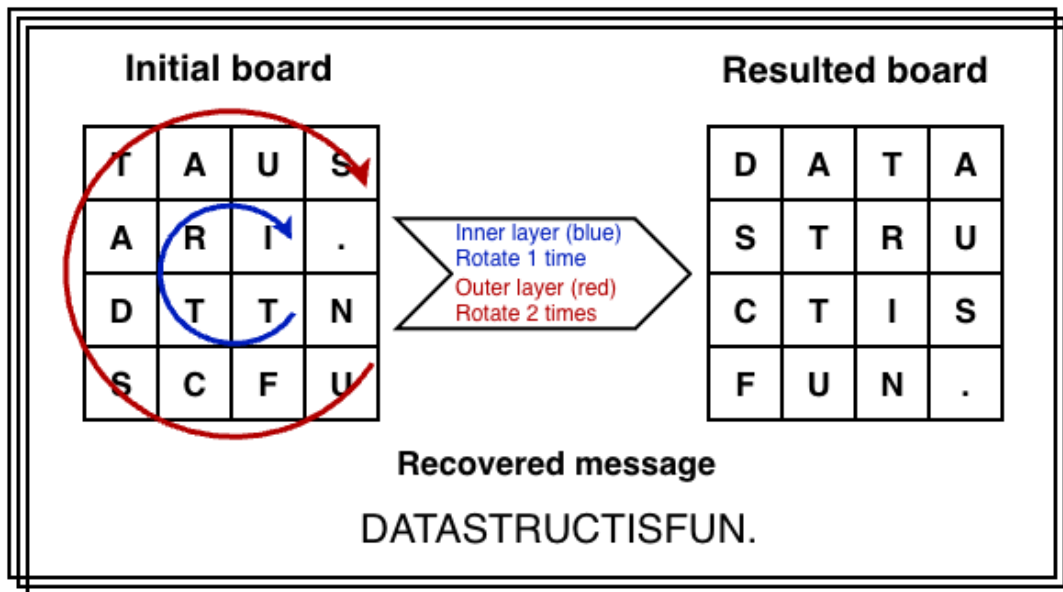
Write a method/function named **rotateSecret()**, which rotate the matrix clockwise :

- Innermost layer rotate 1 time
- Next outer layer rotate 2 times
- Next outer rotate 3 times, and so on.

After all successful rotations, print the recovered message.

Space Complexity of the solution must be O(1)

[can't create new arrays]



Sample Given Matrix 1	After Processing	Output
<pre>board = { {'T','A','U','S'}, {'A','R','I','.'}, {'D','T','T','N'}, {'S','C','F','U'} }</pre>	<pre>board = { {'D','A','T','A'}, {'S','T','R','U'}, {'C','T','I','S'}, {'F','U','N','.'} };</pre>	DATASTRUCTISFUN.
Explanation		
1 clockwise rotation on the innermost (2×2) layer, and 2 clockwise rotations on the outer (4×4) layer, each letter shifts to its correct position after these rotations. Then the message from the board is printed.		

Sample Given Matrix 2	After Processing	Output
<pre>board = { {'O','R','I','R','N','P'}, {'G','S','A','A','L','R'}, {'L','M','N','O','N','Y'}, {'A','H','U','O','O','P'}, {'T','F','C','T','H','S'}, {'E','D','Y','O','C','K'} }</pre>	<pre>{ {'A','L','G','O','R','I'}, {'T','H','M','S','A','R'}, {'E','F','U','N','A','N'}, {'D','C','O','O','L','P'}, {'Y','T','H','O','N','R'}, {'O','C','K','S','P','Y'} }</pre>	ALGORITHMSAREFUNANDCO OLPYTHONROCKSPY
Explanation		
1 clockwise rotation on the innermost (2×2) layer, and 2 clockwise rotations on the outer (4×4) layer, 3 clockwise rotations on the outer (6×6) layer, each letter shifts to its correct position after these rotations. Then the message from the board is printed.		

LAB 2: Node & LinkedList

[CO1]

Instructions for students:

- You may use Java / Python to complete the tasks.

NOTE:

- **YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN THE LINKED LIST YOU'RE CREATING.**
- **IF THE QUESTION ALLOWS YOU TO CREATE OTHER DATA STRUCTURES THEN YOU CAN.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

The Lab Tasks should be completed during the lab class
YOU HAVE TO SUBMIT ONLY THE ASSIGNMENT TASK

Total Lab 2 Assignment Tasks: 4

Total Marks: 20

Converted Marks: 4

Intro Node Tasks

This is an intro task & there isn't any driver code for this

- ❖ For java, create a separate folder for this task and follow the instructions.
- ❖ For Python, create a separate colab/ipynb/py file and follow the instructions.

→ Design a Node class.

- ◆ Declare two instance variables, one called **elem** (Object data type for java) and another one called **next** (Node datatype).
- ◆ Write a constructor that has only one parameter (Object data type for java) and assigns the parameter value to the **elem** instance variable.

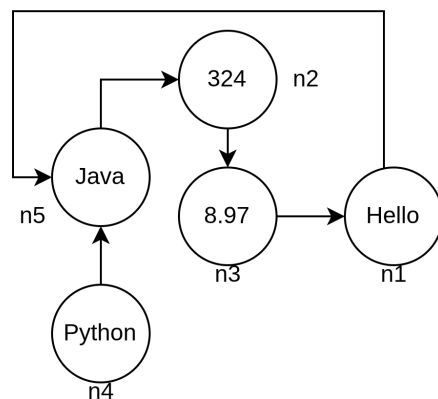
→ Design a Tester class (for java) / Design another code cell (for python)

- ◆ Create 5 different objects of Node class. Assign values as shown in the illustration.
- ◆ Variable names should be: **n1, n2, n3, n4, n5**.
- ◆ Connect the 5 nodes as shown in the illustration.

Now, execute the lines given below and try to understand the output by simulating them using pen & paper

If there are errors, try to figure out why that error occurred and what can be done about it.

Note: Java & Python output won't always be the same.



JAVA	PYTHON
System.out.println(n1); System.out.println(n3.next);	print(n1) print(n3.next)
System.out.println(n3.next.elem);	print(n3.next.elem)
Node x = n4.next; System.out.println(n1.elem + x.elem);	x = n3.next print(x.elem + n4.elem)
x.next = n3; System.out.println(n2.next.next + n5.next);	x.next = n2 print(n2.next.next + n5.next)
n5 = null; System.out.println(n4.next.elem);	n5 = None print(n4.next.elem)
x.next.next = null; n3.next.elem = 321;	x.next.next = None n2.next.elem = 7.98
n4.next = 532; System.out.println(n4.next.elem);	n4.next = 532 print(n4.next.elem)

Now let's design a basic Linked List Class.
Please use the [Java Intro LinkedList Template](#) or [Python Intro LinkedList Template](#) .

Intro Linked List: Design Linked list class

Node class is already given, use that from the template.

1. Complete the **append()** method inside the LinkedList class. This method takes a value as a parameter and inserts it into the back of the linked list.
 - Insert a new node at the end of the list.
 - If the list is empty, the new node becomes the head.
2. Complete the **printList()** method, which Prints all elements starting from head.
 - Traverse until the end of the list.
3. Complete the **prepend()** method inside the LinkedList class. This method takes a value as a parameter and inserts it into the front of the linked list.
 - Update the head reference properly.
4. Complete the **nodeAt()** method that returns the node at the given index (0 based).
 - If the index is invalid, return null.
 - Do NOT print inside this method.
5. Complete the **removeFirst()** method inside the LinkedList class. This should remove the first node from the linked list.
 - If the list is empty, do nothing.
6. Complete the **removeLast()** method inside the LinkedList class. This should remove the last node from the linked list.
 - If the list has only one node, head should become null.
 - If the list is empty, do nothing.

Congratulations! You have successfully designed a LinkedList class.

However, in the upcoming problem solving tasks, you will not be allowed to use this class. Instead, you will be given the head of a linked list and asked to complete specific tasks. If you need any additional helper methods, you must implement them yourself.

Now we can move on to actual problem solving using nodes/linkedlist.
Please use the [Java LabTask Template](#) or [Python LabTask Template](#)

Main Lab Tasks [No Need to Submit]

1. Assemble Conga Line

Have you ever heard the term [conga line](#)? Basically, it's a carnival dance where the dancers form a long line. Everyone holds the waist of the person in front of them and their waists are held in turn by the person to their rear, excepting only those in the front and the back. It kind of looks like [this](#).



pixtastock.com – 11648015

By now, you can quite understand the suitable data structure to represent a conga line. Now you are the choreographer of the Conga Dance in a Summer Festival. You wish to arrange the conga line **ascending** age wise and tell the participants to stand in a line likewise. Now as technical you are, can you write a method that will take the conga line and return True if everyone stands according to your instruction. Otherwise returns False.

Sample Input	Sample Returned Result
10 → 15 → 34 → 41 → 56 → 72	True
10 → 15 → 44 → 41 → 56 → 72	False

2. Word Decoder

Suppose, you have been hired as an cyber security expert in an organization. A mysterious code letter has been discovered by your team, your task is to decode the letter. After lots of research you found a pattern to solve the problem. Problem details are given below:

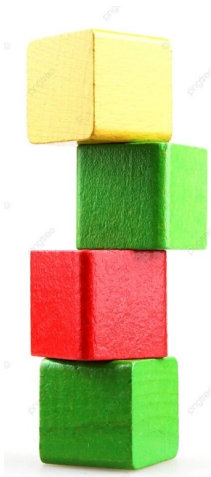
- For each encoded word a linked list will be given, where each node will carry one letter as an element.
- For the decoded word, the letters are at the multiples of $(13 \% \text{length of linkedlist})$ position **[0 indexed]**. For example, if the length of the given linked list is 10, then $13 \% 10 = 3$ and the letters are at positions which are multiples of 3 (i.e. at position 3, 6, 9)
- However, the letters are stored in the given linked list in opposite order. Thus the decoded word has to be reversed.
- After decoding, your program should return a dummy headed singly linked list containing the decoded word.

Sample Input	Sample Output
B→M→D→T→N→O→A→P→S→C	None → C→A→T Explanation: Length of the list= 10 & $13\%10=3$ The letters are T, A, C at 3, 6, 9 positions respectively [0 indexed] . The letters are reversed and the resulting linked list is None → C→A→T
Z→O→T→N→X	None → N Explanation: Length of the list= 5 & $13\%5=3$. The letter is N at position 3 [0 indexed] . The letters are reversed and the resulting linked list is None → N

Assignment Tasks [Need to Submit]

1. Building Blocks

Your twin and you are under an experiment where the amount of thinking similarities you two have is being observed. As per the experiment, you are given a certain number of building blocks of different colors and are told to make a building using those blocks in two different rooms.



After the buildings are finished, the observers check whether the two buildings are the same based on the block colors. Now, you are the tech guy of that team and you are instructed to write a program that will output “Similar” or “Not Similar” given the two buildings. For fun, you decided to represent those buildings as a linked list!

NB: Red means a red block
Blue means a blue block
Yellow means a yellow block
Green means a green block.

Sample Input	Sample Output
<code>building_1 = Red→ Green→ Yellow→ Red→ Blue→ Green building_2 = Red→ Green→ Yellow→ Red→ Blue→ Green</code>	Similar
<code>building_1 = Red→ Green→ Yellow→ Red→ Yellow→ Green building_2 = Red→ Green→ Yellow→ Red→ Blue→ Green</code>	Not Similar
<code>building_1 = Red→ Green→ Yellow→ Red→ Blue→ Green building_2 = Red→ Green→ Yellow→ Red→ Blue→ Green→ Blue</code>	Not Similar

2. Organize Books

You are a system engineer at a university library that maintains its collection of books digitally using a singly linked list. Each node in the linked list represents a book title as a string stored in the catalog.

To improve book recommendations, the library recently collected popularity scores from student surveys. Each score represents how popular a book is, a higher score means greater popularity. Your task is to reorganize the linked list so that books are arranged in descending order of popularity. If multiple books have the same popularity score, they must retain their original order.

Write a function named **organizeBooks(Node head, int[] popularity)**, which will take the **head** of the linked list and an integer array containing the **popularity** score. After organizing the list, **return the head of the modified list**.

- You can not create or take a new linked list, array or any other data structure.

Sample Given LL and Array Example1	Output
Dune → IT → Coraline → Inferno → Twilight [8, 10, 5, 10, 6]	IT → Inferno → Dune → Twilight → Coraline
Explanation Example1: In the given list, Dune has a score of 8, IT has 10, Coraline has 5, Inferno has 10, and Twilight has 6. The list is reorganized so that books with higher popularity appear first. Therefore, IT and Inferno, both with a score of 10, come to the front. Next is Dune with a score of 8, followed by Twilight with 6, and finally Coraline with 5.	
Sample Given LL and Array Example2	Output2
Hamlet → Persuasion → It → Dracula → Beloved [7, 9, 9, 6, 7]	Persuasion → It → Hamlet → Beloved → Dracula
Sample Given LL and Array Example3	Output3
Matilda → Franny → Foundation → Carrie → Misery [5, 8, 8, 10, 6]	Carrie → Franny → Foundation → Misery → Matilda

3. Alternate Merge

You are given two singly non-dummy headed linked lists. Your task is to write a function **alternate_merge(head1, head2)** that takes the heads of the two linked lists and returns the head of a modified linked list with all the elements of the two lists in alternate order. It is guaranteed that alternate placement is always possible. Your resulting linked list will always start with the head of linked list 1.

Input	Output
List1: 1 → 2 → 6 → 8 → 11 → None List2: 5 → 7 → 3 → 9 → 4 → None	1 → 5 → 2 → 7 → 6 → 3 → 8 → 9 → 11 → 4 → None
List1: 5 → 3 → 2 → -4 → None List2: -4 → -6 → 1 → None	5 → -4 → 3 → -6 → 2 → 1 → -4 → None
List1: 4 → 2 → -2 → -4 → None List2: 8 → 6 → 5 → -3 → None	4 → 8 → 2 → 6 → -2 → 5 → -4 → -3 → None

NOTE: The space complexity of the solution must be $O(1)$. Which means, You're **NOT ALLOWED** to create any new linked list for this task.

4. ID Generator

Write a function **idGenerator()** that will take three non-dummy headed linear singly linked lists containing only digits from 0-9 and **generate another singly linked list** with 8 digit student ID from it [The student ID contains only digits from 0 to 9].

The first linked list will have the first four digits of the student id in reverse order. The remaining four digits can be obtained by adding the elements of the second linked list **from the beginning** with the elements of the third linked list **from the beginning**.

If the summation is ≥ 10 , then you have to mod the summation by 10 and insert the result in the output linked list. For example, in sample input 2, the last element of the second list is 7 and the last element of the third linked list is 8. So after adding them we get $7+8=15 > 10$. So the digit we will insert as an element of the Student ID list, will be $15\%10=5$.

Sample Input 1: 0 → 3 → 2 → 2 5 → 2 → 2 → 1 4 → 3 → 2 → 1 Sample output 1: 2 → 2 → 3 → 0 → 9 → 5 → 4 → 2	Sample Input 2: 0 → 3 → 9 → 1 3 → 6 → 5 → 7 2 → 4 → 3 → 8 Sample output 2: 1 → 9 → 3 → 0 → 5 → 0 → 8 → 5
---	---

**You Will Have To Submit Lab 2 and Lab 3 as a single file after next week.
Basically, There will be only 1 combined submission for Lab 2 & Lab 3.**

LAB 3: Doubly LinkedList

[CO1]

Instructions for students:

- You may use Java / Python to complete the tasks.
- If you are using **JAVA**, then follow the [Java Template](#).
- If you are using **PYTHON**, then follow the [Python template](#).

NOTE:

- **YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN THE LINKED LIST YOU'RE CREATING.**
- **IF ONLY THE QUESTION ALLOWS YOU TO CREATE OTHER DATA STRUCTURES, THEN YOU CAN.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

The Lab Tasks should be completed during the lab class
YOU HAVE TO SUBMIT ONLY THE ASSIGNMENT TASK

Total Lab 3 Assignment Tasks: 3

Total Marks: 15

Lab Task

In this part of the lab, we will play with Dummy Headed Doubly Circular Linked List. If you want to read about this type of linked list then check [this file](#).

Note: Even though we're learning Dummy Headed Doubly Circular Linked List, the methods you'll be writing simulate a different data structure. Can you guess what that is? Hint: It's something you face in front of the elevator everyday.

In the first part of this lab, you have to implement a waiting room management system in an emergency ward of a hospital. Your program will serve a patient on a **first-come, first-served** basis. The **simulation of WRM** can be found [here](#).

Solve the above problem using a **Dummy Headed Doubly Circular Linked List**.

1. You need to have a **Patient** class so that you can create an instance of it (patient) by assigning id(integer), name (String), age (integer), and bloodGroup (String).
2. Write a **WRM** (waiting room management) class that will contain the following methods.
 - a. **registerPatient(id, name, age, bloodgroup):** This method will register a patient into your system. The method will create a Patient type object with the information received as a parameter. It means this method will add a patient-type object to your linked list.
 - b. **servePatient():** This method calls a patient to provide hospital service to him/her. In this method, you need to ensure that to serve the patient who was registered first. (serving means removing the patient from line)
 - c. **cancelAll():** This method cancels all appointments of the patients so that the doctor can go to lunch.
 - d. **canDoctorGoHome():** This method returns true if no one is waiting; otherwise, it returns false.
 - e. **showAllPatient():** This method prints all IDs of the waiting patients in sequential order. It means the patient who got registered first will come first, and so on.
 - f. **reverseTheLine():** This method reverses the patient line. It means the patient who got registered last will come first, and so on.

Important Hints:

1. In your program, **there won't be any class called "Node"**. Instead, the **Patient** class will work as the Node class.
2. On the other hand, the **WRM** class will act as the Dummy Headed Doubly Circular Linked List, which will contain all the other functions/methods.

Assignment Tasks **[Need to Submit]**

If you need to use extra helper functions/methods feel free to create & use them. If you use them then make sure to submit those extra functions/methods as well. Here's the [Java Template](#) & [Python Template](#)

[continued from the previous assignment...]

5. Sum Odd Append

Write a method/function called **sumOddAppend()**. The function will take only one parameter, referencing a **dummy-headed singly circular linked list**. The function/method removes all the nodes containing odd values while summing them up. Finally, a node containing the summation is inserted at the end.

NOTE:

- YOU CAN CREATE ONLY 1 EXTRA NEW NODE using the given Node Class.

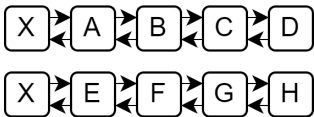
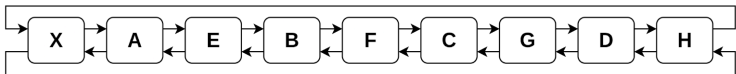
Given Dummy Headed Singly Circular Linked List	Expected Modified Linked List
	

6. Pair Join

Write a method/function called **pairJoin()**. The function takes two parameters, referencing **two dummy heads** of two different Dummy Headed Doubly Linked Lists. The method/function will modify the connections of the two given linked lists and combine them in pairs as shown in the example below. The method/function will not return anything because the first dummy head will naturally become the new head.

NOTE:

1. YOU CANNOT CREATE ANY NEW NODES.
2. YOU CAN ASSUME THE LENGTH OF THE GIVEN TWO LISTS WILL ALWAYS BE EQUAL.

Given Dummy Headed Doubly Linked List	Expected Modified Linked List
	

7. Range Move

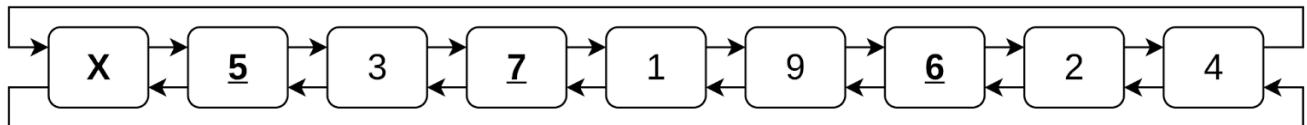
Write a method/function called **rangeMove()**. The function takes three parameters, referencing **one dummy head and two integers**. The method/function will move all the nodes that fall within the given range of integers to the back of the linked list. The method/function will not return anything because the dummy head won't change.

NOTE:

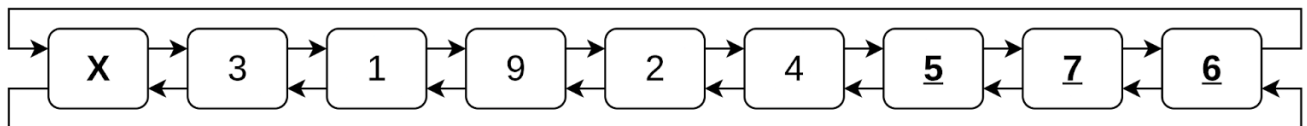
3. YOU **CANNOT** CREATE ANY NEW LINKED LIST FOR THIS TASK
4. YOU **CANNOT** CREATE ANY EXTRA NODE.

Given Dummy Headed Doubly Circular Linked List

Given Range: [start,end] = [5,7]



Expected Modified Linked List



LAB 4 (Part#2)

Secondary Data Structures

[CO1]

Instructions for students:

- You may use Java / Python to complete the tasks.
- If you are using **JAVA**, then follow the [Java Template](#).
- If you are using **PYTHON**, then follow the [Python template](#).
- **YOU CANNOT USE ANY OTHER EXTRA DATA STRUCTURE, UNLESS IT'S MENTIONED IN THE QUESTION.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

Python List, Negative indexing, append() or other types of Built-In functions are STRICTLY prohibited

<u>The Lab Tasks should be completed during the lab class</u> YOU HAVE TO SUBMIT ONLY THE <u>ASSIGNMENT</u> TASK

Total Lab 3 Assignment Tasks: 3

Total Marks: 15

Intro Lab Tasks [No Need to Submit]

Implementing an Array-Based Stack

A stack stores elements such that the most recently inserted element is the first one removed. This follows the **Last-In, First-Out (LIFO)** principle. The **ArrayStack** class has only two **private** instance variables:

- An array of Objects called **'stack'** (You take the size using the constructor).
- An integer called **'top'** which initializes at -1. The top index increases when something is “pushed” and it decreases when something is “popped”.

Complete the **ArrayStack** class by implementing the following methods:

Methods	Description
push(element)	Inserts an element at the top of the stack. If the stack is full, just print stack overflow. The push always increments the top.
pop()	Removes and returns the top element of the stack. If the stack is empty, just print “stack underflow”. The pop() always decrements the top.
peek()	Returns the top element without removing it. If the stack is empty, return null.
isEmpty()	Returns true if the stack contains no elements, otherwise false.

Implementing an Array-Based Queue

A queue stores elements such that the earliest inserted element is the first one removed. This follows the **First-In, First-Out (FIFO)** principle. The **ArrayQueue** class has the following **four private instance variables**:

- An array of Objects called **'queue'** (You take the size using the constructor)
- Three integers:
 - **'front'** – indicates the index of the element at the front of the queue.
 - **'rear'** – indicates the index where the next element will be inserted.
 - [Both front and rear should initialize to 0]
 - **'size'** - indicates the number of elements in the queue

Complete the **ArrayQueue** class by implementing the following methods:

Methods	Description
enqueue(element)	Inserts an element at the rear of the queue. If the queue is full, print "queue overflow". The enqueue always increments the rear. When you reach the end, the value of rear rounds back up to the first index.
dequeue()	Removes and returns the front element of the queue. If the queue is empty, print "queue underflow" (or throw/raise an Exception). The dequeue always increments the front. When you reach the end, the value of rear rounds back up to the first index.
peek()	Returns the front element without removing it. If the queue is empty, return null.
isEmpty()	Returns true if the queue contains no elements, otherwise false.

Lab Tasks **[No Need to Submit]**

1. Stack: Tower of Blocks

A kid is trying to make a tower using blocks with numbers or characters written on them. He puts a block on the top of another to make the tower. As for removing the blocks, he removes the topmost block if needed. He is afraid that removing other blocks from the middle in one go will result in the collapse of his tower.



Now his friend wants your n th block from the top of your tower. He is willing to give him the n th block from the top, preserving the relative position of others.

Following his tower build process, solve this scenario using **Stack** class. You can only use the stack methods.

In the following examples, consider the rightmost element to be the topmost element of the stack. **(Rightmost item is the top)**

Sample Tower	Sample Output	Explanation
Stack: 4 19 23 17 5 n: 2	Stack: 4 19 23 5	In this tower, the 2nd block from the top is the block with 17. After removing the block the stack looks like 4 19 23 5
Stack: 73 85 15 41 n: 3	Stack: 73 15 41	In this tower, the 3rd block from the top is the block with 85. After removing the block the stack looks like 73 15 41

[NOTE: A `LinkedListStack` class is already given and you can create an object of `Stack` to call `push()`, `pop()`, `peek()`, `isEmpty()` as per your will to complete the task]

You cannot use any data structure other than Stack

2. Queue: Remove Consecutive Duplicates

NOTE: YOU MUST USE QUEUE TO SOLVE THIS TASK

Write a method/function called **removeConsecutiveDups()** that takes only one String in its parameter. You are given a String which may or may not contain consecutive duplicate values. Finally the **removeConsecutiveDups()** method returns a new String containing no consecutive duplicate characters. If no duplicates are found then it simply returns the same string.

Sample Given String	Sample Returned String
aabbbccccdd	abcd
aaabbaa	aba
abcabcabc	abcabcabc
aaaaa	a

[Hint: A LinkedListQueue class containing enqueue(), dequeue(), peek() and isEmpty() methods is already given in the driver code, you just need to create an Object of it to solve the task. No need to design a new Queue]

You cannot use any data structure other than Queue

Assignment Tasks [**Need to Submit**]

1. Stack: Conditional Reverse

Consider that a Stack class has been implemented containing the push(element), pop(), peek() and isEmpty() functions.

Complete the function **conditional_reverse()** which will take an object of Stack class as an input that contains some integer values. **The function returns a new stack** which will contain the values in reverse order from the given stack **except the consecutive one's**. You cannot use any other data structure except Stack.

In the following example, consider the rightmost element to be the topmost element of the stack.

Sample Input Stack (Rightmost is the top)	Returned Reversed Stack (Rightmost is the top)
Stack: 10, 10, 20, 20, 30, 10, 50 Top = 50	Stack: 50, 10, 30, 20, 10 Top = 10
Explanation: Consecutive 20 and 10 are not present in the output reversed stack	

[NOTE: A LinkedListStack class is already given and you can create an object of Stack to call push(),pop(),peek(),isEmpty() as per your will to complete the task]

You cannot use any data structure other than Stack

2. Queue: Hot Potato Game

A group of friends are standing in a circle playing the **Hot Potato game**. The students pass an imaginary “potato” to the next person in the circle. After the potato has been passed a fixed number of times k , the student holding the potato is eliminated from the game. The process then continues with the remaining students, starting from the next student in the circle. This continues until only one student remains, who is declared the winner.

Write a method **hotPotato(players, k)** that simulates this game using a queue. The method takes an array of Strings (players) and an integer (k). The method should repeatedly rotate the queue to simulate passing the potato and remove the player who is eliminated. Finally, the method should **return** the name of the winning player.

Sample Input	Sample Output
players = ["Ali", "Ben", "Cia", "Dan", "Eli", "Faye"] k = 3	Dan eliminated Ben eliminated Ali eliminated Cia eliminated Faye eliminated Winner: Eli
<u>Explanation:</u> Queue initially: Ali Ben Cia Dan Eli Faye Passing 3 times → Dan eliminated → Eli Faye Ali Ben Cia Passing 3 times → Ben eliminated → Cia Eli Faye Ali Passing 3 times → Ali eliminated → Cia Eli Faye Passing 3 times → Cia eliminated → Eli Faye Passing 3 times → Faye eliminated → Eli Remaining player → Eli (Winner)	
players = ["Ali", "Rafi", "Sara", "Nina"] k = 2	Sara eliminated Rafi eliminated Nina eliminated Winner: Ali

[NOTE: A LinkedListQueue class is already given, you can use it to create an Object of Queue to call enqueue(), dequeue(), peek(), isEmpty() as per your will to complete the task]

You cannot use any data structure other than Queue

3. Stack and Queue: Dancing Pairs

From a dance school, you are given a **Stack filled with Dancer's Info (Objects of Dancer class)**. The Dancer Class only has three instance variables; name(String), gender(char), id(int). The genders are either "M" or "F" (male or female) and the id can be any random integer.

The stack of dancers need to be paired off as male with female for a competition. But this pairing has be done by following the rules of the competition:

- If *a male & a female* or *a female & a male* are consecutive in the stack, they get paired immediately and popped out of the Stack since they've been processed.
- If two consecutive females are found then the *first spare female* should be popped out of Stack and enqueued in a **femaleQueue** for future pairing.
- If two consecutive males are found, we try to pair the *first spare male* with a spare female from the **femaleQueue**. If no spare female is available in the then the spare male is enqueued in a **maleQueue**.
- Once all the dancers have been processed from the Stack, the dancers in the **maleQueue** or **femaleQueue** need to be processed, in case they're not empty. If a dancer couldn't be paired up they need to be mentioned at the very end.

Your task is to complete the given method **dance_pair()** that takes a **stack** and prints every male-female pair and those who couldn't be paired as well.

NOTE: A LinkedListStack & LinkedListQueue class is already given, you can use it to create an Object of Stack to call push(),pop(),peek(),isEmpty() and Queue to call enqueue(),dequeue(),peek(),isEmpty() to complete the task.

In both classes, isEmpty() returns true/false, peek() and pop() returns null/None for underflow.

You cannot use any data structure other than stack and queue

sample examples are given on the next page

Sample Given Stack	Sample Output
Nina(F-12) <- TOP Omar(M-44) Liam(M-24) Sara(F-15) Maya(F-54) Arif(M-10)	#1 Nina(F-12) & Omar(M-44) #2 Liam(M-24) & Sara(F-15) #3 Maya(F-54) & Arif(M-10)
Rita(F-43) <- TOP Mina(F-54) Lina(F-12) Omar(M-9) Arif(M-51) Sami(M-53) Nina(F-29) Lara(F-43) Tariq(M-45)	#1: Lina(F-12) & Omar(M-09) #2: Arif(M-51) & Rita(F-43) #3: Sami(M-53) & Nina(F-29) #4: Lara(F-43) & Tariq(M-45) Unpaired Female: Mina(F-54)
Sara(F-43) <- TOP Lina(F-52) Omar(M-16) Rafi(M-23) Maya(F-65) Alim(M-11) Kazi(M-32) Nina(F-75) Anan(M-11) Wafi(M-13)	#1: Lina(F-52) & Omar(M-16) #2: Rafi(M-23) & Maya(F-65) #3: Alim(M-11) & Sara(F-43) #4: Kazi(M-32) & Nina(F-75) Unpaired Male: Anan(M-11), Wafi(M-13)

LAB 4 (Part#1)

Secondary Data Structures

[CO1]

Instructions for students:

- If you are using **JAVA**, then follow the [Java Template](#).
- If you are using **PYTHON**, then follow the [Python Template](#).

NOTE:

- **YOU CANNOT USE ANY OTHER EXTRA DATA STRUCTURE, UNLESS IT'S MENTIONED IN THE QUESTION.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

Python List, Negative indexing, append() or other types of Built-In functions are STRICTLY prohibited

The Lab Tasks should be completed during the lab class
YOU HAVE TO SUBMIT ONLY THE ASSIGNMENT TASK

Total Lab 4 [Part#1] Assignment Tasks: 2
Total Marks: 10

Lab Tasks [No Need to Submit]

1. Hashtable: Insert using Forward Chaining:

Write the **insert()** function and **hash_function()** of the HashTable class which uses an array to store a key-value pair, where the key is a string representing a fruit, and the value is an int representing its respective price.

hash_function(key)

Takes a key which is a string, and calculates its length. If length is even, the sum of the ascii values of the even characters is calculated, otherwise, the sum of the ascii values of odd characters is calculated.

Finally, it returns the sum modded by length of the array

If we call `__hash_function("apple")`:

As `len("apple")` is odd, characters in odd indexes (p, l) are taken:

$\text{sum} = \text{ord}(p) + \text{ord}(l) = 112 + 108 = 220$

If the Hashtable object is initialized with length 5, then:

$\text{sum} \% \text{length of Hashtable array} = 220 \% 5 = 0$

So, the function returns 0

insert(key, value)

Creates a node that contains **a tuple(for python) or an array(for java)** which stores the key-value pair in 0,1 index respectively. Finds index by passing key to `hash_function()`. If there is no collision, the node is placed in the index.

If there is a collision, forward chaining is applied. Here, the chain should be arranged in **descending order**, i.e.: if the node being inserted has the highest value (price), it will be the head of the chain. Otherwise, you should iterate the chain and insert it in the appropriate position.

[If the same key is given twice for insertion, update the value of that particular key. No need to create new Node]

Assignment Tasks [**Need to Submit**]

1. Hashtable: Deletion Operation :

You are given the hash function, $h(\text{key}) = (\text{key} + 3) \% 6$ for a hash-table of length 6. In this hashing, forward chaining is used for resolving conflict and a 6-length array of singly linked lists is used as the hash-table. In the singly linked list, each node has a next pointer, an Integer key and a string value, for example: (4 (key) , "Rafi" (value)). The hash-table stores this key-value pair.

Implement a function **remove(hashTable, key)** that takes a key and a hash-table as parameters. The function removes the key-value pair from the aforementioned hashtable if such a key-value pair (whose key matches the key passed as argument) exists in the hashtable.

Consider, Node class and hashTable are given to you. You just need to complete the remove(hashTable, key) function.

```
class Node:
    def __init__(self, key, value, next=None):
        self.key, self.value, self.next = key, value, next
```

Sample Input and Output:

Some Key-value pairs:

(34, "Abid") , (4, "Rafi"), (6, "Karim"), (3, "Chitra"), (22, "Nilu")

HashTable is given in the following:

0: (3, "Chitra")
1: (22, "Nilu") → (4, "Rafi") → (34, "Abid")
2: None
3: (6, "Karim")
4: None
5: None

remove(hashTable, key=4) returns the changed hashTable where (4, "Rafi") is removed.

New HashTable Output:

0: (3, "Chitra")
1: (22, "Nilu") → (34, "Abid")
2: None
3: (6, "Karim")
4: None
5: None

remove(hashTable, key=9) returns the same given hashTable since 9 doesn't exist in the table.

2. Searching in hashtable :

Complete the `hashFunction()` and `searchHashtable()` methods. **Do not** change the given code; implement only the required methods. Creating and Inserting into a hash table using forward chaining is already done inside of the class. Do not initialize any other instance variable other than the given ones.

- a. `search_hashtable()` → this instance method takes a **key value pair** in the parameter then uses the string key to search for the string in the instance variable `ht`. If the **string s** is found, this method returns **'Found'**, else returns **'Not Found'**.
[Do not implement sequential search, implement the hash based search]
- b. `hashFunction()` → this instance method takes a key-value pair (string, int), calculates the hashed index on key and returns the index. This hash function takes consecutive two letters of the key string, concatenates their ascii values into an integer and sums all the concatenated integers. Then it finds out the modulus of the summation (**think for yourself with which number should we mod the summation**) as the hashed index.
For instance, for a string 'Mortis', the consecutive two letters are Mo, rt, is.
The concatenated integer for
Mo is 77111 (Ascii of M is 77, o is 111);
rt is 114116 (Ascii of r is 114, t is 116);
'is' is 105115 (Ascii of i is 105, s is 115).
The summation is = 77111+114116+105115
Mod the summation with ____ and return the answer as the hashed index.
[fill in the ____ gap with your knowledge]

Note: For an odd length string, add the letter 'N' at the end of it. Thus, 'Morti' becomes 'MortiN' and the consecutive two letters are Mo, rt, iN.

LAB 5 Part 1 : Binary Tree

[CO2]

Instructions for students:

- Complete the following methods.
- You may use Java / Python to complete the tasks.
- DO NOT CREATE a separate folder for each task just follow the given template.
- If you are using **JAVA**, then follow the [Java Template](#).
- If you are using **PYTHON**, then follow the [Python Template](#).

NOTE:

- **YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN ARRAY UNLESS MENTIONED IN THE QUESTION.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

**For Python: List, Negative indexing and append()
and**

**For Java: Static or Instance variables
are STRICTLY prohibited**

Lab Tasks: 1-5

Assignment Tasks: 6-8 (3 tasks)

Total Assignment Mark: 3*5=15

Lab Task 0: Basics of Binary Tree

This is an intro task & there isn't any driver code for this

- ❖ For java, create a **separate folder** for this task and follow the instructions.
- ❖ For Python, create a **separate colab/ipynb/py** file and follow the instructions.

→ Design a TreeNode class.

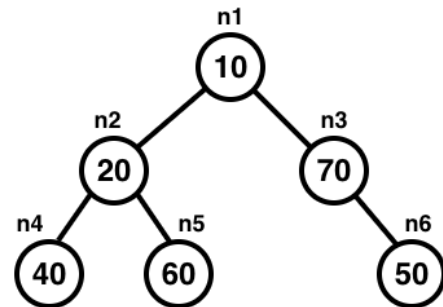
- ◆ Declare three instance variables, one called **elem** (Object data type for java), one called **left** (TreeNode datatype) and another called **right** (TreeNode datatype).
- ◆ Write a constructor that has only one parameter (Object type for Java) and assigns the parameter value to the **elem** instance variable.

→ Design a Tester class (for java) / Design another code cell (for python)

- ◆ Create 6 different objects of TreeNode class. Assign values as shown in the illustration.
- ◆ Variable names should be: **n1, n2, n3, n4, n5, n6**.
- ◆ Connect the 6 TreeNodes as shown in the illustration which will form a binary tree. Here **n1** will be the root of the tree.

Now, execute the lines given below and try to understand the output. You may need pen & paper.

If there are errors, try to figure out why that error occurred and how to fix it.



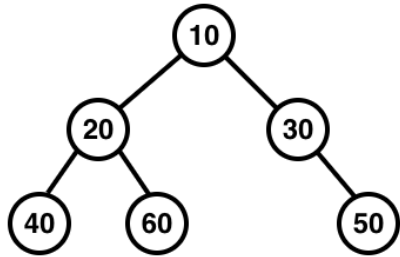
Note: Java & Python output might not always be the same.

JAVA	PYTHON
<code>System.out.println(n1.left);</code>	<code>print(n1.left)</code>
<code>System.out.println(n3.right.elem);</code>	<code>print(n3.right.elem)</code>
<code>TreeNode x = n2.left;</code> <code>System.out.println(n1.elem + x.elem);</code>	<code>x = n2.left</code> <code>print(x.elem + n6.elem)</code>
<code>x = new TreeNode(80);</code> <code>n3.left = x;</code> <code>System.out.println(n1.right.left.elem);</code>	<code>x = TreeNode(80)</code> <code>n3.left = x</code> <code>print(n1.right.left.elem)</code>
<code>System.out.println(n1.left.right + n5.left);</code>	<code>print(n1.left.right + n5.left)</code>
<code>n1.left.right = null;</code> <code>System.out.println(n1.left.right.elem)</code>	<code>n1.left.right = None;</code> <code>print(n1.left.right.elem)</code>

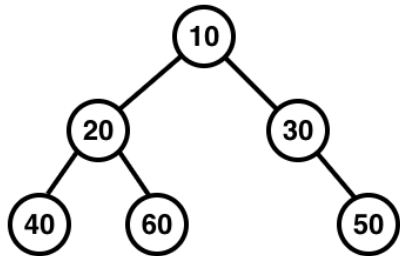
LAB TASKS

1. Basic Traversal:

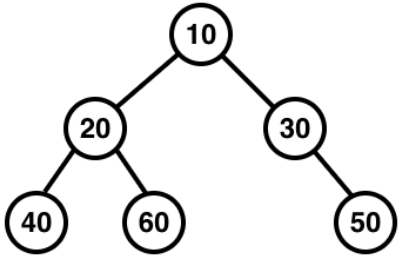
- 1.1. Given the **root** of a binary tree, print the tree **pre-order**.

Sample Given Tree	Sample Output
	10 20 40 60 30 50

- 1.2. Given the **root** of a binary tree, print the tree **post-order** and the levels of each node. The level of the root is 0. You can use a helper.

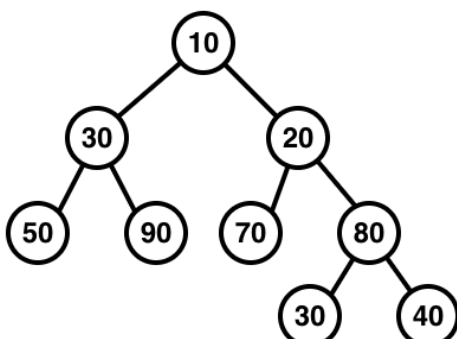
Sample Given Tree	Sample Output
	40:2 60:2 20:1 50:2 30:1 10:0

- 1.3. Given the **root** of a binary tree, print the tree **in-order** and the levels of each node only **if the levels are even**. The level of the root is 0.

Sample Given Tree	Sample Output
	40:2 60:2 10:0 50:2

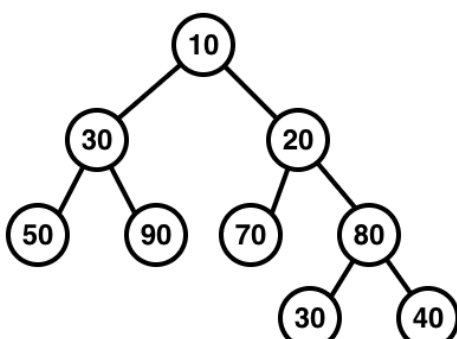
The 3 tasks above should give you a general idea about how traversing a tree works. Try to understand why the root was printed at the very beginning in 1.1 and at the very end in 1.2. You can simulate the recursive calls using pen and paper.

2. Given the **root** of a binary tree, **return** the **total number of nodes** in that tree.

Sample Given Tree	Sample Returned Value
	9

3. Summation of the tree:

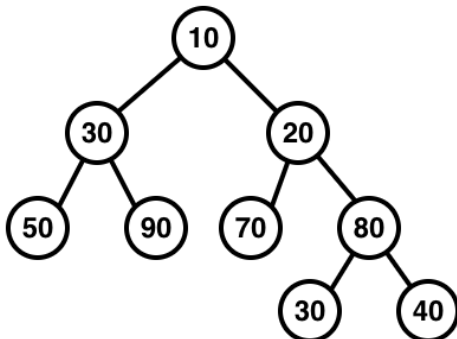
- 3.1. Given the **root** of a binary tree, **return** the summation of nodes in that tree. (You **won't** need helper function)

Sample Given Tree	Sample Output
	420

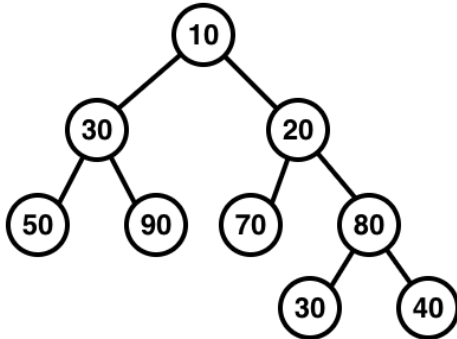
- 3.2. Given the **root** of a binary tree, **print** the summation of nodes in that tree. (think how to re-use 3.1 and try to understand the difference)

Sample Given Tree	Sample Returned Value
Same as above sample	Same as above sample

4. Given the **root** of a binary tree, print all the **leaf** nodes of the tree.

Sample Given Tree	Sample Output
 <pre>graph TD; 10((10)) --> 30((30)); 10 --> 20((20)); 30 --> 50((50)); 30 --> 90((90)); 20 --> 70((70)); 20 --> 80((80)); 80 --> 30((30)); 80 --> 40((40));</pre>	50 90 70 30 40

5. Given the **root** of a binary tree, return the max of all nodes.

Sample Given Tree	Sample Returned Value
 <pre>graph TD; 10((10)) --> 30((30)); 10 --> 20((20)); 30 --> 50((50)); 30 --> 90((90)); 20 --> 70((70)); 20 --> 80((80)); 80 --> 30((30)); 80 --> 40((40));</pre>	90

ASSIGNMENT TASKS

6. Subtraction of Nodes:

Write a **recursive** function **subtract_summation()** that takes the root of a binary tree as a parameter. The function will **subtract** the **summation** of the **right subtree** of the given root **from** the **summation** of the **left subtree** of the given root. Consider, the **Node** class for Binary Tree already defined with **elem**, **left** and **right** variables. You can use helper functions.

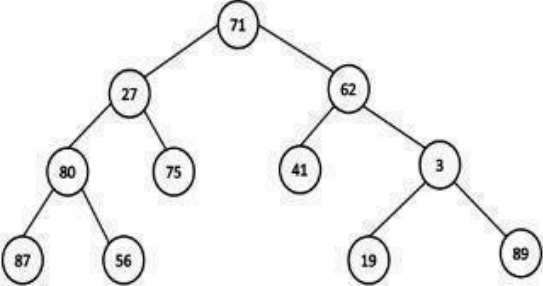
YOU CANNOT USE LIST OR DICTIONARY. You cannot use any built-in function.

Python Notation:

```
def subtract_summation(root):  
    // To do  
    return None
```

Function Call :

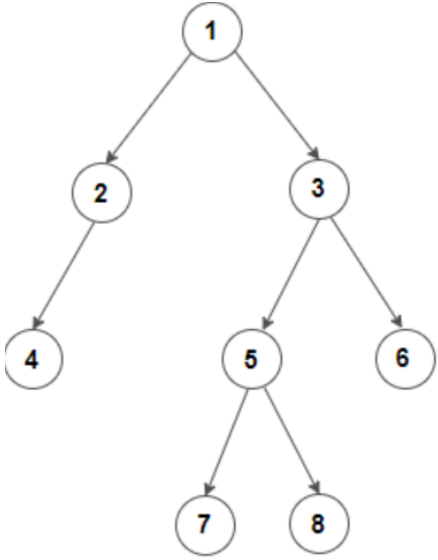
`print(subtract_summation(root))`. Here root refers to the tree below.

Sample Input	Sample Output	Explanation
	111	Summation of left subtree - summation of right subtree = (27+75+80+87+56) - (62+41+3+19+89) = 111

more tasks on the next page>>

7. Difference of Level Sum:

Given a Binary Tree, Write a function that finds the difference between sum of all nodes present at odd and even levels in a binary tree, i.e. sum of all odd level nodes - sum of all even level nodes.

Sample Input:	Sample Output	Explanation
 <pre>graph TD; 1((1)) --> 2((2)); 1 --> 3((3)); 2 --> 4((4)); 3 --> 5((5)); 3 --> 6((6)); 5 --> 7((7)); 5 --> 8((8));</pre>	4	$-1+2+3-4-5-6+7+8 = 4$

more task on the next page>>

8. Swap Children Nodes:

Write a **recursive** function **swap_child()** that takes the root of a binary tree, node's level and a number M as a parameter. The function will swap the left and right children of all the nodes at level M and above. Here, $0 < M < \text{height of the tree (root's height)}$. Consider, the Node class for Binary Tree already defined with elem, left and right variables. **YOU CANNOT USE LIST OR DICTIONARY, any built-in function, global variables.**

Python Notation:

```
def swap_child(root, level, M):  
    # To do
```

Function Call :

swap_child(root, 0, 2). Here root refers to the tree below.

Input Tree	Resulting Tree	Explanation
<pre> A / \ B C / \ \ D E F / \ / \ / G H I J</pre>	<pre> A / \ C B / \ \ F E D / \ / \ / J I G H</pre>	Here $M = 2$ and all the nodes from level 2 and above are swapped left with right. Here above means the level that situated at a higher position of the tree

LAB 5 Part2 : Binary Tree

[CO2]

Instructions for students:

- Complete the following methods.
- You may use Java / Python to complete the tasks.
- DO NOT CREATE a separate folder for each task just follow the given template.
- If you are using **JAVA**, then follow the [Java Template](#).
- If you are using **PYTHON**, then follow the [Python Template](#).

NOTE:

- **YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN ARRAY UNLESS MENTIONED IN THE QUESTION.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

**For Python: List, Negative indexing and append()
and**

**For Java: Static or Instance variables
are STRICTLY prohibited**

Lab Tasks: 3 tasks (1~3)

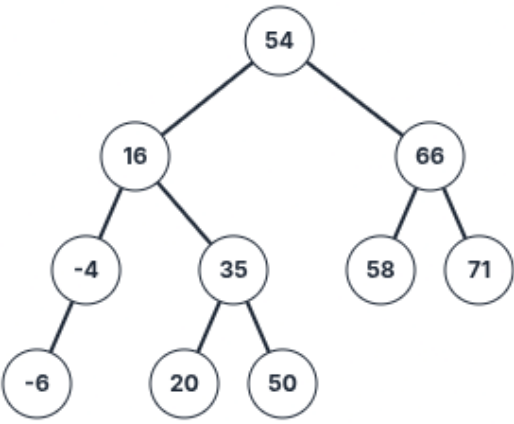
Assignment Tasks: 3 tasks (4~6)

Total Assignment Mark: 3*5=15

LAB TASKS (no need to submit)

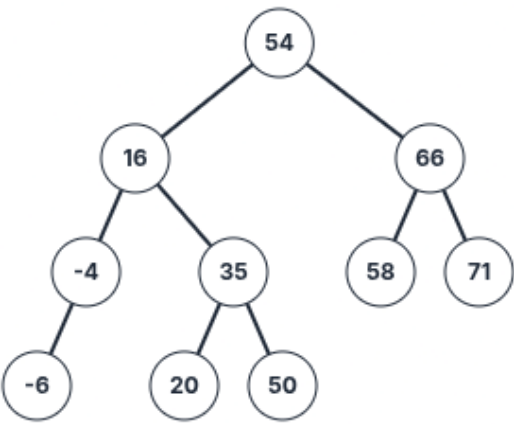
1. Binary Search Tree Minimum and Predecessor:

a. Find the maximum number from a Binary Search Tree

	Sample Call	Result
	getMaximum(root)	71

Note: You have to keep in mind that, the question says it's a Binary Search Tree (not a regular BT). Therefore, you must use BST logic to find the maximum.

b. Find a InOrder Predecessor from a Binary Search Tree

	Sample Call	Result
	inOrderPred(root, 54)	50
	inOrderPred(root, 66)	58
	inOrderPred(root, 20)	16
	inOrderPred(root, 50)	35
	inOrderPred(root, -6)	null/None

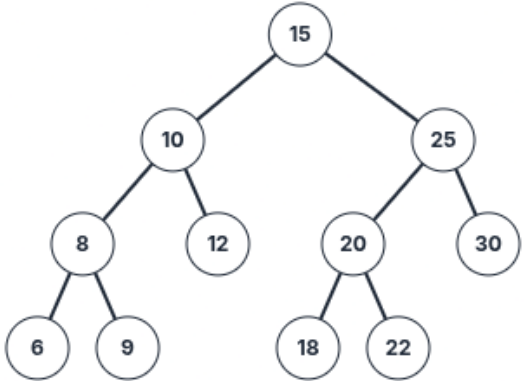
Note: You should reuse the lab task 1a to solve 1b.

You can solve 1a and 1b without using any recursion. Can you guess why we can solve them without recursion even though they are binary tree problems?

2. Find the Lowest Common Ancestor (LCA) of two nodes in a BST

The lowest common ancestor (LCA) of two nodes x and y in the BST is the lowest (i.e., deepest) node that has both x and y as descendants. In other words, the LCA of x and y is the shared ancestor of x and y that is located farthest from the root.

Corner Case: if y lies in the subtree rooted at node x , then x is the LCA; otherwise, if x lies in the subtree rooted at node y , then y is the LCA.

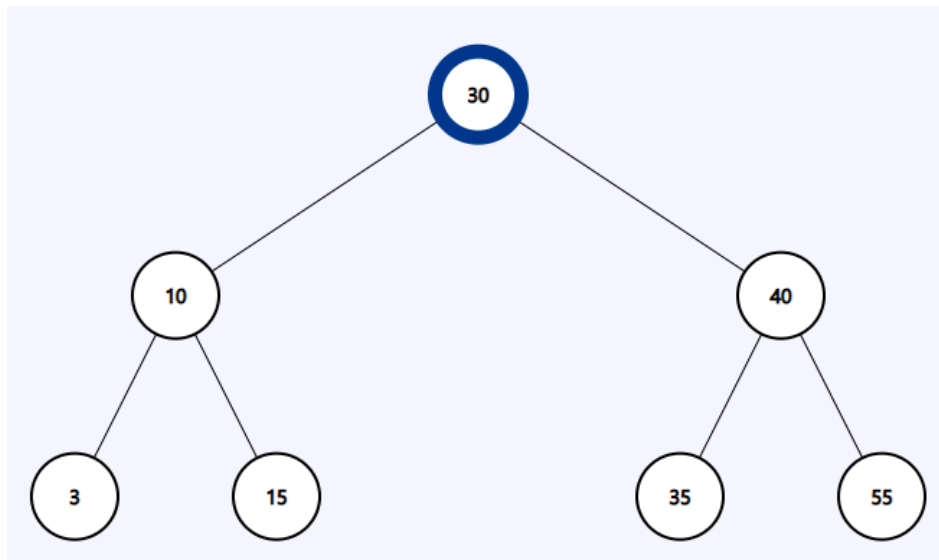
	Sample Call	Result
	LCA(root, 6, 12)	10
	LCA(root, 20, 6)	15
	LCA(root, 18, 22)	20
	LCA(root, 20, 25)	25
	LCA(root, 10, 12)	10

3. Find the path

Faria studies in Rangpur Medical College. During her Eid vacation, she is planning to visit her family and relatives in Chittagong. But she feels really bad during the journey. As a result, you need to help Faria reach her destination.

A Binary Search Tree is given. The root node is the starting position of Faria and a key node will be given as the destination. Now, you have to find if the following key node (which is the destination of Faria) is present in the tree or not. If present, return the path from the root node to the key node else print "No Path Found".

[You can use Python List for this task]

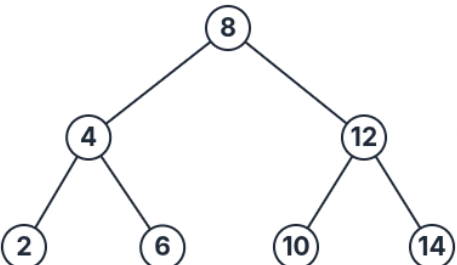
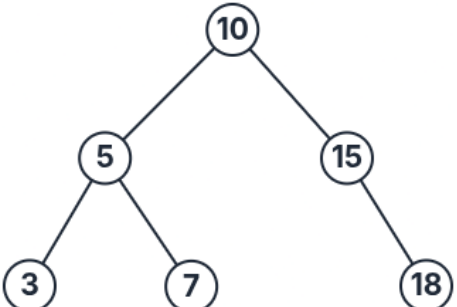


Sample Input	Sample Output
Source node(root): 30 Destination node(key): 15	Path: [30, 10, 15]
Source node(root): 30 Destination node(key): 50	No Path Found

ASSIGNMENT TASKS (**must submit**)

4. Sum of range

Write a function **rangeSum(root,low,high)** that takes the root of a Binary Search Tree as a parameter and sums up the values within the range mentioned in the parameter.

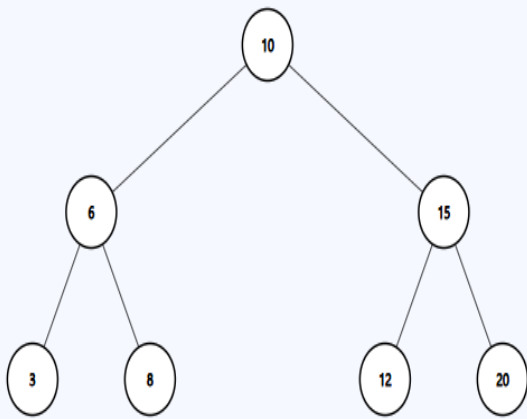
Sample Input	Sample Function Call	Sample Returned Value
	rangeSum(root, 5, 13)	36
		Explanation: 6+8+10+12=36
	rangeSum(root, 7, 15)	32
		Explanation: 7+10+15=32

5. Mirror Sum of a BST

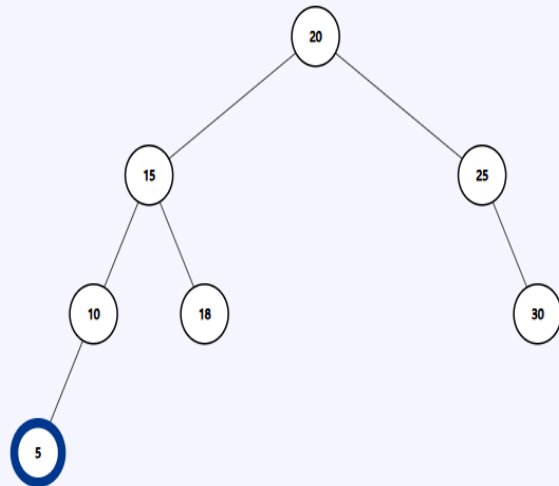
You are given the root node of a Binary Search Tree. You need to calculate the **sum** of the values of the nodes that are **mirrors** of each other then **return** it. Here, mirror means the nodes that are located in corresponding positions in the left and right subtrees of the root. If the mirror doesn't exist then do not sum.

Note: You can use helper functions.

Example Tree input 1



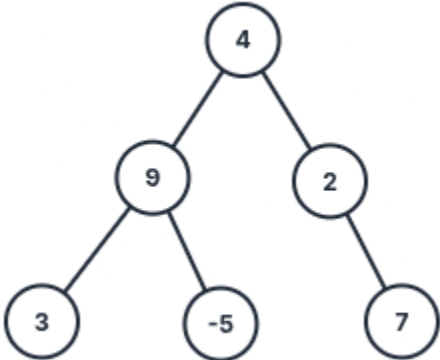
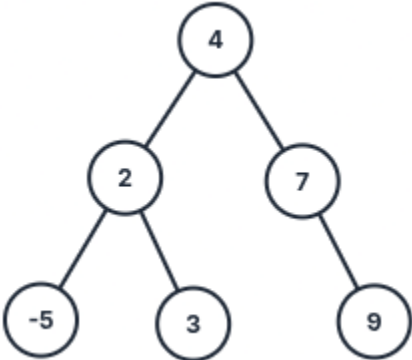
Example Tree input 2



Sample Input 1	Sample Output 1	Explanation 1
mirrorSum(root)	64	For Tree 1 Mirror nodes are: 6 and 15, sum = 6 + 15 = 21 3 and 20, sum = 3 + 20 = 23 8 and 12, sum = 8 + 12 = 20 Total Mirror Node Sum = 21+23+20 = 64
Sample Input 2	Sample Output 2	Explanation 2
mirrorSum(root)	80	For Tree 2 Mirror nodes are: 15 and 25, sum = 15 + 25 = 40 10 and 30, sum = 10 + 30 = 40 Total Mirror Node Sum = 40 + 40 = 80

6. Check BST

Write a function **isBST(root)** that takes the root of a Binary Tree as a parameter and then returns true if its a BST otherwise returns false.

Sample Input	Sample Returned Value
	false
	true

LAB 6 : Heap

[CO2]

Instructions for students:

- Complete the following methods.
- You may use Java / Python to complete the tasks.
- DO NOT CREATE a separate folder for each task.

NOTE:

- YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN ARRAY.
- YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.

Python List, Negative indexing and append() is STRICTLY prohibited

Lab Tasks: 3 tasks (1~3)

Assignment Tasks: 3 tasks (4~6)

Total Assignment Mark: 3*5=15

You were given instructions and driver code for the last 6 lab files. However, for this lab, only Task#1 driver codes will be given. Write the other driver codes yourself in your preferred language (Java / Python).

LAB TASKS

Task1: Implementing an Array-Based Min Heap

Driver Codes for Task#1 only [Java Task#1 Template](#) & [Python Task#1 Template](#)

A **Min Heap** is a specialized binary tree where the value of each node is **less than or equal to its children**. This ensures that the **smallest element is always at the root**.

In this implementation, the heap is represented using an **array**, where the tree structure is maintained through 1-based index relationships.

The MinHeap class contains the following private instance variables:

An Integer array heap	stores the elements of the heap
An integer size	represents the current number of elements in the heap
An integer capacity	represents the maximum number of elements the heap can hold

Instance Methods to Implement	Steps
insert(value) Description: Inserts a new value into the heap while maintaining the heap property.	1. Check if the heap is full (if size is greater or equal to the capacity) → If full, print "Heap is full" and do not insert 2. Insert the value at the end of the array 3. Restore heap property using heapifyUp
extractMin() Description: Removes and returns the smallest element (root of the heap).	1. If the heap is empty, return null 2. Store the root value heap[1] (this will be returned) 3. Move the last element to the root position 4. Decrease the size of the heap 5. Restore heap property using heapifyDown 6. Return the stored minimum value
peek() Description: Returns the smallest element without removing it.	1. If the heap array is empty, return null 2. Return the root element (heap[1])
heapifyUp(index) Description: A private method that restores the heap property by moving an element upward. When used: After insertion	1. Compare the current element with its parent 2. If the parent is greater, swap them 3. Continue until the heap property is satisfied or root is reached

heapifyDown(index) Description: A private method that restores the heap property by moving an element downward. When used: After extracting the minimum element	1. Compare the current element with its children 2. Swap with the smallest child if needed 3. Continue until the heap property is satisfied or reached leaf
isEmpty()	Return True if the heap size is 0, otherwise return False

Task 2:

Complete task 1 for MaxHeap.

Note:

For MaxHeap class, the instance variables & instance methods would be the same as MinHeap except for extractMin(), instead it'll be named extractMax().

The MaxHeap sorts the element in ascending order.

The instance variables must be Private.

Task 3:

Add a **static method** to the class from **Task#1** or **Task#2** (depends on whether you want to use MaxHeap or MinHeap) that performs **out-of-place heap sort** using a heap (min-heap or max-heap). The method will be called **outPlaceHeapsort()** and it takes an array in its parameter. The method will do the following:

- Create a Min-Heap or Max-Heap instance.
- Loop through the given array:
 - Insert each element into the heap.
- Loop until the Max-Heap or Min-Heap is empty:
 - Extract the minimum/maximum element from the minheap/maxheap.
 - put them in their sorted order in the original array

Note: The time complexity of the solution is going to be $n \cdot \log(n)$.

You can utilize the Classes you created for the Lab Task by creating their objects to complete the Assignment Tasks. Just like Stack & Queue tasks, you'll create MaxHeap/MinHeap objects and call their methods. You cannot access the instance variables from outside since they're private.

ASSIGNMENT TASKS

Task 4 [5 marks] :

You are given an array of n tasks, where the value of index i represents the processing time for i -th task. You are also given m machines, each capable of processing one task at a time. Your goal is to distribute the tasks among the machines such that the overall completion time for each machine is minimized. Represent the output as an array where each index corresponds to a machine, and the value at each index is the total processing time of tasks assigned to that machine.

Input Format:

- **tasks:** An array of integers where each integer represents the processing time of a task.
- **m:** An integer representing the number of machines.

Output Format:

- An array of integers of size m , where the value at index i represents the total processing time of tasks assigned to the i -th machine.

Sample Given Input:

tasks = [2, 4, 7, 1, 6]

m = 4

Sample Output:

[2, 4, 7, 7]

Explanation:

To simplify the use of a heap, we focus only on the current load of each machine (not its index). The min-heap will store the current loads of the machines. Initially, all machines will have a load of 0.

Once all tasks are assigned, the heap will contain the loads of all machines. The array of this heap will represent the output.

Steps to Solve:

- Initialize a min-heap object with m length, all set to 0 (representing the load of each machine).
- For each task, do the following:
 - ◆ Extract the smallest load from the heap.
 - ◆ Add the task's processing time to this load.
 - ◆ Reinsert the updated load into the heap.
- The final array representation for your heap is the answer.

Example Walkthrough:

Initial Heap:

Heap Array	[0, 0, 0, 0]
Heap Visual	<pre> 0 / \ 0 0 / 0 </pre>

Task 1 (Processing Time = 2): Assign to the machine with load 0 (current minimum load).

	Extract the smallest	Add the extracted with current & reinsert
Heap Array	0 extracted After extraction: [0, 0, 0, None/null]	0+2=2 is inserted After insertion: [0, 0, 0, 2]
Heap Visual	<pre> 0 / \ 0 0 </pre>	<pre> 0 / \ 0 0 / 2 </pre>

Task 2 (Processing Time = 4): Assign to the machine with load 0 (current minimum load).

	Extract the smallest			Add the extracted with current & reinsert
Heap Array	0 extracted After extraction: [0, 0, 2, None/null]			0+4=4 is inserted After insertion: [0, 0, 2, 4]
Heap Visual	<pre> 0 / \ 0 0 / 2 </pre>	<pre> 2 / \ 0 0 </pre>	<pre> 0 / \ 0 2 / 0 </pre>	<pre> 0 / \ 0 2 / 4 </pre>

Task 3 (Processing Time = 7): Assign to the machine with load 0 (current minimum load)

	Extract the smallest			Add the extracted with current & reinsert
Heap Array	0 extracted After extraction: [0, 4, 2, None/null]			0+7=7 is inserted After insertion: [0, 4, 2, 7]
Heap Visual	<pre> 0 / \ 0 2 / 4 </pre>	<pre> 4 / \ 0 2 </pre>	<pre> 0 / \ 4 2 </pre>	<pre> 0 / \ 4 2 / 7 </pre>

Task 4 (Processing Time = 1): Assign to the machine with load 0 (current minimum load).

	Extract the smallest			Add the extracted with current & reinsert		
Heap Array	0 extracted After extraction: [0, 4, 2, None/null]			0+1=1 is inserted After insertion: [1, 2, 7, 4]		
Heap Visual	<pre> 0 / \ 4 2 / 7 </pre>	<pre> 7 / \ 4 2 </pre>	<pre> 2 / \ 4 7 </pre>	<pre> 2 / \ 4 7 / 1 </pre>	<pre> 2 / \ 1 7 / 4 </pre>	<pre> 1 / \ 2 7 / 4 </pre>

Task 5 (Processing Time = 6): Assign to the machine with the smallest load, that is 1, and add the current processing time (6) with this.

	Extract the smallest			Add the extracted with current & reinsert
Heap Array	1 extracted After extraction: [2, 4, 7, None/null]			1+6=7 is inserted After insertion: [2, 4, 7, 7]
Heap Visual	<pre> 1 / \ 2 7 / 4 </pre>	<pre> 4 / \ 2 7 </pre>	<pre> 2 / \ 4 7 </pre>	<pre> 2 / \ 4 7 / 7 </pre>

Final Output:

Finally, the array representation of the heap is [2, 4, 7, 7]

Hints:

1. Tasks must be assigned one at a time to the machine with the current smallest load.
2. Use a heap (you have built before) to efficiently find the machine with the smallest load.
3. You can create a function that will receive tasks and m and return the final heap for simplicity

Task 5 [5 marks] :

You are given an array of integers and a number k. Your task is to find the top k largest elements from the list. Use a max-heap to solve this problem efficiently.

Input Format:

- nums: An array of integers.
- k: An integer representing the number of largest elements to find.

Output Format:

- An array of k integers representing the largest elements in descending order.

Sample Input:

nums = [4, 10, 2, 8, 6, 7]

k = 3

Output:

[10, 8, 7]

Explanation:

To solve this, we use a max-heap because it allows efficient retrieval of the largest elements.

The process involves:

1. Insert all numbers from the input list into a max-heap.
2. Create a new array with size k.
3. Extract the maximum k times and store it to the new array to get the top k largest elements.
4. Return the result array in descending order.

Steps to Solve:

1. Build a max-heap where the length of the array will be equal to length of nums.
2. Extract the maximum k times and store the values in the newly created result array.
3. Return the result array.

Hints:

1. You must use a max-heap (**created above in Task2**) to solve the problem.
2. k is always smaller than or equal to the size of the array.
3. You can create a function that will receive the nums array and k for simplicity.

For TASK#6 You need build a new custom MaxHeap

Task 6 [5 marks] : Task Scheduler with Priority Queue

You are given two arrays: one containing task names and another containing their corresponding priorities. Your task is to process all tasks in order of their priority using a Max-heap. Tasks with higher priority values should be processed first.

Input Format:

- *task_names*: An array of strings where each element represents a task name.
- *priorities*: An array of integers where each element represents the priority of the corresponding task.

Output Format:

- An array of strings representing the task names in the order they should be processed (highest priority first).

Steps to Solve:

1. Build a customized max-heap where each element contains both the task name and its priority.
2. Iterate through both arrays simultaneously and insert each (task_name, priority) pair into the max-heap (heap should compare based on priority).
3. Extract the maximum repeatedly until the heap is empty, storing each task name in the result array.
4. Return the result array containing all task names in priority order.

Sample Input

task_names = ["Email", "Meeting", "Code Review", "Lunch", "Debug"]
priorities = [2, 5, 3, 1, 4]

Sample Returned Priorities Array

["Meeting", "Debug", "Code Review", "Email", "Lunch"]

Explanation

To solve this problem, we use a max-heap like a priority queue because it allows efficient retrieval of the highest priority task. The process involves:

1. Combine task names with their priorities and insert all tasks into a max-heap based on their priority values.
2. Create a result array to store task names in processing order.
3. Extract the maximum (highest priority task) repeatedly until the heap is empty.
4. Return the result array containing task names in descending order of priority.

LAB 7 : Graph

[CO5]

Instructions for students:

- Complete the following methods.
- You may use Java / Python to complete the tasks.
- DO NOT CREATE a separate folder for each task.

NOTE:

- YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN ARRAY UNLESS MENTIONED IN THE QUESTION.
- YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.

Python List, Negative indexing and append() is STRICTLY prohibited unless mentioned in the question

Lab Tasks: 4 tasks (0~1b)

Assignment Tasks: 6 tasks (2a~4b)

Total Assignment Mark: 6*5=30

Dear Students, you have been given instructions and driver code for the majority of the labs. For this lab no driver code will be given. You will develop everything (necessary functions, class, driver code, etc.) in your preferred language (Java or Python).

To solve the problems, draw a graph (<https://graphonline.top>) according to your preference. However, your graph must have a minimum of **7 vertices** and a minimum of **12 edges**. You can bring the graph you drew on the website to your code by going to Graph menu -> Adjacency Matrix menu and copying the matrix.

Note: Solve all the problems for both adjacency matrix representation and adjacency list representation. You can write different functions for each task to make your life easier.

Task 0: [Lab Task]

Represent the Graph you just drew in your code. Using both **Adjacency Matrix (Task0a)** and [Adjacency List \(Task0b\)](#).

Task 1: [Lab Task]

Given an undirected, unweighted graph as input, write a function to find the vertex with maximum degree and return the degree of that vertex. Solve it for Adjacency Matrix (**Task1a**) & Adjacency List (**Task1b**).

Task 2: [Assignment Task]

Given an undirected, edge-weighted graph as input, write a function to find the vertex whose sum of edge weights is maximum. Solve it for Adjacency Matrix (**Task2a**) & Adjacency List (**Task2b**).

Task 3: [Assignment Task]

Solve Task 1 and Task 2 for directed, edge-weighted graphs considering only outgoing edges. Solve it for Adjacency Matrix (**Task3a**) & Adjacency List (**Task3b**).

Task 4: [Assignment Task]

Write a function that will receive a directed weighted graph and convert it to an undirected weighted graph. Consider the weight of the edges will be unchanged. If there's a case where you have a bidirectional edge then you can sum up their weights. Solve it for Adjacency Matrix (**Task4a**) & Adjacency List (**Task4b**).

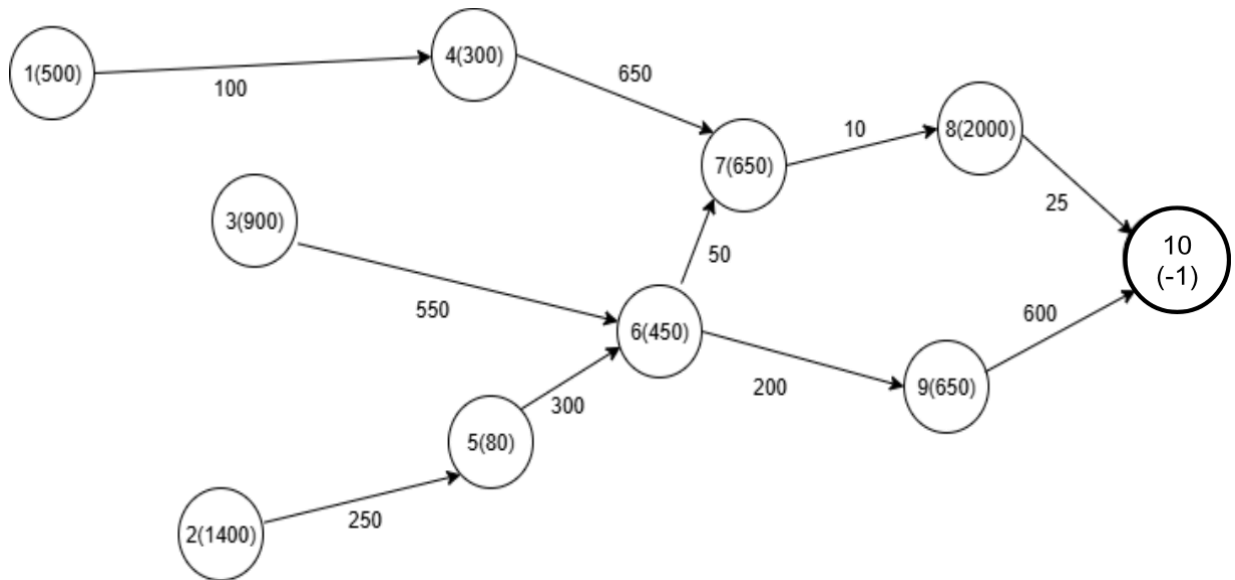
UNGRADED CODING TASK [[Java Template](#),[Python Template](#)]

[**Note:** Try solving this coding question using Adjacency Matrix as well]

Luffy and Shanks start from given islands with initial power and aim to reach **Laugh Tale (vertex warLord power value is -1)**. Between islands, they fight a **Marine Officer (power given as edge weight)**. On each island, they fight a **Warlord (power given as vertex value)**. They always choose the next island with the **minimum (Marine Officer + Warlord) power**. Furthermore, stamina decreases after every single fight. If a pirate's stamina becomes less than the Warlord's power at any island, they cannot continue and fail.

Winner rules:

- If only one of them reaches Laugh Tale and the other can't → the one who reaches wins.
- If both reach Laugh Tale → the one with higher remaining stamina wins.
- If none reach Laugh Tale → Winner is no one.



[You can assume there's won't be any cyclic paths from the starting point to Laugh Tale]

[You can write helper functions if needed]

Python Method Signature

```
def laughTale(wL, adjL, sSt, sP, lSt, lP):  
    #TODO
```

Java Method Signature

```
public static void laughTale(Integer[] warLords, Edge[] adjL, int sSt, int sP, int lSt, int lP){  
    //TODO  
}
```

Warlord Powers, Sample Graph & Edge Class		
warLords powers : [null, 500, 1400, 900, 300, 80, 450, 650, 2000, 650, -1]		
Given Adjacency List : 0 : null 1 : <4,100> -> null 2 : <5,250> -> null 3 : <6,550> -> null 4 : <7,650> -> null 5 : <6,300> -> null 6 : <7,50> -> <9,200> -> null 7 : <8,10> -> null 8 : <10,25> -> null 9 : <10,600> -> null 10 : null	Given Java Edge Class	
	<pre>class Edge{ int toV, weight; Edge next; public Edge(int t, int w){ this.toV = t; this.weight = w; } }</pre>	
	Given Python Edge Class	
	<pre>class Edge: def __init__(self, toV, weight): self.toV = toV self.weight = weight self.next = None</pre>	
Sample Starting Vertex and Graph		Output
sSt = 2, sP = 7000, lSt = 1, lP = 3000		Winner is Shanks
Explanation		
sSt: Shank's start from Island: 2 sP: Shanks's power: 7000 lSt: Luffy starts from Island: 1 lP: Luffy's power: 3000 Luffy's path: vertex 1 -> vertex 4 -> vertex 7 -> vertex 8 Luffy's power level: 3000-500-100-300-650-650-10=790 <i>[no more power left to reach vertex 10]</i> Shanks's path: vertex 2 -> vertex 5 -> vertex 6 -> vertex 7 -> vertex 8 -> vertex 10 Shanks's power level: 7000-1400-250-80-300-450-50-650-10-2000-25= 1785. So, Luffy cannot reach Laugh Tale and the Winner is Shanks.		
Sample Starting Vertex and Graph		Output
sSt = 2, sP = 7000, lSt = 3, lP = 6500		Winner is Luffy
sSt: Shank's start from Island: 2 sP: Shanks's power: 7000 lSt: Luffy starts from Island: 3 lP: Luffy's power: 6500 Luffy's path: vertex 3 -> vertex 6 -> vertex 7 -> vertex 8 -> vertex 10 Luffy's power level: 6500-900-550-450-50-650-10-2000-25=1865		

Shanks's path: vertex 2 -> vertex 5 -> vertex 6 -> vertex 7 -> vertex 8 -> vertex 10

Shanks's power level: $7000 - 1400 - 250 - 80 - 300 - 450 - 50 - 650 - 10 - 2000 - 25 = 1785$

Since $1865 > 1785$. So, the winner is Luffy.

CSE 221

CSE221: Algorithms

Lab Assignment 01

Fall 2023

Submission Guidelines:

1. You can code all of them either in Python, Java or CPP. But you should choose a specific language for all tasks.
 2. For each task write separate python files like task1.py, task3.py and so on.
 3. Add a hand written explanation of 3-4 lines for each of your solutions and submit that as a single document.
 4. For each problem, take input from files called "inputX.txt", and output at "outputX.txt", where X is the task number. So, for problem 2, in your code, the input file is basically this, "input2.txt". Same for the output file.
 5. Finally zip all the files and rename this zip file as per this format: LabSectionNo_ID_CSE221LabAssignmentNo_Fall2023.zip.
[Example: LabSection01_21101XXX_CSE221LabAssignment01_Fall2023.zip]
 6. You MUST follow all the guidelines, and naming/file/ziping convention stated above. **Failure to do so will result in a straight 50% mark deduction.**
 7. Don't copy from your friends. **For plagiarism you will get 0.**
-

Task 1:

- a) You are given a file "**input1a.txt**". The first line of the input file will contain an integer **T**, representing the number of test cases. The next T lines will contain an integer **N**. You have to calculate if the number is Odd or Even. For each test case print the expected output. All the results must be compiled in a single file, "**output1a.txt**."

Sample Input File	Sample Output File
5 10 19 7 3 100	10 is an Even number. 19 is an Odd number. 7 is an Odd number. 3 is an Odd number. 100 is an Even number.

- b) You are given a file “**input1b.txt**”. The first line of the input file will contain an integer **T**, representing the number of test cases. The next T lines will contain a single arithmetic expression. Each arithmetic expression will start with the prefix “calculate”. It is guaranteed that the expression will have exactly two operands and one operator.

Calculate the result of each expression. Your output format should exactly match with the sample output format. All the results must be compiled in a single file, “**output1b.txt**.”

Sample Input File	Sample Output File
15 calculate 67 + 41 calculate 85 / 5 calculate 13 - 56 calculate 99 - 95 calculate 3 / 10 calculate 12 * 19 calculate 14 - 6 calculate 3 * 88 calculate 45 * 68 calculate 81 - 0 calculate 77 + 40 calculate 8 * 84 calculate 73 - 22 calculate 85 - 86 calculate 28 * 58	The result of 67 + 41 is 108 The result of 85 / 5 is 17.0 The result of 13 - 56 is -43 The result of 99 - 95 is 4 The result of 3 / 10 is 0.3 The result of 12 * 19 is 228 The result of 14 - 6 is 8 The result of 3 * 88 is 264 The result of 45 * 68 is 3060 The result of 81 - 0 is 81 The result of 77 + 40 is 117 The result of 8 * 84 is 672 The result of 73 - 22 is 51 The result of 85 - 86 is -1 The result of 28 * 58 is 1624

Task 2:

Here is the code of bubble sort. Its run time complexity is $\theta(n^2)$. Change the code in a way so that its time complexity is $\theta(n)$ for the **best-case** scenario.

You have to **explain how you have achieved the $\theta(n)$ for the best-case scenario in a comment block of your code.**

```
def bubbleSort(arr):  
    for i in range(len(arr)-1):  
        for j in range(len(arr)-i-1):  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]
```

Input 1: 5 3 2 1 4 5	Output 1: 1 2 3 4 5
Input 2: 6 5 10 15 20 25 30	Output 2: 5 10 15 20 25 30

For the input 2, your code should run at $\theta(n)$.

Please note, you have to take the input from an input2.txt file, and show the output in an output2.txt file.

Task 3:

Suppose you are given a task to rank the students. You have gotten the marks and id of the students. Now your task is to rank the students based on their marks using a sorting algorithm.

However, you have to keep in mind that your sorting algorithms perform the minimum number of swapping operations.

Input:

The first line of the input file will contain an integer **N** ($1 \leq N \leq 1000$).

The second line will contain N integers, representing the Student ID, S_i ($1 \leq S_i \leq 1000$).

The next line will contain the N integer, S_m ($1 \leq S_m \leq 1000$), which denotes the obtained mark of the corresponding students.

Output:

You have to show the Student Id and obtained marks in descending order based on their obtained mark. If two or more students get the same mark, then students with the lower ID will get prioritized. See the input and output for a better understanding.

Input 1: 7 7 4 9 3 2 5 1 40 50 50 20 10 10 10	Output 1: ID: 4 Mark: 50 ID: 9 Mark: 50 ID: 7 Mark: 40 ID: 3 Mark: 20 ID: 1 Mark: 10 ID: 2 Mark: 10 ID: 5 Mark: 10
Input 2: 4 7 2 5 3 80 60 80 50	Output 2: ID: 5 Mark: 80 ID: 7 Mark: 80 ID: 2 Mark: 60 ID: 3 Mark: 50

Please note, you have to take the input from an input3.txt file, and show the output in an output3.txt file

Task 4:

You have been recently recruited as the Software Engineer at Jumanji Railway Software System. You have a big task at hand. You will be given the N ($1 \leq N \leq 100$) schedule of the train. The next N line will contain the name of the train and the departure time. See the input format for better understanding.

Your task is to write a sorting algorithm that will group the trains in the lexicographical order based on the name of the trains. If two or more trains have the same name, then

the train with the latest departure time will get prioritized. If there is still a tie, then the train which comes first in the input file will come first.

Sample Input	Sample Output
13 ABCD will departure for Mymensingh at 00:30 DhumketuExpress will departure for Chittagong at 02:30 SubornoExpress will departure for Chittagong at 14:30 ABC will departure for Dhaka at 17:30 ShonarBangla will departure for Dhaka at 12:30 SubornoExpress will departure for Rajshahi at 14:30 ABCD will departure for Chittagong at 01:00 SubornoExpress will departure for Dhaka at 11:30 ABC will departure for Barisal at 03:00 PadmaExpress will departure for Chittagong at 20:30 ABC will departure for Khulna at 03:00 ABCE will departure for Sylhet at 23:05 PadmaExpress will departure for Dhaka at 19:30	ABC will departure for Dhaka at 17:30 ABC will departure for Barisal at 03:00 ABC will departure for Khulna at 03:00 ABCD will departure for Chittagong at 01:00 ABCD will departure for Mymensingh at 00:30 ABCE will departure for Sylhet at 23:05 DhumketuExpress will departure for Chittagong at 02:30 PadmaExpress will departure for Chittagong at 20:30 PadmaExpress will departure for Dhaka at 19:30 ShonarBangla will departure for Dhaka at 12:30 SubornoExpress will departure for Chittagong at 14:30 SubornoExpress will departure for Rajshahi at 14:30 SubornoExpress will departure for Dhaka at 11:30

Please note, you have to take the input from an input4.txt file, and show the output in an output4.txt file.

CSE221

Lab Assignment 02

Fall 2023

Submission Guidelines:

1. You can code all of them either in Python, Java or CPP. But you should choose a specific language for all tasks.
2. For each task write separate python files like task1.py, task3.py and so on.
3. Add a hand written explanation of 3-4 lines for each of your solutions and submit that as a single document.
4. For each problem, take input from files called "inputX.txt", and output at "outputX.txt", where X is the task number. So, for problem 2, in your code, the input file is basically this, "input2.txt". Same for the output file.
5. Finally zip all the files and rename this zip file as per this format:
LabSectionNo_ID_CSE221LabAssignmentNo_Fall2023.zip.
[Example:
LabSection01_21101XXX_CSE221LabAssignment01_Fall2023.zip]
6. You MUST follow all the guidelines, and naming/file/zippping convention stated above. Failure to do so will result in a straight 50% mark deduction.
7. Don't copy from your friends. For plagiarism you will get 0.

Task 01: [Points: 15]

Your little brother, Alice, is very fond of playing with integers. One day, Alice was given a sorted list of **N** integers in ascending order by his school teacher. Now, your brother wants to play a game with you.

Alice will give you an integer, **S**. You have to find if it is possible to find two values from the list (at distinct positions) whose sum is equal to S.

Now you are feeling very tired. So you decided to write a code, so that it can give you the answer very quickly.

- 1) Can you write an $O(N^2)$ Solution to solve the problem? **[Points 5]**
- 2) Come up with an $O(N)$ solution. **[Points 10]**

Input

The first line contains two integers N and S ($1 \leq N \leq 10^5$, $1 \leq S \leq 10^9$), denoting the length of the list, and the target Sum.

In the next line, there will be N integers $a_1, a_2, \dots, a_N$ ($1 \leq a_i \leq 10^9$) separated by space.

Output

Print two integers: **the positions** of the values [**1 based indexing**]. If there are several solutions, you may print any of them. If there are no solutions, print "IMPOSSIBLE".

Sample Input 1	Sample Output 1
4 10 1 3 5 7	2 4
Sample Input 2	Sample Output 2
6 18 1 5 8 9 9 10	3 6 [4 5 is also a valid answer] [print only one output]

Sample Input 3	Sample Output 3
4 7 2 4 6 8	IMPOSSIBLE
Sample Input 4	Sample Output 4
4 10 1 5 6 8	IMPOSSIBLE

Task 02: [Points: 15]

Alice and Bob are two friends. Alice has a sorted list in ascending order of length **N**. On the other hand, Bob has a sorted list of length **M**. Now, they want to make a sorted list of **N+M** length in ascending order. However, they are not very good at algorithms. Hence, they asked for your help.

Since you are a computer science student, your task is to come up with an efficient algorithm. In the following, let $n = N+M$.

- 1) Find a solution which runs in $O(n \log n)$. **[Points 5]**
- 2) Come up with a solution which runs in $O(n)$. **[Points 10]**

Input

The first line contains an integer N ($1 \leq N \leq 10^5$), denoting the length of Alice's sorted list. In the next line, there will be N integers separated by space.

The third line contains another integer M ($1 \leq M \leq 10^5$), denoting the length of Bob's sorted list. In the next line, there will be M integers separated by space.

All the numbers given in the input will fit in a 32-bit signed integer.

It is guaranteed that the given lists will be in sorted order.

Output:

You have to make a sorted list in ascending order from the given lists in ascending order and show the output.

Sample Input/Output:

Sample Input 1	Sample Output 1
4 1 3 5 7 4 2 2 4 8	1 2 2 3 4 5 7 8
Sample Input 2	Sample Output 2
3 2 10 12 6 3 4 6 7 8 9	2 3 4 6 7 8 9 10 12
Sample Input 3	Sample Output 3
4 1 2 3 4 1 10	1 2 3 4 10
Sample Input 4	Sample Output 4
7 2 3 8 8 10 12 14 9 1 1 4 5 6 8 13 15 16	1 1 2 3 4 5 6 8 8 8 10 12 13 14 15 16

Task 3 [10 Marks]

You are a busy person with lots of tasks to do. You have a schedule of tasks represented by intervals of time, where each interval represents a task that you need to complete. However, you can only work on one task at a time, and you want to complete as many tasks as possible.

Given a list of N intervals of time, your task is to determine the maximum number of tasks you can complete and which tasks they are.

Input

The input consists of a single integer N ($1 \leq N \leq 10^5$), the number of tasks, followed by N lines representing the tasks. Each task is represented by two integers S_i and E_i ($0 \leq S_i \leq E_i \leq 10^9$), the start and end times of the task, respectively.

Output

Output a single integer k , the maximum number of tasks you can complete, followed by a line with k intervals of the tasks you can complete.

If there are multiple solutions with the same maximum number of tasks, print any one of them.

Sample Input/Output:

Sample Input 1	Sample Output 1
6 1 3 2 5 3 7 4 6 6 8 7 9	3 1 3 4 6 6 8
Sample Input 2	Sample Output 2
5 1 4 2 5 6 7 4 8 3 6	2 1 4 6 7

Sample Input 3	Sample Output 3
7 0 4 3 4 1 5 9 10 6 9 2 3 1 2	5 1 2 2 3 3 4 6 9 9 10

Task 4 [10 Marks]

Given N tasks and M people, where each task has a start time and end time, implement a greedy algorithm to find the maximum number of tasks that can be completed by M people.

Each task can only be completed by one person and a person can only be assigned one task at a time. Two tasks cannot be completed simultaneously by the same person.

Input

The input consists of two integers N and M ($1 \leq N, M \leq 10^3$), the number of activities and the number of people, respectively. This is followed by N lines representing the activities. Each line contains two integers S_i and E_i ($0 \leq S_i \leq E_i \leq 10^9$), representing the start and end times of the activity, respectively.

Output

Output a single integer representing the maximum number of activities that can be completed.

Sample Input/Output:

Sample Input 1	Sample Output 1
5 2 1 5 3 6 2 5 8 10 6 9	4
Sample Input 2	Sample Output 2
5 2 1 4 2 5 6 7 4 8 3 6	4
Sample Input 3	Sample Output 3
5 2 1 10 2 10 6 7 4 8 3 6	3
Sample Input 4	Sample Output 4
8 3 5 7 2 4 6 8 8 10 1 3 7 9 3 5 2 6	8

Sample Input Explanation:

In sample input 2-

Person 1 will complete the tasks: 1-4, 4-8

Person 2 will complete the tasks: 2-5, 6-7

Brainstroming:

There is no time constraint for this task 4. However, can you solve the problem in $O(n \log n)$?

CSE221
Lab Assignment 03
Fall 2023

Submission Guidelines:

1. You can code all of them either in Python, CPP, or Java. But you should choose a specific language for all tasks.
2. For each task write separate python files like task1.py, task2.py, and so on.
3. For each problem, take input from files called "**inputX.txt**" and output at "**outputX.txt**", where X is the task number.
4. Add a hand written explanation of 3-4 lines for each of your solutions in a separate document. You may compile all of your explanations in a single file.
5. Finally zip all the files and rename this zip file as per this format: **LabSectionNo_ID_CSE221LabAssignmentNo_Fall2023.zip**
[Example: **LabSection01_21101XXX_CSE221LabAssignment03_Fall2023.zip**]
6. Don't copy from your friends.
7. You MUST follow all the guidelines, naming/file/zipping convention stated above.

Failure to follow instructions will result in a straight 50% mark deduction.

Task 01: [Points 10]

In this problem, you will be given a list of numbers. You have to sort the list using the Merge Sort algorithm.

Pseudocode of Merge Sort Algorithm:

```
def merge(a, b):
    # write your code here
    # Here a and b are two sorted list
    # merge function will return a sorted list after merging a and b

def mergeSort(arr):
    if len(arr) <= 1:
        return arr
```

```

else:
    mid = len(arr)//2
    a1 = mergeSort(.....) # write the parameter
    a2 = mergeSort(.....) # write the parameter
    return merge(a1, a2)    # complete the merge function above

```

Note: You already have coded the merge function. Do you know in which task?

Input

The first line contains an integer N ($1 \leq N \leq 10^5$), denoting the length of Alice's sorted list. In the next line, there will be N integers separated by space.

Output:

You have to sort the number using the Merge Sort algorithm and show the sorted list.

Sample Input/Output:

Sample Input 1	Sample Output 1
8 9 5 4 6 1 3 2 9	1 2 3 4 5 6 9 9
Sample Input 2	Sample Output 2
1 10	10
Sample Input 3	Sample Output 3
6 8 1 4 2 1 3	1 1 2 3 4 8
Sample Input 4	Sample Output 4
7 7 6 5 4 3 2 1	1 2 3 4 5 6 7

Task 02 [10 Points]:

Alice and you are playing with a list of **N** non negative integers. Today you will try to find out the maximum number of the list. Alice writes the following code to find the maximum number.

```
maxValue = arr[0]
for i in range(1,N):
    if maxValue < arr[i]:
        maxValue = arr[i]
```

Recently you have learned merge sort. Now, you are thinking if you can use the divide and conquer approach to find out the maximum from the given list.

Please note, you are not allowed to sort the given list. The motive for this task is not sorting but to use the concepts of divide and conquer approach.

Input

The first line contains an integer N ($1 \leq N \leq 10^5$), denoting the length of Alice's list. In the next line, there will be N integers separated by space.

Output:

You have to find out the maximum value from the list using the divide and conquer approach.

Sample Input/Output:

Sample Input 1	Sample Output 1
8 1 7 13 4 5 7 13 12	13
Sample Input 2	Sample Output 2

7 5 15 2 3 10 1 9	15
Sample Input 3	Sample Output 3
1 9	9
Sample Input 4	Sample Output 4
6 5 2 3 10 1 9	10

What is the time complexity of your code?

Task 03 [10 Points]:

Somewhere in the universe, the Biannual Regional Alien Competition is taking place.

There are **N** aliens standing in a line. You will be given a permutation of N, which denotes the height of each alien. A sequence of N numbers is called a permutation if it contains all integers from 1 to N exactly once. For example, the sequences [3,1,4,2], [1] and [2,1] are permutations, but [1,2,1], [0,1] and [1,3,4] – are not.

In the competition, for each alien, the judge wants to count how many aliens are standing on its right side with a strictly smaller height. Then the judge wants to add up all the counts. To do this, the judge writes the following piece of code.

```
count = 0
for i in range(n):
    for j in range(i+1,n):
        if H[i] > H[j]:
            count+=1
```

However, their algorithm wasn't efficient at all. Hence, the alien calls you to write a better solution for the program.

More formally, you have to count how many pairs of aliens are standing in the line such that $H[i] > H[j]$ and $i < j$. Here, A is a permutation of the aliens' heights. And i, j denote the Aliens' positions.

Input

The first line contains a single integer $1 \leq N \leq 10^6$ - the number of total aliens.

The next line contains N integers $H_1, H_2, \dots, H_n (1 \leq H_i \leq N)$ - the height of the i -th alien. It is guaranteed that the given heights will be a permutation of N .

Output

Print a single integer, which denotes the total number of pairs (i, j) such that $i < j$ and $H_i > H_j$.

Sample Input/Output:

Sample Input 1	Sample Output 1
5 1 2 3 4 5	0
Sample Input 2	Sample Output 2
5 5 4 3 2 1	10
Sample Input 3	Sample Output 3
8 2 7 4 1 5 6 8 3	11

Sample Input 3 Explanation:

In the sample input 3, the following pairs on alien's heights satisfy the condition: (2,1), (7,4), (7,1), (7,5), (7,6), (7,3), (4,1), (4,3), (5,3), (6,3), (8,3)

Task 04 [10 points]

You are given a list A of N integers. You have to choose two indices i and j such that $1 \leq i < j \leq N$ and $A[i] + A[j]^2$ is maximum possible. Here, we are considering 1-based indexing.

Write a code which will find the maximum value of $A[i] + A[j]^2$ in $O(N \log N)$.

Input

The first line contains a single integer $1 \leq N \leq 10^6$ - the length of the list.

The next line contains N integers $A_1, A_2, \dots, A_n$ ($-10^8 \leq A_i \leq 10^8$) separated by a space.

Output

Print a single integer - which denotes the maximum possible value of $A[i] + A[j]^2$.

Sample Input/Output:

Sample Input 1	Sample Output 1
5 9 6 5 8 2	73
Sample Input 2	Sample Output 2

8 5 10 4 -3 1 6 -10 2	110
Sample Input 3	Sample Output 3
7 -5 -2 -6 -7 -1 8 2	63

Task 05 [10 Points]

In this problem, you will be given a list of numbers. You have to sort the list using the Quick Sort algorithm in ascending order.

Pseudocode of Quick Sort Algorithm:

QUICKSORT(A, p, r)

```

1  if  $p < r$ 
2       $q = \text{PARTITION}(A, p, r)$ 
3      QUICKSORT( $A, p, q - 1$ )
4      QUICKSORT( $A, q + 1, r$ )

```

PARTITION(A, p, r)

```

1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 

```

[The code snippet has been taken from the book: Introduction to Algorithms]

Input

The first line contains an integer N ($1 \leq N \leq 10^5$), denoting the length of Alice's list. In the next line, there will be N integers separated by space.

Output:

You have to sort the number using the Quick Sort algorithm in ascending order and show the sorted list.

Sample Input/Output:

Sample Input 1	Sample Output 1
8 9 5 4 6 1 3 2 9	1 2 3 4 5 6 9 9
Sample Input 2	Sample Output 2
1 10	10
Sample Input 3	Sample Output 3
6 8 1 4 2 1 3	1 1 2 3 4 8
Sample Input 4	Sample Output 4
7 7 6 5 4 3 2 1	1 2 3 4 5 6 7

Task 06 [10 Points]

In this problem, you will be given a list of numbers. You have to find the k -th smallest value from the list without sorting using the Partition function of Quick sort.

We will consider the 1-based indexing of the list.

Input

The first line contains an integer N ($1 \leq N \leq 10^6$), denoting the length of the list.

The next line contains N integers $A_1, A_2, \dots, A_n$ ($1 \leq A_i \leq 10^6$) separated by a space.

The third line contains a single integer Q ($1 \leq Q \leq 100$) - which denotes the number of queries you have to answer.

Each of the next Q lines will contain a single integer K ($1 \leq K \leq N$).

Output:

For each query, you have to find the K -th smallest number from the given list.

Sample Input/Output:

Sample Input 1	Sample Output 1
9 // Total Elements 10 11 10 6 7 9 8 15 2 4 // Total queries 5 3 2 7	9 7 6 10

CSE221

Lab Assignment 04

Fall 2023

Submission Guidelines :

1. You can code all of them either in Python, CPP, or Java. But you should choose a specific language for all tasks.
2. For each task write separate python files like task2.py, task3.py, and so on. For problems that have subproblems, name those like task1A.py, task1B.py, and so on.
3. Add a hand written explanation of 3-4 lines for each of your solutions and submit that as a single document.
4. For each problem, take input from files called **"inputX_Y.txt"** and output at **"outputX_Y.txt"**, where X is the task number and Y is the sample i/o number. For example, for problem 2 sample 1, the input file is this, "input2_1.txt". For problems that have subproblems, name the files like "input1a_1.txt", "output1a_1.txt" and so on. Same for output.
5. For each task include at least one input file (if any) in the submission folder.
6. Finally zip all the files and rename this zip file as per this
format: **LabSectionNo_ID_CSE221LabAssignmentNo_Fall2023.zip**
[Example: **LabSection01_21101XXX_CSE221LabAssignment05_Fall2023.zip**]
7. Don't copy from your friends.
8. You MUST follow all the guidelines, naming/file/zipping convention stated above.

Failure to follow instructions will result in a straight 50% mark deduction.

A useful tool for making graphs:

https://csacademy.com/app/graph_editor/

You can find all the Input Files of this lab from here:

<https://drive.google.com/drive/folders/10grRR3iRIfVn6kFLyD86-n8xQewUNU9n?usp=sharing>

Task 01 (Graph Representation):

A) You are given a **directed weighted** Graph, G . The graph consists of N vertices and M edges. In this problem, you have to store the graph using Adjacency Matrix and print the matrix.

Input:

The first line contains two integers N and M ($1 \leq N, M \leq 100$) – the number of vertices and the total number of edges.
The next M lines will contain three integers u_i , v_i , and w_i ($1 \leq u_i, v_i \leq N$, $1 \leq w_i \leq 1000$) – denoting there is an edge between node u_i and v_i with cost w_i .

Output:

You have to make the graph using Adjacency Matrix, and print the Adjacency Matrix in output.

Sample Input/Output:

Sample Input 1	Sample Output 1
4 3 1 3 8 3 2 5 1 4 2	0 0 0 0 0 0 0 0 8 2 0 0 0 0 0 0 0 5 0 0 0 0 0 0 0
Sample Input 2	Sample Output 2
6 7 1 5 6 6 3 5 1 3 9 3 4 7 4 6 1 5 6 8 6 1 6	0 0 0 0 0 0 0 0 0 0 9 0 6 0 0 0 0 0 0 0 0 0 0 0 0 7 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 8 0 6 0 5 0 0 0

B) Previously, we have learned how to represent a graph using the Adjacency matrix. Now in this problem, using the information in Task 01 (a), you have to **build the graph using Adjacency List.**

Sample Input/Output:

Sample Input 1	Sample Output 1
4 3 1 3 8 3 2 5 1 4 2	0 : 1 : (3,8) (4,2) 2 : 3 : (2,5) 4 :
Sample Input 2	Sample Output 2
6 7 1 5 6 6 3 5 1 3 9 3 4 7 4 6 1 5 6 8 6 1 6	0 : 1 : (5,6) (3,9) 2 : 3 : (4,7) 4 : (6,1) 5 : (6,8) 6 : (3,5) (1,6)
Sample Input 3	Sample Output 3
9 15 1 9 4 8 7 5 5 4 7 4 3 2 2 1 10 3 1 15 7 1 13 3 9 8 3 8 4 3 7 3 5 3 1 5 8 2 9 4 8 9 2 10 9 2 12	0 : 1 : (9,4) 2 : (1,10) 3 : (1,15) (9,8) (8,4) (7,3) 4 : (3,2) 5 : (4,7) (3,1) (8,2) 6 : 7 : (1,13) 8 : (7,5) 9 : (4,8) (2,10) (2,12)

Brainstorming: [No need to submit the following questions]

- a) What if it was an **undirected** graph? Can you think of where you have to modify in your code for an undirected graph?
- b) In the last input of Task 01 (B), you notice there are two edges from node '9' to node '2' of cost 10 and 12, known as **parallel edges**. If a graph contains parallel edges, then can we represent the graph using Adjacency Matrix?

Task 02 (Graph Traversal Using BFS):

You have most probably heard of Dora The Explorer. She likes to travel from one country to another.

Currently, Dora is at Doraland. Doraland is a beautiful country, consisting of **N cities** and **M bidirectional roads**.

Recently, Dora has started to learn Graph Algorithms. She knows about BFS and DFS. However, since Dora is still learning, she is not feeling so confident and asks for your help.

Dora contacts you and gives you all the necessary information about Doraland. **Dora will start her journey from city 1**. Now, your task will be **to help Dora to find a path which is similar to the path of BFS graph traversal**.

Input

The given map will be an undirected and unweighted graph.

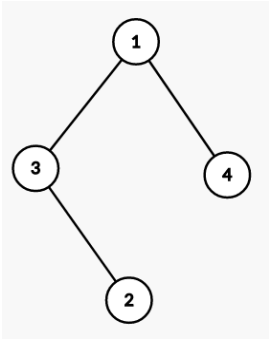
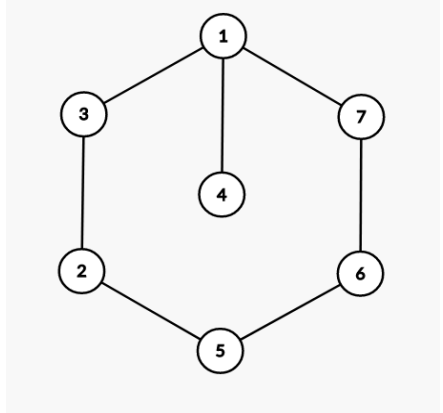
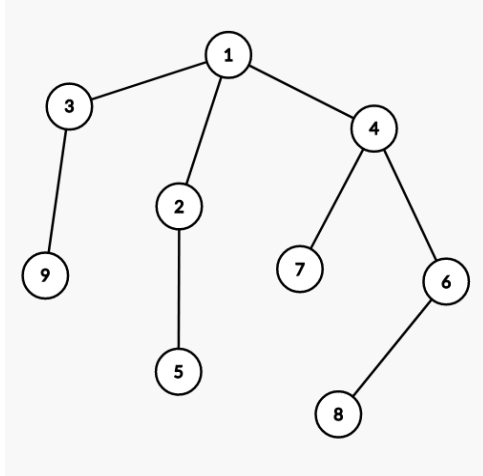
The first line contains two integers N and M ($1 \leq N, M \leq 100$) – the number of cities and the total number of roads.

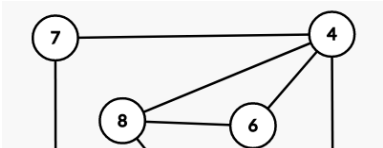
The next M lines will contain two integers u_i, v_i ($1 \leq u_i, v_i \leq N$) – denoting there is a bidirectional road between u_i and v_i .

Output

Print the BFS path traversal which Dora will follow to explore the city, starting from city 1.

Sample Input 1	Sample Output 1	Sample Graph 1
----------------	-----------------	----------------

<pre> 4 3 1 3 3 2 1 4 </pre>	<pre> 1 3 4 2 (Another valid path: 1 4 3 2) </pre>	
Sample Input 2	Sample Output 2	Sample Graph 2
<pre> 7 7 1 3 3 2 1 4 2 5 5 6 1 7 7 6 </pre>	<pre> 1 3 4 7 2 6 5 </pre>	
Sample Input 3	Sample Output 3	Sample Graph 3
<pre> 9 8 1 2 1 3 1 4 2 5 4 6 4 7 6 8 3 9 </pre>	<pre> 1 2 3 4 5 9 6 7 8 </pre>	
Sample Input 4	Sample Output 4	Sample Graph 4

11 14 1 2 1 3 10 4 2 5 4 6 4 7 6 8 3 9 3 7 1 8 4 8 1 10 9 11 10 11	1 2 3 8 10 5 9 7 6 4 11	
--	-------------------------	---

This problem may have multiple valid solutions, however your solution should follow the BFS level order traversal.

Pseudocode of BFS:

The breadth-first-search procedure BFS below assumes that the input graph $G = (V, E)$ is represented using adjacency lists.

```
BFS(G,s):
1 for each vertex u in G.V:
2     u.colour = 0
3 Q = ∅ ;
4 s.colour = 1
5 ENQUEUE(Q,s)
6 while Q ≠ ∅;
7     u = DEQUEUE(Q)
8     for each v in G.Adj[u]:
9         if v.colour == 0:
10             v.colour = 1
11             ENQUEUE(Q,v)
```

Task 03 (Graph Traversal Using DFS):

After successful BFS traversal, Dora wants to visit the whole country again. However, this time Dora wants to traverse the country following the path which is similar to the path of DFS graph traversal.

Yes, you guessed it correctly. You have to help Dora this time also.

The input format remains the same as Task 02. **Dora will start her journey from city 1 again.**

Input

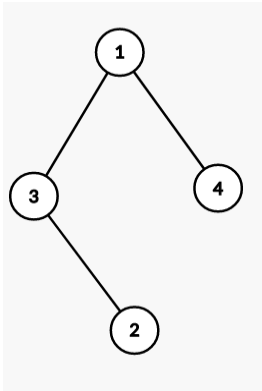
The given map will be an undirected and unweighted graph.

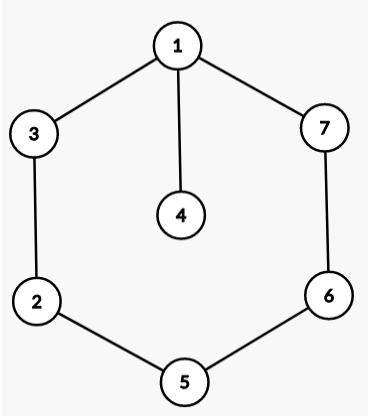
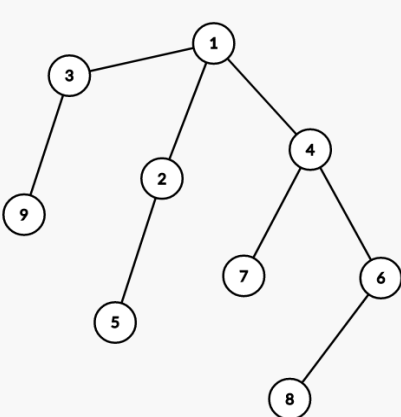
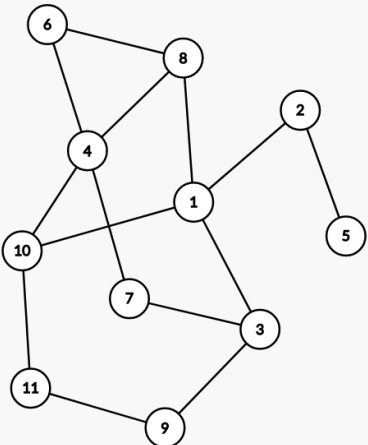
The first line contains two integers N and M ($1 \leq N, M \leq 100$) – the number of cities and the total number of roads.

The next M lines will contain two integers u_i, v_i ($1 \leq u_i, v_i \leq N$) – denoting there is a bidirectional road between u_i and v_i .

Output

Print the DFS path traversal which Dora will follow to explore the city, starting from city 1.

Sample Input 1	Sample Output 1	Sample Graph 1
<pre> 4 3 1 3 3 2 1 4 </pre>	<pre> 1 3 2 4 (Another valid path: 1 4 3 2) </pre>	
Sample Input 2	Sample Output 2	Sample Graph 2

<pre> 7 7 1 3 3 2 1 4 2 5 5 6 1 7 7 6 </pre>	<pre> 1 3 2 5 6 7 4 </pre>	
Sample Input 3	Sample Output 3	Sample Graph 3
<pre> 9 8 1 2 1 3 1 4 2 5 4 6 4 7 6 8 3 9 </pre>	<pre> 1 2 5 3 9 4 6 8 7 </pre>	
Sample Input 4	Sample Output 4	Sample Graph 4
<pre> 11 14 1 2 1 3 10 4 2 5 4 6 4 7 6 8 3 9 3 7 1 8 4 8 1 10 9 11 10 11 </pre>	<pre> 1 2 5 3 9 11 10 4 6 8 7 </pre>	

This problem may have multiple valid solutions. However, your solution should follow the dfs path traversal order.

Pseudocode of DFS:

The depth-first-search procedure DFS below assumes that the input graph $G = (V, E)$ is represented using adjacency lists.

```
colourInitializing(G):
1 for each vertex u in G.V:
2     u.colour = 0

DFS(G,u):
1     u.colour = 1
2     for each v in G.Adj[u]:
3         if v.colour == 0:
4             DFS(G,v)
```

Task 04 (Cycle Finding):

It was a very hectic day for Dora. Before going to sleep, Dora wants to find out if there is any Cycle in the map of the city.

Since Dora has traveled the whole country twice, she is feeling very exhausted and asks you to solve the problem. However, Dora has made the roads of the cities **directed** in her map, so that you don't get bored.

In graph theory, a cycle in a graph is a non-empty trail in which only the first and last vertices are equal. [Wikipedia]

Input:

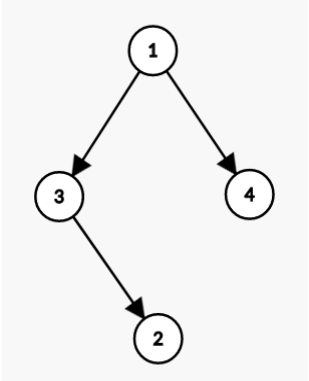
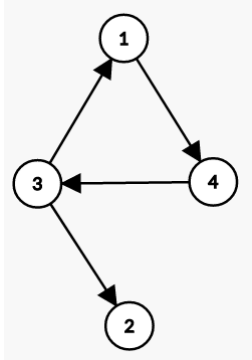
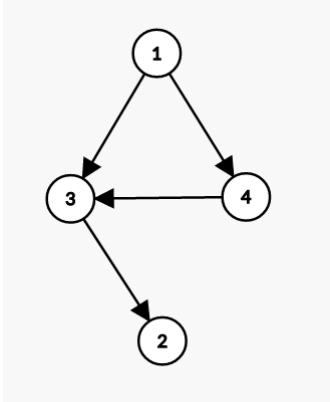
The given map will be a directed and unweighted graph.

The first line contains two integers N and M ($1 \leq N, M \leq 100$) – the number of cities and the total number of roads.

The next M lines will contain two integers u_i, v_i ($1 \leq u_i, v_i \leq N$) – denoting there is a bidirectional road between u_i and v_i .

Output:

Print "YES" if the map contains any Cycle, otherwise print "NO".

Sample Input 1	Sample Output 1	Sample Graph 1
4 3 1 3 3 2 1 4	NO	
Sample Input 2	Sample Output 2	Sample Graph 2
4 4 3 1 3 2 1 4 4 3	YES	
Sample Input 3	Sample Output 3	Sample Graph 3
4 4 1 3 3 2 1 4 4 3	NO	
Sample Input 4	Sample Input 4	Sample Graph 4

<pre> 9 10 1 2 2 3 3 4 4 5 3 5 1 6 2 7 5 8 5 9 8 9 </pre>	NO	<pre> graph TD 1((1)) --> 2((2)) 1((1)) --> 6((6)) 2((2)) --> 7((7)) 2((2)) --> 3((3)) 3((3)) --> 4((4)) 3((3)) --> 5((5)) 4((4)) --> 5((5)) 5((5)) --> 8((8)) 5((5)) --> 9((9)) 8((8)) --> 9((9)) </pre>
Sample Input 5	Sample Input 5	Sample Graph 5
<pre> 5 7 1 2 2 3 1 3 3 4 4 5 3 5 4 2 </pre>	YES	<pre> graph TD 1((1)) --> 2((2)) 1((1)) --> 3((3)) 2((2)) --> 3((3)) 3((3)) --> 4((4)) 3((3)) --> 5((5)) 4((4)) --> 2((2)) 5((5)) --> 4((4)) </pre>

Brainstorming: [No need to submit the following questions]

- Suppose that you have an undirected and unweighted graph, can you find out if the given graph is a tree or not?
- Suppose that you have an undirected and unweighted graph. Can you find out if the graph contains an Odd length Cycle? An odd length cycle means the cycle will contain odd numbers of vertices.

Task 05 (Find the shortest path):

The next day, Dora again called you for helping her. Among all the cities she likes city 'D' the most.

Since you are becoming very much annoyed with Dora, you have planned to reach city D spending the minimum amount of time.

In the belugaland, moving between two cities connected by a road takes one second. You have to find the minimum time to reach D and show the path in the output.

Dora and you will start your journey from city 1.

Input

The given map will be an undirected and unweighted graph.

The first line contains three integers N , M , and D ($1 \leq N, M, D \leq 100$) – the number of cities, the total number of roads and the destination city.

The next M lines will contain two integers u_i, v_i ($1 \leq u_i, v_i \leq N$) – denoting there is a bidirectional road between u_i and v_i .

Output

Print the minimum amount of time to reach city D from city 1.
Next, print the shortest path you will follow.

Sample Input 1	Sample Output 1
4 3 2 1 3 3 2 1 4	Time: 2 Shortest Path: 1 3 2
Sample Input 2	Sample Output 2
6 6 5 1 3 3 2 1 4 2 6 5 6 4 6	Time: 3 Shortest Path: 1 4 6 5
Sample Input 3	Sample Output 3

9 8 7 1 2 1 3 1 4 2 5 4 6 4 7 6 8 3 9	Time: 2 Shortest Path: 1 4 7
Sample Input 4	Sample Output 4
11 14 5 4 2 1 3 10 4 2 5 4 6 4 7 6 8 3 9 3 7 1 8 4 8 1 10 9 11 10 11	Time: 4 Shortest Path: 1 8 4 2 5
Sample Input 5	Sample Output 5
4 3 1 1 3 3 2 1 4	Time: 0 Shortest Path: 1

This problem may have multiple valid shortest paths. However, the minimum time required to reach the destination should be matched exactly with the output.

Task 06 (Flood Fill):

Dora is leaving the country. Before leaving the country, as a token of gesture, Dora gifted you with a map of the jungle "Jumanji".

The map is very interesting. The map is a 2D grid. Some of those cells are occupied by diamonds and obstacles. [See the Sample input for better understanding.]

Now, you are on a journey to Jumanji in search of Diamonds. However, since the jungle is full of ferocious creatures, you can't collect all the diamonds. **You will choose only one start position such that you collect the maximum amount of diamonds. Please note that you can not move to any cell which contains obstacles.**

Input

The first line contains two integers R and H ($1 \leq R, H \leq 100$) – the number of rows and columns respectively in the grid.

The next R line will contain H characters. Each character represents the status of a cell as follows.

- 1) '.': Empty Cell → You can move here.
- 2) 'D': Cell with a Diamond → If you move to the cell containing 'D', you will collect that Diamond.
- 3) '#': Cell with an obstacle → You can't move to this cell.

Output

Print a single integer that denotes the maximum amount of Diamonds you can collect.

[The diamonds are coloured red to demonstrate which diamonds have been collected.]

Sample Input 1	Sample Output 1
4 3 .. D D .. .D. ##.	3
Sample Input 2	Sample Output 2
10 7 ...#...D ...#D.. D ...#.D.	11

<pre> D..#... DDD########.. .#D.#DD .####.. DDD...D </pre>	
Sample Input 3	Sample Output 3
<pre> 9 11 .#...D...D.. .#.#####. D#.##..D..#. D#D#.###.#D .#.##..D#.##. .#.#####.#D D#..D...D#. .#####. ...D..D...D </pre>	15
Sample Input 4	Sample Output 4
<pre> 5 5 ####. #D.#. ####. </pre>	1
Sample Input 5	Sample Output 5
<pre> 5 5 ####. #..#. ####. </pre>	0
Sample Input 6	Sample Output 6
<pre> 1 5 D.... </pre>	1
Sample Input 7	Sample Output 7
<pre> 12 12####..... .#D.#..... .####.....###. </pre>	4

<pre> ...D....#D#.#D#. .#####D#. .#D....##.#. .#.D..D##D#. .#####. </pre>	
---	--

Bonus Tasks: Task 07 and Task 08 [5+5 = 10 Points]

Note that these are bonus problems. Even if you do not attempt these problems, you can get a perfect score. So, do not worry if you find it too hard!

The faculty may ask you viva for these problems. If you are unable to give a correct explanation, you will receive 0 in this lab.

Bonus 1: Task 07:

Dora returns to Doraland once again. She finds that the city has undergone significant improvements since her last visit. The city structure has been reconstructed to make it more beautiful. Currently, **Doraland consists of N cities and all the cities are connected with N-1 bidirectional roads.**

However, this time, Dora will have a very short trip and as usual, she seeks your help. According to Dora's wish, you will choose two cities, let's say city A and city B in such a way that while traveling from city A to city B Dora can pass through a maximum number of cities. You have to keep in mind that it is not possible to visit any roads twice in the journey since Dora is on a short trip.

Can you help Dora find such two cities?

Input

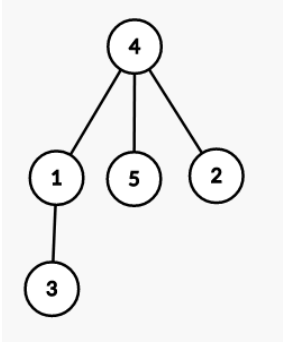
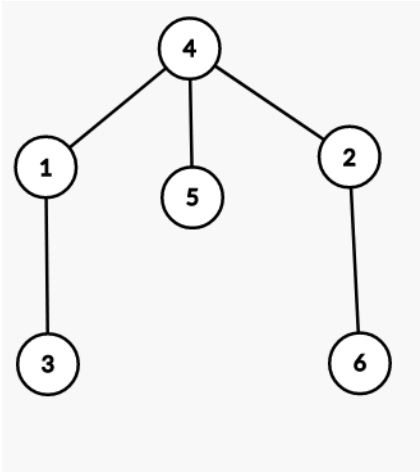
The given map will be an undirected and unweighted graph.

The first line contains one integer N ($2 \leq N \leq 100$) – the number of cities.

The next $N-1$ lines will contain two integers u_i, v_i ($1 \leq u_i, v_i \leq N$) – denoting there is a bidirectional road between u_i and v_i .

Output

Print two cities A and B which satisfy the conditions stated in the problem statement.

Sample Input 1	Sample Output 1	Sample Graph 1
5 4 1 4 2 1 3 4 5	2 3 Or (3 5) [You may print only one valid output]	
Sample Input 2	Sample Output 2	Sample Graph 2
6 4 1 4 2 1 3 4 5 2 6	3 6	
Sample Input 3	Sample Output 3	Sample Graph 3

13 1 2 1 3 1 4 4 5 5 6 5 7 7 8 8 9 9 13 6 10 10 11 11 12	12 13	
--	-------	--

This problem may have multiple valid answers. However, make sure it satisfies the given condition.

Explanation:

In test case 1, if you choose city 2 and city 3, then the path will be

$2 \rightarrow 4 \rightarrow 1 \rightarrow 3$. If you choose city 3 and city 5, then the path will be

$3 \rightarrow 1 \rightarrow 4 \rightarrow 5$.

Since, in both paths you can visit a maximum of four cities by using the roads only one, both the answers are valid.

In test case 2, one can see that visiting from city 3 to city 6 (or vice-versa) is only the valid answer. The path will be $3 \rightarrow 1 \rightarrow 4 \rightarrow 2 \rightarrow 6$.

Bonus 2: Task 08:

As mentioned earlier, Dora is on a short trip and she is leaving the country. Before leaving the country, as a token of gesture, Dora gifted you with a problem this time, which you will find in this link in the lightoj, an online judge.

To open the gift, you have to create an account at lightoj first. [<https://lightoj.com/>]

Problem Link: <https://lightoj.com/problem/back-to-underworld>

CSE221

Lab Assignment 05

Fall 2023

Submission Guidelines :

1. You can code all of them either in Python, CPP, or Java. But you should choose a specific language for all tasks.
2. For each task write separate python files like task2.py, task3.py, and so on. For problems that have subproblems, name those like task1A.py, task1B.py, and so on.
3. Add a hand written explanation of 3-4 lines for each of your solutions and submit that as a single document.
4. For each problem, take input from files called **"inputX_Y.txt"** and output at **"outputX_Y.txt"**, where X is the task number and Y is the sample i/o number. For example, for problem 2 sample 1, the input file is this, "input2_1.txt". For problems that have subproblems, name the files like "input1a_1.txt", "output1a_1.txt" and so on. Same for output.
5. For each task include at least one input file (if any) in the submission folder.
6. Finally zip all the files and rename this zip file as per this
format:**LabSectionNo_ID_CSE221LabAssignmentNo_Fall2023.zip**
[Example:**LabSection01_21101XXX_CSE221LabAssignment05_Fall2023.zip**]
7. Don't copy from your friends.
8. You MUST follow all the guidelines, naming/file/zipping convention stated above.

Failure to follow instructions will result in a straight 50% mark deduction.

A useful tool for making graphs:

https://csacademy.com/app/graph_editor/

Task 01 [30 points]

We all know about the prerequisite course. For example, before taking CSE221, we have to complete CSE220. And prior to taking CSE220 we need to complete CSE111 and CSE110. The course sequence is as follows:

CSE110 → CSE111 → CSE220 → CSE221

In this problem, there are **N** courses in the curriculum. There are **M** prerequisite requirements of the form "course **A** has to be completed before course **B**".

Your task is to find an order in which you can complete the courses.

A) Solve it using the DFS approach. [15 points]

B) Solve it using the BFS approach. [15 points]

Input:

The first input line has two integers **N** ($1 \leq N \leq 1000$) and **M** ($1 \leq M \leq N^2$) - the number of courses and prerequisite requirements. The courses are numbered 1,2,3,...,N.

Next, there are M lines describing the requirements. Each line has two integers **A, B** ($1 \leq A, B \leq N$)- course A has to be completed before course B.

Output:

Print an order in which you can complete the courses. Please note, there could be multiple correct sequences. You can print any valid order that includes all the courses.

If there are no solutions, print "IMPOSSIBLE".

Sample Input 1	Sample Output 1
5 3 3 1 1 2 4 5	3 4 1 5 2

Sample Input 2	Sample Output 2
6 6 1 2 2 3 4 3 4 5 5 6 6 4	IMPOSSIBLE
Sample Input 3	Sample Output 3
8 10 1 2 1 4 2 4 2 5 2 3 4 6 4 5 6 5 5 3 7 8	1 7 2 8 4 6 5 3

Task 02[10 points]

The problem statement of this problem is the same as **Task 01**. The only difference is that in this task, you have to find the lexicographically smallest valid course sequence.

If you have two sequences, for example: A: $3 \rightarrow 1 \rightarrow 2 \rightarrow 4$ and B: $3 \rightarrow 1 \rightarrow 4 \rightarrow 2$. Then path A is lexicographically smaller than path B.

Input:

The first input line has two integers **N** ($1 \leq N \leq 1000$) and **M** ($1 \leq M \leq N^2$) - the number of courses and prerequisite requirements. The courses are numbered $1, 2, 3, \dots, N$.

Next, there are M lines describing the requirements. Each line has two integers **A, B** ($1 \leq A, B \leq N$)- course A has to be completed before course B.

Output:

Print the lexicographically smallest valid course sequence in which you can complete the courses.

If there are no solutions, print "IMPOSSIBLE".

Sample Input 1	Sample Output 1
5 3 3 1 1 2 4 5	3 1 2 4 5
Sample Input 2	Sample Output 2
6 6 1 2 2 3 4 3 4 5 5 6 6 4	IMPOSSIBLE
Sample Input 3	Sample Output 3
8 10 1 2 1 4 2 4 2 5 2 3 4 6 4 5 6 5 5 3 7 8	1 2 4 6 5 3 7 8

Task 03[10 points]

You are given a Directed Graph consisting of **N** vertices and **M** edges. You have to find the strongly connected components of the given graph.

Input:

The given map will be a directed and unweighted graph.

The first line contains two integers N and M ($1 \leq N, M \leq 100$) – the number of vertices and the total number of edges.

The next M lines will contain two integers u_i, v_i ($1 \leq u_i, v_i \leq N$) – denoting there is a road between u_i to v_i .

Output

Print all the strongly connected components of the given graph. See the output for better understanding.

Sample Input 1	Sample Output 1
5 5 1 2 2 3 2 4 3 1 4 5	1 2 3 4 5
Sample Input 2	Sample Output 2
8 9 1 2 1 6 2 3 3 4 4 5 4 6 5 7 6 2 7 5	1 2 3 4 6 5 7 8
Sample Input 3	Sample Output 3
4 3 1 2	1 2

2 3 2 4	3 4
------------	--------

CSE221

Lab Assignment 06

FALL 2023

Submission Guidelines :

1. You can code all of them either in Python, CPP, or Java. But you should choose a specific language for all tasks.
2. For each task write separate python files like task2.py, task3.py, and so on. For problems that have subproblems, name those like task1A.py, task1B.py, and so on.
3. Add a hand written explanation of 3-4 lines for each of your solutions and submit that as a single document.
4. For each problem, take input from files called "**inputX_Y.txt**" and output at "**outputX_Y.txt**", where X is the task number and Y is the sample i/o number. For example, for problem 2 sample 1, the input file is this, "input2_1.txt". For problems that have subproblems, name the files like "input1a_1.txt", "output1a_1.txt" and so on. Same for output.
5. For each task include at least one input file (if any) in the submission folder.
6. Finally zip all the files and rename this zip file as per this format:**LabSectionNo_ID_CSE221LabAssignmentNo_FALL2023.zip**
[Example:**LabSection01_21101XXX_CSE221LabAssignment06_FALL2023.zip**]
]
7. Don't copy from your friends.
8. You MUST follow all the guidelines, naming/file/zipping convention stated above.

Failure to follow instructions will result in a straight 50% mark deduction.

A useful tool for making graphs:

https://csacademy.com/app/graph_editor/

Task 01:[10 Marks]

You are given a weighted, directed graph with N nodes and M edges. Each edge is represented as a triple (u, v, w) , where u and v are the nodes connected by the edge and w is the weight of the edge.

Your task is to find the shortest path from a source node S to all other nodes in the graph using **Dijkstra's algorithm**. You should output the shortest distance from the source node to each of the other nodes in the graph. If a node is unreachable from the source node, its distance should be represented as -1 .

Input

The first line of the input contains two integers, N and M ($1 \leq N \leq 1000$, $1 \leq M \leq 100000$) denoting the number of nodes and edges in the graph, respectively.

The next M lines each contain three integers, u , v ($1 \leq u, v \leq N$), and w ($1 \leq w \leq 100$) denoting an edge from node u to node v with weight w .

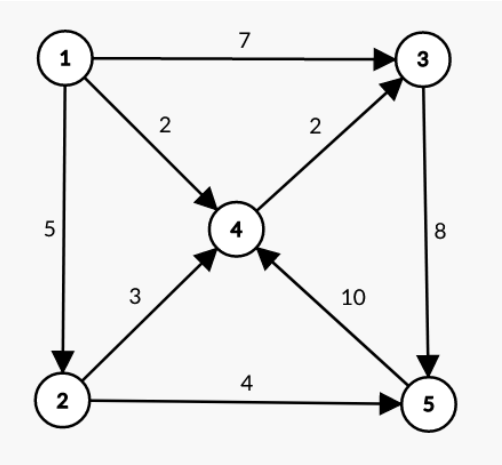
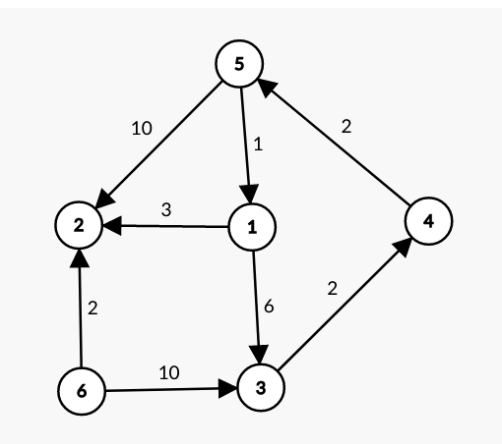
The last line of the input contains an integer S ($1 \leq S \leq N$) denoting the source node.

Output

Output N space-separated integers, where the i -th integer represents the shortest distance from the source node to node i .

If a node is not reachable from the source node, output -1 instead.

Sample Input 1	Sample Output 1	Sample Graph 1
----------------	-----------------	----------------

5 8 1 2 5 1 3 7 1 4 2 2 4 3 2 5 4 3 5 8 4 3 2 5 4 10 1	0 5 4 2 9	
Sample Input 2	Sample Output 2	Sample Graph 2
6 8 1 2 3 1 3 6 3 4 2 4 5 2 5 2 10 5 1 1 6 2 2 6 3 10 3	5 8 0 2 4 -1	

Explanation of Sample Input 1

Using Dijkstra's algorithm, the shortest path from node 1 to all other nodes in the given graph is:

- Distance from node 1 to node 1 is 0.
- Distance from node 1 to node 2 is 5.
- Distance from node 1 to node 3 is 4.
- Distance from node 1 to node 4 is 2.
- Distance from node 1 to node 5 is 9.

Task 02:[10 Marks]

Alice and Bob are in a hurry to meet each other and have to traverse through a directed graph with weighted edges. Alice starts from node S and Bob starts from node T. They want to find a common node in the graph, where they can meet each other in the minimum amount of time. Alice or Bob can wait at any node if they want to.

Input

The first line contains two integers N ($1 \leq N \leq 10000$) and M ($1 \leq M \leq 100000$) separated by a space, denoting the number of nodes and edges in the graph, respectively.

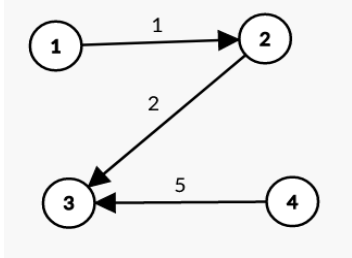
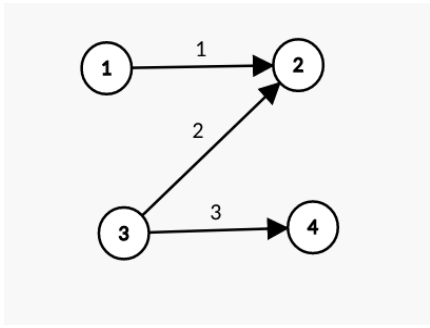
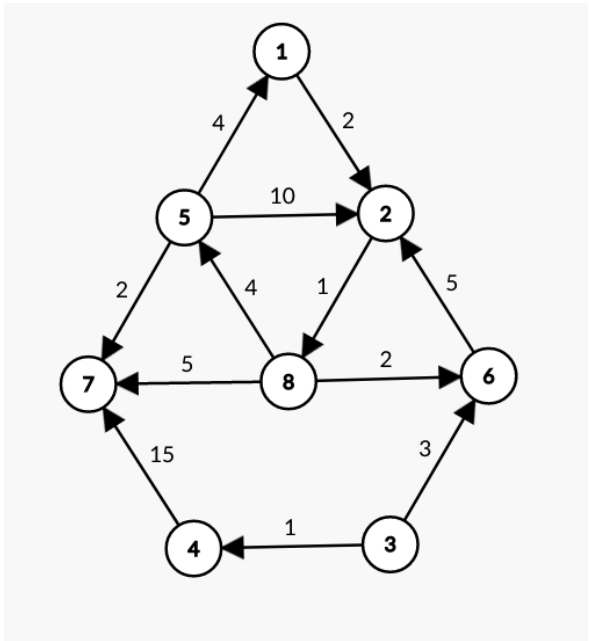
The next M lines each contain three integers, u, v ($1 \leq u, v \leq N$), and w ($1 \leq w \leq 100$) denoting an edge from node u to node v with weight w .

The last line contains two integers S, T ($1 \leq S, T \leq N$) which denotes the starting node of Alice and the starting node of Bob respectively.

Output

Output two integers separated by a space. The first integer is the minimum time it takes for Alice and Bob to meet. The second integer is the vertex where they will meet. If there is no such node, print "Impossible".

Sample Input 1	Sample Output 1	Sample Graph 1
----------------	-----------------	----------------

4 3 1 2 1 2 3 2 4 3 5 1 4	Time 5 Node 3	
Sample Input 2	Sample Output 2	Sample Graph 2
4 3 1 2 1 3 2 2 3 4 3 1 4	Impossible	
Sample Input 3	Sample Output 3	Sample Graph 3
8 12 3 6 3 3 4 1 6 2 5 4 7 15 2 8 1 8 6 2 8 7 5 8 5 4 5 2 10 5 1 4 1 2 2 5 7 2 3 5	Time 8 Node 2	

Task 3 [10 Marks]

Study material: <http://www.shafaetsplanet.com/?p=763>

There is a group of N people living in a small village. They live in their own house. Although they are all neighbors, they don't all know each other very well.

Each person in that village has their own unique identity - labeled with an integer value between 1 to N . Initially, the villagers don't have any friends. As time passes by, they begin to make friendship between themselves.

In this problem, you will be given a description of K friendships. You have to print an integer value which denotes the size of their friend circle.

Suppose, there are five people living in the village labeled with 1,2,3,4 and 5. Initially, the size of each friend circle is one, since no friendship has been created yet.

One day, person 1 and person 2 become friends. So the size of their friend circle becomes two. Next day, person 3 and person 4 become friends and the size of their friendship becomes two as well. After a few days, person 1 and person 4 become friends. Now the size of their friend circle becomes four consisting of persons 1,2,3 and 4.

Input Format:

The input consists of two integers, separated by a space, denoting the number of people in the village, N ($1 \leq N \leq 10^5$) and the number of queries that will follow, K ($1 \leq K \leq 10^5$).

The next K lines contain two integers A_i and B_i each ($1 \leq A_i, B_i \leq N$ and $A_i \neq B_i$), separated by a space, representing two people who have become friends as a result of the query.

Output Format:

For each query, output a single integer on a new line representing the size of the friend circle that the two people belong to after becoming friends.

Sample Input/Output:

Sample Input 1	Sample Output 1
5 3 1 2 3 4 1 4	2 2 4
Sample Input 2	Sample Output 2
8 7 2 4 4 5 3 6 4 7 3 1 2 7 6 2	2 3 2 4 3 4 7

Sample Input Explanation:

In sample input 2,

Query 0: Initially, there are 8 people in the village who do not know each other.

{1} {2} {3} {4} {5} {6} {7} {8}

Query 1: After person 2 and person 4 becoming friends:

{1} {2,4} {3} {5} {6} {7} {8}

The output is 2, since the size of the friends circle {2,4} is 2.

Query 2: After person 4 and person 5 becoming friends:

{1} {2,4,5} {3} {6} {7} {8}

The output is 3, since the size of the friends circle {2,4,5} is 3.

Query 3: After person 3 and person 6 becoming friends:

{1} {2,4,5} {3,6} {7} {8}

The output is 2, since the size of the friends circle {3,6} is 2.

Query 4: After person 4 and person 7 becoming friends:

{1} {2,4,5,7} {3,6} {8}

The output is 4, since the size of the friends circle {2,4,5,7} is 4.

Query 5: After person 3 and person 1 becoming friends:

{2,4,5,7} {1,3,6} {8}

The output is 3, since the size of the friends circle {1,3,6} is 3.

Query 6: Since the person 2 and person 7 are already in the same friend circle, nothing changes:

{2,4,5,7} {1,3,6} {8}

The output is 4, since the size of the friends circle {2,4,5,7} is 4.

Query 7: After person 6 and person 2 becoming friends:

{2,4,5,7,1,3,6} {8}

The output is 7, since the size of the friends circle {2,4,5,7,1,3,6} is 7.

Task 4 [10 Marks]

In the kingdom of Beluga, there are N cities connected by M roads, each with a maintenance cost associated with it. There is at least one path between any two cities. The king is concerned about the increasing maintenance cost and decides to take action.

He calls upon his council, and they suggest that they find a minimum-cost set of roads that connects all cities while minimizing the maintenance cost. Then the king decides to reduce the total maintenance cost by destroying some of the existing roads, instead of building new ones.

Since you are a very good programmer the king calls you. He asks you to find out what the lowest maintenance cost can be achieved after destroying a few roads while ensuring there still exists a path from each city to another.

Input

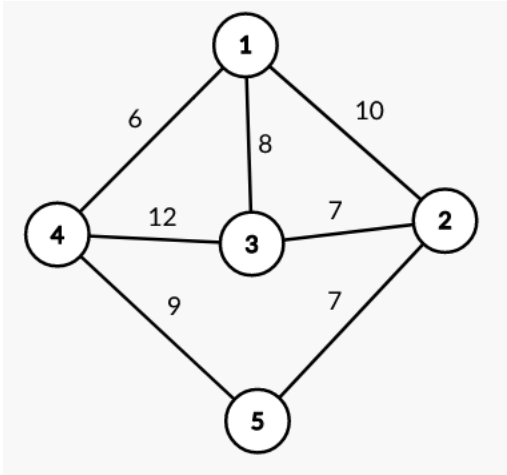
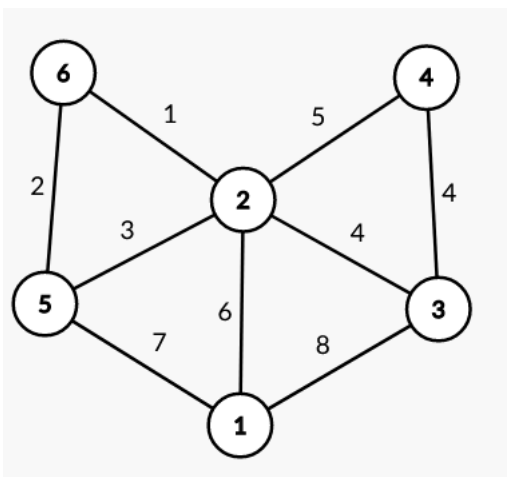
The first line of the input contains two space-separated integers, N and M ($1 \leq N \leq 10^5$, $1 \leq M \leq 10^6$), representing the number of cities and roads in the kingdom of Beluga, respectively.

The next M lines each contain three space-separated integers, u , v , and w ($1 \leq u, v \leq N$, $1 \leq w \leq 10^9$), where u and v denote the endpoints of a road and w represents its maintenance cost.

Output

The output should contain a single integer, the minimum total maintenance cost achievable.

Sample Input/Output:

Sample Input 1	Sample Output 1	Sample Graph 1
5 7 1 2 10 1 3 8 1 4 6 2 3 7 2 5 7 3 4 12 4 5 9	28	
Sample Input 2	Sample Output 2	Sample Graph 2
6 9 1 2 6 2 4 5 2 3 4 1 3 8 4 3 4 2 6 1 2 5 3 5 6 2 5 1 7	17	

Time complexity, Searching, Sorting

1. Solve the following questions about time complexity.

a. Calculate the time complexity of the following code snippet:

```
int p = 0;
for (i = n / 2; i > 1; i /= 6) {
    for (j = 2; j <= n; j *= 4) {
        for (k = 0; k <= j; k *= 3) {
            p = p + n / 2;
        }
    }
}
```

b. Calculate the time complexity of the following code snippet:

```
int p = 0;
for (i = n; i > 1; i -= 1) {
    for (j = 2; j <= n; j += 1) {
        p = p + n;
        if (p > (n ^ i)) // ^ sign means XOR in code
        {
            break;
        }
    }
}
for (i = n / 2; i > 1; i /= 6) {
    for (j = 2; j <= i; j *= 4) {
        for (k = 0; k <= j; k *= 3) {
            p = p + n / 2;
        }
    }
}
```

2. After working with the binary search algorithm, as a CSE221 student you want to explore its strength. You have several ideas in your mind. Try to modify the algorithm to implement your ideas.

- What if you want to return the first index of the search key in case there are duplicates. For example: your search key is 13 and there are three 13s in the list.
- Now, you also want to return the number of times the key appears in the list. Say, the search key is 54 and it occurs 4 times in indices 6,7,8,9. Then you should return (6,4).

3.

Consider the following list:

Index	0	1	2	3	4	5	6	7
Number	23	2	19	3	7	11	5	13

Will the algorithm given on the right be able to find the search value $T=2$ for the list above?

If yes, **show** the value of L , R , m in each step of the algorithm. If no, **explain** why not.

```
function binary_search(A, n, T) is
    L := 0
    R := n - 1
    while L ≤ R do
        m := floor((L + R) / 2)
        if A[m] < T then
            L := m + 1
        else if A[m] > T then
            R := m - 1
        else:
            return m
    return unsuccessful
```

Here, A denotes the list, and n denotes its size.

4. You are given an array containing N distinct integers in a wave-like sequence. Meaning, the numbers in the beginning are in ascending order, and after a specific position, they are in descending order. For example: [1, 3, 4, 5, 9, 6, 2, -1]
You have to find the maximum number of this sequence. Can you devise an efficient algorithm such that the time complexity will be less than $O(N)$?
- Present your solution idea as a pseudocode/ python code/ flowchart/ step-by-step instructions/ logical explanation in one-two paragraphs.
 - Write the time complexity of your algorithm.
5. [Mid - Summer '24] Your friend gave you an integer (key) and asked you to find its integer square root without using built-in functions like `sqrt()` or exponentiation. For example, if the input is 25, the output should be 5 because $5^2=25$. If the square root is not an integer, return the floor value of the actual square root. For example, for an input of 10, the output should be 3.
Your friend's initial approach is to use linear search (time complexity $O(n)$) as shown below.

```
def LinearSearchToFindSquareRoot (key) :
    result = -1
    for i in range(1, key, 1):
        if i * i <= key:
            result = i
    return result
```

- Present an $O(\sqrt{n})$ time algorithm with pseudocode/programmable code/step-by-step logical instructions.
 - Present an $O(\log_2 N)$ time algorithm with pseudocode/programmable code/step-by-step logical instructions.
6. For different tasks, we often need to search for things. And most of the time, we sort the dataset before performing search operations. Once, you found the following list of numbers.
[2, 5, 1.2, 6.7, 1.7, 9.3, 2.2, 7.7, 0, -4, -5.1, 2, 5, 5.2]
- To search an element in an unsorted array, we need $O(N)$ time using linear search. Binary search works in $O(\log N)$ time but the array needs to be sorted beforehand which takes at least $O(N \log N)$ time in general. Why would you then sort the array first and then perform binary search instead of just performing linear search?
 - Can you modify count sort so that it may work with negative integers as well?
 - Can you modify count sort so that it may work with the given list?
 - Say, you are working in a system where you need to sort a very big dataset. The memory available to you can barely accommodate the data. You have two options – merge sort & quick sort. Which one should you choose? Why?
 - Construct an array where quick sort fails to work in $O(N \log N)$ time.
7. [Mid - Fall '22] Jack loves to play with integers. He created a list of n integers where the even indices hold numbers in decreasing order and the odd indices hold numbers in increasing order. For example, this is a list of n=8 integers Jack made.

Index	0	1	2	3	4	5	6	7
Number	23	2	19	3	7	11	5	13

[Explanation:

The indices 1, 3, 5, and 7 have numbers 2, 3, 11, and 13 in increasing order.

The indices 0, 2, 4, and 6 have numbers 23, 19, 7 and 5 in decreasing order.]

- a. Present a linear time algorithm to sort the list in ascending order. Write your algorithm with pseudocode/programmable code/step-by-step logical instructions.

Graph Theory - I

Lecture material:

<https://drive.google.com/file/d/1OkLO91GV4M23e-1EtvFr9yeGWgk6V91q/view?usp=sharing>

For the questions that ask you to propose/present an algorithm, you can write either a programmable code or a pseudocode or step-by-step instructions.

Problem #1:

Answer the following general questions regarding the basic terminology of Graph Theory:

1. In a directed graph, if there is an edge from Node A to Node B, and not the other way around, can A and B be called adjacent/neighbor to each other?
2. In an undirected graph of n vertices, how many edges can we construct without allowing loops (edge to self) and multiple edges between a pair of vertices?
3. Same question for directed graphs.
4. How many edges can we construct in an undirected graph with n vertices without introducing any cycle? Loops can also be considered as cycles.
5. Same question for directed graphs.
6. Take any graph, try to construct a path that starts from a vertex, then visits all the edges of the graph exactly once. It will not be possible in all graphs, only in some having a special property. What's that property?
7. Take any graph, try to construct a path that starts from a vertex, then visits all other vertices of the graph exactly once (some edges might be unvisited). Like the previous question, it will not be possible in all graphs. Just find some examples with both cases.
8. Analyze the efficiency of using adjacency list over adjacency matrix for different types of graphs: undirected, directed, weighted, unweighted, sparse, dense...

Problem #2:

Suppose we have the following social network, Friendsbook where:

Person A is friends with Person B, C, D, and E.

Person B is friends with Person A, C, and F.

Person C is friends with Person A, B, D, and G.

Person D is friends with Person A, C, E, G, and H.

Person E is friends with Person A, D, and H.

Person F is friends with Person B and G.

Person G is friends with Person C, D, F, and H.

Person H is friends with Person D, E, and G.

- A. Draw a graph to represent the network using each user as a node or vertex and the friendships as edges between the nodes.
- B. Create the adjacency list and adjacency matrix for the graph representation

- C. For each pair of users, calculate the number of friends they have in mutual.
- D. In Friendsbook, a user can share posts. Suppose the CEO of Friendsbook hired you for developing their post display system. You were given the task to implement a system that displays posts from users who are at most two degrees of connection away from the one who is scrolling his feed. After looking at the requirements you came up with an idea to implement the system using the BFS algorithm. Now, for person F, simulate your algorithm. Can F see the posts of all other users in his feed?

Problem #3:

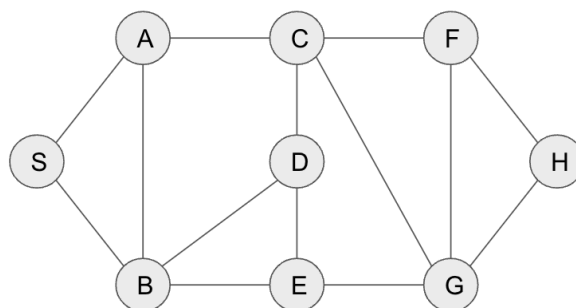
The following information is stored after implementing the DFS algorithm on an undirected graph.

Nodes	1	2	3	4	5	6	7
Parent							
Starting Time	8	11	5	6	4	1	2
Finish Time	9	12	10	7	13	14	3
Distance from Root							

- A. Draw the DFS tree.
- B. Fill up the empty rows of the table.

Problem #4:

Consider this graph:



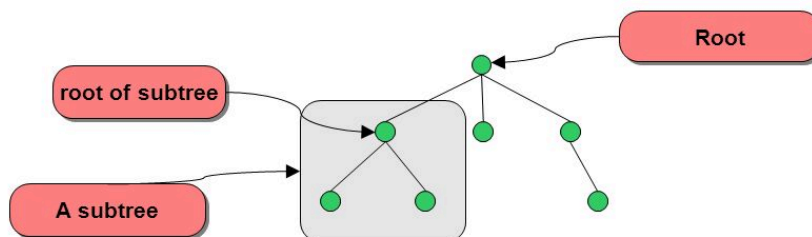
- A. Justify $\sum_{v \in V} \deg(v) = 2m$, where m = number of edges in the graph.
- B. Calculate the number of additional edges that can be added between the nodes randomly, keeping the graph 'simple' (without adding multiple edges between any two nodes, loops in a single node).

Problem #5:

Trees and subtrees



- In a rooted tree, any node is a root of a **subtree** consisting of itself and the nodes "below" it
- By definition there is only one path between the root and each of the other nodes.
 - Because a root is also a node the main property applies
- The convention used in computer science is that roots are drawn on the top. It may seem strange at the beginning but you'll get used to it



Source: <https://slideplayer.com/slide/7322529/>

You are given a Binary Tree with N nodes. That means each of the nodes has zero to two children, and its own subtree. Nodes are numbered from 1 to N .

You will be given Q queries. Each of them tells you a node number P , and asks you to find the size of its subtree (number of nodes in the subtree including P as the root).

- How would you build the adjacency list/matrix of this tree? Show with an example tree.
- You have to answer each query in constant time, $O(1)$. However, you are allowed to do a preprocessing of $O(V+E)$. Propose a suitable algorithm that satisfies this.

Problem #6:

Suppose you are given a directed graph where the number of vertices is V and the number of edges is E . The vertices are numbered from 1 to V . You are given the task to find out whether vertex '1' is reachable from all the other vertices, i.e. there exist paths to go from '2' to '1', '3' to '1', ... and so on.

Propose an $O(V+E)$ algorithm to find the answer to this question.

Clarification: You cannot run DFS from each source. That would take the complexity to $O(V \cdot E)$.

The End

Graph Theory II: Shortest Paths, MST

Deadline: <check announcement in your section's slack channel>

Instructions:

1. Prepare a single pdf file containing your works
 2. Submit through the google form: <check announcement in your section's slack channel>
 3. You need to sign in to your bracu g-suite email account to see the form
-

Assume that you are given a function called $dijkstra(G, a)$ which takes a graph G , and a source vertex a as input, then finds the shortest path (distances, $dist$ and parents, par) from a to every other vertex in the graph. Use this function to solve problems #1 and #2. Present your solution idea with a pseudocode, accompanying necessary explanation.

Problem #1: Optimum meeting place

You, and your friend Benji, live in a city which has 50 areas numbered from 1 to 50. Some of the areas are connected with bi-directional roads of various lengths. Your house is in 1 and Benji's is in 50. Both of you want to meet. Now propose an algorithm to choose the meeting point that minimizes the total travel time. In other words, if the meeting area you chose is X , it has the minimum $dist(1, X) + dist(50, X)$ among all possible area choices.

Expected time complexity: $O(E * \log_2(V))$

Problem #2: Meet in the middle!

Continuing on the scenario of Problem #1, but assuming all the roads are one-way (representing a directed graph)...

Benji now has asked you to visit his house in area 50. He likes KFC, so you are thinking about getting a bucket of fried chicken for him. There are KFC outlets available in K different areas in the city. You have to make a stop at one of them. Now propose an algorithm to choose the area from the available K options that minimizes your total travel time. In other words, if the area you chose is X , it has the minimum $dist(1, X) + dist(X, 50)$ among all possible choices.

Expected time complexity: $O(E * \log_2(V))$

*Note: If you have to run Dijkstra's algorithm from K different sources, the complexity is $O(K * E * \log_2(V))$, which exceeds the expected complexity for this problem.*

Problem #3: Dijkstra's algorithm in a 'negative' weight graph!!

Draw a directed graph as described below.

No. of vertices: 7, No. of edges: 9

List of vertices, V : {1, 2, 3, 4, 5, 6, 7}

List of edges, E [in (u, v, w) format which means there is an outgoing edge from u to v with weight w] :

{(1, 2, 8), (2, 3, - 8), (1, 3, 1) (3, 4, 4), (4, 5, - 4), (3, 5, 1), (5, 6, 2), (6, 7, - 2), (5, 7, 1)}

- a. Show a simulation of Dijkstra's algorithm to find the *shortest path costs* from vertex 1 to all others. You can use the following pseudocode as reference.

Note: Dijkstra's algorithm inserts all vertices in a priority-queue/min-heap at the beginning and then repeatedly extracts the vertex with minimum distance. It assumes, whenever a vertex is extracted from the queue, its shortest path cost from the source is finalized.

```
DIJKSTRA( $G, s$ )
1  for each  $v \in V[G]$ 
2      do  $d[v] \leftarrow \infty$ 
3       $\pi[v] \leftarrow \text{NIL}$ 
4   $d[s] \leftarrow 0$ 
5   $S \leftarrow \emptyset$ 
6   $Q \leftarrow V[G]$ 
7  while  $Q \neq \emptyset$ 
8      do  $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
9       $S \leftarrow S \cup \{u\}$ 
10     for each vertex  $v \in \text{Adj}[u]$ 
11         do if  $d[v] > d[u] + w(u, v)$ 
12             then  $d[v] \leftarrow d[u] + w(u, v)$ 
13                  $\pi[v] \leftarrow u$ 
```

- b. Explain why the algorithm could not find the correct shortest path costs for some of the vertices.
- c. Imagine you have a modified version of Dijkstra's algorithm with the following changes:
- At the beginning, it inserts only the source vertex in the priority-queue.
 - Whenever the distance of a vertex is updated, it inserts that vertex into the priority-queue.
 - It does not assume that extracting a vertex from the queue means that its shortest path cost is finalized.

Will this modified Dijkstra's algorithm be able to find the correct shortest path costs in the graph given above? Explain with reasoning or a simulation.

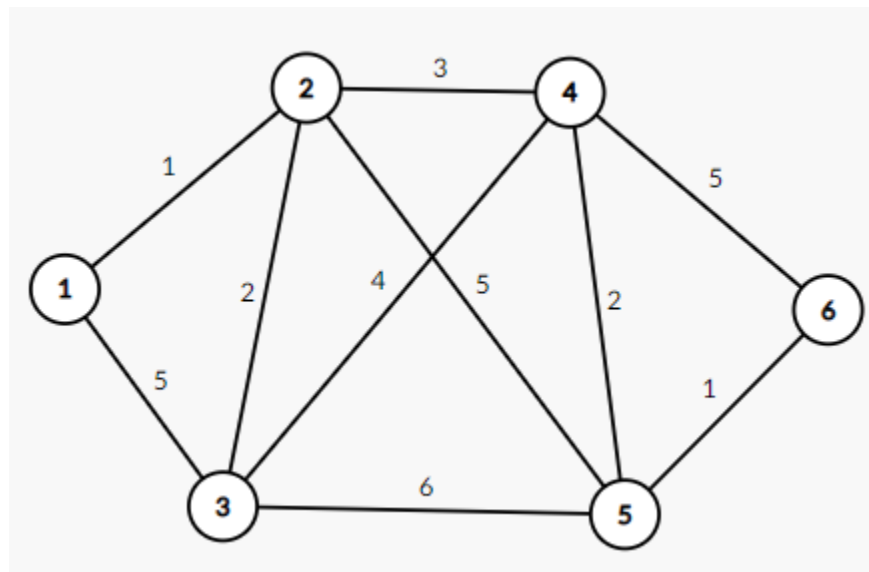
- d. Analyze the time-complexity of this modified algorithm.

Problem #4: A divide-and-conquer MST algorithm?!

Your friend Mriaslun loves the divide and conquer strategy! He recently invented a divide-and-conquer algorithm to find the Minimum Spanning Tree of a given undirected weighted graph. The algorithm is quite simple, just having 4 steps in it:

1. Divide the set of vertices into two equal halves (one side can have an extra, like we do in merge sort). Assume that this divides the graph in three parts:
 - a. G_{left} which contains all the vertices in the left half and the edges between them
 - b. G_{right} containing the vertices in the right half, and the corresponding edges
 - c. $G_{remains}$ containing the edges where one vertex is from the left half, and another one is from the right half
2. Use a recursive approach (repeating steps 1 to 4) to find the MST of G_{left} , let's call it MST_{left}
3. Do the same to find MST_{right}
4. Now you only need to select one edge from $G_{remains}$ to connect MST_{left} to MST_{right} . So you pick the one with the lowest weight value and you have the MST of the original graph!

Let's see an example.



In G_{left} , we have three vertices $\{1, 2, 3\}$ and three edges $\{(1, 2), (1, 3), (2, 3)\}$

In G_{right} , we have vertices $\{4, 5, 6\}$ and edges $\{(4, 5), (4, 6), (5, 6)\}$

And $G_{remains}$ is defined by the edges $\{(2, 4), (2, 5), (3, 4), (3, 5)\}$

If we can find MST_{left} and MST_{right} separately, then we can connect them with the minimum weight edge from $G_{remains}$, which is (2, 4). And we can use the same divide and conquer approach recursively to know that MST_{left} consists of the edges {(1, 2), (2, 3)} and MST_{right} contains the edges {(4, 5), (5, 6)}.

So, the MST of the original graph is {(1, 2), (2, 3), (2, 4), (4, 5), (5, 6)} which is the correct answer!

Now your job is to prove that this algorithm will not always give a correct answer. Provide necessary reasonings to do that. You can also create an input graph and show that the algorithm fails or provides a wrong answer for this input.

Problem #5: Coin Change

There are a few types of banknotes available in your country's currency system. Any amount of taka that you want to give someone has to be constructed using these notes. Suppose you are given an infinite supply of these banknotes. You have to find the minimum number of notes needed to construct the amount.

Now, propose an algorithm applying your knowledge on graph theory which will take the currency notes and the amount to make, K as input. Then as output, return the minimum number of notes and the value of the notes.

Examples:

Input	Output
Notes: 1, 2, 5, 10 Amount to make: 21	3 notes needed: 10, 10, 1
Notes: 3, 5, 8 Amount to make: 20	4 notes needed: 5, 5, 5, 5
Notes: 3, 5, 8 Amount to make: 7	Impossible

The following questions might help:

1. Which graph theory algorithm might be relevant here?
2. How do you convert this into a graph? What are the vertices? What are the edges?

END!
